

Cyclone4 Mini_FPGA

2025.11 (Rev 2.1)

Оглавление

Первая часть: Общее описание платы MINI_FPGA

1.1	Архитектура платы MINI_FPGA	1
1.2	Введение в FPGA-чип	2
1.2.1	Введение в FPGA-чип	2
1.2.2	Способы конфигурации Cyclone IV E	2
1.2.3	Конфигурация платы MINI_FPGA	3
1.3	Обзор периферийных модулей	6
1.3.1	Модули ввода	7
1.3.2	Модули отображения	9
1.3.3	Аудиомодули	12
1.3.4	Тактовые модули	13
1.3.5	Порты расширения IO	14
1.3.6	Модуль хранения данных	14
1.3.7	Интерфейсные модули	16
1.3.8	Блок питания	17
1.4	Создание проекта в Quartus	17
1.5	Генерация ROM таблиц (IP ядра Altera)	34
1.6	Генерация PLL (IP ядра Altera)	40
1.7	Использование SignalTap	43
1.8	Генерация FIR-фильтра (IP ядра)	49

Базовые эксперименты: проектирование и реализация . 54

Эксперимент 1 — Бегущие огни 54

1.	Цели эксперимента	54
2.	Основная идея проектирования	54
3.	Функциональная схема модулей и описание входных/выходных выводов	54
4.	Разработка программы	54
5.	Конфигурация выводов FPGA	55
6.	Результаты эксперимента	56
7.	Вопросы и расширенные задания	56

Эксперимент 2 — Интегральные логические элементы и их базовые применения 57

实验 2.1 实现基本逻辑门功能	57
实验 2.2 利用门电路设计实现全加器功能	60
实验三 译码器 编码器	64
实验 3.1 实现 3-8 译码器	64
实验 3.2 实现 8-3 优先编码器	68
实验四 数据选择器	73
一、实验设计目标	73
二、实验设计思路	73
三、功能模块图与输入输出引脚说明	74
四、程序设计	75
五、FPGA 管脚配置	76
六、实验结果	77
七、实验小结	77
实验五 触发器	79
实验 5.1 使用门级结构描述 D 触发器	79
实验 5.2 使用行为描述语句实现 8D 触发器	84
实验 5.3 实现 4JK 触发器	88
实验六 加法计数器	93
一、实验设计目标	93
二、实验设计思路	93
三、功能模块图与输入输出引脚说明	93
四、程序设计	94
五、FPGA 管脚配置	97
六、实验结果	97
七、思考与拓展	98
实验七 抢答器	98
一、实验设计目标	98
二、实验设计思路	99
三、功能模块图与输入输出引脚说明	100
四、程序设计	101
五、FPGA 管脚配置	102
六、实验结果	103

七、思考与拓展.....	104
实验八 功能数字钟.....	104
一、实验设计目标	104
二、实验设计思路	105
三、功能模块图与输入输出引脚说明.....	106
四、程序设计	107
五、FPGA 管脚配置	108
六、实验结果	108
七、思考与拓展.....	109
第三篇 综合性实验设计与实现.....	110
实验一 PWM.....	110
一、实验设计目标	110
二、实验设计思路	110
三、功能模块图与输入输出引脚说明.....	111
四、程序设计	113
五、FPGA 管脚配置	114
六、实验结果	115
七、思考与拓展.....	115
实验二 DA 及 DDS	117
实验 2.1 DA 输出实验.....	117
实验 2.2 DDS 实验	122
实验三 AD.....	128
实验 3.1 AD 采样实验.....	128
实验 3.2 时序分析实验	135
实验四 UART 串行通信.....	141
实验 4.1 UART 串口接收实验.....	141
实验 4.2 UART 串口发送实验.....	147
实验五 FIR 滤波器.....	153
一、实验设计目标	153
二、实验设计思路	153
三、功能模块图与输入输出引脚说明.....	154
四、程序设计	156

五、FPGA 管脚配置156

六、实验结果157

七、思考与拓展161

Первая часть — Общее введение в плату MINI_FPGA

Данная цифровая экспериментальная платформа разработана специально для учебного курса «Цифровые схемы».

Основные требования к системе:

охват базовых функций, простота использования,

наличие разумных возможностей расширения,

отсутствие стремления к «максимальной универсальности».

Основываясь на учебном опыте вузов и практических требований лабораторных работ

, в качестве основного чипа была выбрана FPGA EP4CE6F17C8N.

При проектировании периферийных модулей ключевыми целями были:

простота и наглядность, компактность и удобство переноски.

Плата включает следующие основные категории модулей:

Встроенный USB-Blaster

Позволяет осуществлять питание и программирование по одному USB-кабелю.

Модули вывода, такие как светодиоды (LED) и семисегментные индикаторы.

Модули ввода, такие как кнопки и тумблеры (переключатели).

Аудио-модули, например пьезодинамик (buzzer).

Интерфейсные модули, такие как преобразователь UART-to-USB.

Модули памяти, включая внешнюю Flash-память и EEPROM.

1.1 Архитектура (структура) платы MINI_FPGA

Структурная схема платы MINI_FPGA показана на рис. 1.1.

Она состоит из трёх основных частей:

(1) Основной FPGA-чип

Используется FPGA серии Cyclone IV — EP4CE6F17C8 в корпусе BGA с 256 выводами.

(2) Периферийные устройства Включают:

светодиоды (LED), семисегментные индикаторы, пьезодинамик, кнопки,

тумблеры (SW), разъёмы JTAG и UART,

76 линий универсальных GPIO на верхней, нижней и левой сторонах платы,

несколько линий GND и питания, схему USB-Blaster (встроенный программатор),

внешнюю память, блок питания, кварцевый генератор 50 МГц.

Все это интегрировано для уменьшения габаритов и удобства использования.

(3) USB-кабель для загрузки

Используется для передачи данных и питания между ПК и платой MINI_FPGA.

Рисунки 1.1 и 1.2 показывают внешний вид платы MINI_FPGA —

вид спереди и сзади соответственно.

Подробное описание FPGA-чипа приведено в разделе 1.2,

а описание периферийных модулей — в разделе 1.3.

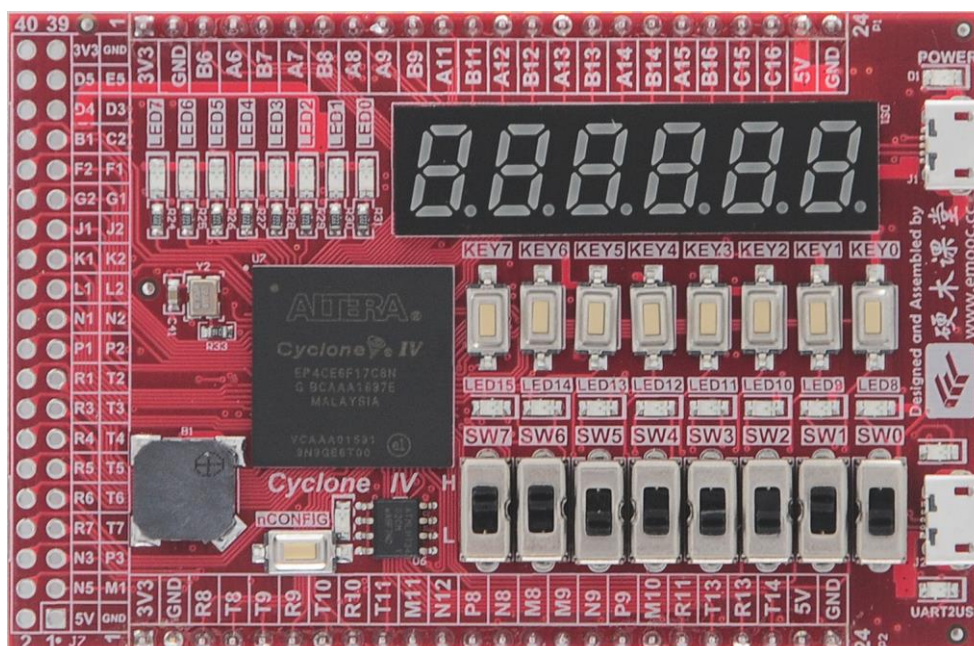


Рисунок 1.1

1.2 Краткое описание основного FPGA-чипа

1.2.1 Ресурсы серии Cyclone IV E

FPGA серии Cyclone IV демонстрируют сильные стороны компании Altera в области разработки эн

По сравнению с предыдущим поколением Cyclone III, серия Cyclone IV получила:

улучшенную архитектуру,

оптимизированную кристалльную структуру,

более продвинутый полупроводниковый технологический процесс,

расширенные инструменты управления энергопотреблением.

Благодаря этим улучшениям Altera снизила энергопотребление на 25%.

Основные характеристики серии Cyclone IV E:

Архитектура FPGA с низкой стоимостью и низким энергопотреблением

Диапазон логических элементов от 6K до 150K

До 6.3 Мбит встроенной оперативной памяти

До 360 умножителей 18×18

— поддержка вычислительно-интенсивных DSP-приложений

Поддержка протокольных мостов (protocol bridging)

— позволяет реализовать системные решения с энергопотреблением менее 1.5 Вт

Основной чип данной учебной платформы

Плата MINI_FPGA использует микросхему EP4CE6F17C8N, которая имеет:

256 выводов,

6272 логических элемента,

корпус BGA,

встроенную память и DSP-блоки.

Подробная компоновка ресурсов FPGA отображена на рисунке 1.3.

Resources	EP4CE6	EP4CE10	EP4CE15	EP4CE22	EP4CE30	EP4CE40	EP4CE55	EP4CE75	EP4CE115
Logic elements (LEs)	6,272	10,320	15,408	22,320	28,848	39,600	55,856	75,408	114,480
Embedded memory (Kbits)	270	414	504	594	594	1,134	2,340	2,745	3,888
Embedded 18 × 18 multipliers	15	23	56	66	66	116	154	200	266
General-purpose PLLs	2	2	4	4	4	4	4	4	4
Global Clock Networks	10	10	20	20	20	20	20	20	20
User I/O Banks	8	8	8	8	8	8	8	8	8
Maximum user I/O	179	179	343	153	532	532	374	426	528

所选芯片资源

1.2.2 Способы конфигурации устройств серии Cyclone IV E

Устройства серии Cyclone IV E поддерживают пять режимов конфигурации:

AS — Active Serial (активная последовательная) AP — Active Parallel (активная параллельная)

PS — Passive Serial (пассивная последовательная) FPP — Fast Passive Parallel (быстрая пассивная параллельная)

JTAG — конфигурация через интерфейс JTAG

При использовании разработчик может выбрать один или несколько доступных режимов —

в зависимости от требований проекта.

Однако важно учитывать:

При каждой загрузке FPGA можно использовать только один конкретный режим конфигурации.

Одновременное использование нескольких режимов недопустимо.

Далее в документе (рис. 1.7) приведена официальная таблица конфигурационных режимов Cyclone IV E,

в которой указано:

какие устройства конфигурации поддерживаются, каким образом данные передаются в FPGA,

различия между активными и пассивными режимами.

Смысл активных и пассивных режимов

Активные режимы (Active Serial / Active Parallel)

В них сама FPGA инициирует процесс загрузки, считывая данные из внешнего Flash-памяти.

Пассивные режимы (PS / FPP)

В этих режимах прошивка передаётся внешним контроллером (например, микроконтроллером или ПЛИС).

JTAG

Особый независимый режим, позволяющий:

программировать FPGA напрямую,

программировать внешнюю Flash (EPCS/EPCQ),

отлаживать проект.

Режим JTAG можно использовать в любое время, даже если аппаратные переключатели (MSEL) настроены под другой режим.

В активном режиме загрузки FPGA самостоятельно считывает конфигурационные данные из внешней памяти и загружает их во внутреннюю конфигурационную схему FPGA.

В пассивном режиме конфигурация осуществляется внешним контроллером (например, MCU), который формирует требуемые временные последовательности и подаёт конфигурационные данные во вход FPGA снаружи.

Разница между последовательным (serial) и параллельным (parallel) режимами определяется шириной данных, которые передаются в FPGA за один тактовый цикл конфигурации:

последовательный режим (1 бит) — данные идут по одному биту за такт;

параллельный режим (8/16 бит) — ширина шины составляет 8 или 16 бит, скорость конфигурации значительно выше. JTAG — самый простой и самый распространённый режим

Режим JTAG — наиболее простой, самый часто используемый и обладает более высоким приоритетом, чем другие режимы конфигурации.

Из примечаний (2) и (3) на рисунке 1.4 (в оригинале) видно:

Выбор режима JTAG не зависит от уровней на конфигурационных выводах MSEL.

Все остальные режимы (AS, PS, FPP, AP) требуют настройки конфигурационных выводов MSEL в определённые состояния для определения способа загрузки.

Иначе говоря:

JTAG можно использовать всегда

независимо от того, как установлены MSEL.

Остальные режимы работают только при правильной комбинации MSEL-пинов.

Configuration Scheme	MSEL3	MSEL2	MSEL1	MSEL0	POR Delay	Configuration Voltage Standard (V) ⁽¹⁾
AS	1	1	0	1	Fast	3.3
	0	1	0	0	Fast	3.0, 2.5
	0	0	1	0	Standard	3.3
	0	0	1	1	Standard	3.0, 2.5
AP	0	1	0	1	Fast	3.3
	0	1	1	0	Fast	1.8
	0	1	1	1	Standard	3.3
	1	0	1	1	Standard	3.0, 2.5
	1	0	0	0	Standard	1.8
PS	1	1	0	0	Fast	3.3, 3.0, 2.5
	0	0	0	0	Standard	3.3, 3.0, 2.5
FPP	1	1	1	0	Fast	3.3, 3.0, 2.5
	1	1	1	1	Fast	1.8, 1.5
JTAG-based configuration ⁽²⁾	⁽³⁾	⁽³⁾	⁽³⁾	⁽³⁾	—	—

Notes
(1) Configuration voltage standard applied to the V_{CCIO} supply of the bank in which the configuration pins reside.
(2) JTAG-based configuration takes precedence over other configuration schemes, which means the MSEL pin settings are ignored.
(3) Do not leave the MSEL pins floating. Connect them to V_{CCA} or GND. These pins support the non-JTAG configuration scheme used in production. Altera recommends connecting the MSEL pins to GND if your device is only using JTAG configuration.

На плате MINI_FPGA, поскольку используется только один FPGA-чип, нет необходимости синхронизировать конфигурацию с другими FPGA или выполнять распределённую загрузку. Кроме того, чтобы обеспечить:

удобную загрузку прошивки через JTAG, возможность одновременной отладки во время разработки, в качестве основных методов конфигурации для платы выбраны:

AS-режим (Active Serial) и JTAG-режим.

1.2.3 Конфигурирование платы MINI_FPGA

Плата MINI_FPGA содержит одну серийную Flash-память (Serial Flash Memory), в которой хранится конфигурационный файл FPGA.

Каждый раз при включении питания:

FPGA автоматически считывает данные конфигурации из этой Flash-памяти и загружает себя.

Используя программное обеспечение Quartus II, пользователь может в любой момент: перепрограммировать FPGA,

изменить содержимое Serial Flash Memory, записав туда новый конфигурационный файл.

Таким образом, изменённая прошивка будет автоматически загружаться при следующем включении питания.

На рисунке 1.5 приведена схема цепей, связанных с конфигурацией платы MINI_FPGA. В ней показаны:

сигнальные линии, относящиеся к AS-режиму, линии интерфейса JTAG, назначение конфигурационных выводов FPGA.

(1) Вывод состояния конфигурации — nSTATUS

Это двунаправленный (bidirectional) вывод.

Во время процесса конфигурации FPGA, если сигнал nSTATUS переходит: из высокого уровня в низкий, то это означает:

Произошла ошибка конфигурации, и требуется повторно выполнить загрузку FPGA.

(2) Вывод управления конфигурацией — nCONFIG

Это входной вывод, управляющий процессом конфигурации.

В данной плате его поведение следующее:

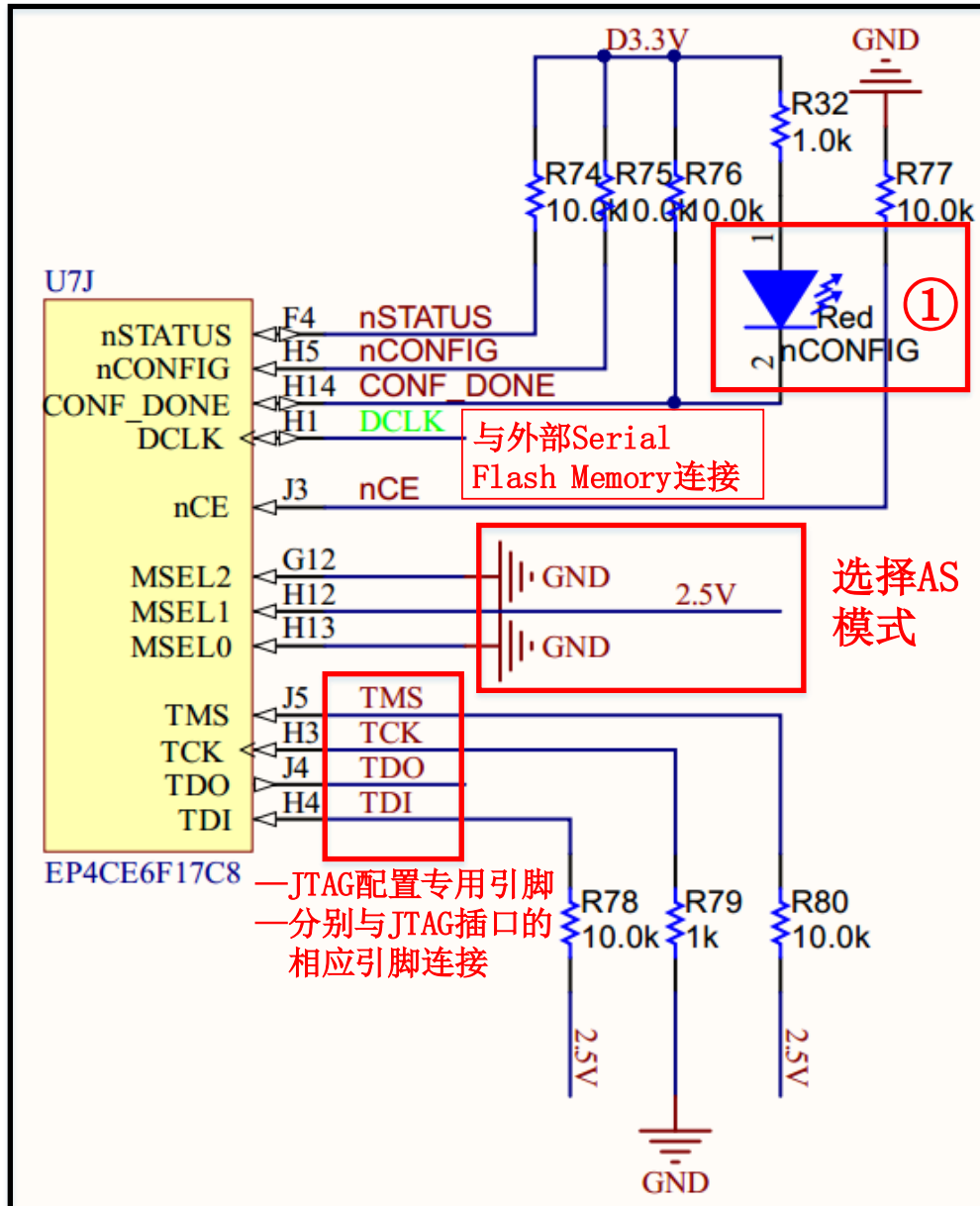
Когда nCONFIG меняется с высокого уровня на низкий,

→ FPGA выполняет сброс (reset). Когда nCONFIG меняется с низкого уровня на высокий,

→ начинается процесс конфигурации FPGA.

То есть: Падение nCONFIG = аппаратный сброс FPGA

Возврат в «1» = запуск процедуры конфигурации



При обычной загрузке (нормальном включении питания) процесс конфигурации выполняется так: FPGA самостоятельно считывает конфигурационные данные из внешней Serial Flash Memory (серийной флеш-памяти) и загружает их внутрь себя. Это и называется:

AS-режим конфигурации (Active Serial).

(3) Вывод завершения конфигурации — CONF_DONE

Это двунаправленный вывод, который показывает, завершён ли процесс конфигурации.

Поведение в данной плате: Когда нажимается кнопка nCONFIG на плате MINI_FPGA (при нажатии уровень сигнала nCONFIG = 0), → происходит перезапуск конфигурации FPGA.

Когда FPGA успешно завершает загрузку конфигурации из Flash,

→ вывод CONF_DONE переключается в высокий уровень.

То есть:

Высокий уровень CONF_DONE = конфигурация завершена успешно

Низкий уровень / отсутствие перехода = конфигурация не завершена

Когда входной сигнал nCONFIG переходит в низкий уровень, FPGA выполняет сброс и входит в процесс AS-конфигурации.

Когда же используется интерфейс JTAG для загрузки скомпилированных данных внутрь FPGA, устройство находится в режиме JTAG-конфигурации.

Как показано на рисунке 1.5, пометка :

до начала конфигурации и в процессе загрузки

→ FPGA удерживает сигнал CONF_DONE в низком уровне,

→ соответствующий индикатор на плате горит красным.

после того как конфигурационные данные успешно загружены и FPGA переходит в стадию инициализации

→ сигнал CONF_DONE поднимается в высокий уровень,

→ индикатор гаснет,

→ конфигурация завершена.

(4) Вывод конфигурационного тактового сигнала — DCLK

Это двунаправленный сигнал.

В режиме AS-конфигурации

FPGA использует DCLK как выход, формируя тактовый сигнал для Serial Flash Memory, обеспечивая ей требуемую временную последовательность.

В режиме JTAG-конфигурации

FPGA использует входной тактовый сигнал, который поступает от кварцевого генератора на плате (50 МГц).

Для этого используется входной вывод E1.

(5) Вывод разрешения конфигурации — nCE

Это входной сигнал, активный по низкому уровню.

В данной плате он подключён к земле через подтягивающий резистор (pull-down), чтобы удерживать его в активном состоянии.

(6) Выводы выбора режима конфигурации — MSEL[3:0]

Все режимы конфигурации, кроме JTAG, определяются состоянием выводов MSEL.

Согласно рисунку 1.4:

для режима AS (Active Serial)

необходимо установить MSEL[3:0] = 0010

Это соответствует:

стандартной задержке сброса при включении питания;

стандарту конфигурационного напряжения 3.3 В.

(7) Вывод конфигурационных данных — DATA

В режиме последовательной конфигурации (serial configuration):

FPGA получает конфигурационные данные через вывод DATA0.

Как показано на рисунке 1.17:

вывод DATA0 FPGA соединён с выводом DO / IO1 внешней Serial Flash Memory (они используют один и тот же вывод).

(8) Выводы граничного сканирования (JTAG) — TMS, TCK, TDO, TDI

Это четыре сигнала, предназначенные исключительно для JTAG-конфигурации:

TMS — Test Mode Select (выбор тестового режима)

TCK — Test Clock (тестовый тактовый сигнал)

TDO — Test Data Out (выход тестовых данных)

TDI — Test Data In (вход тестовых данных)

См. рисунок 1.9.

В данной плате эти выводы должны быть подключены к соответствующим выводам JTAG-разъёма. Схема подключения приведена на рисунке 1.20.

Для обеспечения корректного «обходного режима» (bypass mode) JTAG-интерфейса, когда FPGA работает в нормальном режиме, необходимо:

подтянуть TMS к высокому уровню (pull-up),

подтянуть TDI к высокому уровню (pull-up),

подтянуть TCK к низкому уровню (pull-down).

Pin Name	Pin Type	Description
TDI	Test data input	Serial input pin for instructions as well as test and programming data. Data shifts in on the rising edge of TCK. If the JTAG interface is not required on the board, the JTAG circuitry is disabled by connecting this pin to V _{CC} . TDI pin has weak internal pull-up resistors (typically 25 k Ω).
TDO	Test data output	Serial data output pin for instructions as well as test and programming data. Data shifts out on the falling edge of TCK. The pin is tri-stated if data is not being shifted out of the device. If the JTAG interface is not required on the board, the JTAG circuitry is disabled by leaving this pin unconnected.
TMS	Test mode select	Input pin that provides the control signal to determine the transitions of the TAP controller state machine. Transitions in the state machine occur on the rising edge of TCK. Therefore, TMS must be set up before the rising edge of TCK. TMS is evaluated on the rising edge of TCK. If the JTAG interface is not required on the board, the JTAG circuitry is disabled by connecting this pin to V _{CC} . TMS pin has weak internal pull-up resistors (typically 25 k Ω).
TCK	Test clock input	The clock input to the BST circuitry. Some operations occur at the rising edge, while others occur at the falling edge. If the JTAG interface is not required on the board, the JTAG circuitry is disabled by connecting this pin to GND. The TCK pin has an internal weak pull-down resistor.

AS-конфигурация (Active Serial)

AS-режим является одним из наиболее часто используемых способов конфигурации FPGA. В этом режиме плата MINI_FPGA должна соединить FPGA с внешней Serial Flash Memory (эквивалент чипа EPCS16) по четырём сигналам:

DCLK — тактовый сигнал nCSO — сигнал выбора кристалла (chip select output)

DATA0 — вход данных конфигурации

ASDO — выход данных из FPGA для управления Serial Flash
(как показано на рисунке 1.17)

Назначение сигналов:

DCLK — обеспечивает Serial Flash тактовым сигналом.

nCSO — активирует Serial Flash, выбирая её как источник данных.

DATA0 — Serial Flash передаёт FPGA конфигурационные данные через этот вывод.

ASDO — FPGA передаёт во Flash команды чтения/записи, адреса, а также данные при программировании Flash.

Полный процесс конфигурации

(1) AS-режим (обычная загрузка при включении питания)

После подачи питания сигнал nCONFIG сначала переходит из высокого уровня в низкий — FPGA выполняет сброс.

Затем nCONFIG возвращается в высокий уровень, и FPGA начинает загрузку в режиме AS.

FPGA самостоятельно читает конфигурационные данные из Serial Flash (EPCS16).

После завершения загрузки FPGA входит в стадию инициализации, а затем переходит в пользовательский режим.

(2) JTAG-режим

Через интерфейс JTAG в FPGA загружается скомпилированная конфигурация (SOF-файл).

После завершения загрузки FPGA начинает работать нормально.

В этом режиме конфигурационные данные не сохраняются:

если питание отключить — FPGA теряет содержимое (т. е. SOF живёт только до выключения).

(3) Повторная конфигурация через nCONFIG

Если нажать кнопку сброса на плате и отпустить её:

уровень сигнала nCONFIG снова падает в «0»,

затем возвращается в «1»,

и весь процесс конфигурации повторяется с шага (1).

Статус конфигурации можно определить по индикатору рядом с кнопкой reset:

горит красный светодиод — идёт процесс конфигурации (FPGA загружает данные)

светодиод погас — конфигурация завершена корректно

1.3 Введение в периферийные функциональные модули

После того как определены:
модель FPGA-чипа
и способы конфигурации платы,
следующим шагом является разработка периферийных функциональных модулей.
Периферия платы MINI_FPGA включает четыре основные категории:
Операционные модули — кнопки, тумблеры
Модули отображения — LED, семисегментные индикаторы
Аудио-модули — пьезодинамик (buzzer)
Внешние интерфейсы — UART, GPIO, память и др.
Полное описание каждой категории приведено далее в разделе 1.3.

1.3.1 Модули ввода

Эти модули используются для подачи в систему операционных сигналов или сигналов прерывания. К ним относятся кнопочные переключатели и тумблерные переключатели.

(1) Кнопочные переключатели Плата MINI_FPGA предоставляет:

8 кнопок: KEY7—KEY0 одну кнопку сброса (RESET)(показано на рис. 1.10)

Схема подключения кнопки RESET:

одна сторона кнопки соединена с землёй (GND)

другая сторона — непосредственно подключена к выводу nCONFIG FPGA

Поведение сигнала:

Когда кнопка RESET нажата → на вход FPGA подаётся низкий уровень,
и сигнал nCONFIG = 0 (активный низкий уровень вызывает аппаратный сброс).

Когда кнопка RESET отпущена → вывод nCONFIG соединён с подтягивающим резистором,
и сигнал nCONFIG = 1 (высокий уровень), FPGA выходит из сброса.

Схема подключения кнопок KEY7—KEY0:

Кнопки KEYx подключены аналогично кнопке RESET: одна сторона — к GND,

другая — напрямую к соответствующим входным пинам FPGA.

Поэтому:

при нажатии кнопки на вход FPGA подаётся низкий уровень,

при ненажатой кнопке вход удерживается на высоком уровне через подтягивающий резистор.

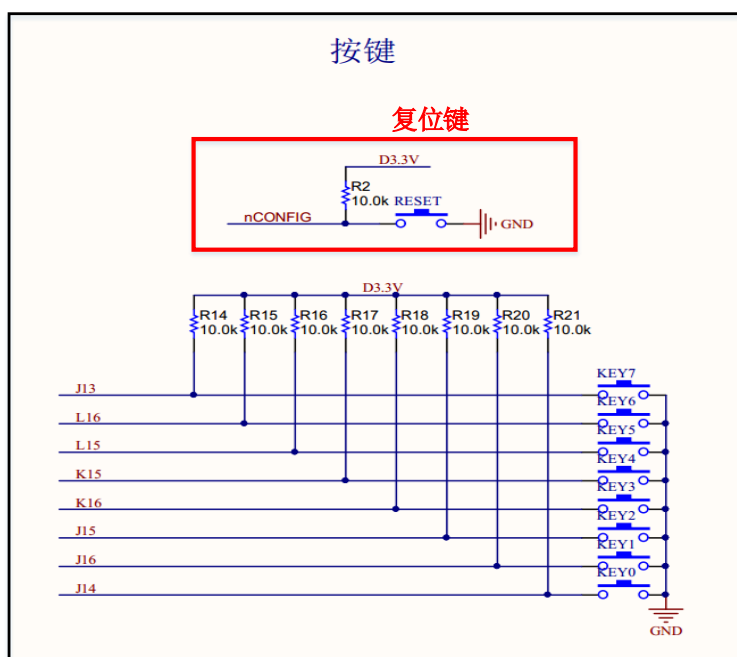


Рис. 1.10 — Схема подключения кнопочных переключателей

(2) Тумблерные переключатели (SW7—SW0)

На плате MINI_FPGA также расположены 8 тумблерных переключателей SW7—SW0, как показано на рисунке 1.11.

Поведение переключателей:

Когда тумблер находится в положении DOWN

(то есть ближе к краю платы),

→ на соответствующий вход FPGA подаётся низкий уровень (0).

Когда тумблер находится в положении UP,

→ на вход FPGA подаётся высокий уровень (1).

Таблицы 1.1 и 1.2

Таблица 1.1 содержит информацию о подключении всех кнопок KEY7—KEY0.

Таблица 1.2 содержит информацию о подключении всех тумблеров SW7—SW0.

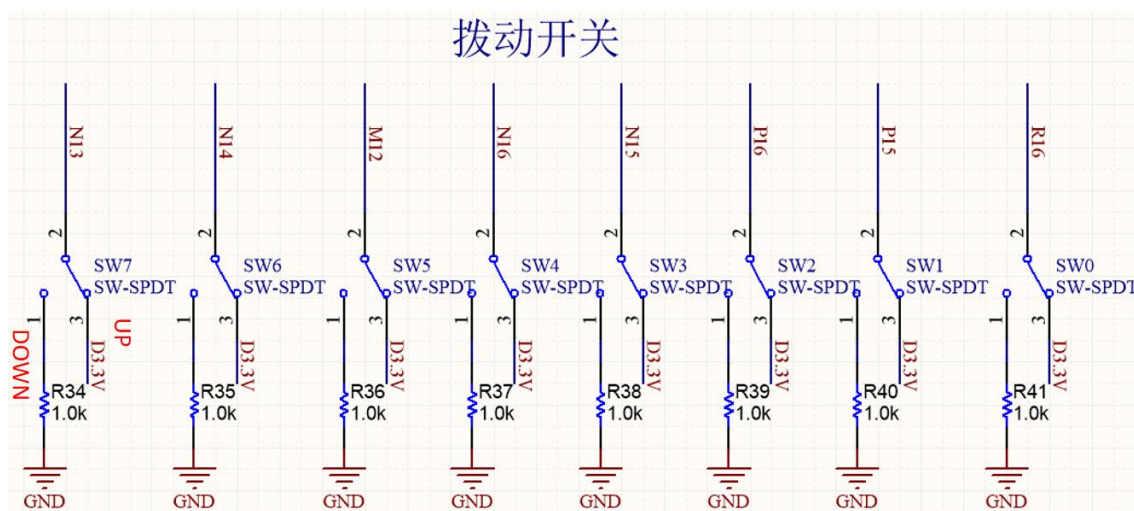


Рис. 1.11 — Схема подключения тумблерных переключателей

Таблица 1.1 — Назначение выводов кнопочных переключателей

Имя сигнала	FPGA Номер вывода (пина)	Описание
Key_In[0]	J14	KEY0
Key_In[1]	J16	KEY1
Key_In[2]	J15	KEY2
Key_In[3]	K16	KEY3
Key_In[4]	K15	KEY4
Key_In[5]	L15	KEY5
Key_In[6]	L16	KEY6
Key_In[7]	J13	KEY7

Таблица 1.2 — Конфигурация выводов тумблерных переключателей

Имя сигнала	FPGA Номер вывода (пина)	Описание
SW_In[0]	R16	SW0

SW_In[1]	P15	SW1
SW_In[2]	P16	SW2
SW_In[3]	N15	SW3
SW_In[4]	N16	SW4
SW_In[5]	M12	SW5
SW_In[6]	N14	SW6
SW_In[7]	N13	SW7

1.3.2 Модули вывода и отображения

Эти модули предназначены для визуального отображения результатов работы системы — с помощью светодиодов или семисегментных индикаторов.

(1) LED-индикаторы

Плата MINI_FPGA содержит 16 светодиодов LED15–LED0, которые непосредственно управляются FPGA.

Каждый светодиод подключён к отдельному выводу FPGA (см. рисунок 1.12).

Принцип работы:

когда FPGA выводит высокий уровень, соответствующий светодиод загорается;

когда вывод имеет низкий уровень, светодиод гаснет.

В таблице 1.3 приведена информация о том, какие светодиоды подключены к каким выводам FPGA.

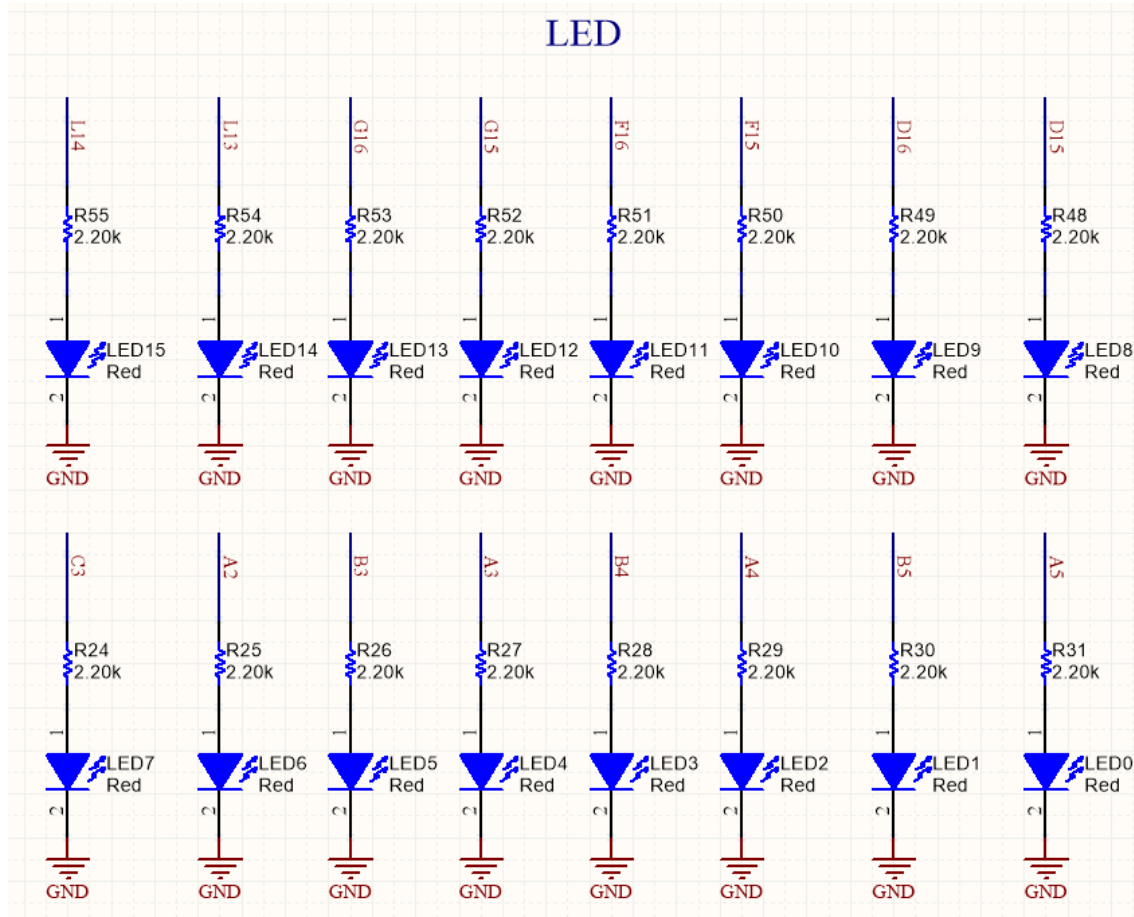


Рис. 1.12 — Схема подключения светодиодов (LED)

Имя сигнала	FPGA Номер вывода (пина)	Описание
LED_Out[0]	A5	LED0
LED_Out[1]	B5	LED1
LED_Out[2]	A4	LED2
LED_Out[3]	B4	LED3
LED_Out[4]	A3	LED4
LED_Out[5]	B3	LED5
LED_Out[6]	A2	LED6
LED_Out[7]	C3	LED7
LED_Out[8]	D15	LED8
LED_Out[9]	D16	LED9

LED_Out[10]	F15	LED10
LED_Out[11]	F16	LED11
LED_Out[12]	G15	LED12
LED_Out[13]	G16	LED13
LED_Out[14]	L13	LED14
LED_Out[15]	L14	LED15

(2) Семисегментные индикаторы ()

На плате MINI_FPGA установлено 6 семисегментных индикаторов

DIG1–DIG6 (если смотреть на плату в нормальном положении — считаются слева направо).

Каждый индикатор управляется отдельным сигналом выбора (chip select):

DigCS1 — DigCS6 (см. рисунок 1.13)

Все выводы семисегментных индикаторов выполнены в конфигурации общий катод (, common cathode). Каждый сегмент подключён непосредственно к выводам FPGA, и управляется логическим уровнем, подаваемым с этих выводов.

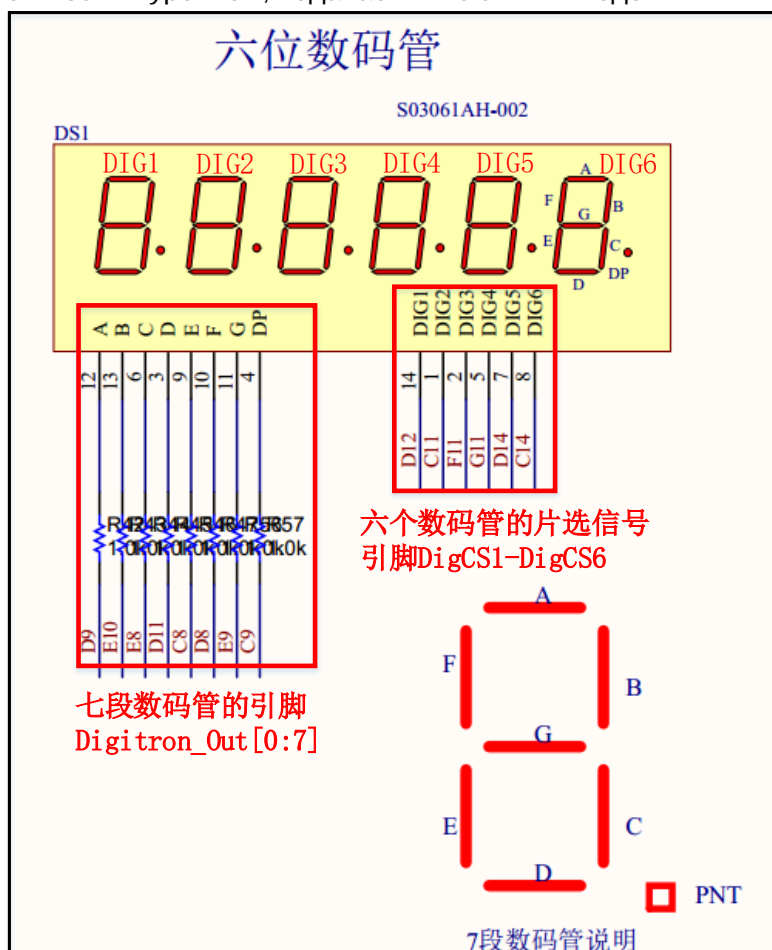


Рис. 1.13 — Схема подключения семисегментных индикаторов

Семисегментные индикаторы

Выводы сегментов каждого семисегментного индикатора напрямую подключены к FPGA (микросхема Cyclone IV EP4CE6F17C8).

Когда FPGA выводит высокий уровень, соответствующий сегмент загорается.

Когда FPGA выводит низкий уровень, сегмент гаснет.

Сигналы выбора индикаторов (chip select) также подключены непосредственно к выводам FPGA:

Когда FPGA выводит низкий уровень на линию выбора конкретного индикатора (DigCSx), этот индикатор становится активным (включён).

Когда на выходе высокий уровень, соответствующий индикатор не выбирается (отключён).

Таким образом: Сегменты — активны по высокому уровню

Выбор индикатора (CS) — активен по низкому уровню

Таблица 1.4 содержит информацию о соответствии: сегментов (A, B, C, D, E, F, G, DP), линий выбора DigCS1–DigCS6, и их назначенных пинов FPGA.

Конфигурация выводов семисегментных индикаторов

Имя сигналов	FPGA Номер пина	Описание
Digitron_Out[0]	D9	Сегмент A
Digitron_Out[1]	E10	Сегмент B
Digitron_Out[2]	E8	Сегмент C
Digitron_Out[3]	D11	Сегмент D
Digitron_Out[4]	C8	Сегмент E
Digitron_Out[5]	D8	Сегмент F
Digitron_Out[6]	E9	Сегмент G
Digitron_Out [7]	C9	Сегмент DP
DigitronCS_Out[0]	C14	Chip Select DigCS6
DigitronCS_Out[1]	D14	Chip Select DigCS 5
DigitronCS_Out[2]	G11	Chip Select DigCS 4
DigitronCS_Out[3]	F11	Chip Select DigCS 3
DigitronCS_Out[4]	C11	Chip Select DigCS 2
DigitronCS_Out[5]	D12	Chip Select Dig CS1

1.3.3 Аудио- или звуковые модули

(1) Пьезодинамик (буззер)

На плате MINI_FPGA размещён один звуковой излучатель. После прохождения через усилительный каскад, сигнал от FPGA подаётся на буззер (см. рисунок 1.14).

Принцип работы:

Когда FPGA выводит низкий уровень, буззер начинает звучать.

Изменяя частоту переключения высокого и низкого уровней на управляющем выводе, можно регулировать частоту звучания буззера.

Другими словами:

Звук издаётся при генерации ШИМ/прямоугольного сигнала.
Чем выше частота переключений — тем выше тон звука.

В таблице 1.5 указано, какие выводы FPGA соединены с буззером.

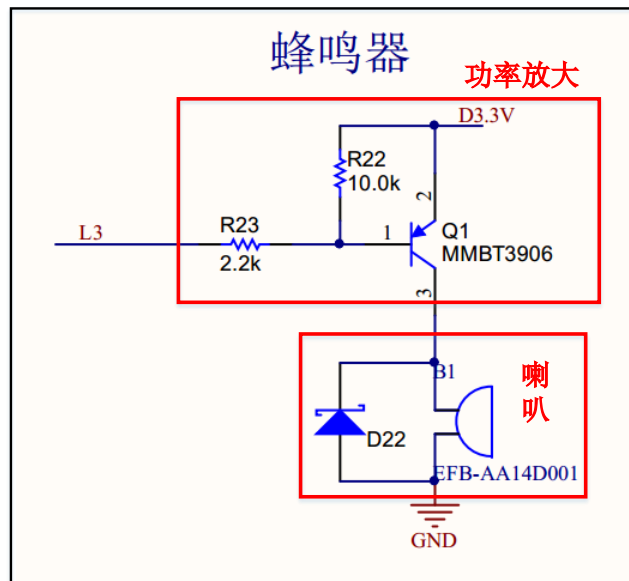


Схема подключения буззера (пьезодинамика)

Таблица 1.5 — Конфигурация выводов буззера

Имя сигналов	FPGA Номер пина	Описание
Buzzer_Out	L3	Буззер / пьезодинамик

1.3.4 Тактовый модуль

На плате MINI_FPGA установлен кварцевый генератор, формирующий тактовый сигнал частотой 50 МГц, как показано на рисунке 1.15.

Этот тактовый сигнал подключён напрямую к выводу FPGA и используется для работы всей пользовательской логики внутри микросхемы.

В таблице 1.6 приведена информация о подключении выводов, связанных с тактовым сигналом.

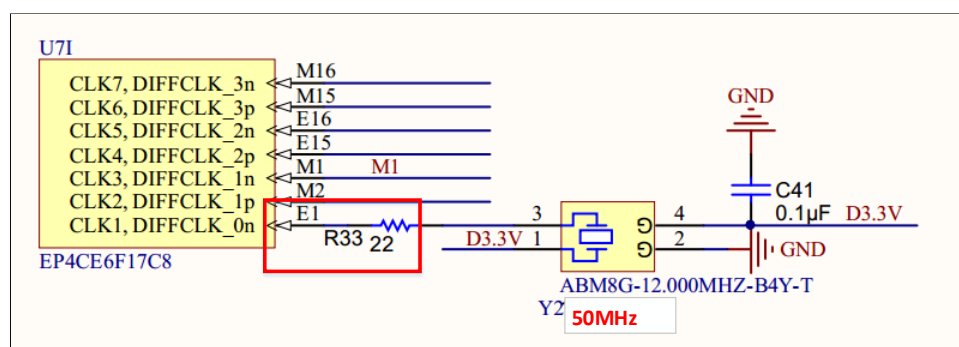


Рис. 1.15 — Схема подключения кварцевого генератора

Таблица 1.6 — Конфигурация выводов тактового сигнала

Имя сигналов	FPGA	Номер пина	Описание
CLK	E1		50 MHz clock input

1.3.5 Модуль расширения ввода-вывода (IO Expansion Port)

Эти модули предназначены для:
 подачи сигналов из внешних устройств в систему,
 вывода сигналов, генерируемых системой,
 то есть для расширения возможностей взаимодействия FPGA с внешними компонентами.

(1) IO-разъёмы на плате MINI_FPGA

Плата MINI_FPGA содержит:
 один 40-пиновый IO-разъём J7,
 два 24-пиновых IO-разъёма P1 и P2,
 как показано на рисунке 1.16.

Разъём J7:

36 выводов подключены напрямую к FPGA Cyclone IV EP4CE6F17C8
 присутствуют выводы питания Vbus и 3.3 V,
 а также два вывода GND.

Пояснение:

Vbus — это 5-вольтовое питание, поступающее с USB-интерфейса.

Разъём P1:

21 вывод подключён напрямую к FPGA,
 имеются выводы 3.3 V, Vbus (5 В) и GND.

Разъём P2:

по структуре полностью аналогичен разъёму P1.

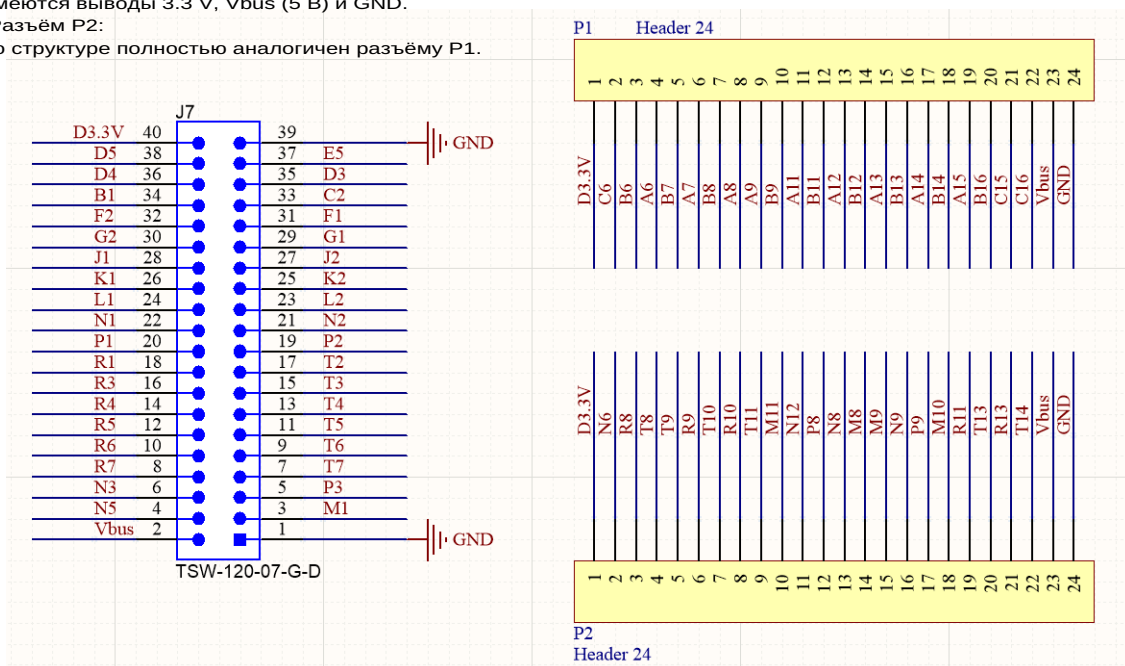


Рис. 1.16 — Схема подключения универсальных IO-разъёмов

1.3.6 Модули памяти

Память используется для задач встроенных систем и хранения данных.
 На практике память обычно делится на две большие категории:

1) RAM (Random Access Memory) — оперативная память

относится к типам SDRAM и SRAM,
 обеспечивает высокую скорость доступа,
 теряет данные при отключении питания,
 обычно используется как:

системный буфер,
 рабочая память (оперативная память).

2) ROM (Read Only Memory) — постоянная память

Основана на технологиях:

Flash

EEPROM (Electrically Erasable Programmable Read-Only Memory)

Flash и EEPROM сохраняют данные при выключенном питании,
 и широко применяются для:

хранения конфигурационных данных FPGA (EPQ / EPCQ),

хранения прошивок,

хранения постоянных таблиц, шрифтов, LUT и др.

(1) Serial Flash Memory (серийная флэш-память)

На плате MINI_FPGA установлен чип серийной флэш-памяти (Serial Flash Memory). Он используется для хранения конфигурационных данных FPGA, которые автоматически загружаются в микросхему при подаче питания.

(см. рисунок 1.17)

Эта Flash-память...

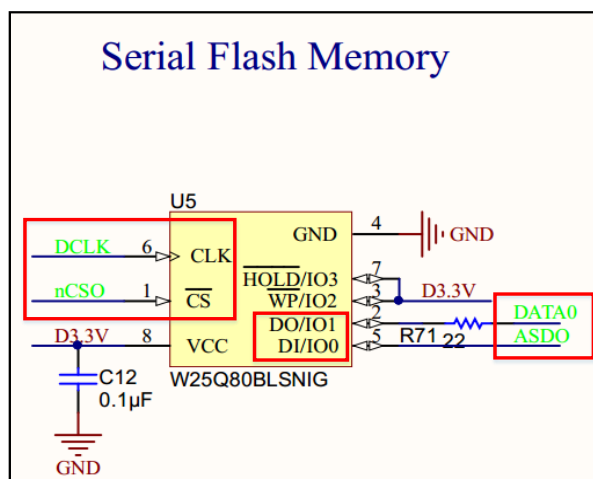


Рис. 1.17 — Схема подключения Serial Flash Memory

Линии CLK, CS, DO и DI этого чипа Serial Flash Memory соединены с соответствующими конфигурационными выводами FPGA. Эти соединения уже подробно объяснялись в разделе 3.2.3, поэтому здесь повторно не рассматриваются.

(2) EEPROM-память с интерфейсом I²C

На плате MINI_FPGA также установлена микросхема EEPROM, работающая по протоколу I²C, как показано на рисунке 1.18.

EEPROM — это электрически стираемая и перезаписываемая постоянная память (Electrically Erasable Programmable ROM).

Выводы микросхемы:

SCL — линия тактового сигнала
→ подключена к выводу FPGA R14

SDA — линия данных
→ подключена к выводу FPGA T15

Эта память может использоваться как постоянная системная память для хранения данных, программных параметров или пользовательской информации.

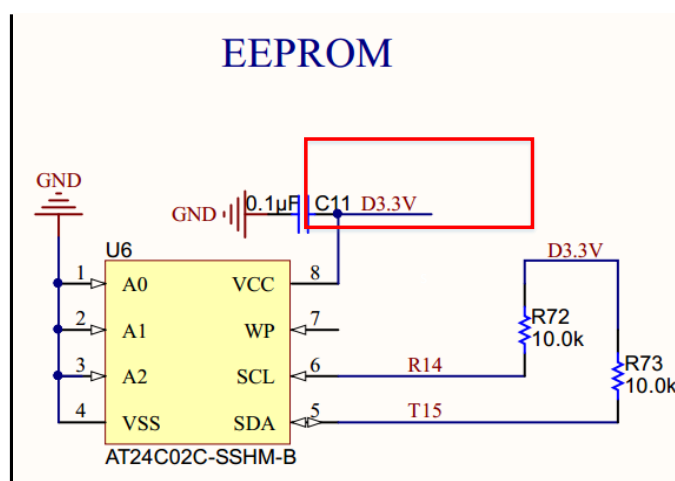


Рис. 1.18 — Схема подключения EEPROM

1.3.7 Модуль UART-коммуникации

На плате MINI_FPGA интегрирована схема преобразования UART ↔ USB, как показано на рисунке 1.21.

Благодаря этому обмен данными между FPGA и персональным компьютером становится очень простым: нужно лишь подключить ПК к разъёму UART2USB на плате (см. рисунок 1.1) с помощью обычного USB-кабеля.

После подключения USB-данные проходят через преобразовательную схему и конвертируются в формат протокола UART, который затем поступает в FPGA.

Важно учитывать направление данных

На рисунке 1.21:

TXD — это вывод, по которому преобразователь передаёт данные В FPGA (то есть TXD → FPGA_RX)

RXD — это вывод, по которому преобразователь принимает данные ОТ FPGA (то есть FPGA_TX → RXD)

Поэтому при назначении выводов в проекте Quartus необходимо правильно понимать, какой пин FPGA является входом, а какой — выходом.

В таблице 1.7 приведены номера выводов FPGA, которые используются в лабораторной работе по UART-коммуникации.

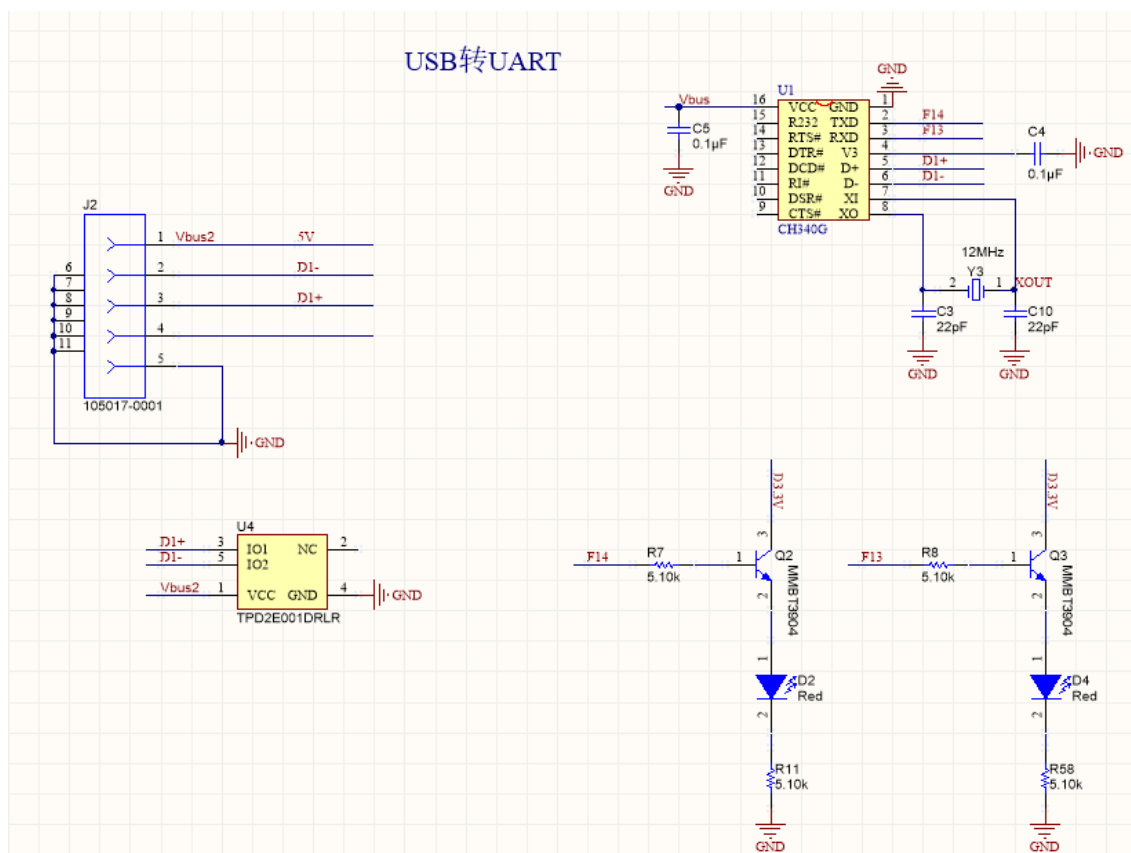


Рис. 1.21 — Схема подключения преобразователя USB-в-UART

Таблица 1.7 — Конфигурация выводов для UART-коммуникации

Имя сигналов	FPGA Номер пина	Описание
RX_In	F14	FPGA Приём
TX_Out	F13	FPGA Передача

1.3.8 Модуль питания

В кабеле USB находятся четыре провода:

два провода передают дифференциальные сигналы данных,
два других — подают питание 5 В.

Если мощность периферийного устройства невелика, его можно питать непосредственно от USB, без использования внешнего блока питания.

Чтобы уменьшить необходимость носить с собой дополнительные адаптеры, на плате MINI_FPGA интегрирована собственная схема питания, как показано на рисунке 1.22.

Vbus — это 5-вольтовое питание, поступающее по USB-линии.

Плата использует импульсный стабилизатор RT8059, который понижает напряжение 5 В и формирует следующие уровни питания:

3.3 В

2.5 В

1.2 В

Эти напряжения используются для различных подсистем FPGA и периферии.

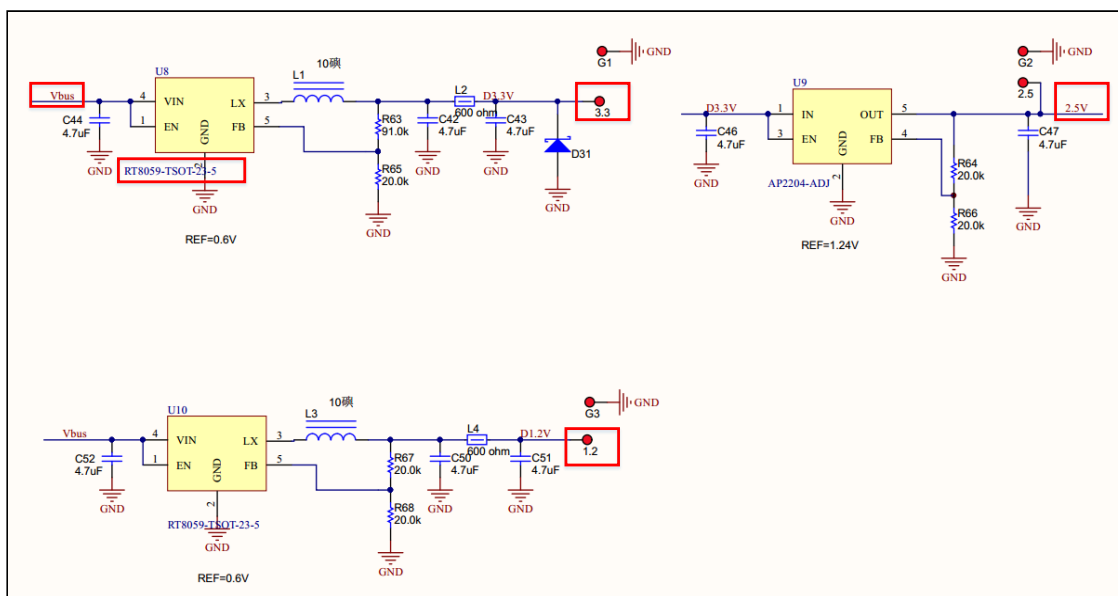


Рис. 1.22 — Схема питания (Power Supply Circuit)

1.4 — Как создать проект в Quartus

Ниже приведён полный процесс создания проекта “Бегущие огни” в Quartus и загрузки его на плату MINI_FPGA для проверки работы.

1. Создание нового проекта

Как показано на рисунке 1.23:

открой Quartus II,

затем нажми кнопку, выделенную красным кружком,
— это позволит создать новый проект.

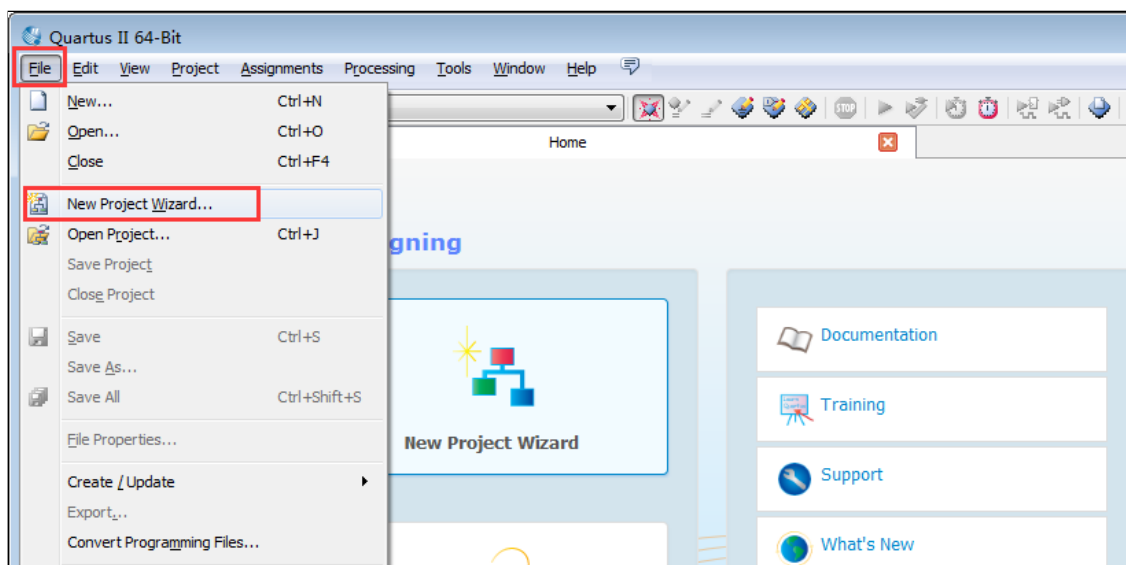


Рис. 1.23 — Создание нового проекта

2. Указание имени проекта

Как показано на рисунке 1.24, в открывшемся окне необходимо выбрать:

путь к проекту,

имя проекта,

имя верхнего уровня (top-level) проекта.

В данном примере путь к проекту —

E:\MINI_FPGA\run_led

Важно: в пути Quartus недопустимо использовать китайские символы. При выборе директории можно создать собственную папку с английским названием.

Имя проекта и имя верхнего уровня также задаются как:

run_led

После этого нажмите кнопку "Next".

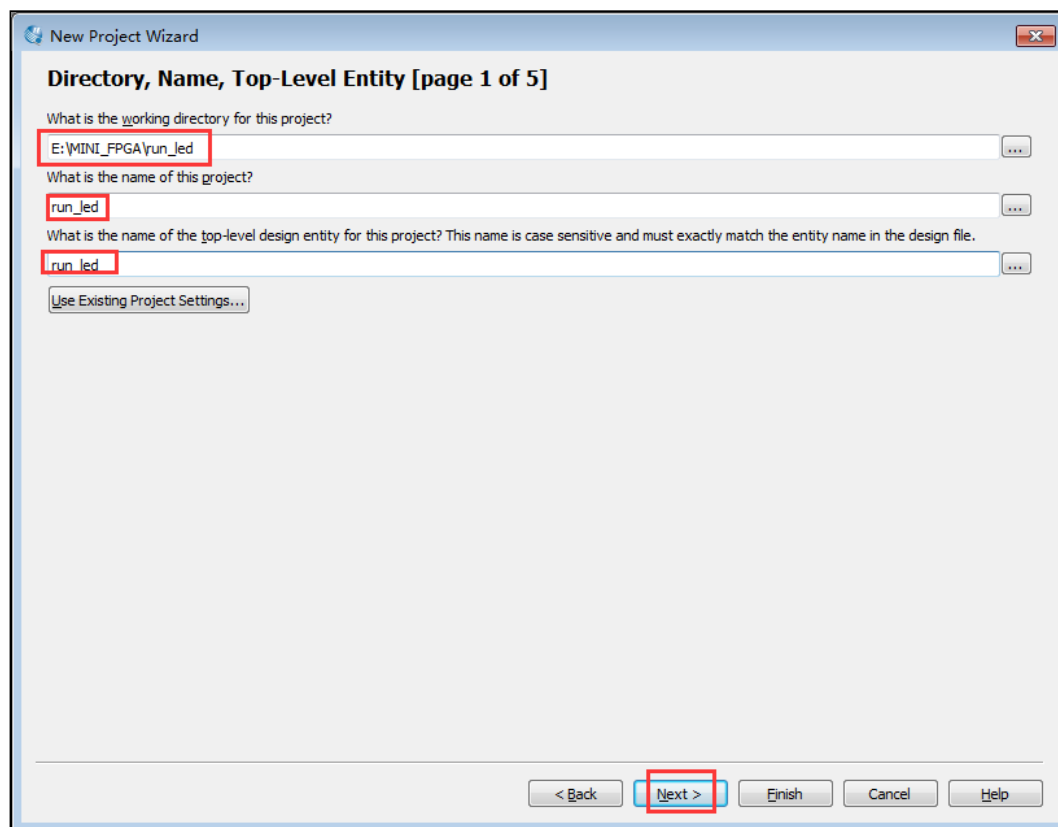


Рис. 1.24 — Указание имени проекта

3. Добавление существующих файлов проекта

Как показано на рисунке 1.25, на этом шаге можно добавить уже существующие файлы проекта. Поскольку на данный момент никаких файлов ещё нет, этот шаг можно пропустить.

Просто нажмите кнопку “Next”.

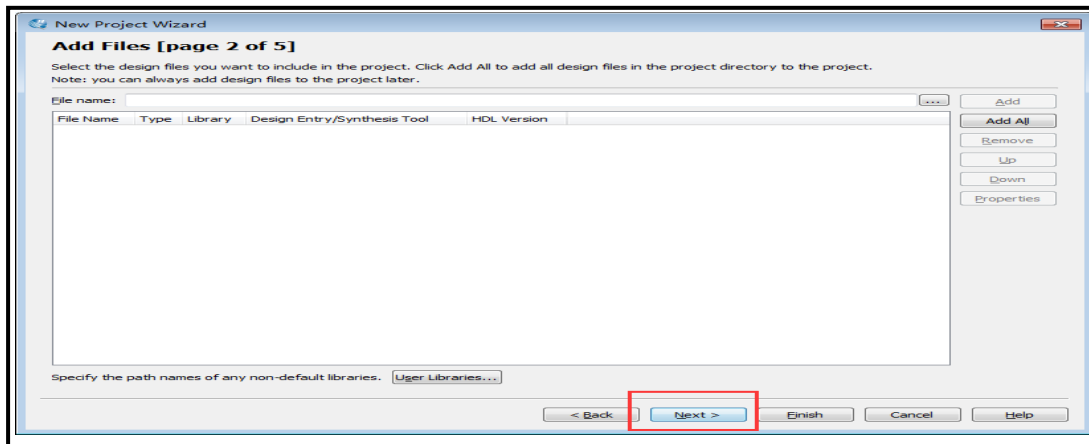


Рис. 1.25 — Добавление существующих файлов проекта

4. Выбор модели устройства

Как показано на рисунке 1.26, на этом шаге необходимо выбрать тип используемого устройства.

В качестве модели следует выбрать EP4CE6F17C8, после чего нажать кнопку “Next”.

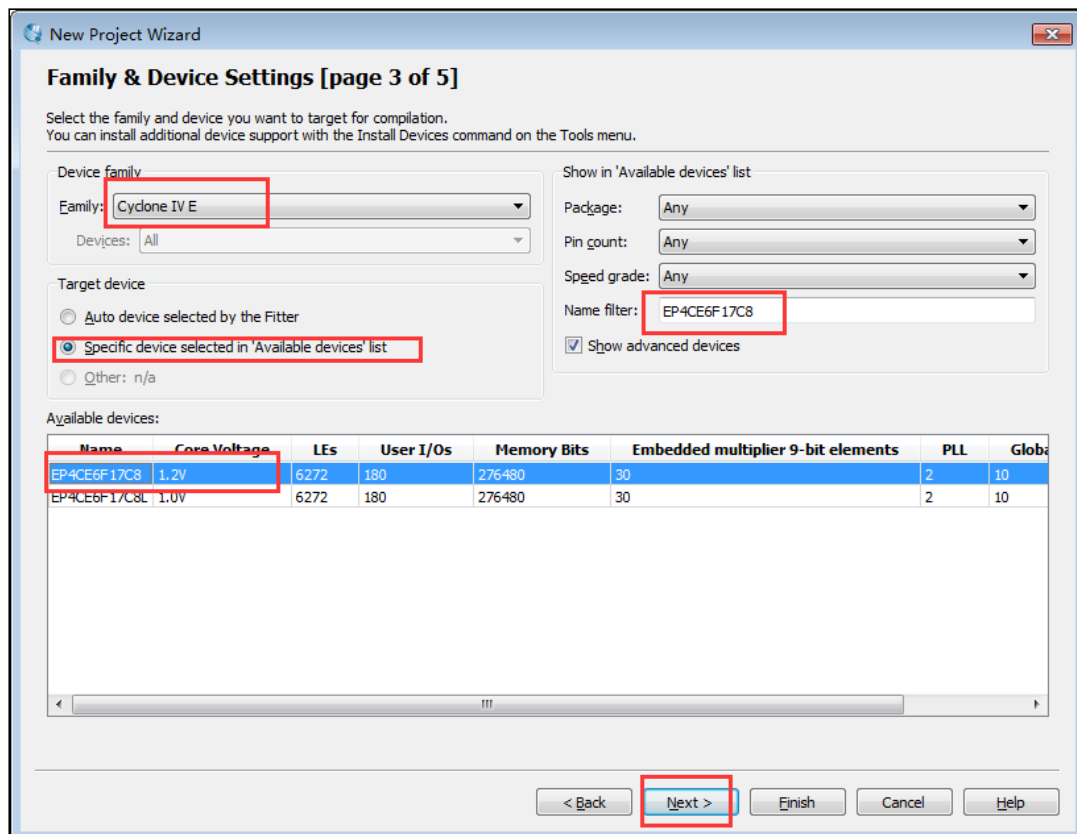


Рис. 1.26 — Выбор модели устройства

5. Выбор EDA-инструментов

Как показано на рисунке 1.27, на этом шаге выбираются инструменты EDA. Необходимо указать только инструмент симуляции (simulation tool). Здесь выбирается:

Modelsim-Altera — как средство моделирования,

Verilog HDL — как язык описания аппаратуры.

После выбора нажмите кнопку “Next”.

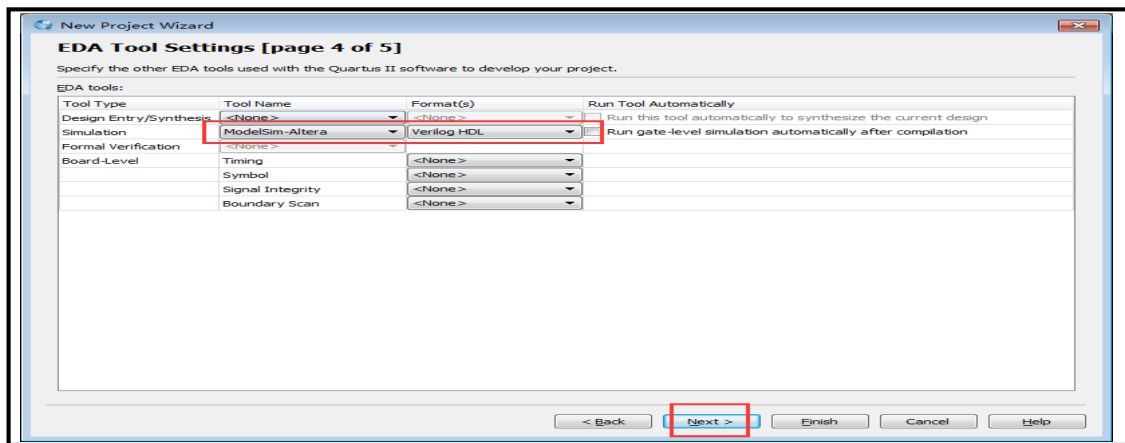


Рис. 1.27 — Выбор EDA-инструментов

6. Затем нажмите “Next”, а после — “Finish”.

Этим завершается процесс создания нового проекта в мастере (New Project Wizard).

7. Создание верхнего файла проекта

После выполнения всех предыдущих шагов можно приступить к написанию модуля run_led на Verilog.

Как показано на рисунке 1.28:

в главном окне Quartus II нажмите кнопку «Создать новый файл» (New), расположенную в левом верхнем углу;

затем в списке типов файлов выберите создание нового Verilog-файла.

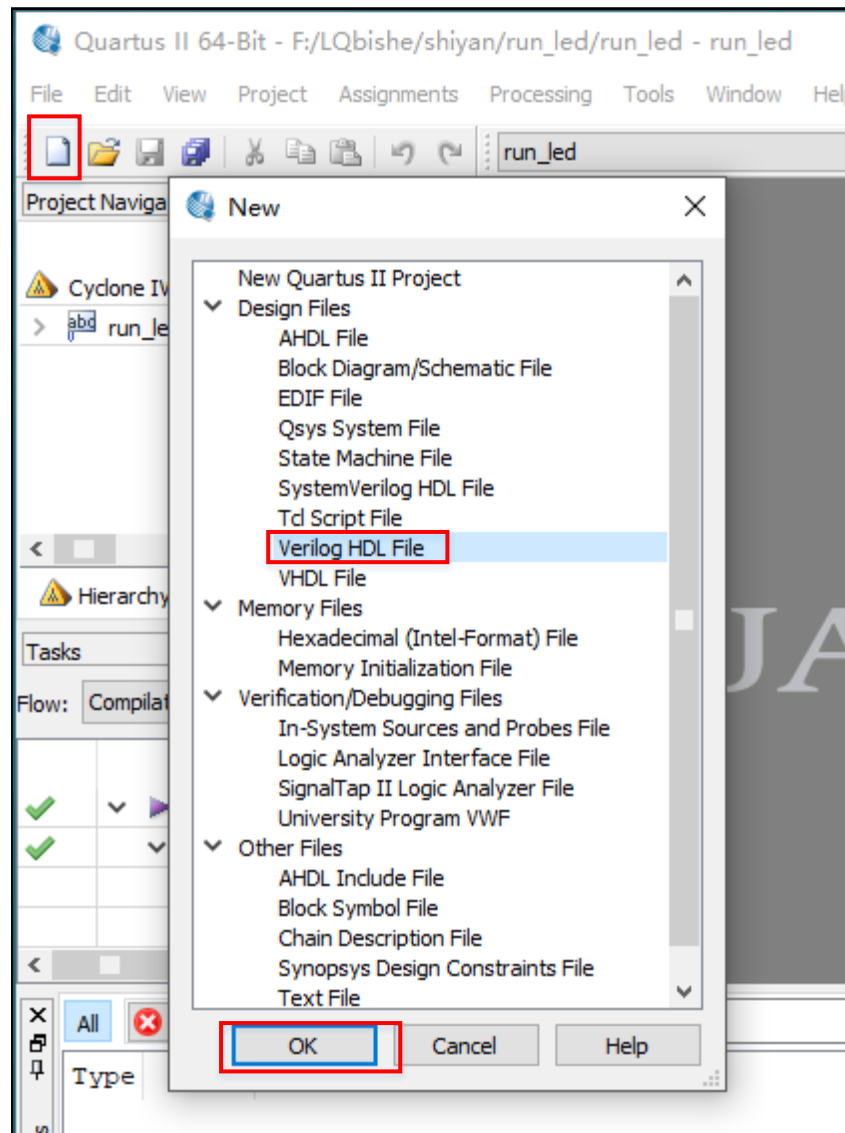


Рис. 1.28 — Создание верхнего файла проекта

8. Разработка файла верхнего уровня

В только что созданный файл необходимо ввести код, показанный на рисунке 1.29, и сохранить его под именем `run_led.v`.

Важно: имя файла должно совпадать с именем модуля (module) внутри него.

Этот модуль является верхним модулем проекта.

Как правило, в верхнем модуле размещаются только экземпляры (instance) других модулей, а основная логика реализуется в нижележащих модулях

```

1  module run_led
2  (
3      CLK, RSTn, LED_Out
4  );
5
6      input CLK;    //Clock
7      input RSTn;  //Reset
8      output [7:0] LED_Out;
9
10 endmodule

```

Рис. 1.29 — Новый файл верхнего уровня

9. Разработка нижнего (внутреннего) модуля

Повторите шаг 7, чтобы создать новый файл.

В новом файле введите код, показанный на рисунке 1.30, и сохраните его под именем `led8_module.v`.

В отличие от верхнего уровня, для нижнего модуля имя файла не обязательно должно совпадать с именем модуля (module) внутри него.

```
1  module led8_module
2  (
3      CLK, RSTn, LED_Out
4  );
5
6      input CLK;
7      input RSTn;
8      output [7:0]LED_Out;
9
10     parameter T100MS = 23'd5_000_000;
11     reg [22:0]Count;    //Delay
12     reg [7:0]rLED_Out;
13
14     always @ ( posedge CLK or negedge RSTn )
15         if( !RSTn )
16         begin
17             Count <= 23'd0;
18             rLED_Out <= 8'b0000_0001;
19         end
20     else if( Count == T100MS - 1'b1)
21     begin
22         Count <= 23'd0;
23         if( rLED_Out == 8'b0000_0000 )
24             rLED_Out <= 8'b0000_0001;
25         else
26             rLED_Out <= {rLED_Out[0],rLED_Out[7:1]};
27         end
28     else
29         Count <= Count + 1'b1;
30
31     assign LED_Out = rLED_Out;
32
33 endmodule
```

Рис. 1.30 — Разработка нижнего модуля

10. Предкомпиляция (Analysis & Elaboration)

В главном окне Quartus II нажмите кнопку предкомпиляции, как показано на рисунке 1.31.

Если в проекте нет ошибок, появится диалоговое окно, показанное на рисунке 1.32. Нажмите ОК для продолжения.

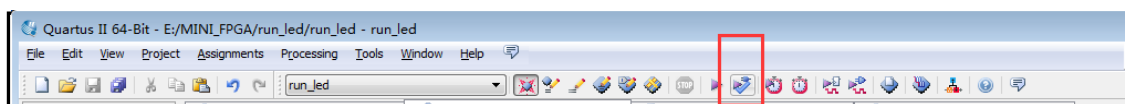


Рис. 1.31 — Кнопка предкомпиляции

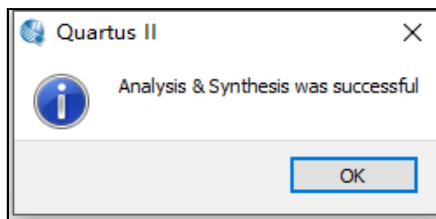


Рис. 1.32 — Диалоговое окно предкомпиляции

11. Создание файла инстанцирования (instance) с помощью инструмента Quartus II

Как показано на рисунке 1.33:

выберите файл `led8_module.v`,

затем нажмите кнопку, выделенную красным кружком, чтобы автоматически сгенерировать файл инстанцирования на языке Verilog.

Quartus создаёт такой файл по следующему правилу: к имени исходного файла добавляется суффикс `_inst`.

В данном примере итоговый файл будет называться:

`led8_module_inst.v`

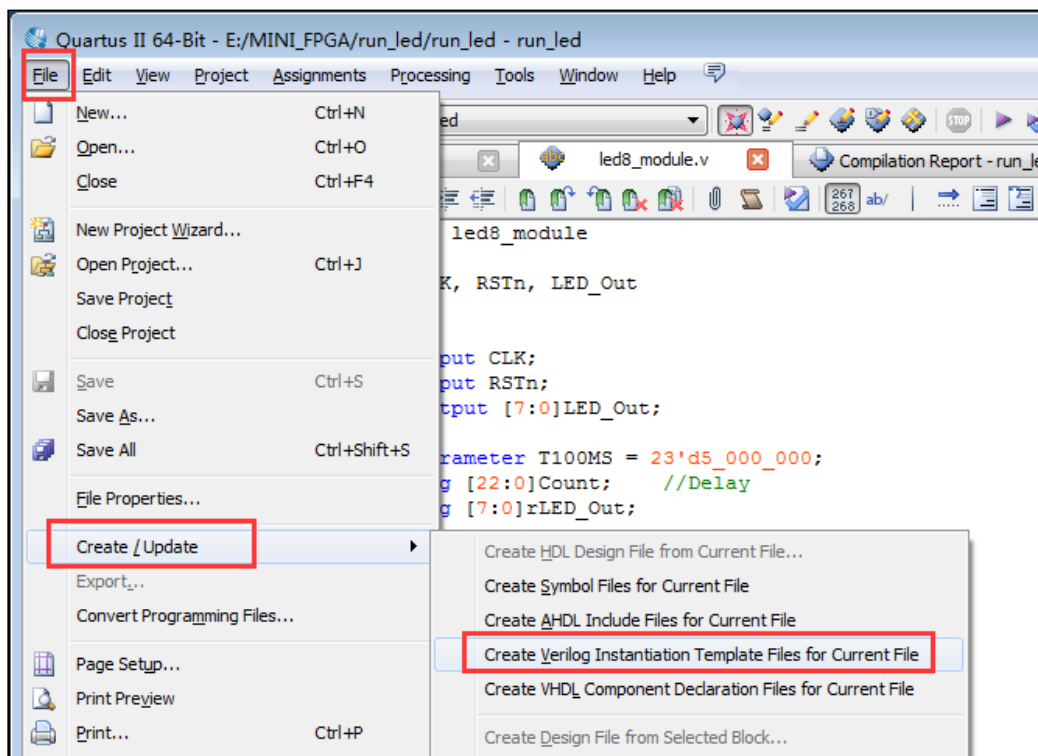


Рис. 1.33 — Создание файла инстанцирования

12. Использование (вставка) инстанцированного модуля

Как показано на рисунке 1.34, откройте созданный ранее файл инстанцирования. Затем скопируйте интерфейс инстанцирования (см. рисунок 1.35) и вставьте его в файл `run_led.v`.

После вставки необходимо:

изменить имя экземпляра модуля (instance name),

выполнить корректное подключение всех интерфейсов (портов), как показано на рисунке 1.36.

При этом внимательно проверяйте:

точное совпадение букв верхнего/нижнего регистра в именах,

ширину шин данных (bit-width),

соответствие входов и выходов.

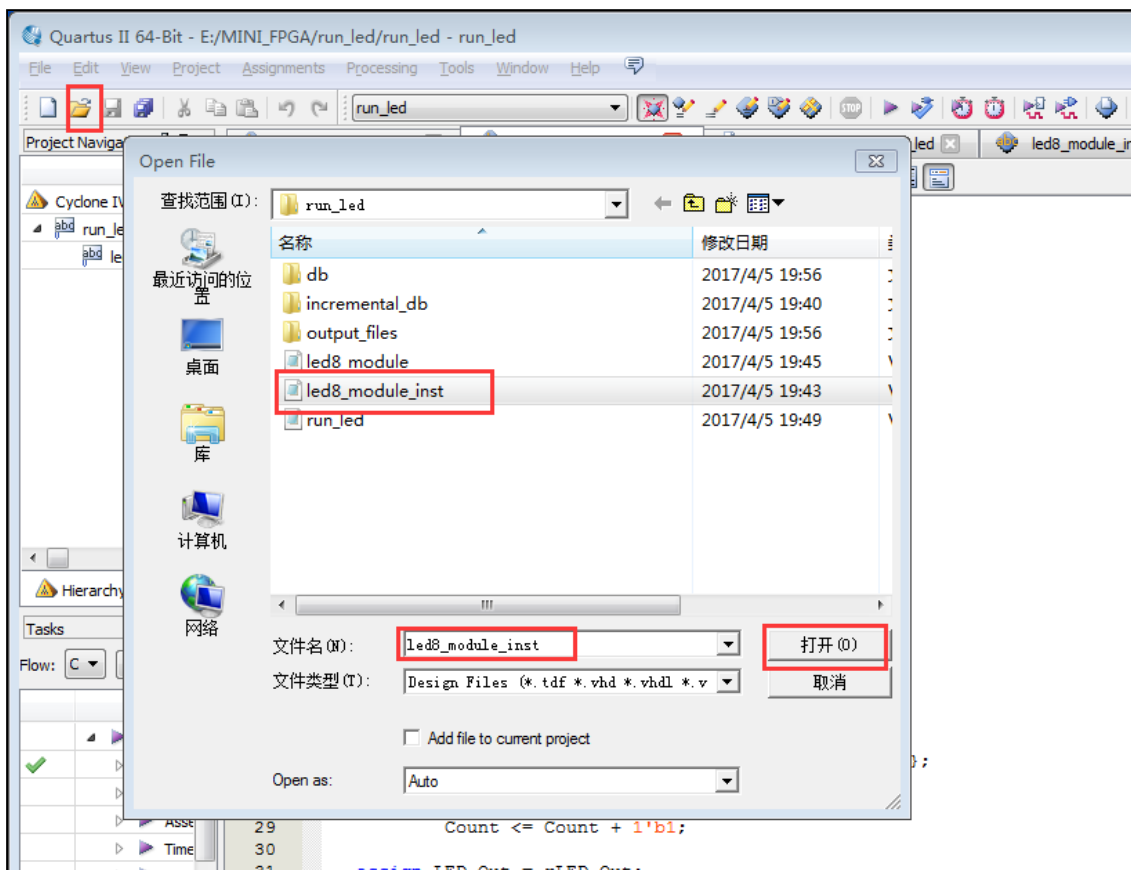


Рис. 1.34 — Использование инстанцированного компонента

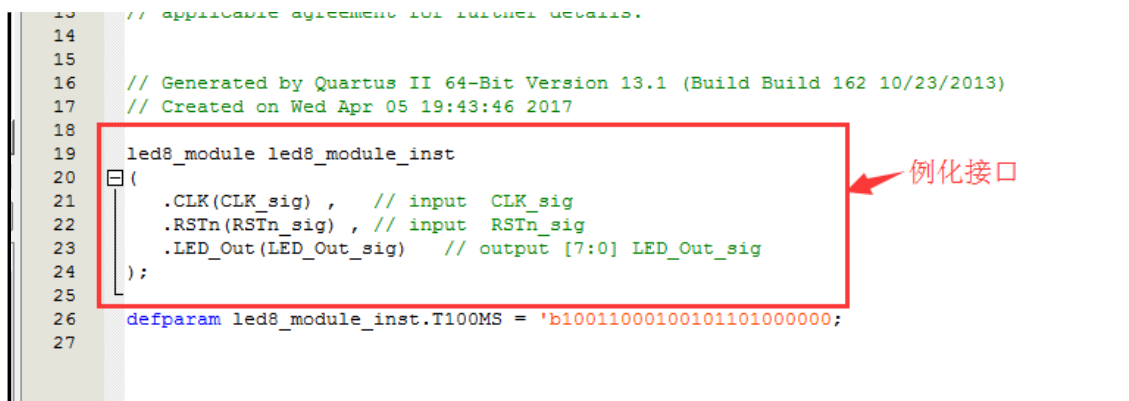


Рис. 1.35 — Интерфейс инстанцированного модуля

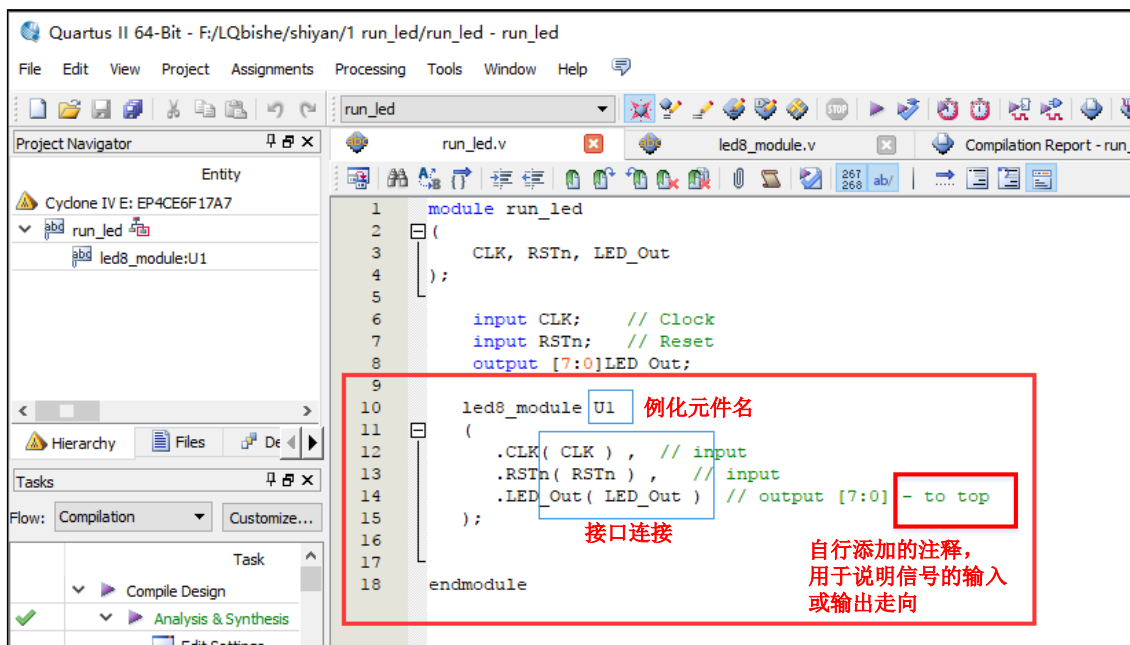


Рис. 1.36 — Изменение имени экземпляра и подключение интерфейсов

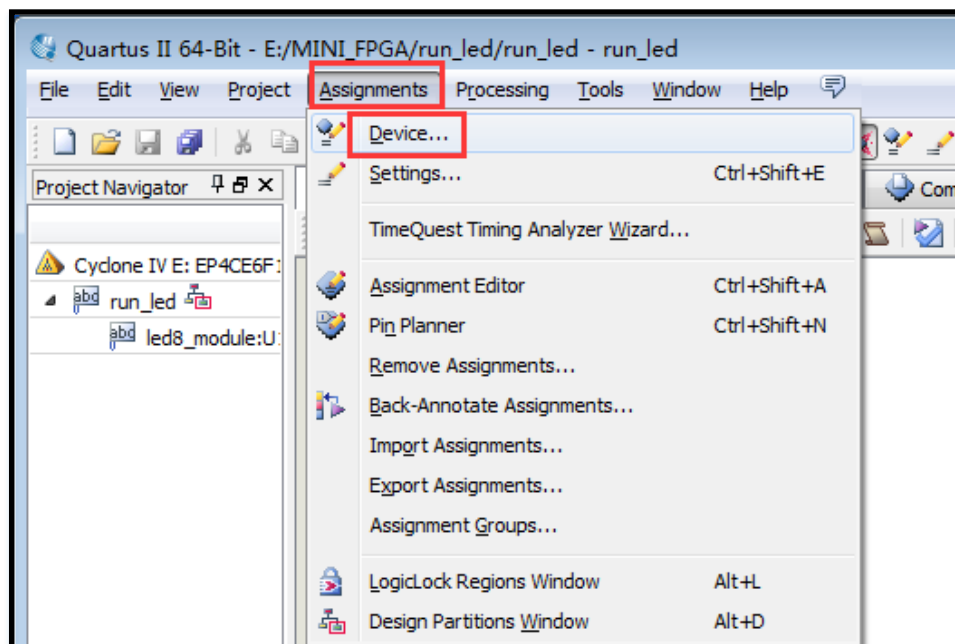


Рис. 1.37 — Настройка состояния выводов (Pin State Settings)

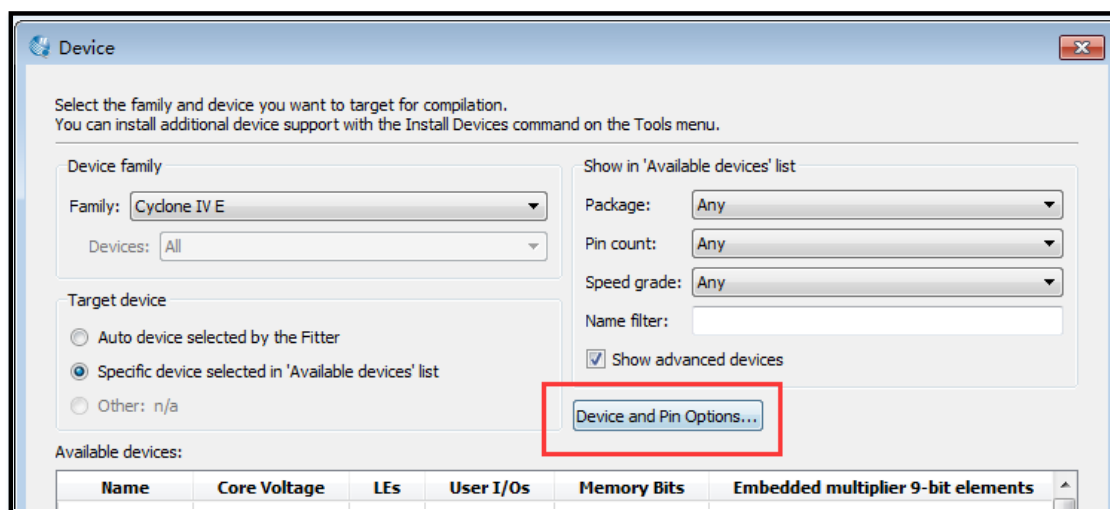


Рис. 1.38 — Настройка состояния выводов (Pin State Settings)

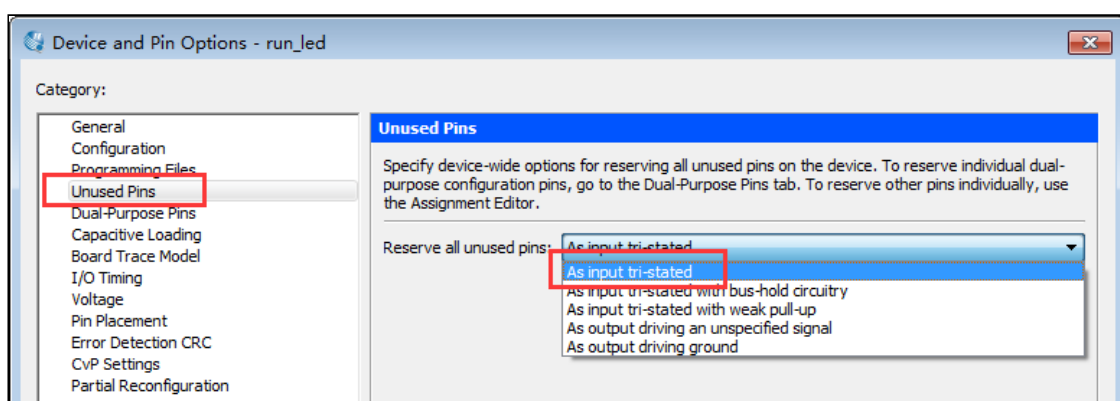


Рис. 1.39 — Настройка состояния выводов

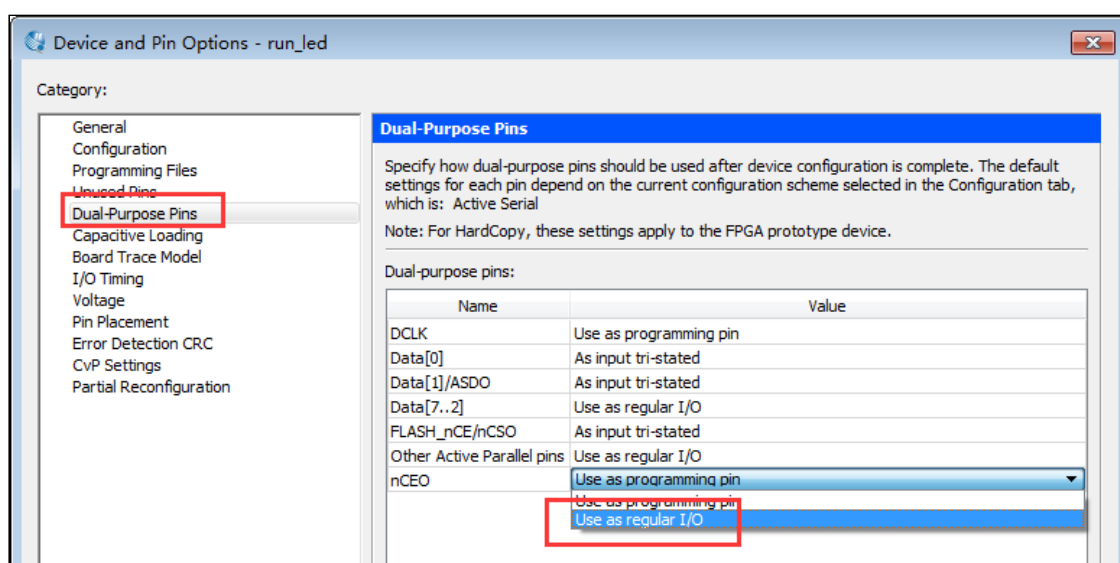


Рис. 1.40 — Настройка состояния выводов

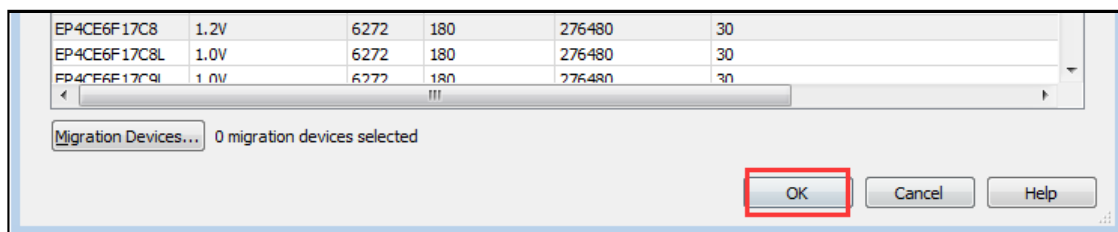


Рис. 1.41 — Настройка состояния выводов

15. Назначение выводов FPGA

Как показано на рисунке 1.42, нажмите кнопку, выделенную красным кружком, чтобы открыть Pin Planner, инструмент для настройки выводов FPGA.

Далее показано, как выполнять назначение выводов (pin assignment) с использованием схемы платы MINI_FPGA.

Пример: назначение вывода для сигнала CLK

Чтобы определить правильный вывод FPGA для сигнала CLK, необходимо:

Понять, что такое CLK

CLK — это входной системный тактовый сигнал.

Найти его в документации

Сигнал CLK можно найти:

в схеме MINI_FPGA, рисунок 1.15,

или в таблице 1.6 первой части документа.

Из схемы видно, что тактовый генератор на плате выдаёт сигнал 50 МГц, и этот сигнал поступает на вывод E1 FPGA.

Указать вывод E1 в Pin Planner

В Pin Planner:

найдите строку сигнала CLK,

дважды щёлкните по ячейке Location,

введите E1 (регистр букв не имеет значения — можно e1 или E1),

нажмите Enter.

Таким же образом необходимо назначить выводы и для остальных сигналов проекта.

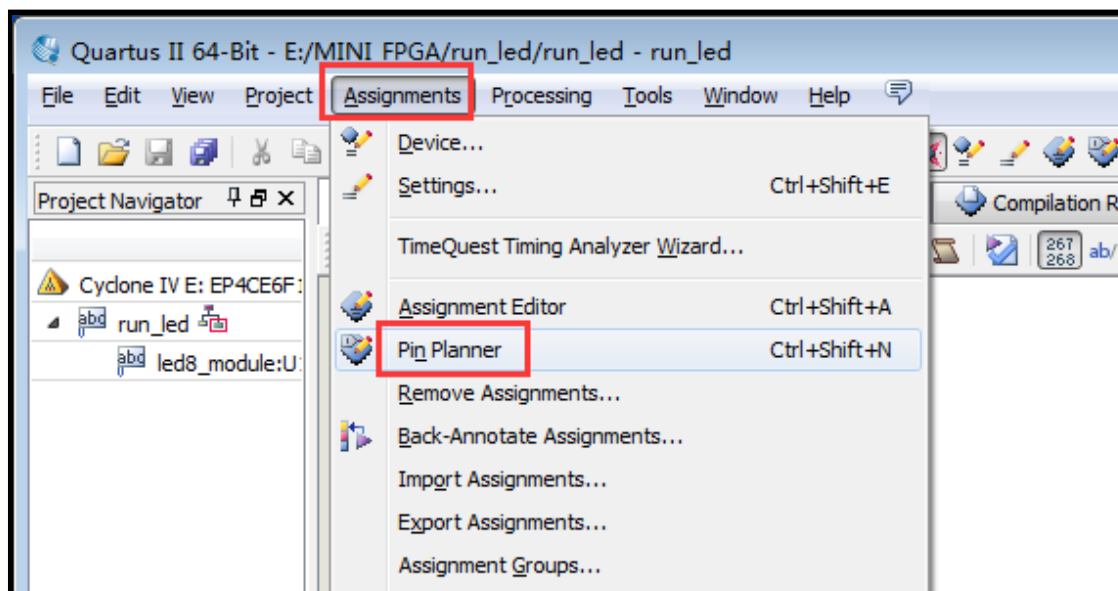


Рис. 1.42 — Подключение выводов FPGA

16. Установка драйвера USB-Blaster

Сначала щёлкните правой кнопкой мыши по значку «Мой компьютер / Компьютер» на рабочем столе,

затем выберите «Свойства», после чего откройте Диспетчер устройств.

Как показано на рисунке 1.43:

в разделе «Контроллеры USB» нажмите правой кнопкой мыши и выберите пункт «Сканировать на наличие изменений оборудования».

Как показано на рисунке 1.44: в разделе «Другие устройства» () появится устройство «USB-Blaster». Щёлкните по нему правой кнопкой мыши...

Щёлкните правой кнопкой мыши по устройству «USB-Blaster»,
затем выберите «Обновить драйвер»
Как показано на рисунке 1.45:
выберите пункт «Выполнить поиск драйверов на этом компьютере»

Как показано на рисунке 1.46:
нажмите кнопку «Обзор»,
затем в каталоге установки Quartus найдите папку drivers,
нажмите ОК, поставьте галочку «Включить вложенные папки»,
нажмите «Далее».
Если появится окно, показанное на рисунке 1.47,
это означает, что драйвер установлен успешно.



Рис. 1.43 — Установка драйвера USB-Blaster

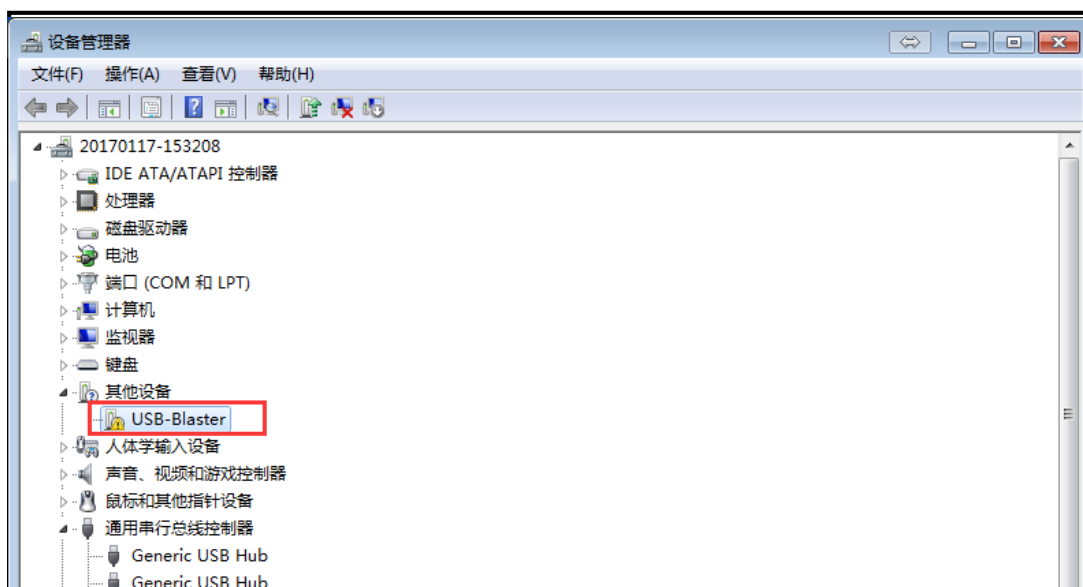


Рис. 1.44 — Появление устройства «USB-Blaster» в разделе «Другие устройства»



Рис. 1.45 — Поиск драйверов на этом компьютере

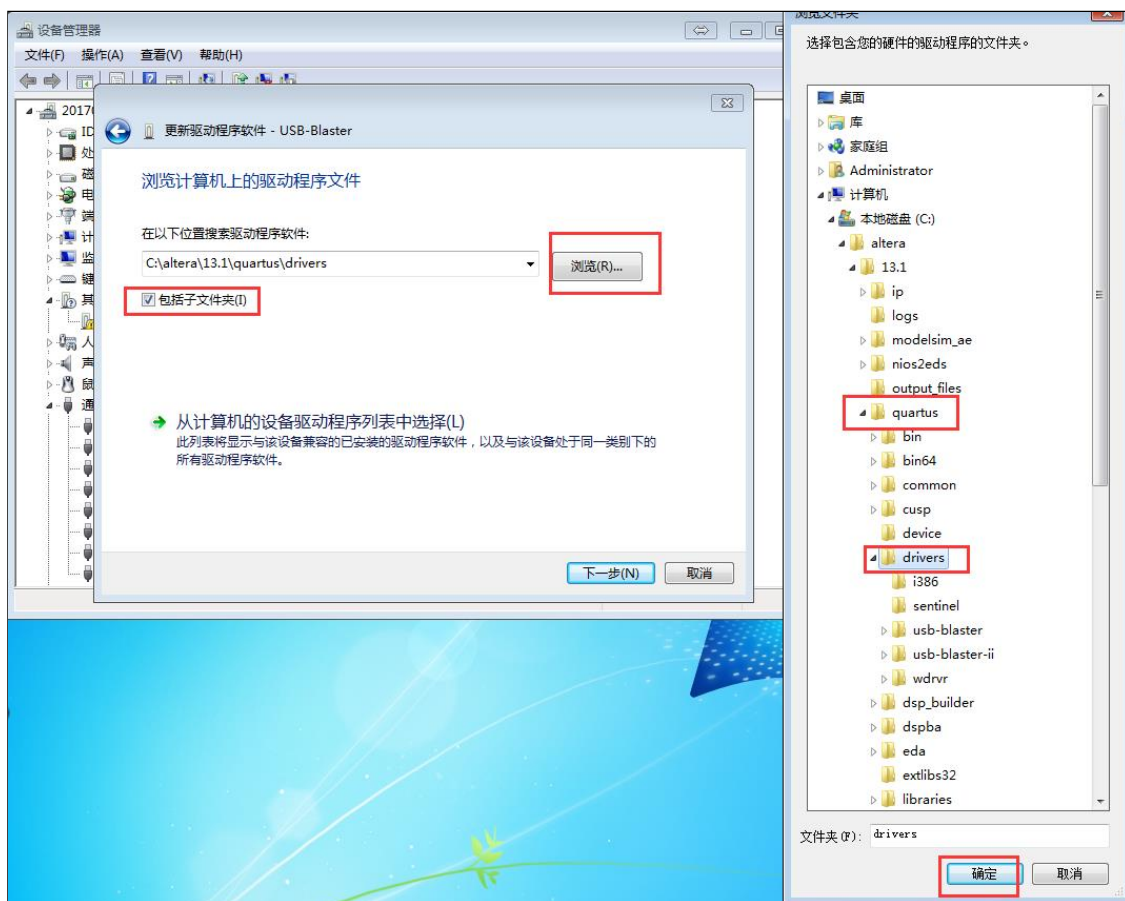


Рис. 1.46 — Выбор папки «drivers»

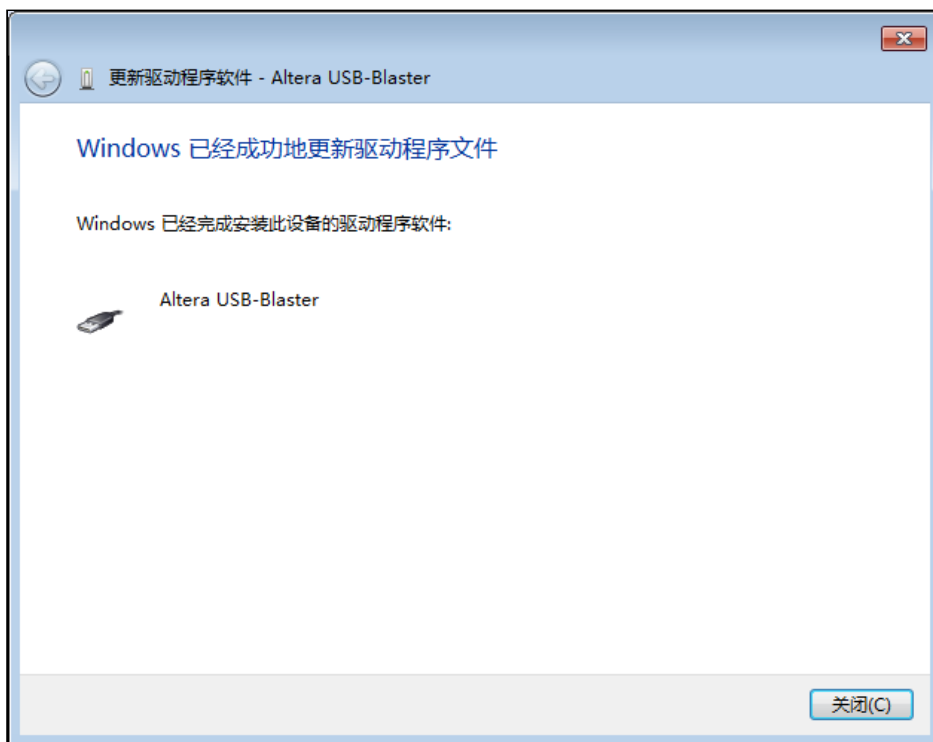


Рис. 1.47 — Драйвер успешно установлен

17. Компиляция проекта

Нажмите кнопку, отмеченную красным кольцом на рисунке 1.48, чтобы выполнить полную компиляцию проекта.

Если компиляция прошла успешно, появится диалоговое окно, показанное на рисунке 1.49.

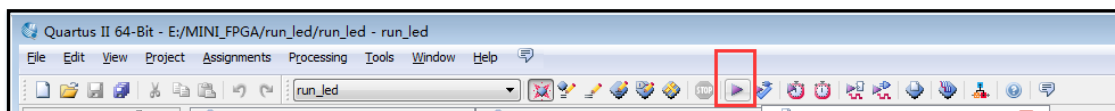
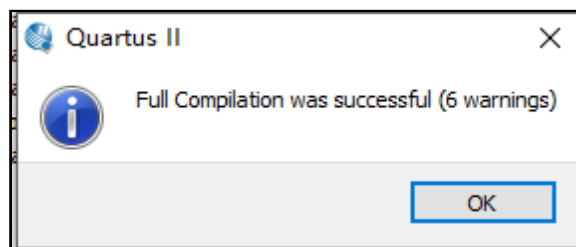


Рис. 1.48 — Компиляция проекта



8. Загрузка программы на плату MINI_FPGA

Подключите компьютер к плате MINI_FPGA с помощью USB-кабеля. Затем нажмите кнопку, выделенную красным кружком на рисунке 1.50, чтобы начать процесс загрузки (программирования).

Как показано на рисунке 1.51, в появившемся диалоговом окне нажмите кнопку "Hardware Setup", чтобы открыть окно "Hardware Setup".

Далее, как показано на рисунке 1.52, ...

Если драйвер USB-Blaster был установлен успешно, в окне Hardware Setup появится устройство USB-Blaster.

Нажмите на раскрывающийся список, выберите USB-Blaster [USB-0], затем нажмите “Close”.

Как показано на рисунке 1.53, режим программирования должен быть установлен в JTAG.

Если файл конфигурации был загружен автоматически можно сразу нажать кнопку “Start”, чтобы начать прошивку FPGA.

Если файл не был загружен автоматически его нужно добавить вручную.

Как показано на рисунке 1.54:

Нажмите “Add File”.

В открывшемся окне перейдите в папку output_files.

Как показано на рисунке 1.55, выберите файл с расширением .sof.

Нажмите “Open”,

Затем нажмите “Start”, чтобы начать процесс программирования FPGA.

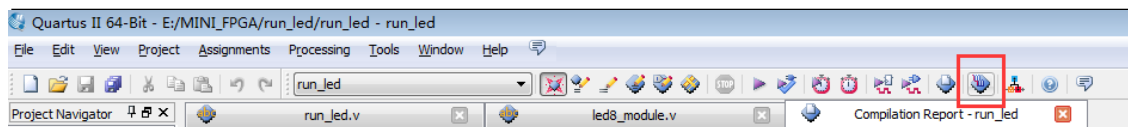


Рис. 1.50 — Загрузка программы

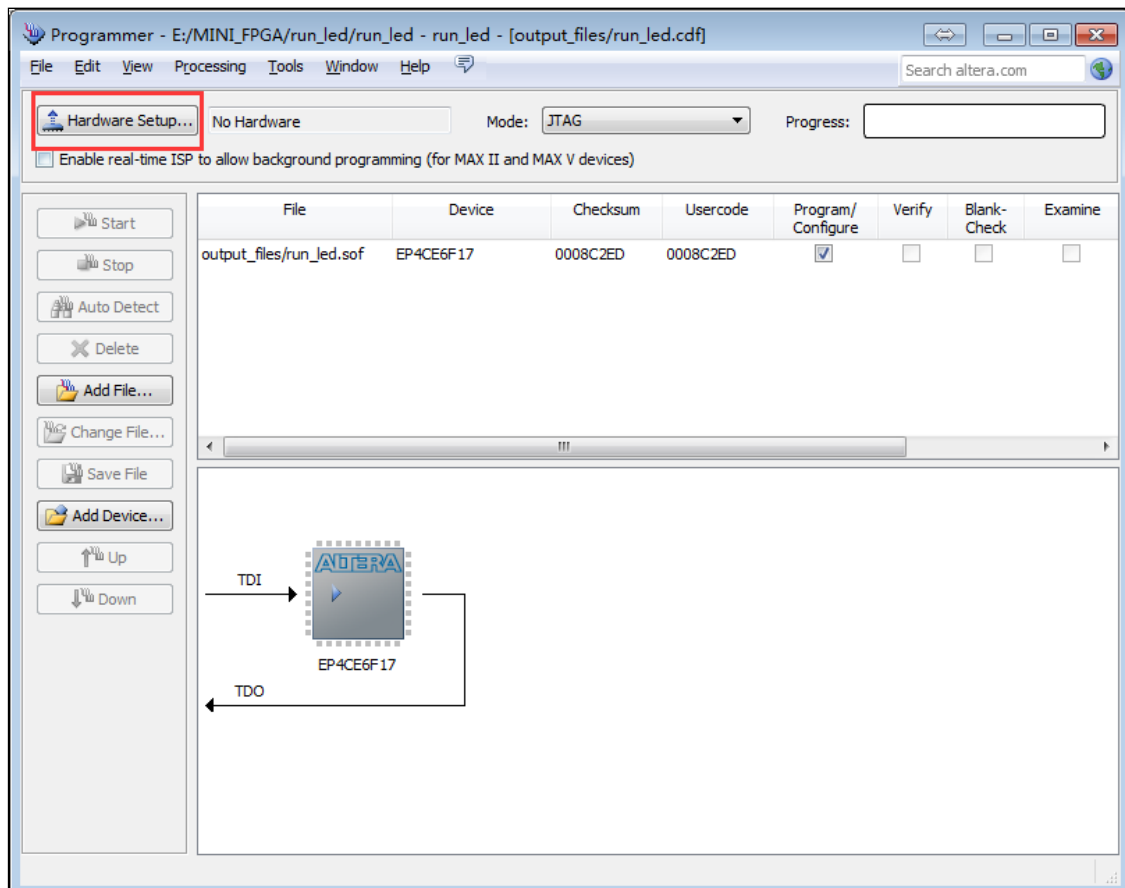


Рис. 1.51 — Диалоговое окно «Hardware Setup»

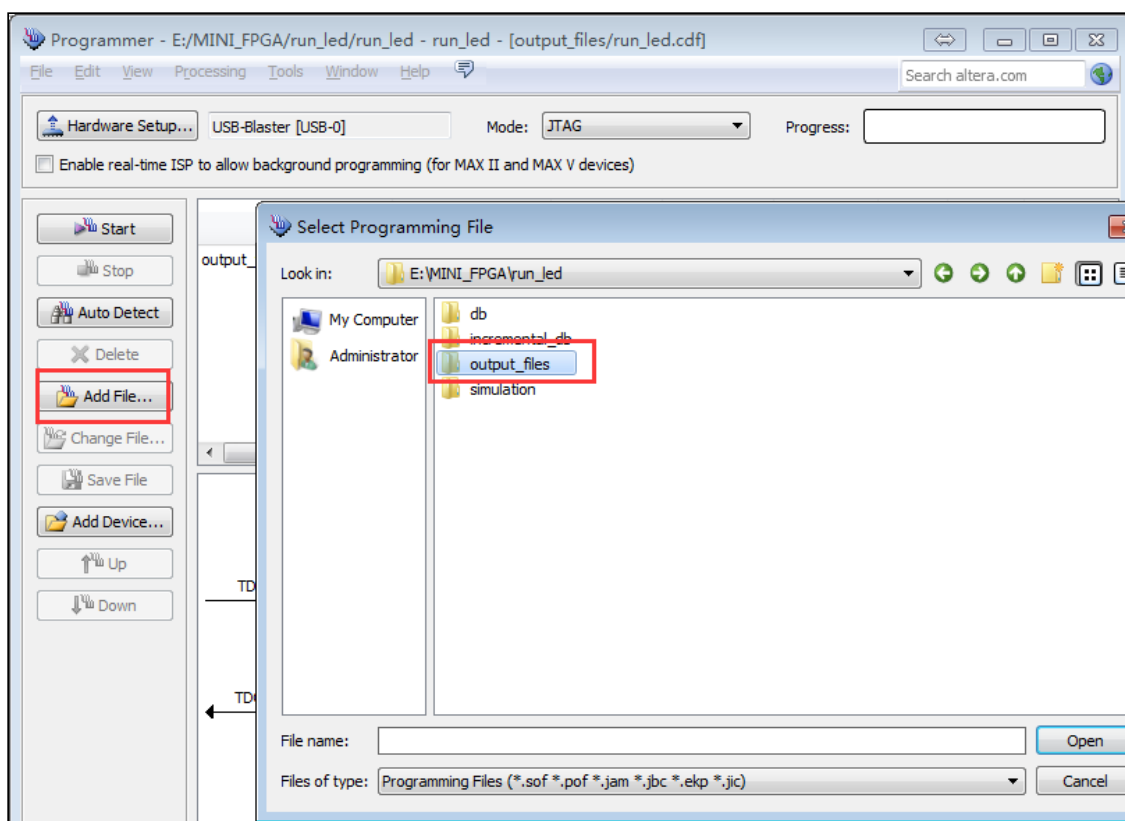


Рис. 1.54 — Загрузка файла

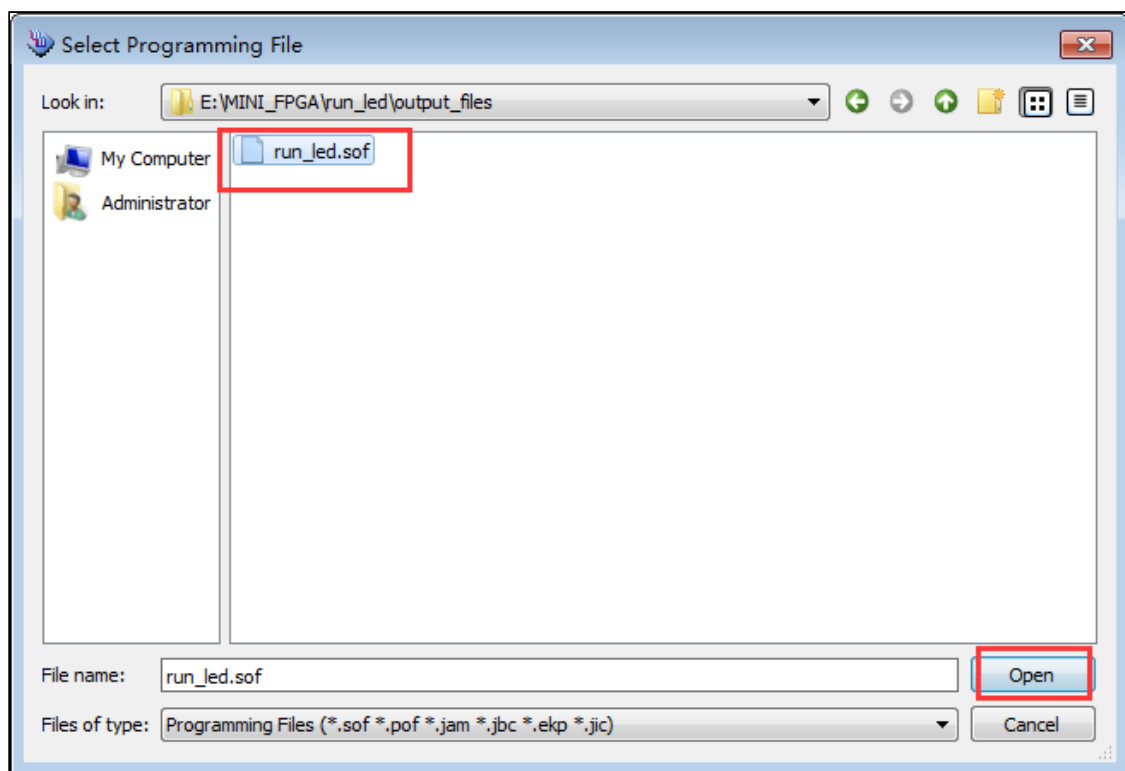


Рис. 1.55 — Выбор файла

1.5 Как с помощью IP-ядер Altera создать ROM-таблицу поиска (Lookup Table)

В Quartus для создания ROM-таблицы с использованием IP-ядра сначала необходимо сгенерировать файл данных, который это IP-ядро будет использовать.

В качестве примера далее будет показано создание ROM-таблицы значений синусоиды.

Сначала при помощи Matlab создаётся таблица синусоиды с шириной адреса 12 бит и шириной данных 8 бит.

Пример программы выглядит следующим образом:

1. Как показано на рисунке 2.15, откройте Matlab

После запуска Matlab:

нажмите кнопку, отмеченную красным кружком, чтобы создать новый скрипт (script).

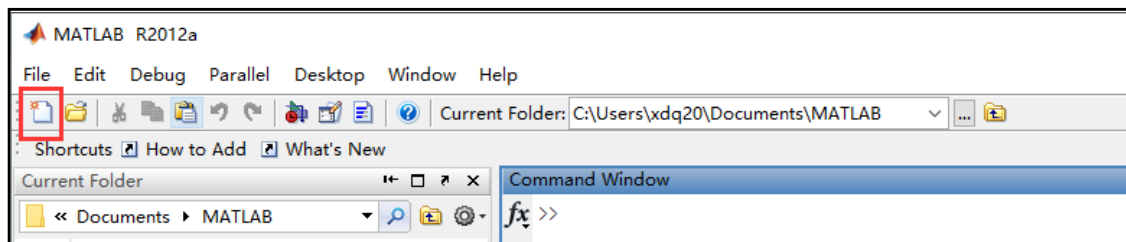


Рис. 2.15 — Создание нового скрипта в Matlab

2. В новом скрипте Введите код, показанный на рисунке 2.16, и сохраните файл под именем "DDS_sin.m" (имя можно выбрать любое, путь хранения — также произвольный), что показано на рисунке 2.17.

3. Затем, как показано на рисунке 2.18 Нажмите кнопку, выделенную красным кружком, чтобы запустить выполнение файла "DDS_sin.m". Если появится окно, подобное рисунку 2.19, выберите пункт "" ("Изменить рабочую папку").

4. После успешного выполнения в каталоге, где расположен файл "DDS_sin.m", появится новый файл "sinrom1.mif", как показано на рисунке 2.20. Этот .mif-файл и будет содержать ROM-таблицу синусоиды для последующего использования в IP-ядре.

```
1 ADDR_WIDTH=12;
2 DATA_WIDTH=16;
3 depth=2^ADDR_WIDTH;
4 x=ceil((2^DATA_WIDTH/2-1)*sin(0:pi*2/depth:2*pi)+2^DATA_WIDTH/2);
5 fid=fopen('sinrom1.mif','w');
6 fprintf(fid,'width=%d;\n',DATA_WIDTH);
7 fprintf(fid,'depth=%d;\n',depth);
8 fprintf(fid,'address_radix=uns;\n');
9 fprintf(fid,'data_radix=uns;\n');
10 fprintf(fid,'Content Begin\n');
11 for(k=1:depth)
12     fprintf(fid,'%d:%d;\n',k-1,x(k));
13 end
14 fprintf(fid,'end;');
```

Пример кода Matlab для генерации ROM-таблицы синусоиды

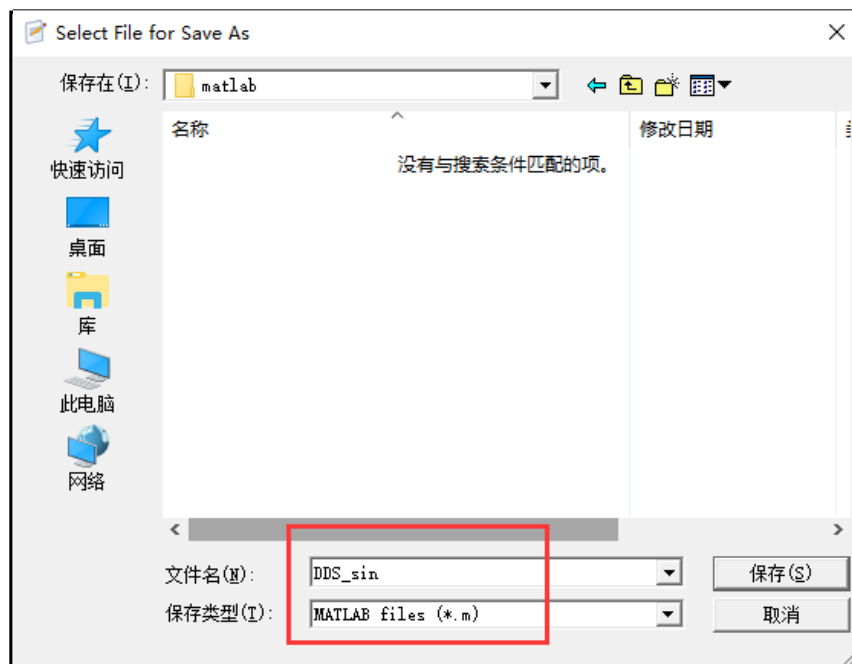


Рис. 2.17 — Сохранение скрипта Matlab ("DDS_sin.m")

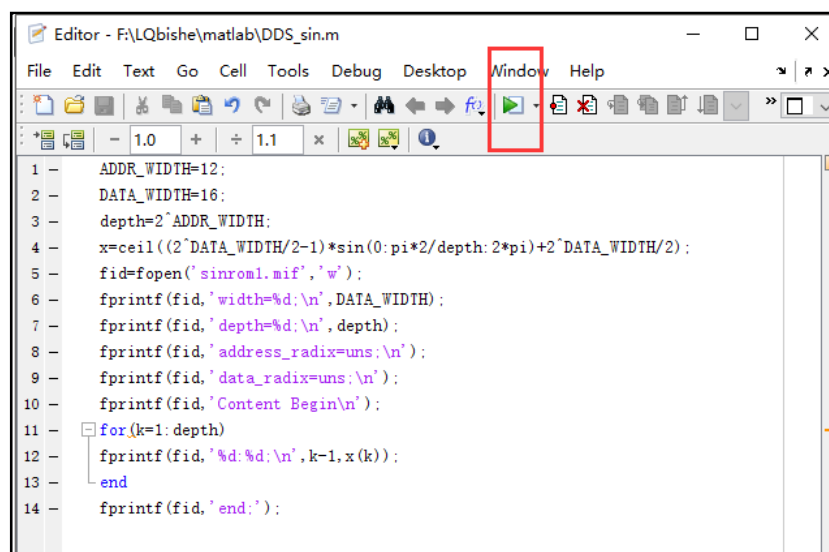


Рис. 2.18 — Запуск скрипта в Matlab

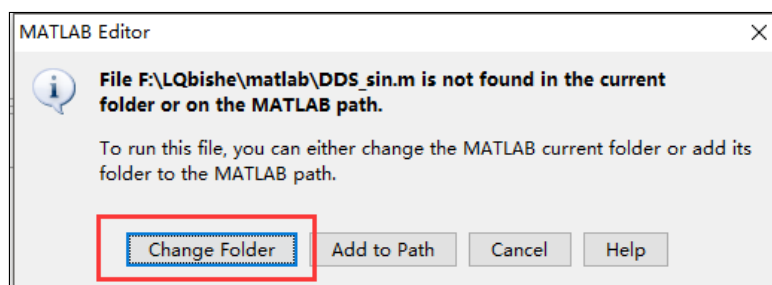


Рис. 2.19 — Диалоговое окно с запросом на изменение рабочей папки

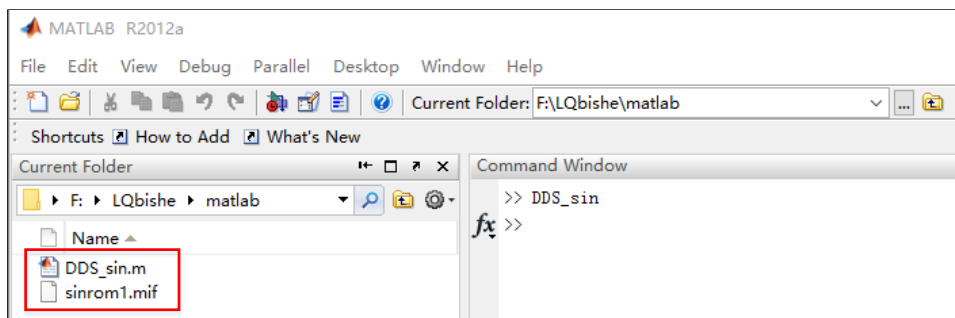


Рис. 2.20 — Сгенерированный файл синусоидальной таблицы "sinrom1.mif"

После того как файл sinrom1.mif был создан, его необходимо поместить в папку проекта DDS, после чего можно использовать IP-ядро Altera для создания ROM-таблицы поиска.

Дальнейшие шаги:

5. Открыть менеджер IP-ядер в Quartus

Как показано на рисунке 2.21, в Quartus откройте IP Catalog / MegaWizard Plug-In Manager.

6. Выбрать создание нового IP-ядра

Как показано на рисунке 2.22:

выберите пункт "Create a new custom megafunction variation" (создать новый вариант пользовательского мегафункционального блока), нажмите "Next".

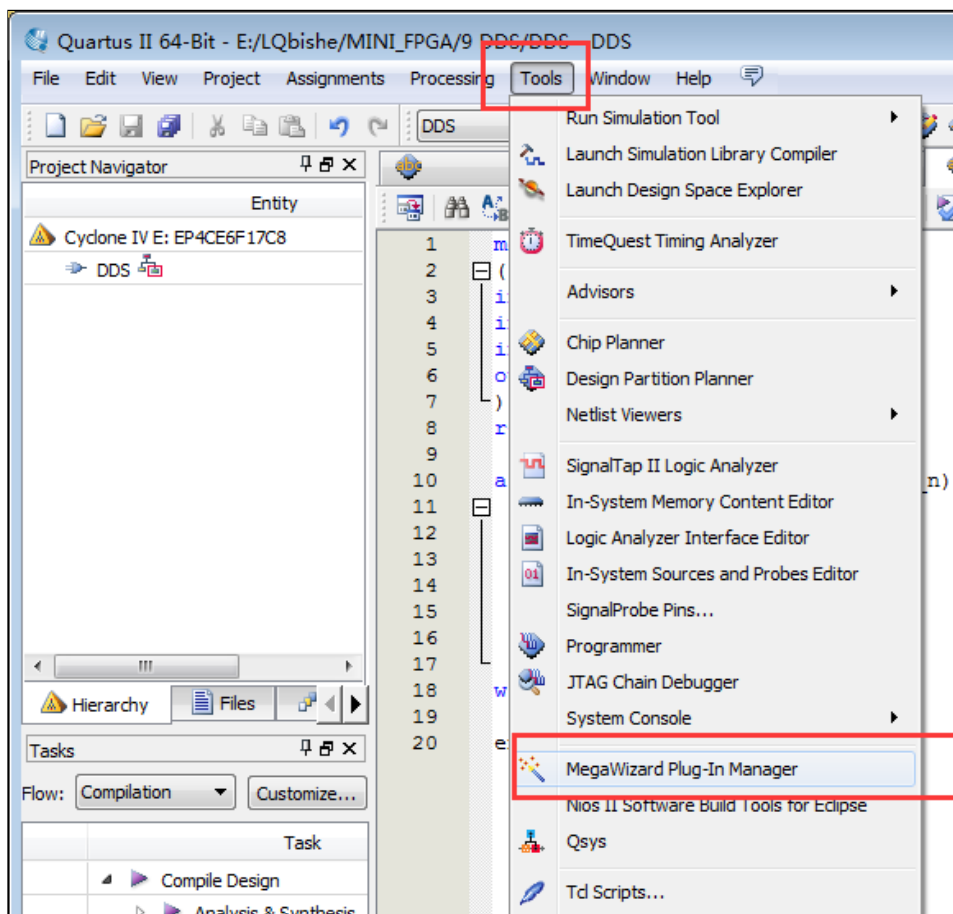


Рис. 2.21 — Открытие менеджера IP-ядер в Quartus

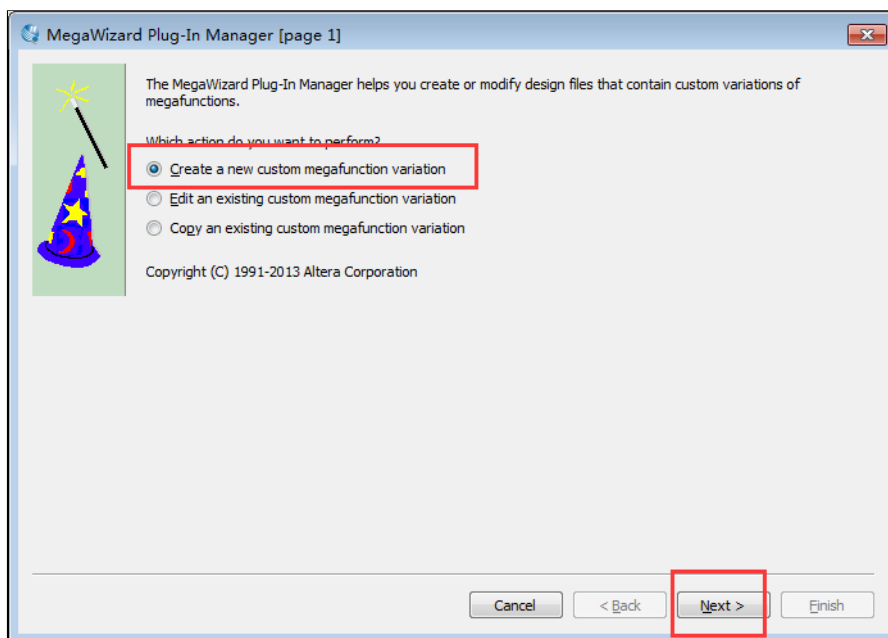


Рис. 2.22 — Создание нового пользовательского варианта IP-ядра

7. В открывшемся менеджере IP-ядер

Откройте папку Memory, затем выберите Single-Port ROM (однопортовый ROM), и задайте имя модуля:

dds_sin_rom

Как показано на рисунке 2.23.

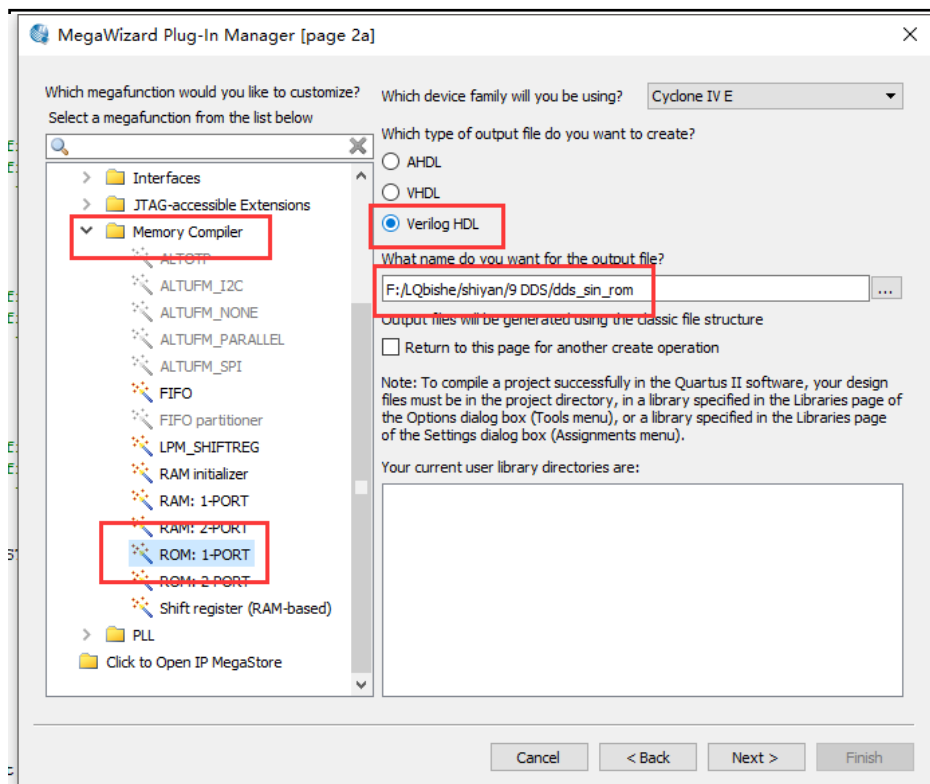


Рис. 2.23 — Выбор Single-Port ROM и задание имени dds_sin_rom

8. Нажмите “Next”, чтобы перейти к настройке параметров однопортового ROM. В окне параметров IP-ядра Single-Port ROM выполните следующие настройки:
 ширина выходных данных (Output width): 8 бит
 размер ROM (Number of words): 4096
 Это соответствует ранее созданной в Matlab таблице:
 данные 8-битные,
 адресная ширина 12 бит $\rightarrow 2^{12} = 4096$ значений.
 После указания параметров нажмите “Next”.
 (см. рисунок 2.24)

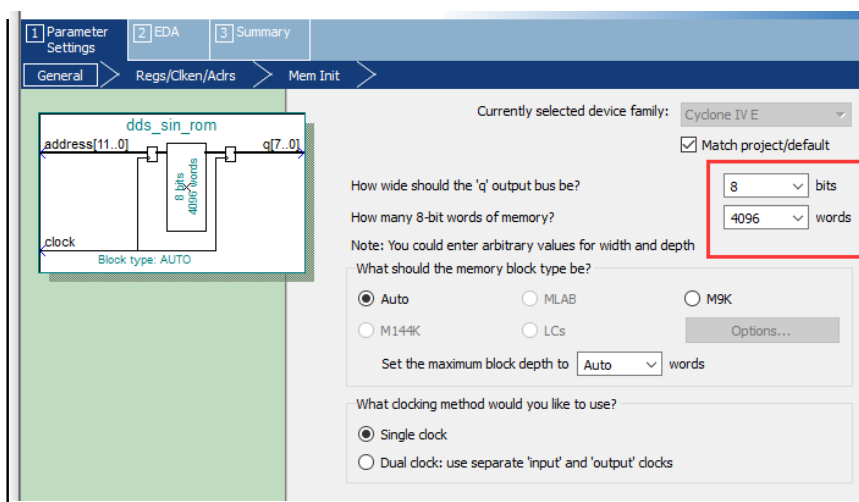


Рис. 2.24 — Настройка параметров однопортового ROM (8 бит, 4096 слов)

9. Далее выбрать конфигурацию выходов (регистры/триггеры). На следующем шаге необходимо указать, какие выходные порты должны иметь регистры (триггеры). В данном примере отмечается только порт q (то есть регистрируется только выходные данные ROM). После этого нажмите “Next”. (см. рисунок 2.25)

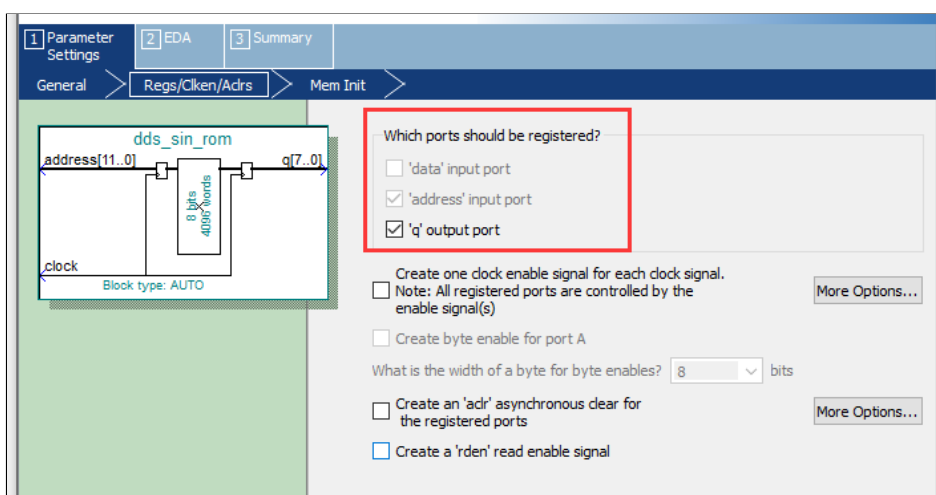


Рис. 2.25 — Настройка регистров (выбор регистрации порта q)

10. Инициализация содержимого ROM

Скопируйте ранее созданный в Matlab файл sinrom1.mif в рабочую папку проекта DDS.

Затем, в окне параметров IP-ядра ROM, нажмите кнопку “Browse” и выберите файл sinrom1.mif (см. рисунок 2.26).

После выбора файла нажмите “Finish”, чтобы завершить создание ROM-таблицы.

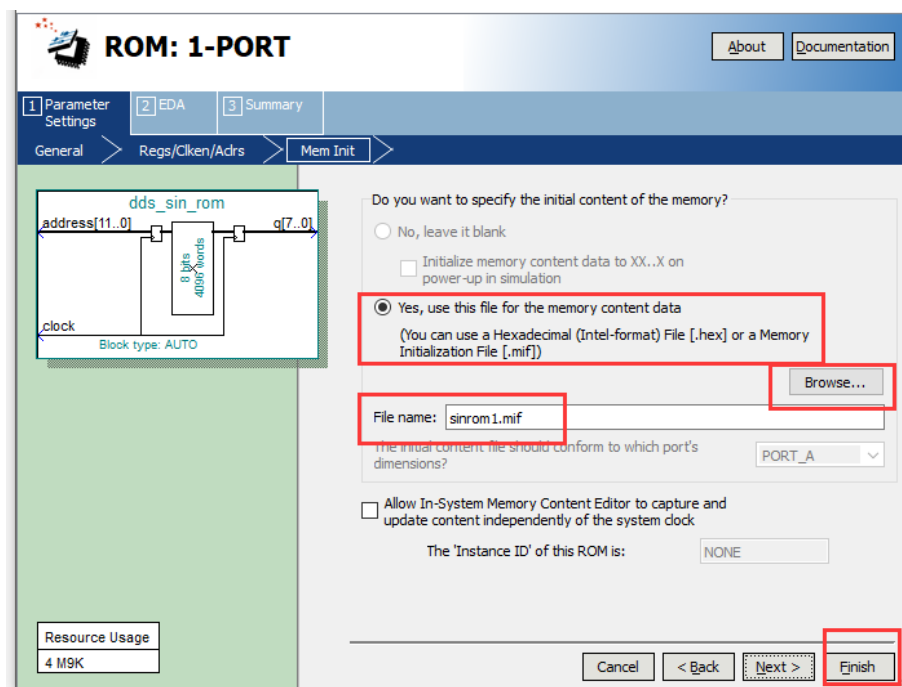


Рис. 2.26 — Загрузка файла sinrom1.mif для инициализации ROM

11. На финальном экране Summary

На последнем шаге мастера (окно Summary) необходимо поставить галочку на пункте `dds_sin_rom_inst.v` — это позволит автоматически сгенерировать инстанцированный файл ROM.

После выбора опции нажмите “Finish”.

(см. рисунок 2.27)

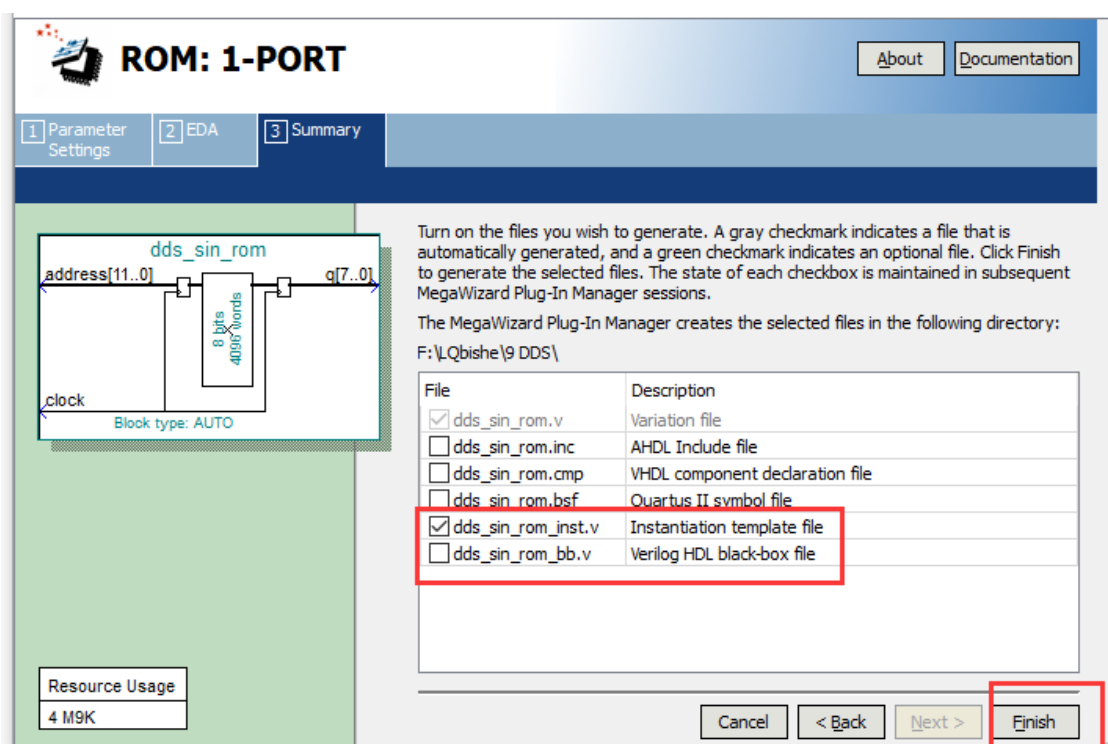


Рис. 2.27 — Генерация файла `dds_sin_rom_inst.v` на этапе Summary

Таким образом, инстанцированный файл ROM-таблицы синусоиды успешно создан.

Метод использования (вставки) этого инстанцированного компонента совпадает с тем, что был описан в Эксперименте 1 второй части:

просто откройте сгенерированный файл инстанцирования,

скопируйте содержащийся в нём код,

вставьте его в нужный модуль,

и при необходимости измените имена интерфейсов (портов).

После этого ROM можно использовать в проекте как обычный модуль.

1.6 Как с помощью IP-ядер Altera создать модуль PLL (фазовой автоподстройки частоты)

Начало раздела о создании PLL для SignalTap

В данном эксперименте рабочая частота SignalTap выбирается равной 100 кГц.

Этот тактовый сигнал будет получен напрямую из модуля PLL, созданного с помощью IP-ядра.

Шаги создания PLL аналогичны тем, что описаны в пункте «8» эксперимента 2.2, но здесь кратко поясняются параметры, которые требуется настроить.

1. В менеджере IP-ядер

В открывшемся IP Catalog:

найдите и откройте папку I/O,

выберите элемент ALTPLL,

задайте имя модуля:

pll_100K

Как показано на рисунке 3.10.

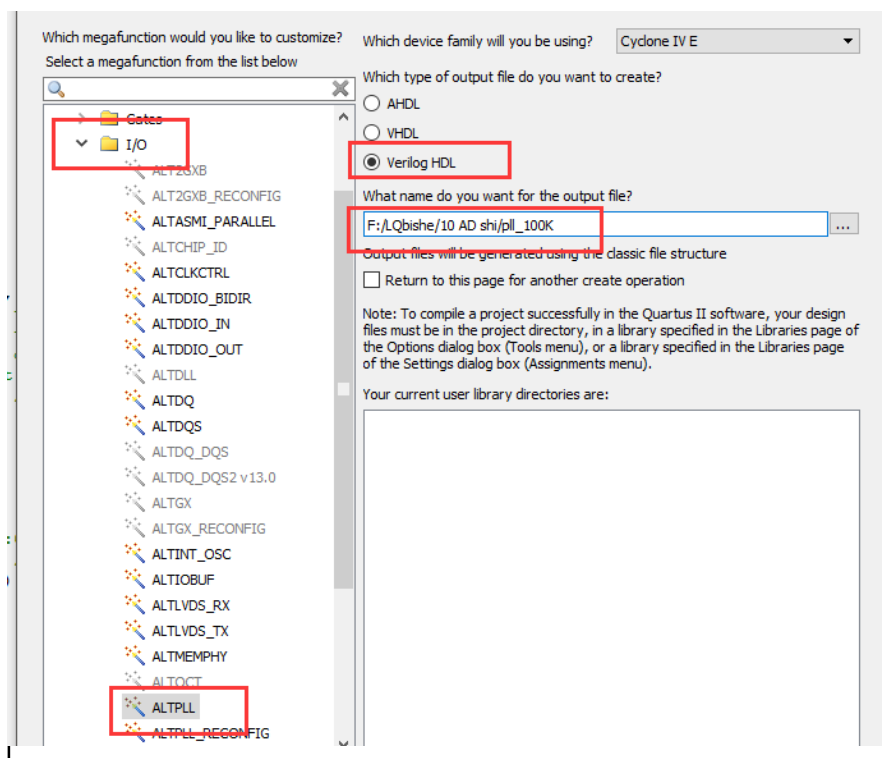


Рис. 3.10

2. Нажмите “Next”, чтобы перейти к окну настройки параметров IP-ядра ALTPLL

В появившемся окне настройки IP-ядра ALTPLL необходимо указать частоту входного тактового сигнала.

Выберите:

Входная частота (Input Clock Frequency): 50 MHz

(см. рисунок 3.11)

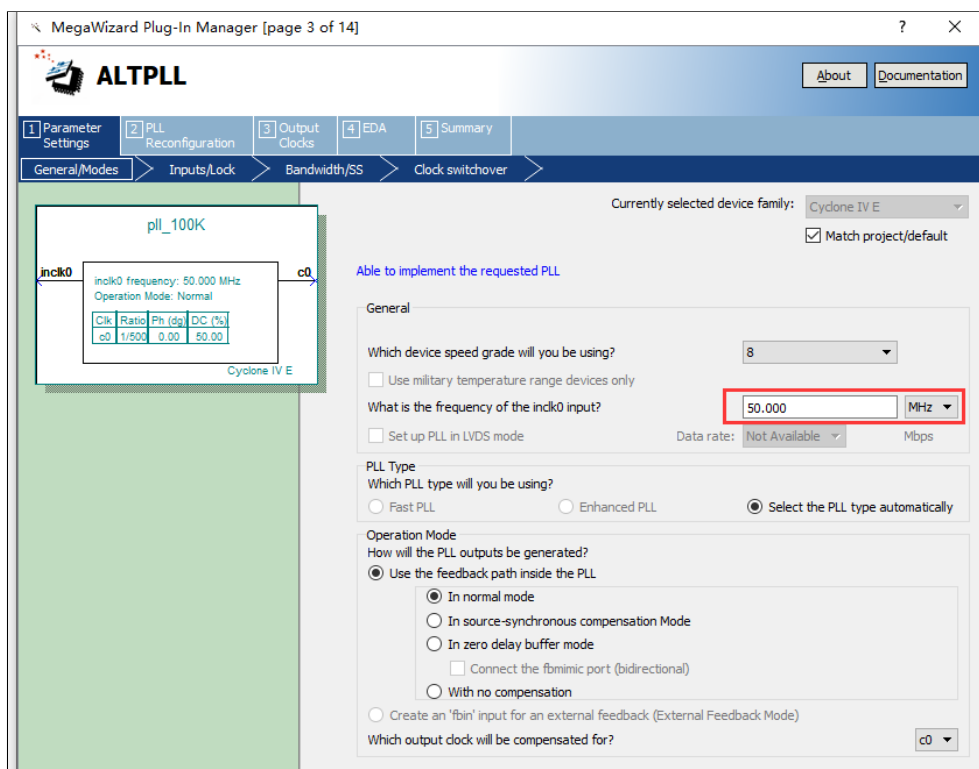


Рис. 3.11

3. Далее выбираются входные и выходные сигналы

На этом этапе необходимо установить параметры входов и выходов PLL. В окне выбора входных/выходных сигналов не нужно отмечать ни одного параметра — оставьте все галочки снятыми.

После этого нажмите “Next”.

(см. рисунок 3.12)

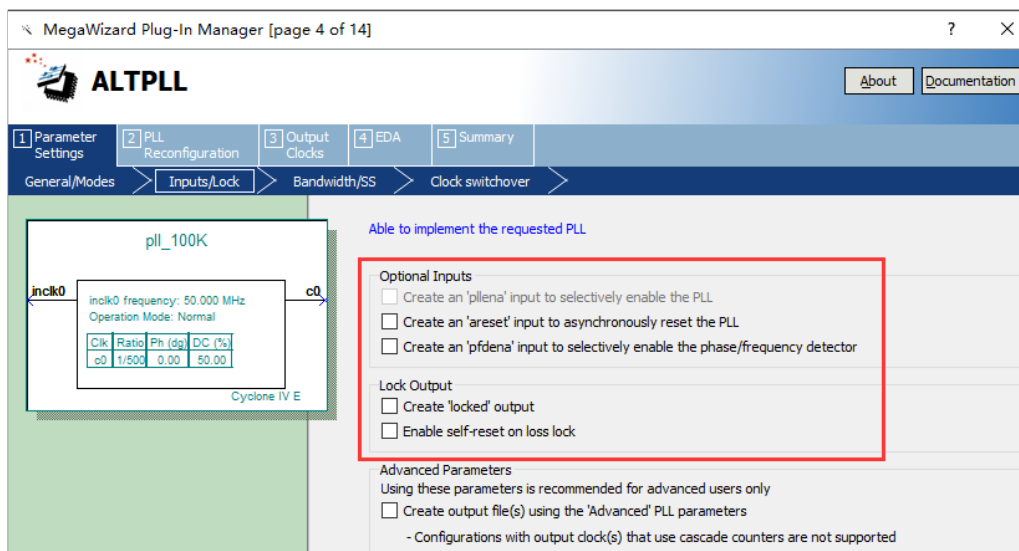


Рис. 3.12

4. Последовательно нажимайте “Next”, оставляя параметры по умолчанию

Продолжайте переходить по страницам мастера настройки PLL, не изменяя параметры, пока не дойдёте до страницы:

“page 8 of 14”

На этой странице необходимо установить частоту выходного тактового сигнала:

Output Clock Frequency = 100 kHz

(см. рисунок 3.13)

После выбора частоты нажмите “Finish”, чтобы завершить создание PLL-модуля.

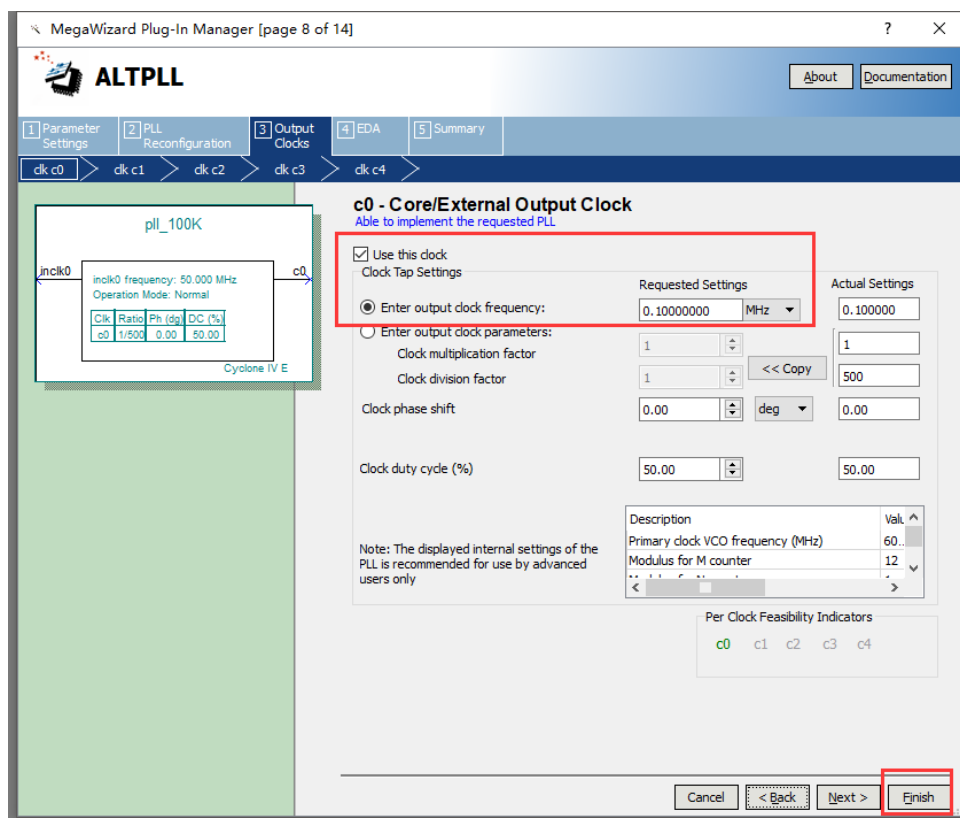


Рис. 3.13

5. На финальном экране Summary

На последнем шаге мастера необходимо отметить пункт, который отвечает за создание файла инстанцирования PLL:

pll_100K_inst.v

(в тексте у тебя ошибочно повторено dds_sin_rom_inst.v, но по логике здесь должен быть именно файл инстанцирования PLL).

После установки галочки нажмите “Finish”.

(см. рисунок 3.14)

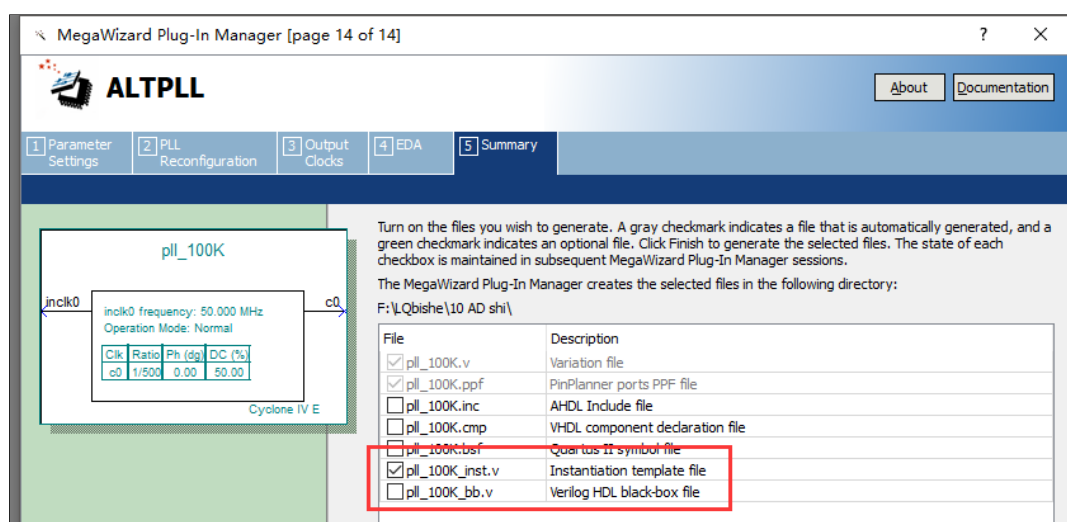


Рис. 3.14

Таким образом, инстанцированный файл модуля PLL успешно создан.

Использовать (вставлять) этот инстанцированный компонент можно точно так же, как и любой другой модуль в проекте.

1.7 Как использовать SignalTap

В данном эксперименте используется встроенный в Quartus инструмент SignalTap (логический анализатор) для анализа временных характеристик сигналов.

Процесс работы с ним следующий:

1. Открытие логического анализатора

Порядок действий показан на рисунке 3.15.

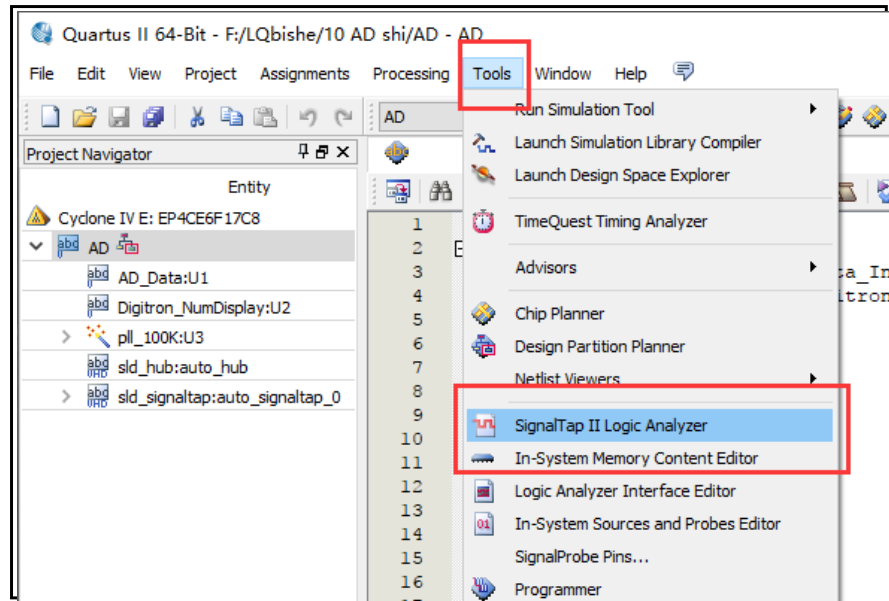


Рис. 3.15

2. Основной интерфейс SignalTap

Главное окно SignalTap показано на рисунке 3.16. Оно содержит несколько областей:

Область управления состоянием

Используется для запуска, остановки и паузы логического анализатора.

Область сигналов

Здесь отображаются все сигналы, которые вы добавите для наблюдения.

Область настройки тактового сигнала и глубины выборки

Позволяет выбрать тактовый сигнал (sampling clock) и задать глубину буфера (sample depth).

Область расширенных функций триггера

Используется для настройки более сложных условий запуска анализа.

3. После открытия SignalTap первым делом нужно выбрать тактовый сигнал

В данном эксперименте в качестве тактового сигнала используется 100 kHz, созданный модулем PLL.

Как показано на рисунке 3.17:

нажмите кнопку справа от поля Clock,
откроется диалоговое окно Node Finder.

4. Настройка Node Finder

Как показано на рисунке 3.18,
в открывшемся окне Node Finder:

в поле Filter выберите пункт:

SignalTap II: pre-synthesis

Это означает, что будут отображаться сигналы перед синтезом, то есть напрямую из исходных файлов.

Почему рекомендуется именно так:

при синтезе Quartus оптимизирует схему;

некоторые сигналы могут быть удалены или объединены;

поэтому, если выбирать post-synthesis, часть нужных сигналов может не отображаться.

Поэтому правильное решение — pre-synthesis.

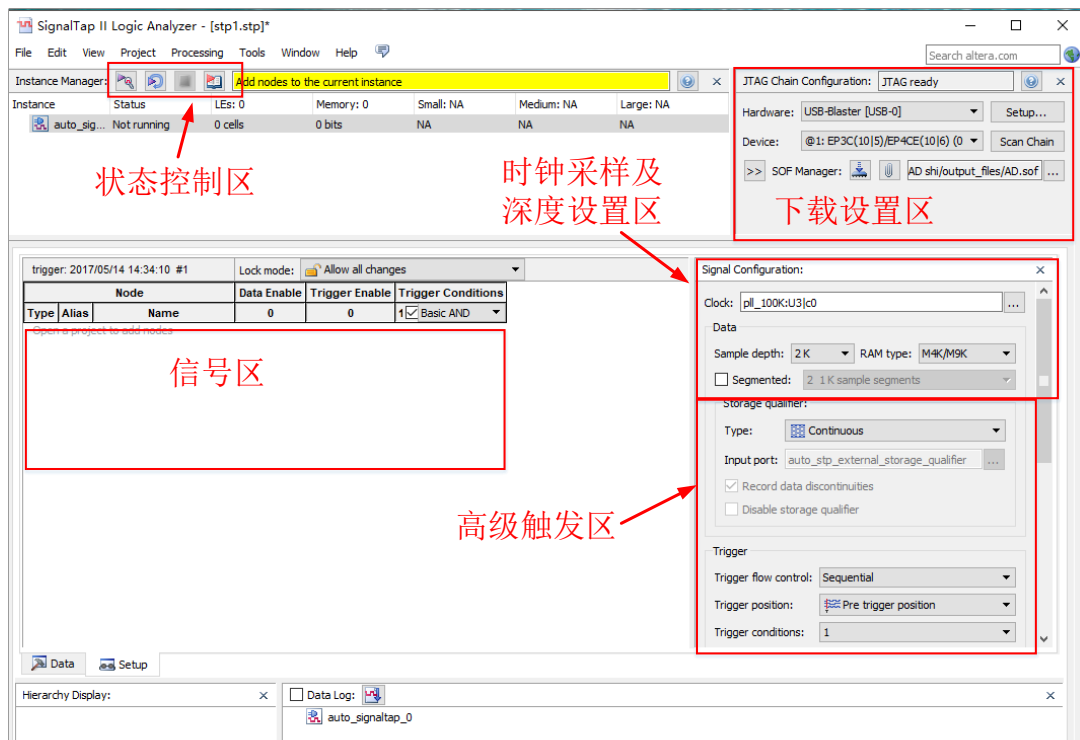


Рис. 3.16

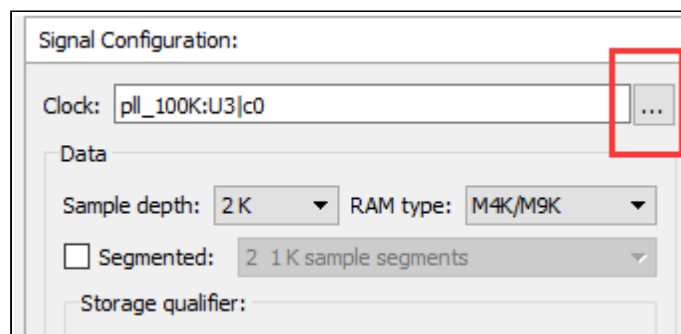


Рис. 3.17

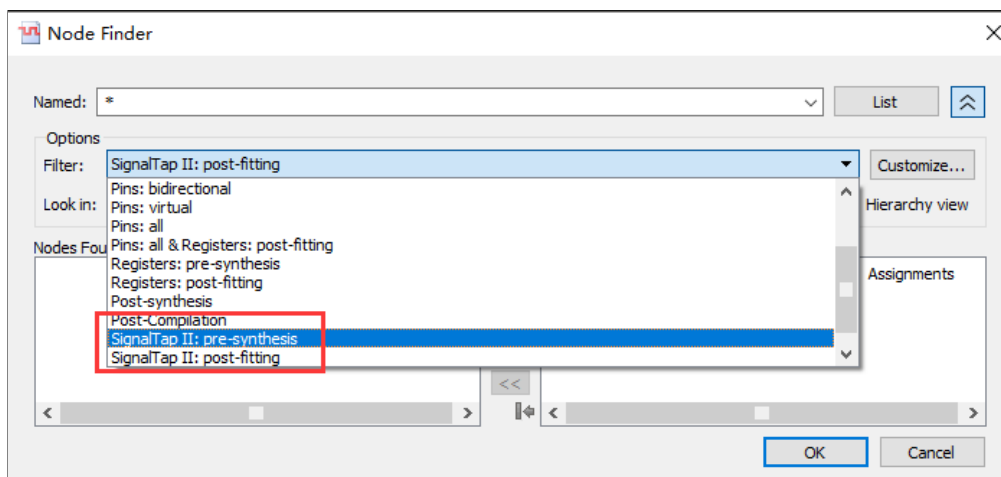


Рис. 3.18

5. Добавление сигнала тактового генератора в SignalTap

Нажмите кнопку List в правом верхнем углу окна Node Finder (см. рисунок 3.18).

В поле Nodes Found появится список всех сигналов, которые можно добавить в SignalTap. Обычно там находятся:

интерфейсные сигналы модуля,

регистры,

выходы/входы подмодулей,

внутренние сигналы, доступные до синтеза.

Так как в данном эксперименте SignalTap должен использовать тактовый сигнал c0, который является выходом PLL-модуля pll_100K, необходимо найти этот сигнал в списке Nodes Found.

Далее:

выберите сигнал c0,

добавьте его в SignalTap двойным щелчком или выполните действия, показанные на рисунке 3.19,

нажмите Finish,

чтобы завершить добавление тактового сигнала.

После этого SignalTap будет использовать c0 как рабочий (sampling) clock.

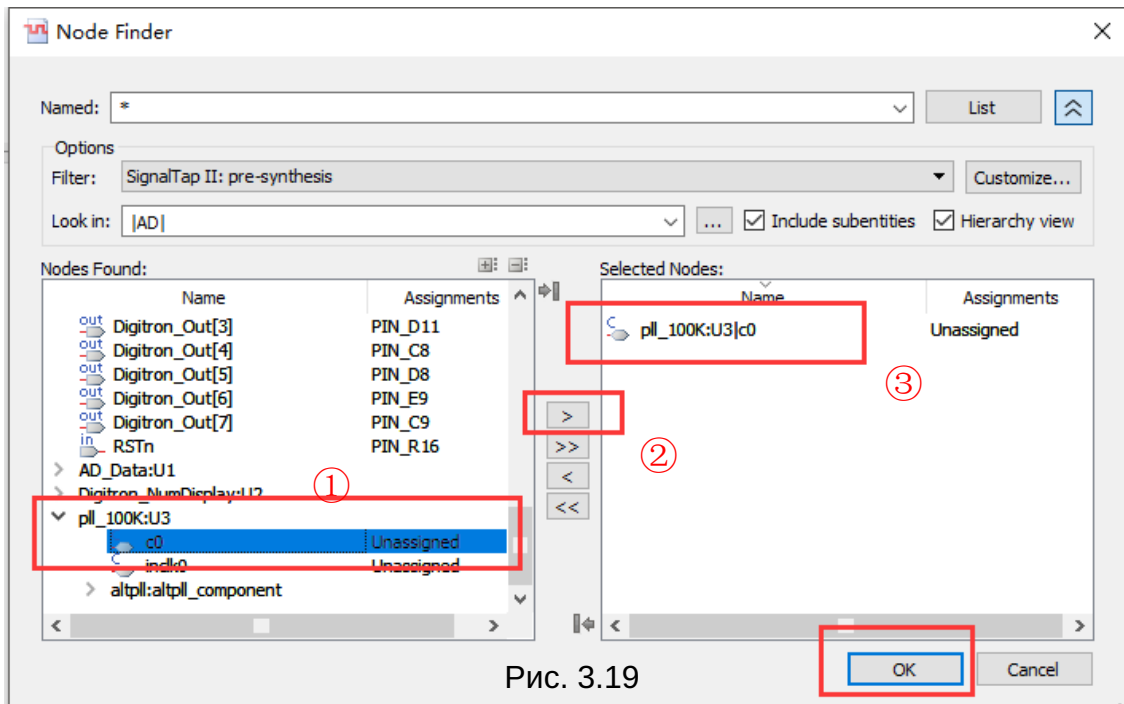


Рис. 3.19

6. Добавление наблюдаемых сигналов в SignalTap

Щёлкните левой кнопкой мыши по пустой области в списке сигналов (Signal List). Откроется окно Node Finder.

Так как в данном эксперименте нас интересуют сигналы из модуля AD_Data, необходимо:

в поле Look in выбрать уровень иерархии AD_Data,

затем нажать кнопку List.

После этого в окне Nodes Found отобразятся только сигналы модуля AD_Data.

Порядок действий показан на рисунке 3.20.

7. Добавление сигналов Data_Out и других

По аналогии с пунктом 5:

выберите в списке нужные сигналы (например, Data_Out и другие),

добавьте их в список SignalTap двойным щелчком или как показано на рисунке 3.21.

После добавления нажмите OK.

8. Настройка глубины выборки (Sampling Depth)

Установите глубину буфера семплирования равной:

2K (2048 выборок)

После этого сохраните проект SignalTap в файл:

stp1.stp

(имя может быть произвольным).

В появившемся диалоговом окне выберите Yes, чтобы добавить файл stp1.stp в проект Quartus (см. рисунок 3.22).

После этого выполните полную компиляцию проекта.

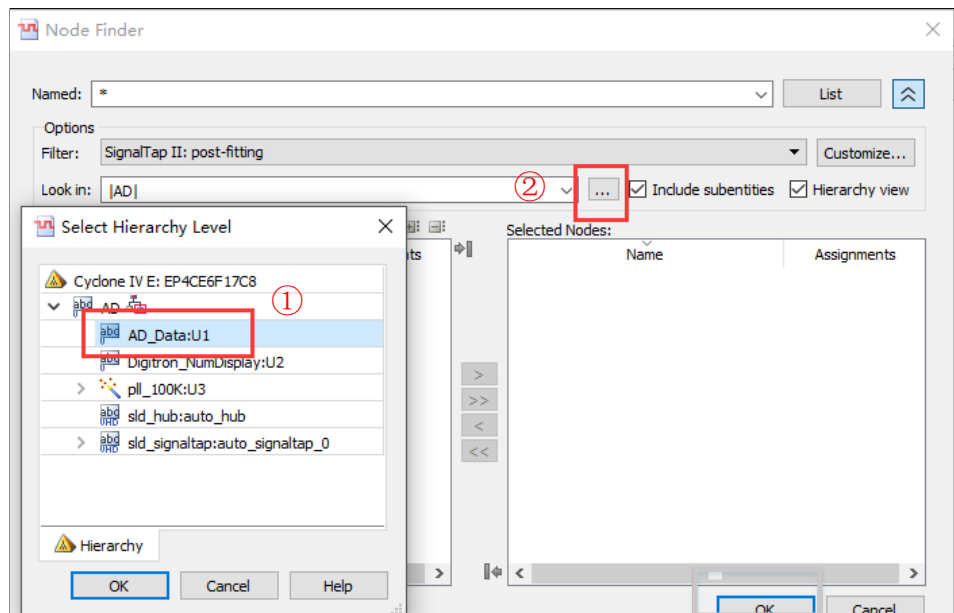


Рис. 3.20

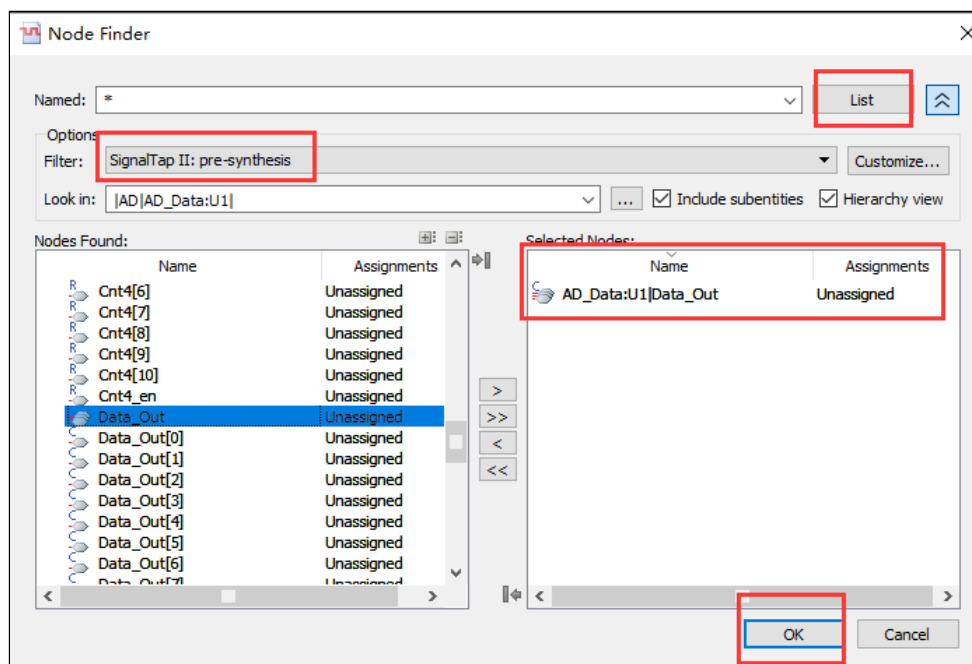


Рис. 3.21

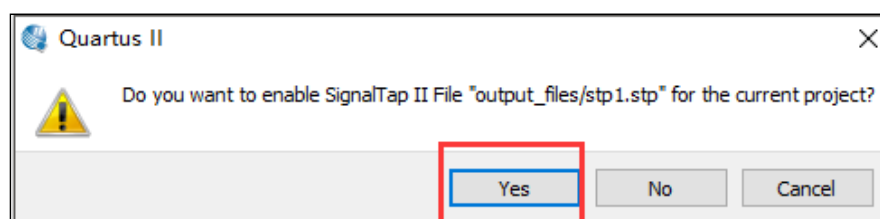


Рис. 3.22

9. После завершения компиляции

Откройте окно SignalTap.

В области загрузки (Download Configuration) необходимо настроить устройство USB-Blaster.

Порядок действий показан на рисунке 3.23.

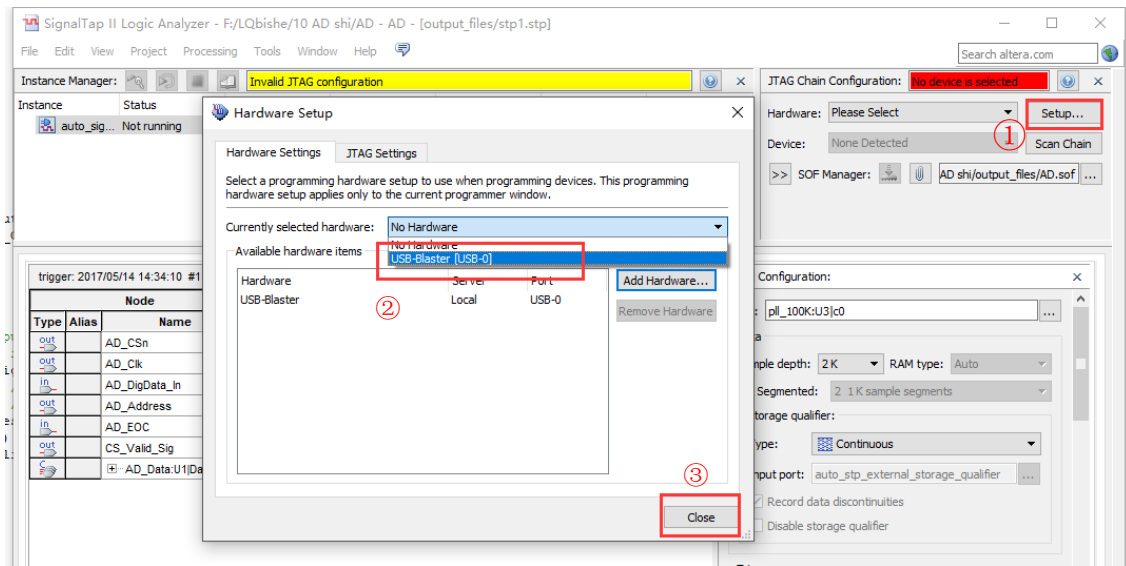


Рис. 3.23

10. Как показано на рисунке 3.24

Чтобы получить информацию об устройстве (Device Info), необходимо сначала подать питание на отладочную плату.

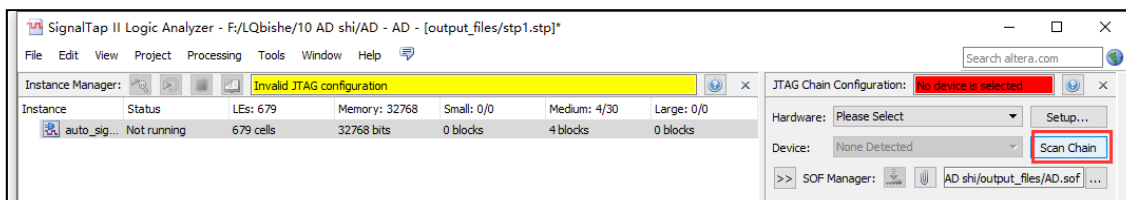


Рис. 3.24

11. Получение информации о SOF-файле

Как показано на рисунке 3.25, нажмите кнопку, выделенную красным кружком, и выберите файл конфигурации .sof.

После успешного выбора SOF-файла область загрузки (Download Area) будет выглядеть так, как показано на рисунке 3.26.

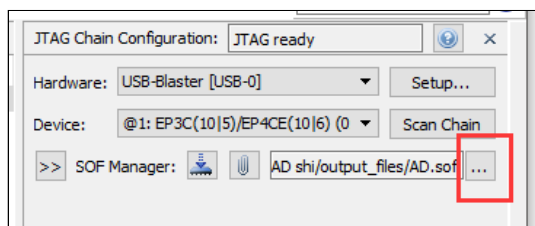


Рис. 3.25

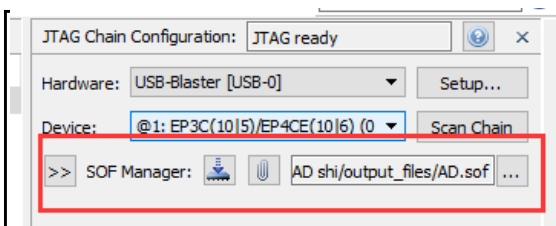


Рис. 3.26

12. Загрузка проекта в FPGA и запуск SignalTap

Нажмите кнопку загрузки (Download), чтобы выполнить прошивку FPGA, как показано на рисунке 3.27.

После завершения загрузки:

нажмите значок в области управления состоянием (левый верхний угол окна),

чтобы запустить сбор данных в SignalTap, как показано на рисунке 3.28.

После запуска вы сможете наблюдать, как выбранные сигналы непрерывно изменяются во времени.

Иными словами, логический анализатор работает — на экране отображается временная диаграмма сигналов в реальном времени.

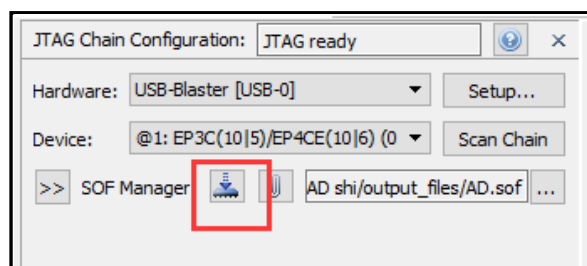


Рис. 3.27

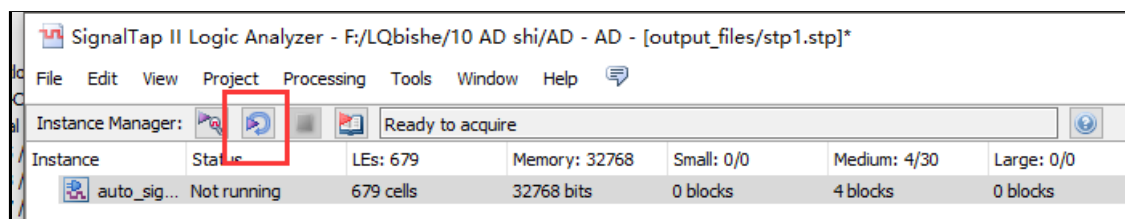


Рис. 3.28

13. Различные режимы отображения сигналов в SignalTap

Один и тот же сигнал может отображаться в SignalTap разными способами. Возьмём для примера сигнал Data_Out.

Выберите сигнал Data_Out, затем щёлкните по нему правой кнопкой мыши — появится меню выбора формата отображения сигнала.

Наиболее часто используемые режимы:

Hexadecimal — шестнадцатеричное отображение

Unsigned Decimal — беззнаковое десятичное отображение

Binary — двоичное отображение

Unsigned Line Char — графическое отображение беззнаковых данных (линейный график)

Signed Line Char — графическое отображение со знаком

В данном эксперименте сигнал Data_Out представляет собой 8-битное беззнаковое значение, поэтому нужно выбрать режим:

Unsigned Line Char

После выбора этого режима (см. рисунок 3.29) можно увидеть отображение сигнала Data_Out в виде синусоидальной кривой.

Заключение

Другие функции SignalTap студенты могут изучить самостоятельно, используя данное руководство.

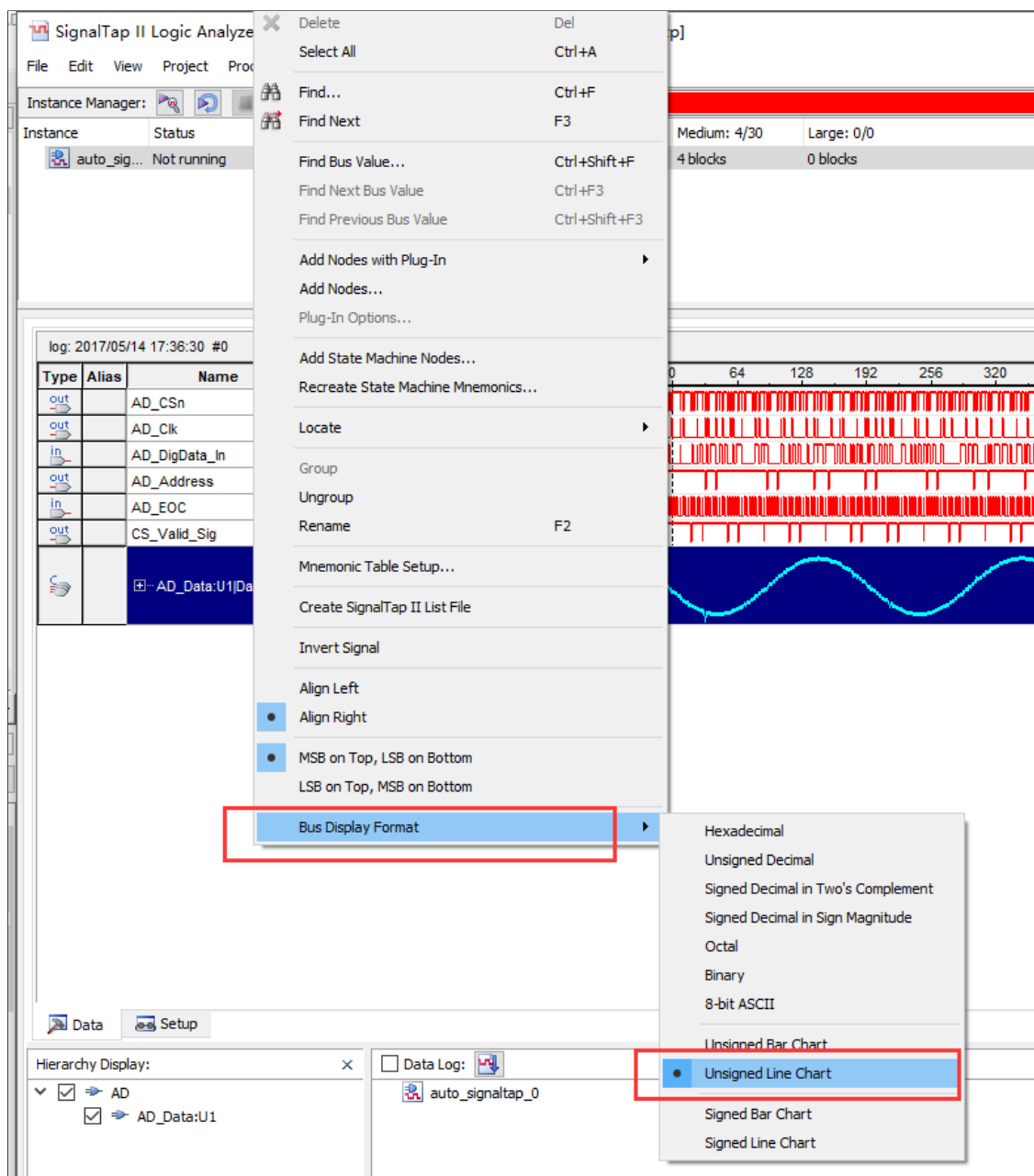


Рис. 3.29

1.8 Как с помощью IP-ядер Altera создать цифровой низкочастотный фильтр (Low-Pass Filter, LPF)

В данном эксперименте для настройки FIR низкочастотного фильтра используется IP-ядро. Ниже приведено краткое описание ключевых параметров и порядка настройки.

1. Откройте менеджер IP-ядер

Способ открытия такой же, как в пункте «8» эксперимента 2.2 (использование MegaWizard / IP Catalog).

2. В открывшемся менеджере IP-ядер

откройте папку DSP,

выберите компонент FIR Compiler,

задайте имя модуля:

low_pass_filter1

Как показано на рисунке 3.29

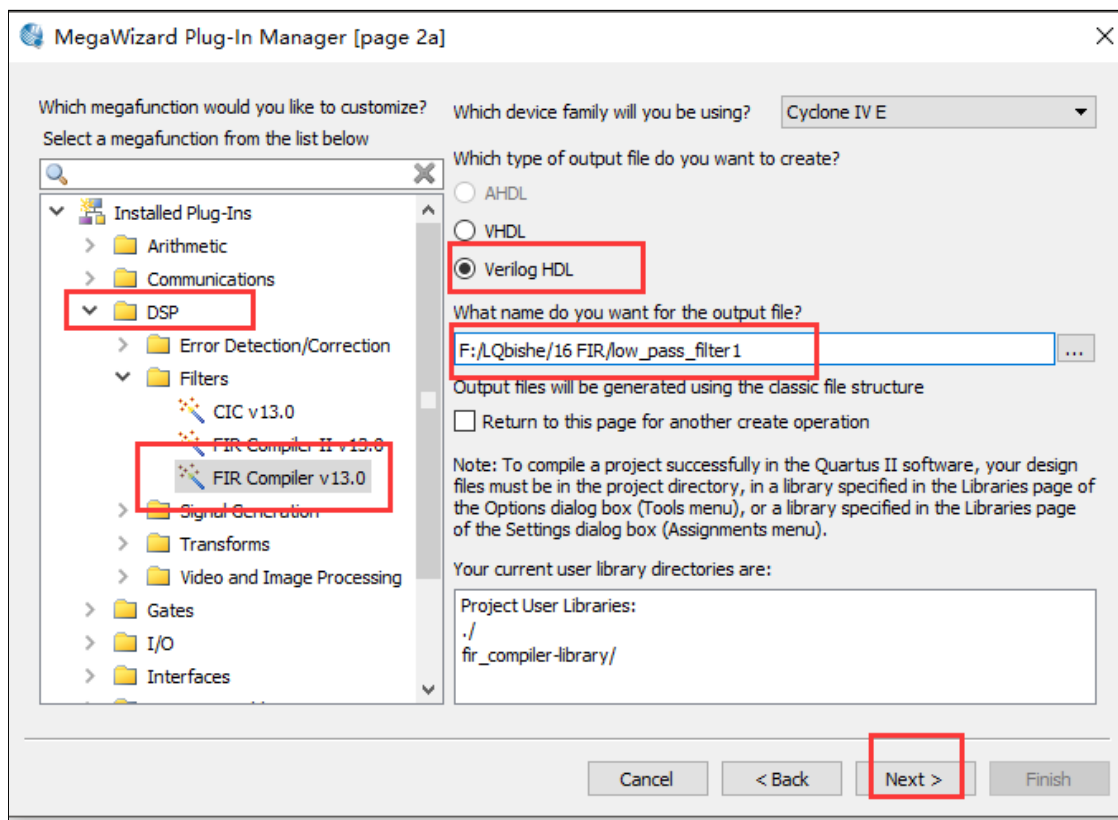


Рис. 3.29

2. В появившемся диалоговом окне выберите “Parameterize”

После выбора компонента FIR Compiler откроется диалоговое окно. В этом окне необходимо нажать кнопку Parameterize, чтобы перейти к экрану настройки параметров IP-ядра FIR.

Как показано на рисунке 3.30.

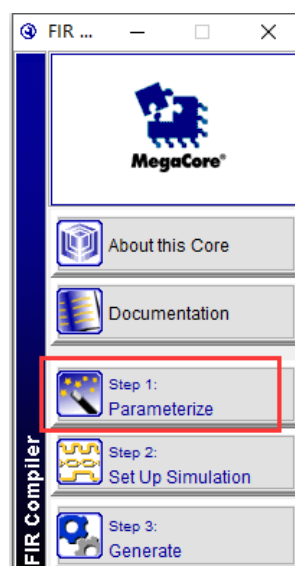


Рис. 3.30

3. Нажмите кнопку, отмеченную красным кружком

Как показано на рисунке 3.31, необходимо нажать указанную кнопку, чтобы перейти к следующим параметрам настройки FIR-фильтра.

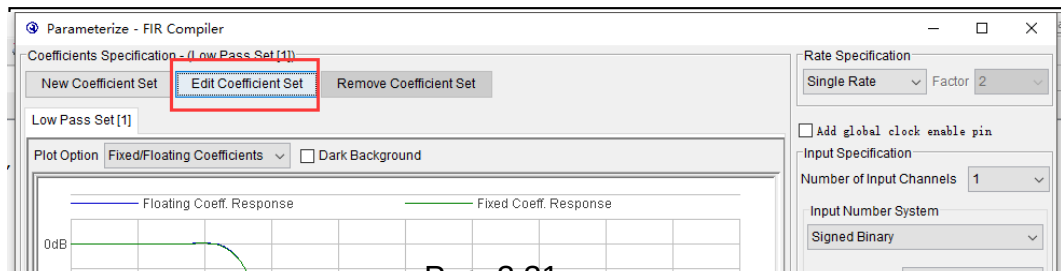


Рис. 3.31

4. Настройка основных параметров FIR-фильтра

Следуя шагам, показанным на рисунке 5.19, необходимо последовательно задать параметры: тип фильтра, тип окна, порядок (число коэффициентов / taps), частоту дискретизации, частоту среза (cutoff frequency).

В данном эксперименте используются следующие настройки:

тип фильтра: низкочастотный фильтр (Low-Pass Filter) тип окна: окно Хэмминга (Hamming Window)

порядок фильтра: 50 частота дискретизации: 50 kHz частота среза: 10 kHz

После завершения настройки нажмите Apply, затем OK.

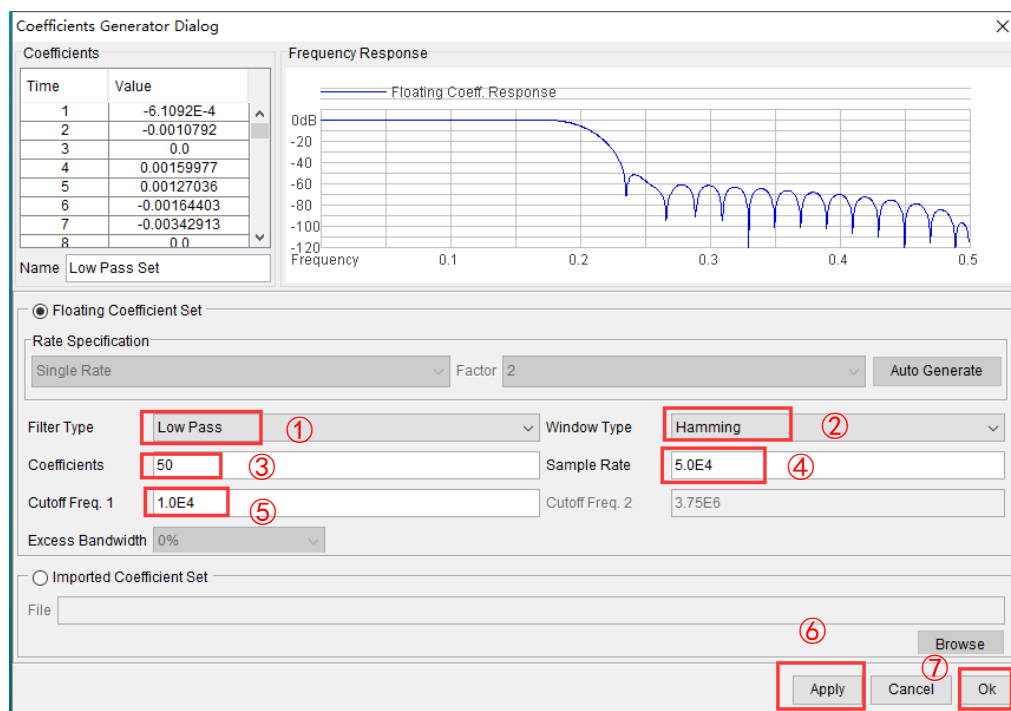


Рис. 3.32

5. Настройка параметров входа/выхода, коэффициентов и устройства

Как показано на рисунке 3.33, необходимо указать следующие параметры:

Формат входных данных

Выберите: Unsigned binary input (входные данные — беззнаковое двоичное число)

Разрядность входных данных

Установите: 8 бит (так как в эксперименте на вход фильтра подаются 8-битные данные)

Формат выхода и насыщение

Выберите: Пользовательская настройка насыщения (Custom Saturation)

и установите разрядность выходных данных = 23 бита

Параметры коэффициентов FIR-фильтра

расширение коэффициентов (Coefficient Expansion) — Auto

разрядность коэффициентов (Coefficient Width) — 12 бит

Тип устройства (Device Family)

Выберите нужную семейство/модель FPGA, соответствующую проекту.

Pipeline Level

Установите: 1 (включает конвейеризацию 1 уровня для повышения частоты работы)

После завершения настройки нажмите Finish.

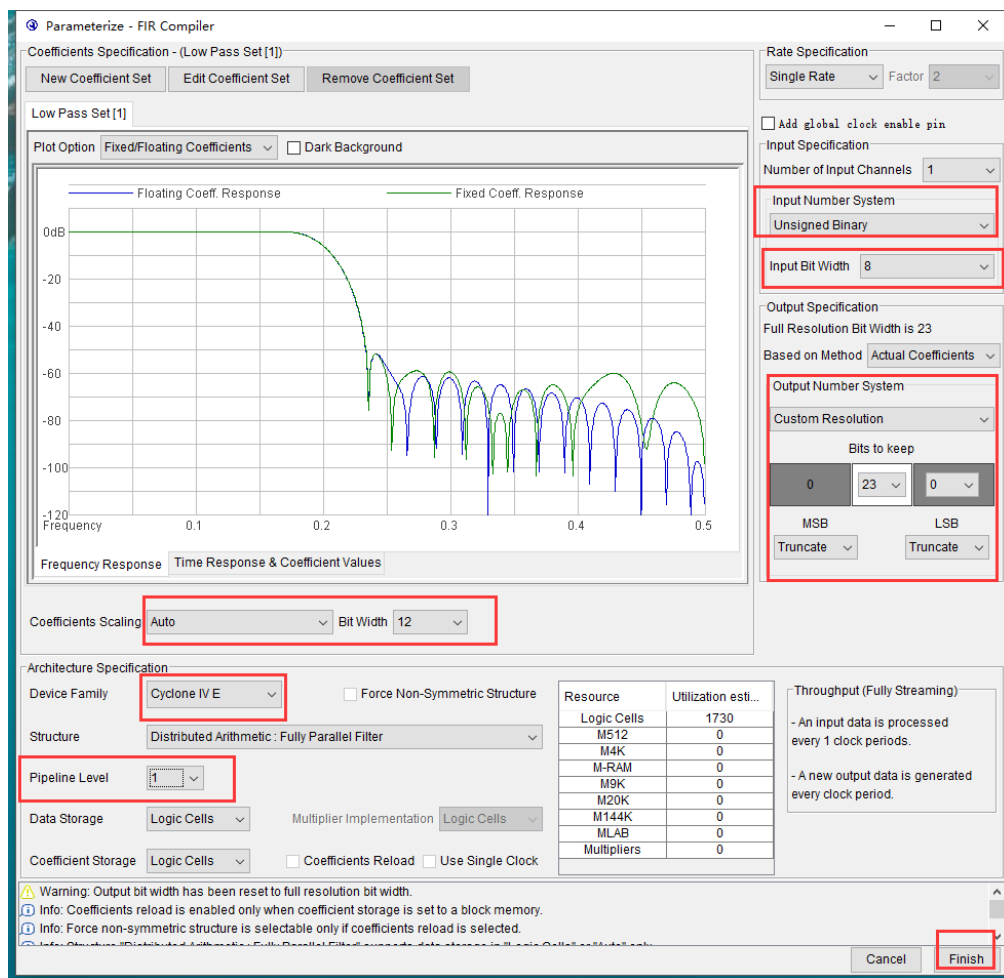


Рис. 3.33

6. В появившемся диалоговом окне выберите “Generate”

После настройки всех параметров, в открывшемся диалоговом окне нажмите кнопку Generate, чтобы сгенерировать модуль FIR-фильтра.

Как показано на рисунке 3.34

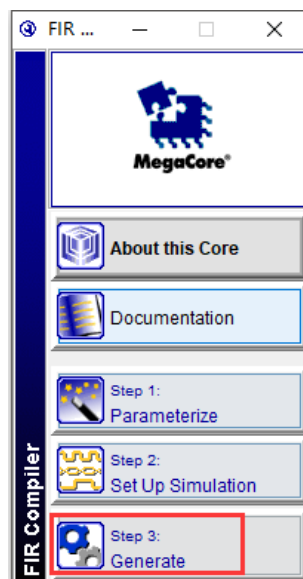


Рис. 3.34

7. Если на экране появилось содержимое, выделенное красной рамкой в рисунке 3.35

это означает, что настройка фильтра завершена успешно.

После этого нажмите Exit, чтобы закрыть окно мастера.

Вызов (инстанцирование) сгенерированного FIR-модуля выполняется таким же образом, как и для других IP-ядер: откройте файл инстанцирования, скопируйте интерфейс и подключите его в нужном модуле проекта.

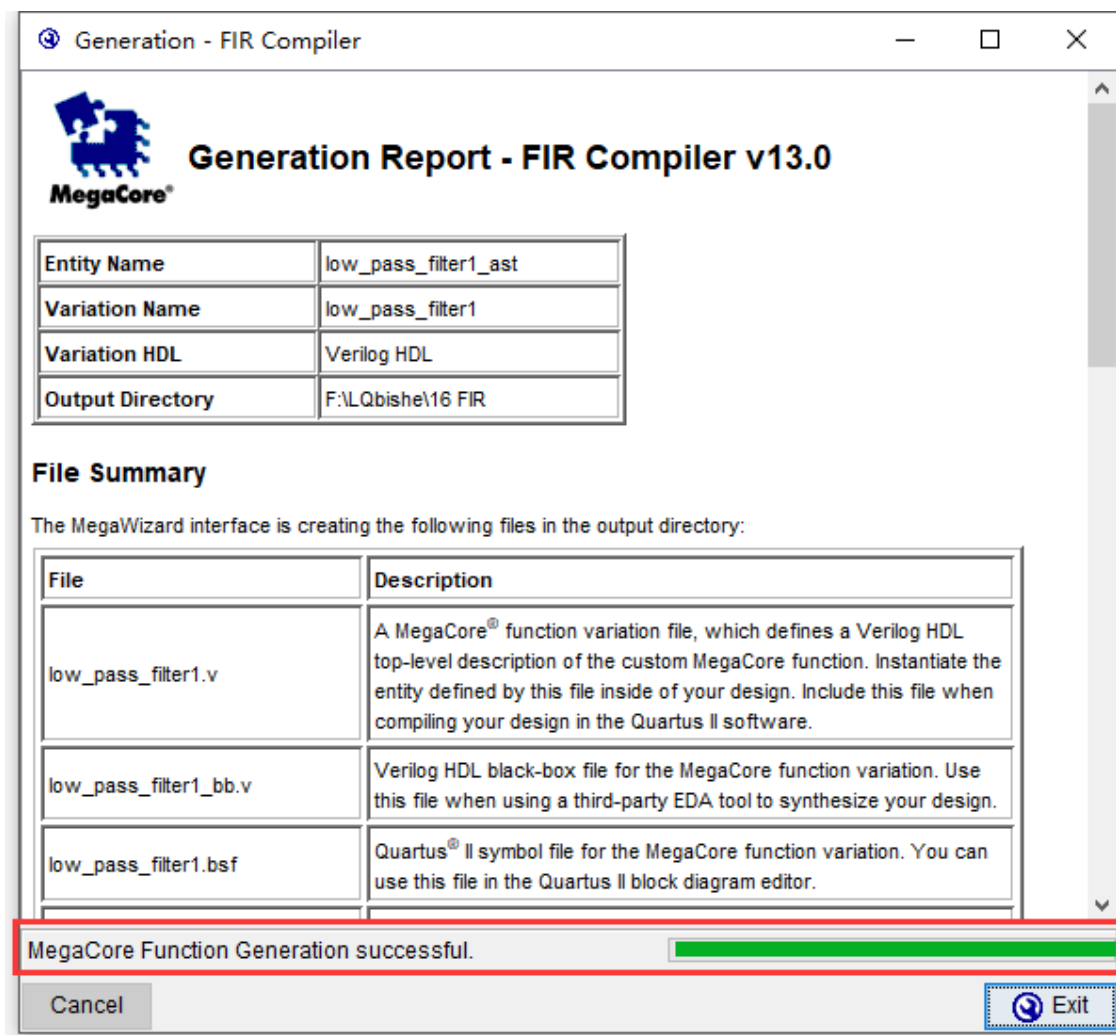


Рис. 3.35

Эксперимент 1 — Бегущие огни

I. Цели эксперимента

- (1) С помощью программирования последовательно зажигать 8 светодиодов, создавая эффект «бегущих огней».
- (2) В процессе выполнения работы освоить создание нового проекта в Quartus II.

II. Идея (принцип) проектирования

В разделе 1.3.2 первой части уже упоминалось, что светодиоды на плате зажигаются высоким уровнем, то есть: вывод = 1 → светодиод зажён
вывод = 0 → светодиод выключен

Следовательно, сущность бегущих огней — это перемещение единицы.

Суть реализации:

В регистре хранится значение 0000_0001, где младший бит (LSB) равен “1”.

При каждом тактовом импульсе, который определяет частоту бегущих огней, это значение циклически сдвигается вправо на 1 бит.

Таким образом создаётся эффект «движения» единицы слева направо.

Через выводы I/O содержимое регистра подаётся на светодиоды.

Это и формирует визуальный эффект бегущих огней слева направо.

III. Функциональная схема модулей и описание сигналов I/O

Проект «бегущие огни» содержит: верхний модуль run_led

нижний модуль led8_module

На рис. 1.1 приведён BSF-файл верхнего уровня, созданный Quartus.

На рис. 1.2 — функциональная блок-схема всего проекта.

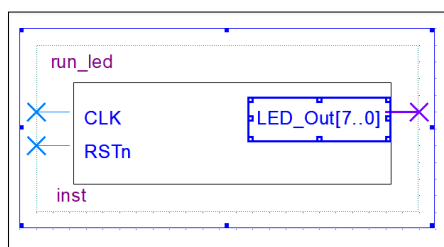


Рис. 1.1 — BSF-файл верхнего уровня

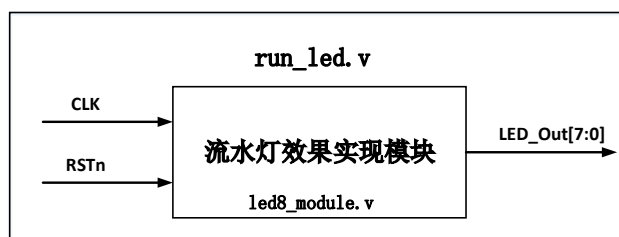


Рис. 1.2 — Функциональная схема модуля бегущих огней

Описание основных входов и выходов

- (1) CLK Входной системный тактовый сигнал 50 МГц. При делении частоты на 5 000 000 получаем 10 Гц. Это означает, что состояние бегущих огней меняется каждые 0,1 секунды.
- (2) RST Входной сигнал сброса по низкому уровню (активный “0”). После сброса: система возвращается в начальное состояние, внутренний счётчик обнуляется, LED7 ~ LED1 — выключены, LED0 — включён.
- (3) LED_Out Выход на светодиоды — 8-битная шина. Когда система работает: значение регистра передаётся на эти выводы, светодиоды отображают текущую «позицию» бегущей единицы.

IV. Программная часть

На рис. 1.3 показан фрагмент кода нижнего модуля led8_module.

Строки 10–12:
Объявление параметров и регистровых сигналов.

Строки 15–19:
Описание поведения при сбросе.
После сброса:

счётчик Count обнуляется,

rLED_Out устанавливается в 00000000.

Строки 20–22 и 28–29:
Это счётчик.

При каждом фронте (положительном переходе) сигнала CLK счётчик Count увеличивается на 1. Когда Count достигает 4999999, он сбрасывается при следующем фронте CLK.

Цель — получить управляющую частоту 10 Гц, которая определяет скорость смены эффекта бегущих огней.

Строки 25–26:

Это ключевые строки всего эксперимента.

Они выполняют циклический сдвиг вправо, благодаря чему «1» перемещается по 8-битному регистру rLED_Out — создавая эффект движения.

```
10      parameter T100MS = 23'd5_000_000;
11      reg [22:0]Count;    //Delay
12      reg [7:0]rLED_Out;
13
14      always @ ( posedge CLK or negedge RSTn )
15      if( !RSTn )
16      begin
17          Count <= 23'd0;
18          rLED_Out <= 8'b0000_0001;
19      end
20      else if( Count == T100MS - 1'b1)
21      begin
22          Count <= 23'd0;
23          if( rLED_Out == 8'b0000_0000 )
24              rLED_Out <= 8'b0000_0001;
25          else
26              rLED_Out <= {rLED_Out[0],rLED_Out[7:1]};
27      end
28      else
29          Count <= Count + 1'b1;
```

Рис. 1.3 — основной код бегущих огней

V. Конфигурация выводов FPGA

На рис. 1.4 показано изображение Pin Planner, сгенерированное Quartus.

Входной сигнал CLK подключен к 50 МГц кварцевому генератору на плате MINI_FPGA.

Выходная шина LED_Out[7:0] подключена к светодиодам LED7...LED0.

Вход RSTn соединён с переключателем SW0.

Когда SW0 установлен в положение DOWN, система выполняет сброс.

Все эти соединения уже были подробно описаны в разделе 1.3 первой части документа.

	Node Name	Direction	Location
in	CLK	Input	PIN_E1
out	LED_Out[7]	Output	PIN_C3
out	LED_Out[6]	Output	PIN_A2
out	LED_Out[5]	Output	PIN_B3
out	LED_Out[4]	Output	PIN_A3
out	LED_Out[3]	Output	PIN_B4
out	LED_Out[2]	Output	PIN_A4
out	LED_Out[1]	Output	PIN_B5
out	LED_Out[0]	Output	PIN_A5
in	RSTn	Input	PIN_R16

Рис. 1.4 — Схема подключения выводов (Pin Planner) для проекта “Бегущие огни”

VI. Результаты эксперимента

Сигнал сброса RSTn подключён к переключателю SW0.

Когда SW0 установлен в положение “DOWN”, происходит сброс системы.

Индикатор у переключателя SW0 на плате гаснет.

LED0 загорается.

LED7–LED1 остаются погашенными.

Феномен показан на рис. 1.5.

Когда SW0 установлен в положение “UP”,

индикатор SW0 на плате загорается,

LED7–LED0 начинают работать в режиме бегущих огней,

последовательное зажигание слева направо.

Рис. 1.6 показывает момент одного из состояний бегущего свечения.

Полную динамику можно наблюдать на практике.

(Рис. 1.5 и 1.6 — изображения с примерами результата.)

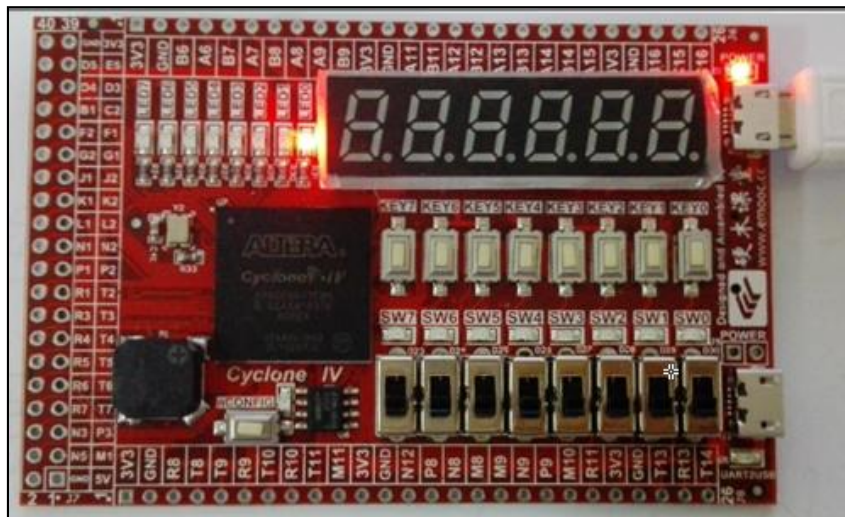


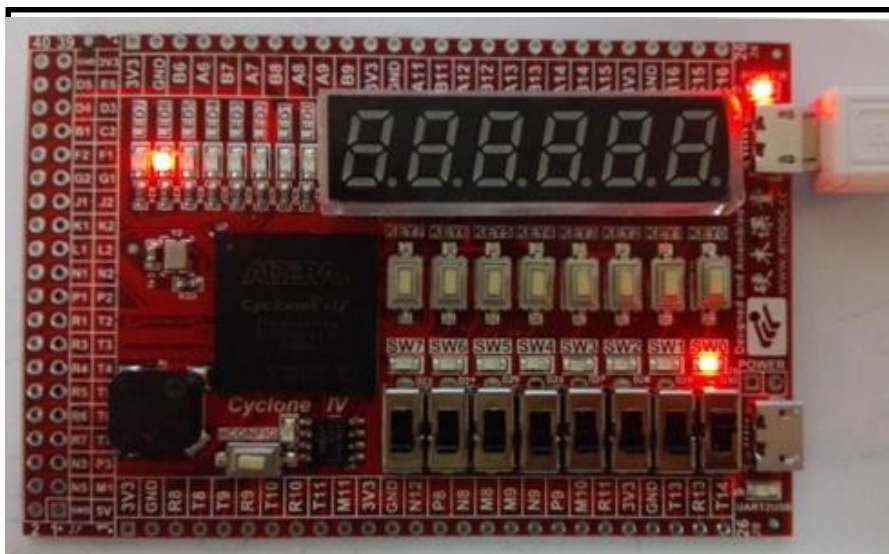
Рис. 1.5 — изображения с примерами результата.

VII. Вопросы и расширение

Какова функция строк 23–24 в коде на рис. 1.3?

Какое преимущество даёт такое проектное решение?

Измените значения параметров в программе, чтобы добиться эффекта: одна смена светодиода каждые 1 секунду.



1.6 — изображения с примерами результата

实验二 集成逻辑门及其基本应用

实验 2.1 实现基本逻辑门功能

一、实验设计目标

在 FPGA 中实现基本逻辑门并验证其功能。

二、实验设计思路

本实验涉及到的基本逻辑门有“与门”、“与非门”、“或门”、“或非门”、“异或门”和“同或门”，功能简单，实验时使用 2 个拨动开关模拟逻辑门的输入信号，通过 LED 灯验证逻辑门的功能。

三、功能模块图与输入输出引脚说明

逻辑门工程包含顶层模块 gate 与底层模块 Gate_module，图 2.1 是使用 Quartus 生成的顶层文件的 BSF 图，图 2.2 是整个工程的模块功能图。下面介绍一下各主要引脚的功能：

(1) SW_In：拨动开关输入，共有两位总线。SW_In[1]和 SW_In[0]分别连接“两输入逻辑门”的输入信号。

(2) LED_Out：输出到 LED 灯，共有六位总线。LED_Out[5:0]分别连接“同或门”、“异或门”、“或非门”、“或门”、“与非门”和“与门”的输出，通过 LED 灯

的点亮或熄灭来表示逻辑门输出的“高”和“低”电平。

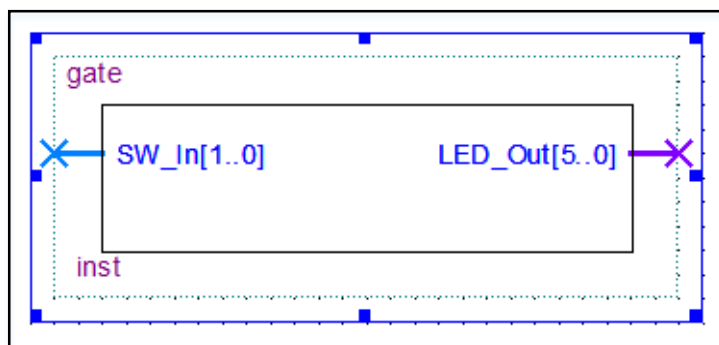


图 2.1 逻辑门顶层文件 BSF 图

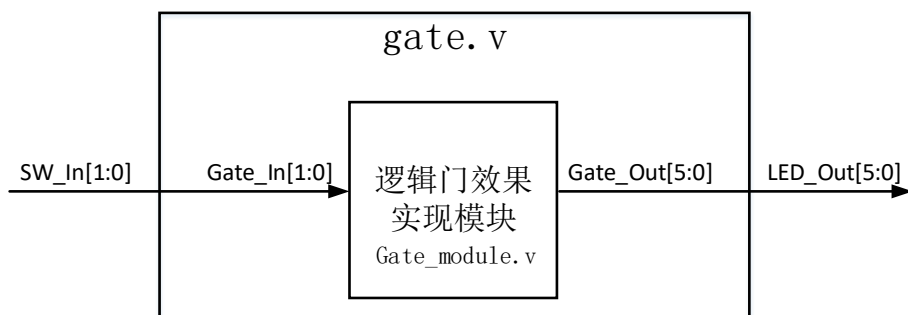


图 2.2 逻辑门模块功能图

四、程序设计

图 2.3 是截取自底层模块 Gate_module 的部分代码：

```

6      input [1:0]Gate_In;
7      output reg [5:0]Gate_Out;
8
9      always @ ( Gate_In[0] or Gate_In[1] )
10     begin
11         Gate_Out[0] = Gate_In[0] & Gate_In[1];
12         Gate_Out[1] = ~ (Gate_In[0] & Gate_In[1]);
13         Gate_Out[2] = Gate_In[0] | Gate_In[1];
14         Gate_Out[3] = ~ (Gate_In[0] | Gate_In[1]);
15         Gate_Out[4] = Gate_In[0] ^ Gate_In[1];
16         Gate_Out[5] = Gate_In[0] ~^ Gate_In[1];
17     end
    
```

图 2.3 逻辑门实验核心代码

6-7：输入输出信号声明。

11-16：这是本次实验的重要程序，11-16 行依次实现“与门”、“与非门”、“或

门”、“或非门”、“异或门”和“同或门”功能。

五、FPGA 管脚配置

图 2.4 是使用 Quartus 生成的 Pin Planner 管脚图，SW_In[1:0]输入信号分别与 MINI_FPGA 开发板上的 SW1 和 SW0 相连；LED_Out[5:0]输出信号分别与开发板上的 LED5~LED0 相连。引脚连接信息已经在第一篇的 1.3 节中做过详细介绍，这里就不再赘述。

六、实验结果

图 2.5 是当 SW1 与 SW0 分别拨至“UP”和“DOWN”位置时的实验现象。当逻辑门两输入分别为“1”和“0”时，“与门”、“与非门”、“或门”、“或非门”、“异

Node Name	Direction	Location
LED_Out[5]	Output	PIN_B3
LED_Out[4]	Output	PIN_A3
LED_Out[3]	Output	PIN_B4
LED_Out[2]	Output	PIN_A4
LED_Out[1]	Output	PIN_B5
LED_Out[0]	Output	PIN_A5
SW_In[1]	Input	PIN_P15
SW_In[0]	Input	PIN_R16

图 2.4 逻辑门实验 Pin Planner

或门”和“同或门”输出分别为“0、1、1、0、1、0”，即 LED_Out[0]~LED_Out[5] 分别为“0、1、1、0、1、0”，则 LED0~LED5 分别为“灭、亮、亮、灭、亮、灭”，如图 2.5 所示。因篇幅有限，其他情况请自行验证。

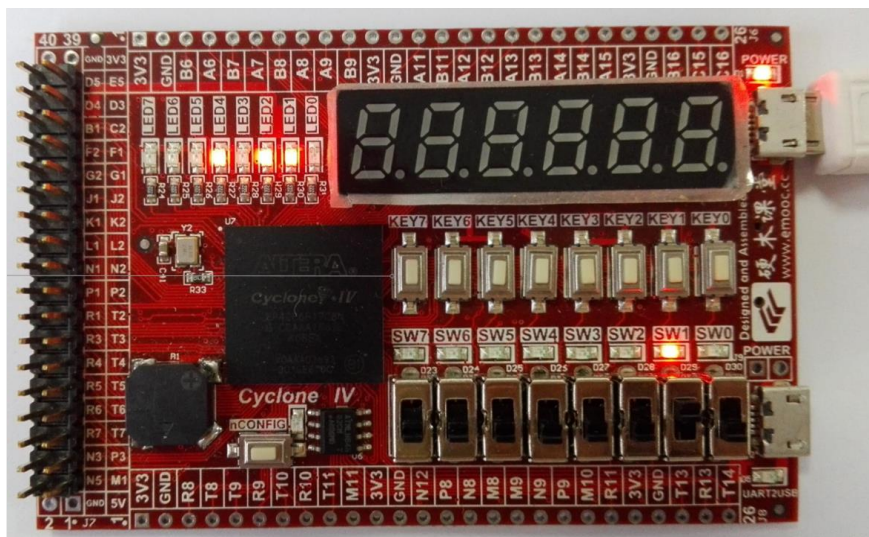


图 2.5 逻辑门实验结果

实验 2.2 利用门电路设计实现全加器功能

一、实验设计目标

- (1) 利用门电路设计实现全加器功能。
- (2) 通过此实验初步掌握连续赋值语句——assign 语句的使用方法。

二、实验设计思路

在电路中，算术运算中的加减乘除运算，往往是分解转化为加法运算，因此，加法器是运算电路的核心。在做二进制的加法时，必须考虑低位向高位的进位。本实验设计实现一个全加器，输入信号包含两加数 a_i 、 b_i ，以及进位输入 c_i ，输出信号包含结果位 s_i 以及进位输出 c_{i+1} 。其中，进位输入 c_i 为低一位加法运算产生的进位，进位输出 c_{i+1} 将作为高一位加法运算的进位输入。表 2.1 给出了对应的真值表。从真值表可以写出 s_i 与 c_{i+1} 的逻辑表达式：

$$s_i = \sim c_i \& \sim a_i \& b_i \mid \sim c_i \& a_i \& \sim b_i \mid c_i \& \sim a_i \& \sim b_i \mid c_i \& a_i \& b_i \quad (2.1)$$

$$c_{i+1} = a_i \& b_i \mid c_i \& \sim a_i \& b_i \mid c_i \& a_i \& \sim b_i \quad (2.2)$$

化简为：

$$s_i = c_i \wedge (a_i \wedge b_i) \quad (2.3)$$

$$c_{i+1} = a_i \& b_i \mid c_i \& (a_i \wedge b_i) \quad (2.4)$$

表 2.1 全加器真值表

c_i	a_i	b_i	s_i	c_{i+1}
-------	-------	-------	-------	-----------

0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

三、功能模块图与输入输出引脚说明

全加器工程包含顶层模块 `adder` 与底层模块 `Adder_module`，图 2.6 是使用 Quartus 生成的顶层文件的 BSF 图，图 2.7 是整个工程的模块功能图。下面介绍一下顶层模块各引脚的功能：

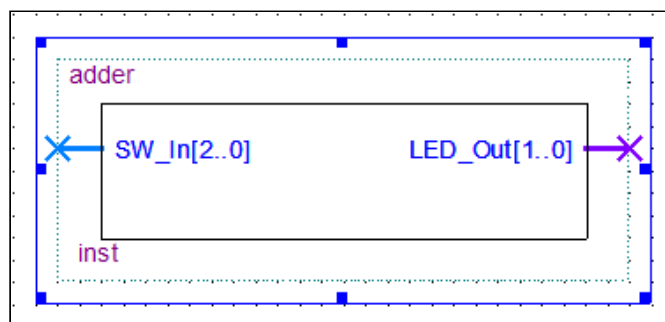


图 2.6 全加器顶层文件 BSF 图

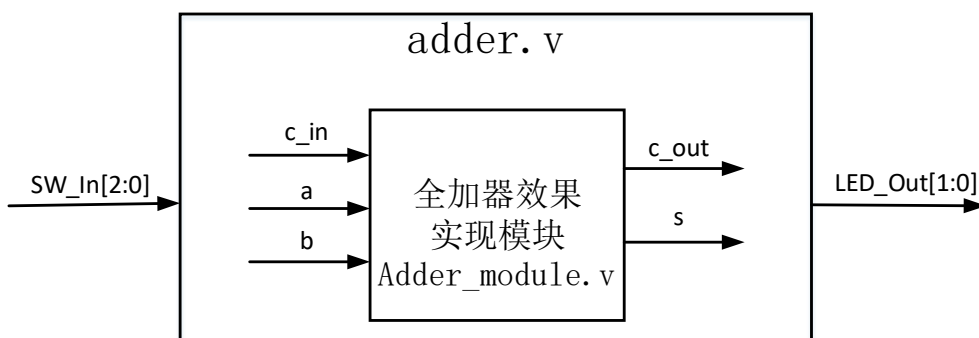


图 2.7 全加器模块功能图

(1) `SW_In`：拨动开关输入，共有三位总线。`SW_In[2]`、`SW_In[1]`和`SW_In[0]`分别连接“全加器”的输入信号 `c_in`、`a` 和 `b`，用于模拟输入信号。

(2) LED_Out: 输出到 LED 灯, 共有两位总线。LED_Out[1]和 LED_Out[0]分别连接“全加器”的输出信号 c_out 和 s, 通过 LED 灯的点亮或熄灭来表示全加器的运算结果。

四、程序设计

图 2.8 是截取自底层模块 Adder_module 的部分代码:

6-10: 信号输入输出端口及类型声明。

12-13: 这是本次实验的重要程序, 用于实现全加器功能, 见公式 (2.3) 和公式 (2.4)。

```

6      input wire a;
7      input wire b;
8      input wire c_in;
9      output wire s;
10     output wire c_out;
11
12     assign s = c_in ^ a ^ b ;
13     assign c_out = (a & b) | (c_in & (a ^ b));
    
```

图 2.8 全加器实验核心代码

五、FPGA 管脚配置

图 2.9 是使用 Quartus 生成的 Pin Planner 管脚图, SW_In[2:0]输入信号分别与 MINI_FPGA 开发板上的 SW2、SW1 和 SW0 相连; LED_Out[1:0]输出信号分别与开发板上的 LED1~LED0 相连。引脚连接信息已经在第一篇的 1.3 节中做过详细介绍, 这里就不再赘述。

	Node Name	Direction	Location
out	LED_Out[1]	Output	PIN_B5
out	LED_Out[0]	Output	PIN_A5
in	SW_In[2]	Input	PIN_P16
in	SW_In[1]	Input	PIN_P15
in	SW_In[0]	Input	PIN_R16

图 2.9 全加器实验 Pin Planner

六、实验结果

图 2.10 是当 SW2、SW1 和 SW0 全部拨至“UP”位置时的实验现象。此时 a_i 、 b_i 、

c_i 均为 1，全加器运算结果为 $s_i = 1$ ， $c_{i+1} = 1$ ，即 LED1 和 LED0 均点亮。因篇幅有限，其他情况请自行验证。

七、思考与拓展

用四个全加器级联可以构成一个四位的加法器，只需将低位全加器的进位输出连接到高位全加器的进位输入即可，请编写程序在 FPGA 中实现四位加法器。

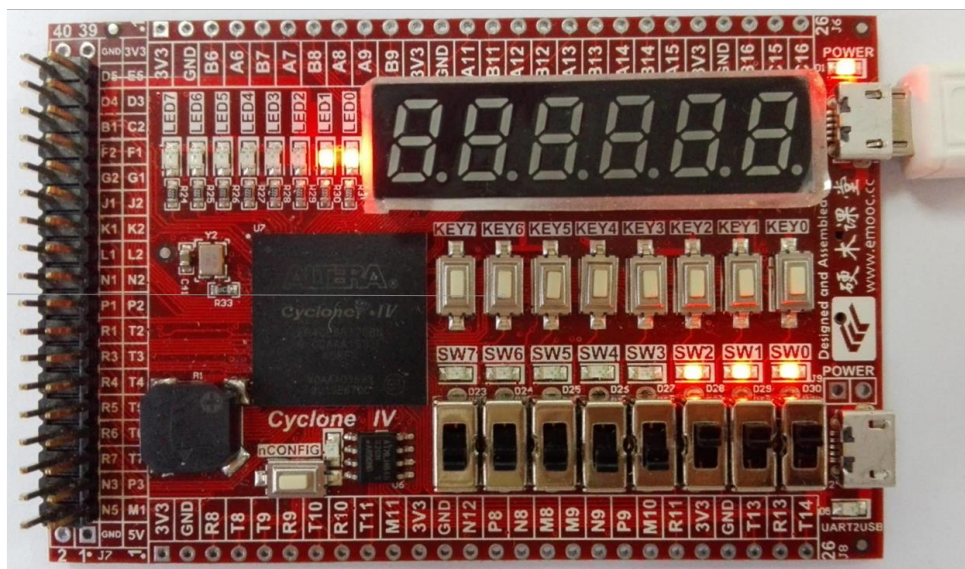


图 2.10 全加器实验结果

实验三 译码器 编码器

实验 3.1 实现 3-8 译码器

一、实验设计目标

在 FPGA 中使用行为描述语句实现 3-8 译码器。

二、实验设计思路

译码器电路有 n 个输入和 2^n 个输出，每个输出都对应着一个可能的二进制输入。本实验设计实现一个 3-8 译码器，表 3.1 给出了该译码器的真值表。从真值表可以写出 $y_7 \sim y_0$ 的逻辑表达式：

$$\begin{aligned}
 y_7 &= a_2 \& a_1 \& a_0 \\
 y_6 &= a_2 \& a_1 \& \sim a_0 \\
 y_5 &= a_2 \& \sim a_1 \& a_0 \\
 y_4 &= a_2 \& \sim a_1 \& \sim a_0 \\
 y_3 &= \sim a_2 \& a_1 \& a_0 \\
 y_2 &= \sim a_2 \& a_1 \& \sim a_0 \\
 y_1 &= \sim a_2 \& \sim a_1 \& a_0 \\
 y_0 &= \sim a_2 \& \sim a_1 \& \sim a_0
 \end{aligned} \tag{3.1}$$

表 3.1 3-8 译码器真值表

a_2	a_1	a_0	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0
0	0	0	0	0	0	0	0	0	0	1

0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

三、功能模块图与输入输出引脚说明

译码器工程包含顶层模块 decode38 与底层模块 decode_module，图 3.1 是使用 Quartus 生成的顶层文件的 BSF 图，图 3.2 是整个工程的模块功能图。下面介绍一下顶层模块各引脚的功能：

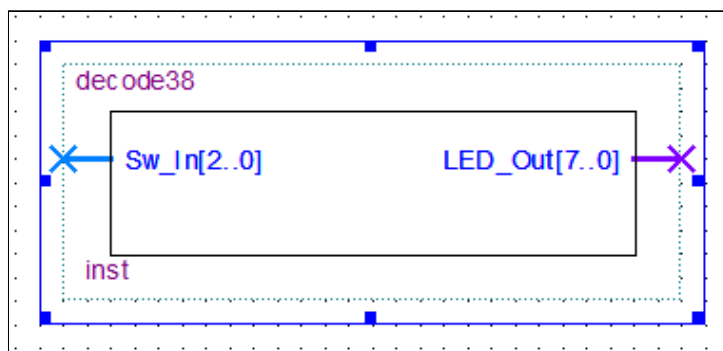


图 3.1 逻辑门顶层文件 BSF 图

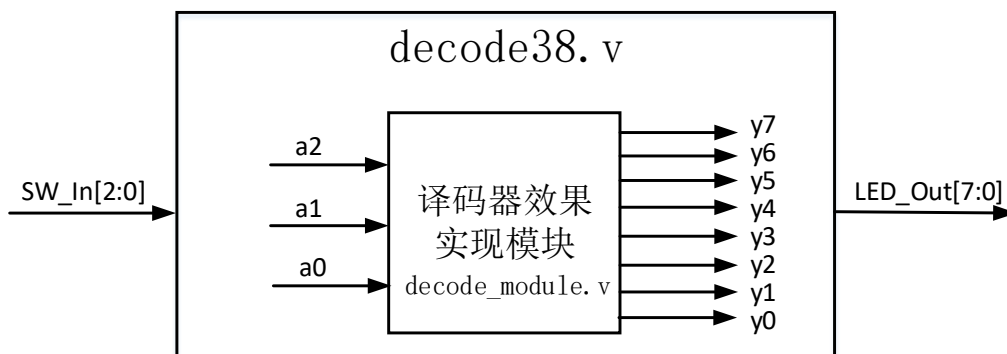


图 3.2 译码器模块功能图

(1) SW_In: 拨动开关输入，共有三位总线。SW_In[2:0] 分别连接 “3-8 译码

器”的输入“a2、a1、a0”。

(2) LED_Out: 输出到 LED 灯, 共有八位总线。LED_Out[7:0]分别连接“3-8 译码器”的输出“y7~y0”, 通过 8 个 LED 灯的亮灭情况来判断二进制输入。

四、程序设计

图 3.3 是截取自底层模块 decode_module 的部分代码:

5-6: 输入输出信号声明。

8-15: 使用连续赋值语句 assign 实现 y7~y0 的逻辑表达式, 见公式 (3.1)。

```

5      input wire a2, a1, a0;
6      output wire y7, y6, y5, y4, y3, y2, y1, y0;
7
8      assign y0 = ~a2 & ~a1 & ~a0;
9      assign y1 = ~a2 & ~a1 & a0;
10     assign y2 = ~a2 & a1 & ~a0;
11     assign y3 = ~a2 & a1 & a0;
12     assign y4 = a2 & ~a1 & ~a0;
13     assign y5 = a2 & ~a1 & a0;
14     assign y6 = a2 & a1 & ~a0;
15     assign y7 = a2 & a1 & a0;

```

图 3.3 逻辑门实验核心代码

五、FPGA 管脚配置

图 3.4 是使用 Quartus 生成的 Pin Planner 管脚图, SW_In[2:0]输入信号分别与 MINI_FPGA 开发板上的 SW2~SW0 相连; LED_Out[7:0]输出信号分别与开发板上的 LED7~LED0 相连。引脚连接信息已经在第一篇的 1.3 节中做过详细介绍, 这里就不再赘述。

Node Name	Direction	Location
 LED_Out[7]	Output	PIN_C3
 LED_Out[6]	Output	PIN_A2
 LED_Out[5]	Output	PIN_B3
 LED_Out[4]	Output	PIN_A3
 LED_Out[3]	Output	PIN_B4
 LED_Out[2]	Output	PIN_A4
 LED_Out[1]	Output	PIN_B5
 LED_Out[0]	Output	PIN_A5
 Sw_In[2]	Input	PIN_P16
 Sw_In[1]	Input	PIN_P15
 Sw_In[0]	Input	PIN_R16

图 3.4 译码器实验 Pin Planner

六、实验结果

图 3.5 是当 SW2、SW1、SW0 分别拨至“UP、DOWN、DOWN”位置时的实验现象。当输入信号“a2、a1、a0”分别为“1、0、0”时，根据表 3.1，输出信号中 y4 为“1”，其余全为“0”。即 LED4 亮，其余全灭，如图 3.5 所示。因篇幅有限，其他情况请自行验证。

七、思考与拓展

自行建立工程，使用 case 语句实现 3—8 译码器。

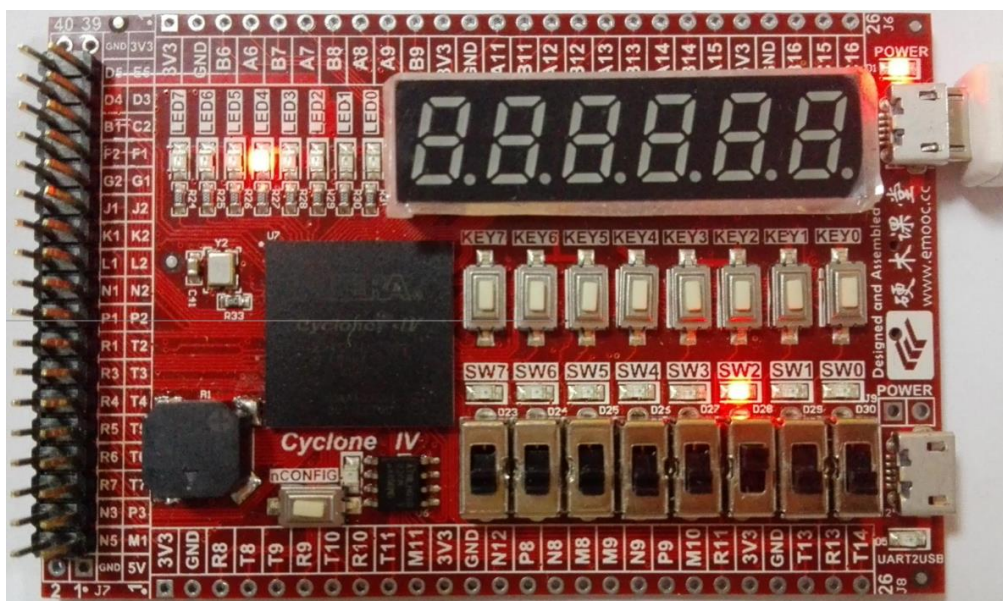


图 3.5 译码器实验结果

实验 3.2 实现 8-3 优先编码器

一、实验设计目标

- (1) 使用 for 循环语句设计实现 8-3 优先编码器。
- (2) 通过此实验初步掌握 for 语句的使用方法。

二、实验设计思路

编码器就是译码器的反向器件，有 2^n 个输入和 n 个输出。编码器常用来告知计算机当前请求中断的外部设备是哪个，当有多个外部中断请求时，计算机将响应优先级高的那个中断。本实验设计实现一个 8-3 优先编码器，假定输入信号中不会出现高阻态或不定值，如果编码器的多个输入同时为高电平，它将选择优先级高的那个输入。表 3.2 给出了对应的真值表。

表 3.2 8-3 优先编码器真值表

x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	y_2	y_1	y_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	X	0	0	1
0	0	0	0	0	1	X	X	0	1	0
0	0	0	0	1	X	X	X	0	1	1
0	0	0	1	X	X	X	X	1	0	0
0	0	1	X	X	X	X	X	1	0	1
0	1	X	X	X	X	X	X	1	1	0
1	X	X	X	X	X	X	X	1	1	1

三、功能模块图与输入输出引脚说明

编码器工程包含顶层模块 pencode83 和底层模块 pencode_module，图 3.6 是使用 Quartus 生成的顶层文件的 BSF 图，图 3.7 是整个工程的模块功能图。下面介绍一下顶层模块各引脚的功能：

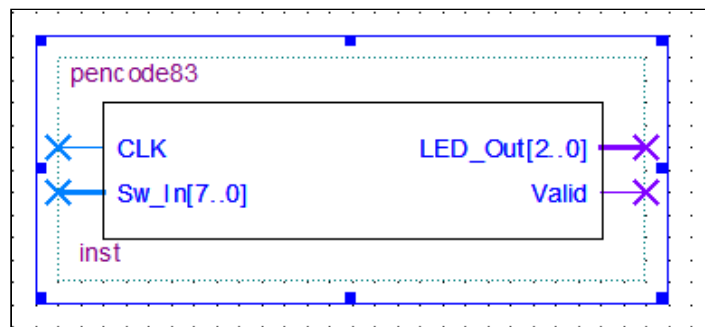


图 3.6 编码器顶层文件 BSF 图

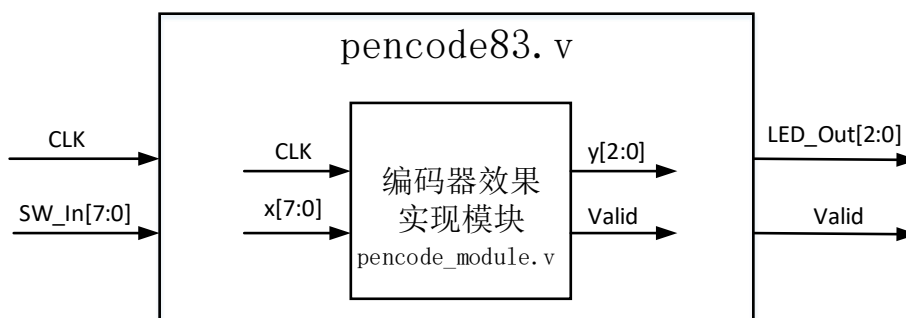


图 3.7 编码器模块功能图

(1) CLK: 50MHz 的系统基准时钟输入。用 CLK 的上升沿作为 always 模块的触发信号。

(2) SW_In: 拨动开关输入，共有八位总线。SW_In[7: 0] 分别连接“编码器”的输入信号 $x_7 \sim x_0$ ，用于模拟二进制输入。

(3) LED_Out: 输出到 LED 灯，共有三位总线。LED_Out[2: 0] 分别连接“编码器”的输出信号 $y_2 \sim y_0$ ，通过 LED 灯的点亮或熄灭来表示编码器的二进制输出。

(4) Valid: 输出有效标志位，用于表明编码器的输出是否有效。只要输入的 8 个元素中有一个为 1，输出 Valid 的值就为 1，否则为 0。该信号输出到 LED 灯，共有一位总线。

四、程序设计

图 3.8 是截取自底层模块 pencode_module 的部分代码：

5-8：输入输出信号声明，always 块的输出信号均必须定义为 reg 型。

14-15：在 always 块中，将 y 和 Valid 的值初始化为“0”，这样它们就总是被赋予一定值的，否则，可能会生成一个锁存器。

16-21：使用 for 循环实现优先编码器功能。因为 for 循环是从 0 到 7 的，判断 x[i] 是否等于 1，这将把最终 i 的值赋给 y，因此，x[7] 具有最高优先级。

```

5      input CLK;
6      input wire [7:0]x;
7      output reg [2:0]y;
8      output reg Valid;
9
10     integer i;
11
12     always @ ( posedge CLK )
13     begin
14         y <= 'b0;
15         Valid <= 'b0;
16         for( i = 0; i <= 7; i = i+1 )
17             if( x[i] == 1 )
18             begin
19                 y <= i;
20                 Valid <= 1;
21             end
22     end

```

图 3.8 编码器实验核心代码

20：当 $x[7:0] \neq 8'b0$, $Valid = 1'b1$ ，此时的编码输出才是在有输入时的有效输出。如果输出 y[2: 0] 为“000”且 Valid 的值为 1，那么就意味着 x[7: 0] 的输入为“0000_0001”；如果输出 y[2: 0] 为“000”且 Valid 的值为 0，那么就意味着 x[7:0] 的输入为“0000_0000”，即没有有效输入。

五、FPGA 管脚配置

图 3.9 是使用 Quartus 生成的 Pin Planner 管脚图，CLK 时钟输入信号与 MINI_FPGA 开发板上的 50MHz 的晶振时钟相连；SW_In[7:0] 输入信号分别与开发板上的 SW7~SW0 相连；LED_Out[2:0] 输出信号分别与 LED2~LED0 相连；Valid 输出信号与 LED3 相连。引脚连接信息已经在第一篇的 1.3 节中做过详细介绍，这里就不再赘述。

Node Name	Direction	Location
in CLK	Input	PIN_E1
out LED_Out[2]	Output	PIN_A4
out LED_Out[1]	Output	PIN_B5
out LED_Out[0]	Output	PIN_A5
in Sw_In[7]	Input	PIN_N13
in Sw_In[6]	Input	PIN_N14
in Sw_In[5]	Input	PIN_M12
in Sw_In[4]	Input	PIN_N16
in Sw_In[3]	Input	PIN_N15
in Sw_In[2]	Input	PIN_P16
in Sw_In[1]	Input	PIN_P15
in Sw_In[0]	Input	PIN_R16
out Valid	Output	PIN_B4

图 3.9 编码器实验 Pin Planner

六、实验结果

如图 3.10 与图 3.11，当 SW7~SW0 均拨至 DOWN 时，LED2~LED0 全灭，且此时 LED3 灯灭，表明无有效输入；当 SW7~SW1 均拨至 DOWN，SW0 拨至 UP 时，LED2~LED0 全灭，且此时 LED3 灯亮，表明存在有效输入 $Sw_In[7:0] = 8'b00000001$ 。

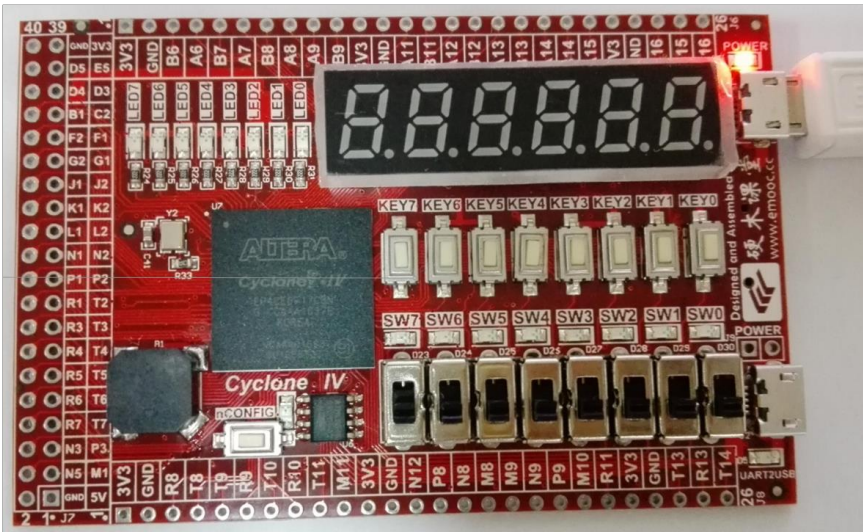


图 3.10 编码器实验结果 1

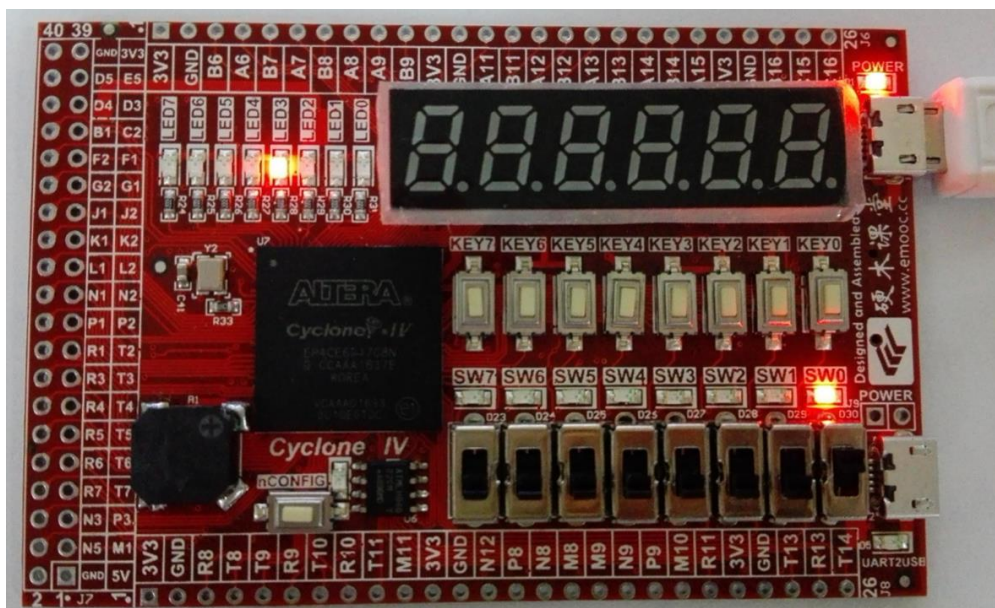


图 3.11 编码器实验结果 2

如图 3.12, 当 SW6, SW2, SW0 均拨至 UP 时, 因为 SW6 具有最高优先级, 编码器输出 $LED_Out[2:0] = 3'b110$, 即 LED2、LED1 亮, LED0 灭, 且此时 LED3 亮。

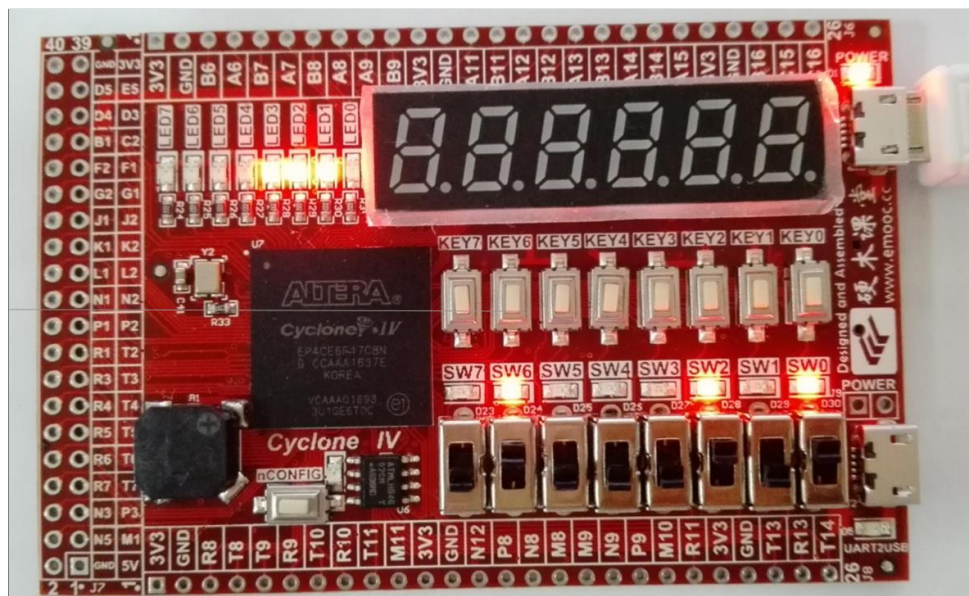


图 3.12 编码器实验结果 3

七、思考与拓展

(1) 使用逻辑方程能更简单的实现不带优先编码功能的 8-3 编码器, 请自行建立工程, 实现其功能。

(2) 请解释图 3.9 中第十行代码的含义; 如果将 i 定义为 reg 型, 预编译能

通过吗？程序功能能实现吗？

八、实验小结

在定义信号类型时，一般有 reg 型与 wire 型。reg 型即寄存器型信号，always 模块的输出信号均必须定义为 reg 型；wire 可以想象为电路内部连线，若不说明信号类型，则默认为 wire 型。

实验四 数据选择器

一、实验设计目标

- (1) 使用 case 语句设计实现自定义数据位宽的 8 选 1 数据选择器。
- (2) 通过此实验初步掌握 case 语句的使用方法。
- (3) 通过此实验初步掌握参数 (parameter) 型常数的定义和使用方法。

二、实验设计思路

数据选择器又称多路转换器或称多路开关，其功能是根据地址码的不同，从多个输入数据流中选择一个送往公共的输出端^[11]。根据数据输入端的个数的不同，可分为 16 选 1、8 选 1、4 选 1 等数据选择器。

本实验设计使用 case 语句实现 8 选 1 数据选择器功能，包含 8 个数据输入端 D7~D0，3 个地址输入端 A2~A0，一个输入使能控制端 CSn 和一个数据输出端 Y。当 CSn 为低电平时允许数据选择器工作；每组输入信号 (D7~D0) 的数据位宽是可自定义的，因按键开关个数限制，在验证实验时将其位宽定为 1 位。表 4.1 给出了对应的真值表。

表 4.1 8 选 1 数据选择器真值表

地址			使能	数据输入								输出
A_2	A_1	A_0	CSn	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	Y
X	X	X	1									0
0	0	0	0									D_0
0	0	1	0									D_1
0	1	0	0									D_2
0	1	1	0									D_3
1	0	0	1									D_4
1	0	1	X									D_5
1	1	0	X									D_6
1	1	1	X									D_7

三、功能模块图与输入输出引脚说明

数据选择器工程包含顶层模块 mux81 与底层模块 mux81_module，图 4.1 是使用 Quartus 生成的顶层文件的 BSF 图，图 4.2 是整个工程的模块功能图。下面介绍一下各主要引脚的功能：

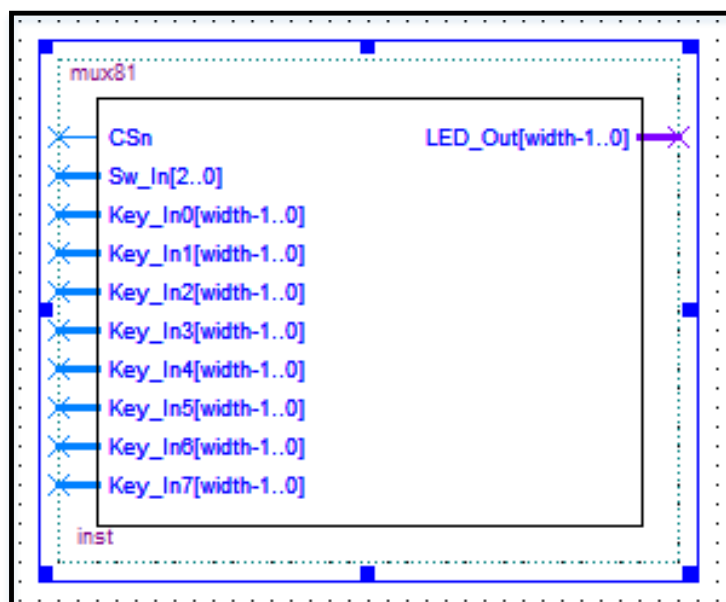


图 4.1 数据选择器顶层文件 BSF 图

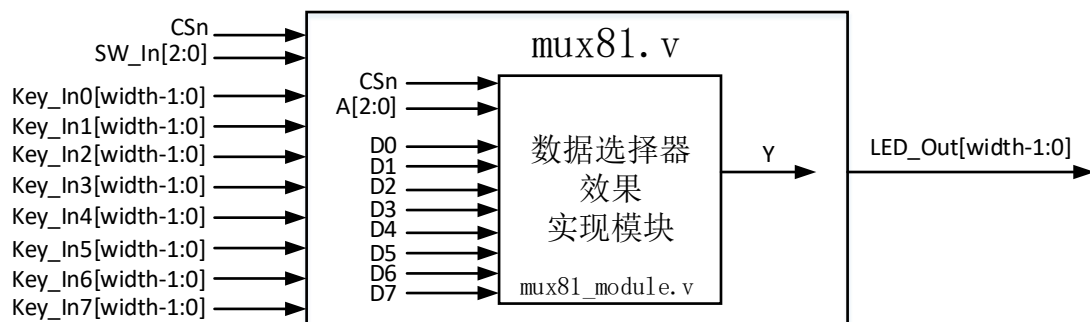


图 4.2 数据选择器模块功能图

(1) Width: 数据位宽，在程序中定义，不属于输入输出信号。实验时，将 width 定为 1，即每组输入信号和输出端信号的位宽都为 1 位。

(2) CSn: 输入使能控制信号，低电平有效。当 CSn=0 时，允许数据选择器工作；当 CSn=1 时，禁止数据选择器工作。

(3) SW_In: 拨动开关输入，共有三位总线。SW_In[2: 0] 分别连接“数据选择器”的地址输入信号 A2~A0，用于模拟地址选择。

(4) Key_In0~Key_In7: 按键开关输入，均只有一位总线。Key_In0~Key_In7 分别连接“数据选择器”的数据输入信号 D0~D7，用于模拟数据输入。

(5) LED_Out: 输出到 LED 灯，共有一位总线。LED_Out 连接“数据选择器”的数据输出端 Y，通过 LED 灯的亮灭情况来显示结果。

四、程序设计

图 4.3 是截取自底层模块 mux81_module 的部分代码：

```

5      input CSn;
6      input [2:0]A;
7      input [width-1:0]D0, D1, D2, D3, D4, D5, D6, D7;
8      output reg [width-1:0]Y;
9
10     parameter width = 1;
11
12     always @ ( * )
13     begin
14         if( !CSn )
15         case( A )
16             3'b000: Y <= D0;
17             3'b001: Y <= D1;
18             3'b010: Y <= D2;
19             3'b011: Y <= D3;
20             3'b100: Y <= D4;
21             3'b101: Y <= D5;
22             3'b110: Y <= D6;
23             3'b111: Y <= D7;
24         endcase
25     else
26         Y <= 0;
27     end

```

图 4.3 数据选择器实验核心代码

5-8: 输入输出信号声明。

10: 定义一个参数型常量 width, 用于表征数据的位宽。此处定义为 1, 即 Y[width-1:0]等同于 Y[0]。

12: 敏感事件程序清单中的*号将自动包含 always 块中的语句或条件表达式中的所有信号, 也可直接写 always @ (A or CSn)。

15-24: 使用 case 语句实现数据选择, 选择方式请参考真值表 4.4。

26: 当使能控制信号 CSn=1 时, 数据选择器不工作, 输出恒为 0。

五、FPGA 管脚配置

图 4.4 是使用 Quartus 生成的 Pin Planner 管脚图, 输入控制使能信号 CSn 与 MINI_FPGA 开发板上 SW3 相连; Key_In0~ Key_In7 分别与开发板上的 KEY0~KEY7 相连; SW_In[2:0] 分别与开发板上的 SW2~SW0 相连; LED_Out[0]直接与开发板上的 LED0 相连。引脚连接信息已经在第一篇的 1.3 节中做过详细介绍, 这里就不再赘述。

	Node Name	Direction	Location
in	CSn	Input	PIN_N15
in	Key_In0[0]	Input	PIN_J14
in	Key_In1[0]	Input	PIN_J16
in	Key_In2[0]	Input	PIN_J15
in	Key_In3[0]	Input	PIN_K16
in	Key_In4[0]	Input	PIN_K15
in	Key_In5[0]	Input	PIN_L15
in	Key_In6[0]	Input	PIN_L16
in	Key_In7[0]	Input	PIN_J13
out	LED_Out[0]	Output	PIN_A5
in	Sw_In[2]	Input	PIN_P16
in	Sw_In[1]	Input	PIN_P15
in	Sw_In[0]	Input	PIN_R16

图 4.4 数据选择器 Pin Planner

六、实验结果

如图 4.5 所示，当拨动开关 SW3 拨至“DOWN”，且 SW2~SW0 均拨至“UP”，选中输入 KEY7 时，LED0 输出 KEY7 的状态，当按下 KEY7 时，LED0 灭，反之则亮。具体实验现象请参照真值表 4.1 自行验证。

七、实验小结

(1) if 语句必须包含在一个 always 块中，always 块中的语句按它们出现的顺序执行。

(2) 用 parameter 来定义一个标识符代表一个常量，称为符号常量，即标识符形式的常量，采用符号常量可提高程序的可读性和可维护性。

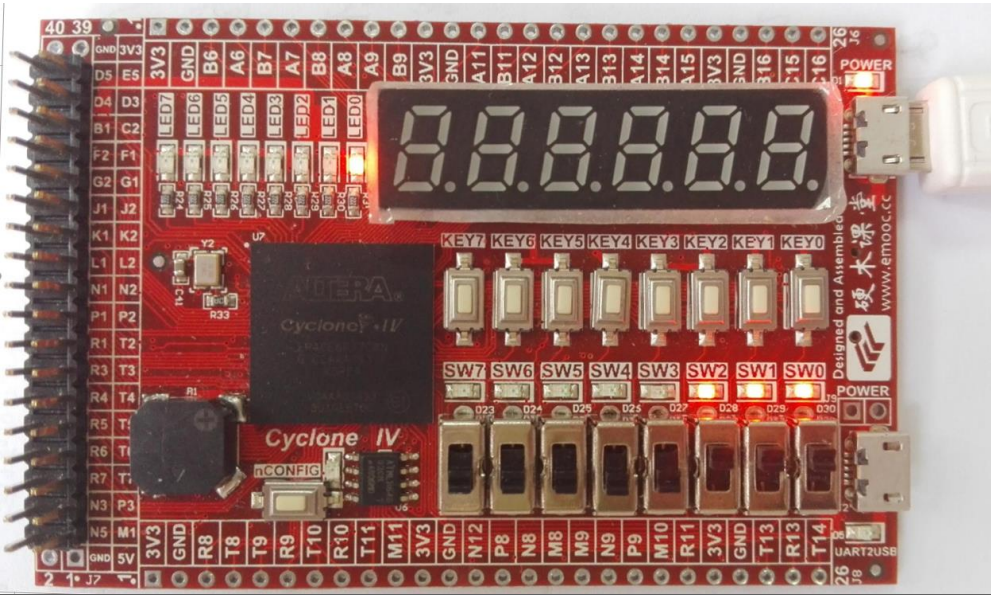


图 4.5 数据选择器实验结果

实验五 触发器

实验 5.1 使用门级结构描述 D 触发器

一、实验设计目标

- (1) 在 FPGA 中使用门级结构描述 D 触发器。
- (3) 通过此实验学习门级结构建模的基本方法。

二、实验设计思路

一个逻辑电路是由许多逻辑门和开关组成的，因此用基本逻辑门的模型来描述逻辑电路结构是最直观的。本实验设计使用结构描述语句实现 D 触发器功能，采用带异步置位和清零端的正边沿触发方式，输入信号包含时钟信号 CLK、置位端 Setn、清零端 Clnr 和一个数据输入 D，输出信号包含数据输出 Q 和 $\sim Q$ 。当 Setn 为低电平时输出 Q 恒为 1；当 Setn 为高电平且 Clnr 为低电平时输出恒为 0；当 Setn 和 Clnr 都为高电平时，输出 Q 在时钟信号 CLK 的上升沿处被赋予输入 D 的值。

图 5.1 是带异步置位和清零端的正边沿触发的 D 触发器的电路结构图，该逻辑电路的行为分析如下：

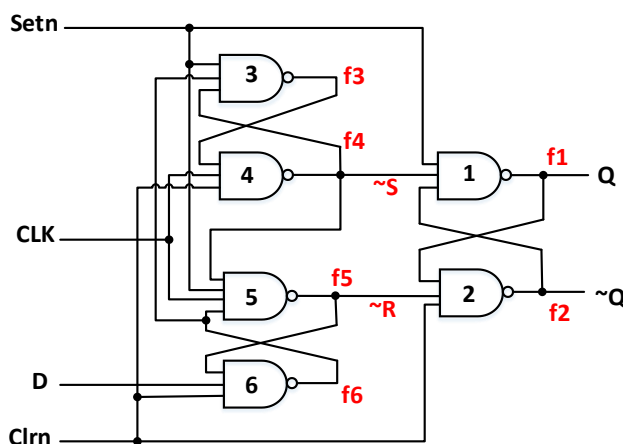


图 5.1 带异步置位和清零端的正边沿触发的 D 触发器

当 Setn 和 Clnr 都为 1 时，可不考虑 Setn 和 Clnr 引脚，此时与非门 1 和与非门 2 构成一个 SR 锁存器。当 $\sim S$ 和 $\sim R$ （即图 5.1 中的反馈信号 f4 和 f5）都为 1 时，该锁存器处于存储状态。假设，时钟信号 CLK 为 0，D 为 1，则信号 f4 和 f5 都为 1，SR 锁存器处于存储状态。同时，信号 f6 将变为 0，f3 将为 1。假设 CLK 变为 1，

这将使 f4 变为 0，输出 Q 被置为 1。如果现在输入 D 变为 0，时钟信号 CLK 仍为 1，则 f6 将变为 1，只要时钟信号 CLK 保持为 1，f3 也将保持为 1。则 f4 仍为 0，输出 Q 保持为 1 不变。而当时钟信号变为 0 时，f4 和 f5 都将变为 1，SR 锁存器再次处于存储状态，输出 Q 保持为不变。也就是说输出 Q 只在时钟信号 CLK 的上升沿被置为输入 D 的值。其他情况也可类似讨论。

当置位信号 Setn（清零信号 Cln）为 0 时，输出 Q 立即变为 1(0)，而不用等到下一个时钟上升沿的到来，此即为异步置位和清零的特点。并且对于输出 Q 来说，Setn 的优先级高于 Cln。

三、功能模块图与输入输出引脚说明

该工程包含顶层模块 triggerD1 与底层模块 trigger_module，图 5.2 是使用 Quartus 生成的顶层文件的 BSF 图，图 5.3 是整个工程的模块功能图。本实验仅验证了输出 Q 而未验证输出 $\sim Q$ ，下面介绍一下顶层模块各引脚的功能：

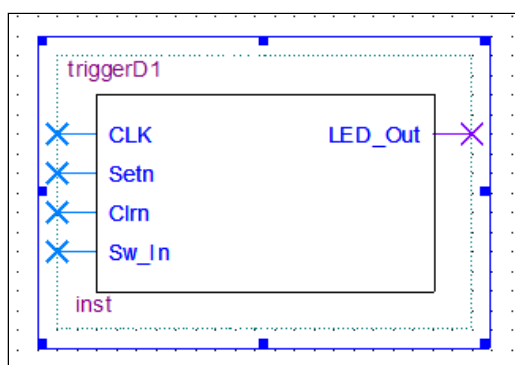


图 5.2 D 触发器顶层文件 BSF 图

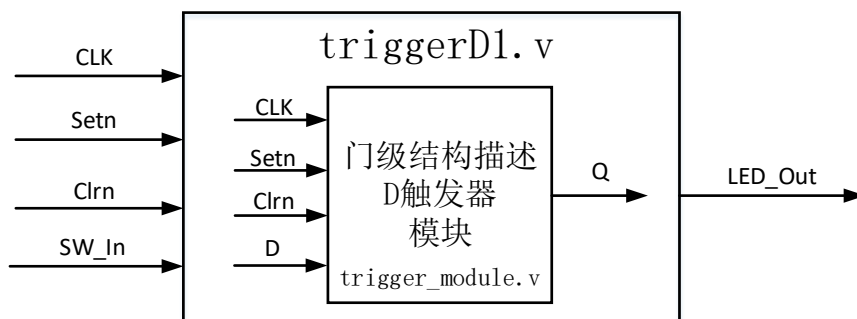


图 5.3 D 触发器模块功能图

- (1) CLK: 50MHz 的时钟信号输入。用 CLK 的上升沿作为 D 触发器的触发信号。
- (2) Setn: 置位输入信号。当 Setn 为低电平时输出 Q 恒为 1。

(3) Clrn: 清零输入信号。当 Setn 为高电平且 Clrn 为低电平时输出 Q 恒为 0。对输出 Q 来说, Setn 信号的优先级高于 Clrn 信号。

(4) SW_In: 拨动开关输入, 共有一位总线。SW_In 直接连接 “D 触发器” 的输入 “D”, 用于模拟输入信号。

(5) LED_Out: 输出到 LED 灯, 共有一位总线。LED_Out 直接连接 “D 触发器” 的输出 “Q”, 通过 LED 灯的亮灭情况来显示触发器的输出 Q。

四、程序设计

图 5.4 是截取自底层模块 trigger_module 的部分代码:

```

5      input wire CLK;
6      input wire Setn;
7      input wire Clrn;
8      input wire D;
9      output wire Q;
10
11     nand U1 ( f1, Setn, f4, f2 ),
12           U2 ( f2, f1, f5, Clrn ),
13           U3 ( f3, Setn, f6, f4 ),
14           U4 ( f4, f3, CLK, Clrn ),
15           U5 ( f5, f4, Setn, CLK, f6 ),
16           U6 ( f6, f5, D, Clrn );
17
18     assign Q = f1;

```

图 5.4 D 触发器实验核心代码

5-9: 输入输出信号声明。

11-21: 使用门级结构描述 D 触发器的电路结构图 (图 5.1)。门声明语句的格式为:

<门的类型>[<驱动能力><延时>]<门实例 1>[, <门实例 2>, ..., <门实例 n>];

门的类型是门声明语句必须的; 驱动能力和延时是可选项; 门实例 1 是在本模块中所引用的第一个这种类型的门, 而门实例 n 是引用的第 n 个这种类型的门, 且在结束时使用逗号, 最后才用分号。

在本例中, 代码第 11 行使用了一个名为 U1 的与非门, 对应图 5.1 中的与非门 1, 输入为 Setn、f4 和 f2, 输出为 f1。输出与输入无延时。

五、FPGA 管脚配置

图 5.5 是使用 Quartus 生成的 Pin Planner 管脚图, CLK 时钟输入信号与

MINI_FPGA 开发板上的 50MHz 的晶振时钟相连；置位输入信号 Setn 与开发板上的 KEY0 相连；清零输入信号 Clrn 与开发板上的 KEY1 相连；LED_Out 输出信号与 LED0 相连；SW_In 输入信号与开发板上的 SW0 相连。引脚连接信息已经在第一篇的 1.3 节中做过详细介绍，这里就不再赘述。

Node Name	Direction	Location
in CLK	Input	PIN_E1
in Clrn	Input	PIN_J16
out LED_Out[1]	Output	PIN_B5
out LED_Out[0]	Output	PIN_A5
in Setn	Input	PIN_J14
in Sw_In	Input	PIN_R16

图 5.5 D 触发器实验 Pin Planner

六、实验结果

图 5.6 是当 KEY0，KEY1 均不按下的实验现象。

当按键开关 KEY0 按下时，LED0 点亮；当按键开关 KEY0 不按下，KEY1 按下时，LED0 熄灭；当按键开关 KEY0，KEY1 均不按下时，在每个时钟的上升沿，LED0 输出 SW0 的状态，SW0 拨至 DOWN 时，LED0 熄灭，反之点亮。因篇幅有限，置位与清零功能请自行验证。

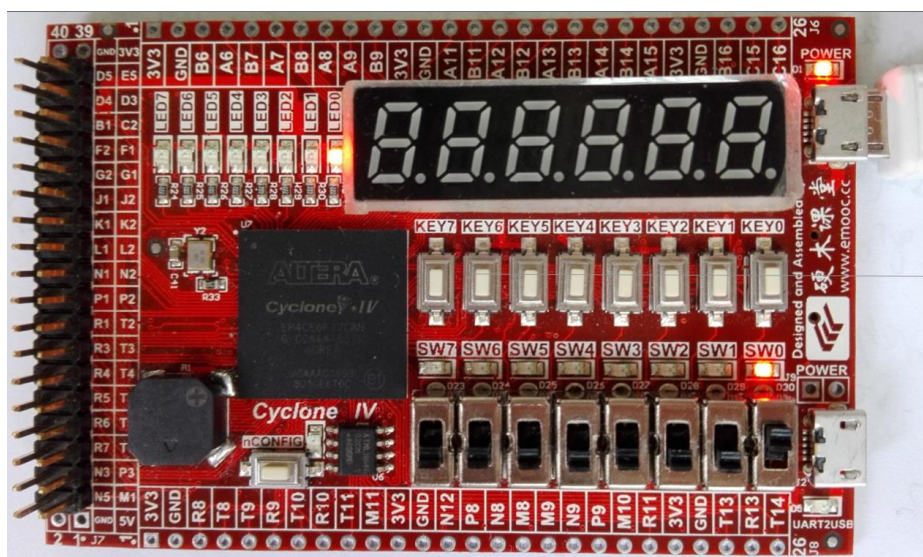


图 5.6 D 触发器实验结果

七、思考与拓展

(1) 本实验仅验证了输出 Q 而未验证输出 $\sim Q$ 。对于输出 Q 来说, $Setn$ 信号的优先级高于 Clr_n 信号, 那么对于输出 $\sim Q$ 又是怎样的呢? 请适当修改程序同时观察输出 Q 和 $\sim Q$, 并结合图 5.1 解释实验现象。

八、实验小结

Verilog 既可以是一种行为描述的语言也可以是一种结构描述的语言。Verilog 模型可以是实际电路的不同级别的抽象, 这些抽象的级别包括: 系统级、算法级、RTL (Register Transfer Level) 级、门级和开关级。前三种都属于行为描述, 后两种属于结构描述, RTL 级是描述数据在寄存器之前流动和如何处理、控制这些数据流动的模型, 门级是描述逻辑门及逻辑门之间的连接的模型。本实验使用门级结构描述 D 触发器, 通过此实验可以学习门级结构建模的基本方法, 在下一节实验中将使用行为描述语句实现 8D 触发器功能。

实验 5.2 使用行为描述语句实现 8D 触发器

一、实验设计目标

(1) 在 FPGA 中使用行为描述语句实现 8D 触发器功能。

(3) 结合实验 5.1 与实验 5.1, 对比理解行为描述语句和结构描述语句的不同及各自的优点。

二、实验设计思路

输出在时钟信号某个特定时刻随输入改变的器件称为触发器。本实验设计实现一个带异步置位与清零端的正边沿触发的 8D 触发器, 除包含 8 个数据输入端 $D_7 \sim D_0$ 、和 8 个数据输出端 $Q_7 \sim Q_0$ 外, 其他功能均与实验 5.1 相同, 此处就不再赘述。

表 5.1 是第 i 组输入输出信号的真值表, 从真值表可以写出 Q_i 的逻辑表达式:

$$Q_i = D_i \bullet Setn \bullet Clrn + \overline{Setn} \quad (0 \leq i \leq 7, i \in N) \quad (5.1)$$

表 5.1 8D 触发器真值表

置位	清零	输入		输出
$Setn$	$Clrn$	CLK	D_i	Q_i
0	X	X	X	1
1	0	X	X	0
1	1	↑	0	0
1	1	↑	1	1

三、功能模块图与输入输出引脚说明

该工程包含顶层模块 triggerD2 与底层模块 trigger_module, 图 5.7 是使用 Quartus 生成的顶层文件的 BSF 图, 图 5.8 是整个工程的模块功能图。下面介绍一下顶层模块各引脚的功能:

(1) CLK 信号、Setn 信号和 Clrn 信号的功能均与实验 5.1 中 D 触发器相同,

此处不再赘述。

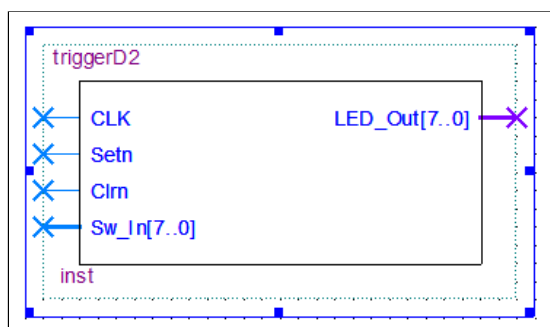


图 5.7 8D 触发器顶层文件 BSF 图

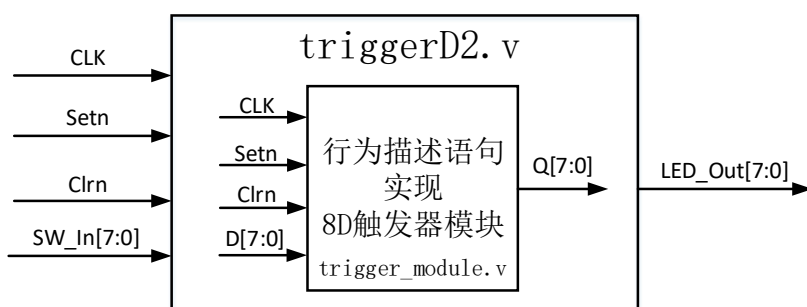


图 5.8 8D 触发器模块功能图

(2) SW_In: 拨动开关输入，共有八位总线。SW_In[7:0]分别连接“8D 触发器”的数据输入“D7~D0”，用于模拟输入信号。

(5) LED_Out: 输出到 LED 灯，共有八位总线。LED_Out[7:0]分别连接“8D 触发器”的输出“Q7~Q0”，通过 8 个 LED 灯的亮灭情况来显示触发器的输出结果。

四、程序设计

图 5.9 是截取自底层模块 trigger_module 的部分代码：

```

5      input wire CLK;
6      input wire Setn;
7      input wire Clrn;
8      input wire [7:0]D;
9      output reg [7:0]Q;
10
11     always @(posedge CLK or negedge Setn or negedge Clrn)
12     begin
13         Q[0] <= ( D[0] & Setn & Clrn ) | ~Setn;
14         Q[1] <= ( D[1] & Setn & Clrn ) | ~Setn;
15         Q[2] <= ( D[2] & Setn & Clrn ) | ~Setn;
16         Q[3] <= ( D[3] & Setn & Clrn ) | ~Setn;
17         Q[4] <= ( D[4] & Setn & Clrn ) | ~Setn;
18         Q[5] <= ( D[5] & Setn & Clrn ) | ~Setn;
19         Q[6] <= ( D[6] & Setn & Clrn ) | ~Setn;
20         Q[7] <= ( D[7] & Setn & Clrn ) | ~Setn;
21     end
    
```

图 5.9 8D 触发器实验核心代码

5-9: 输入输出信号声明。

11-21: 使用行为描述语句实现带异步置位和清零端的 8D 触发器的逻辑表达式, 见公式 (5.1)。Setn 信号的优先级高于 Clnr 信号。

五、FPGA 管脚配置

图 5.10 是使用 Quartus 生成的 Pin Planner 管脚图, CLK 信号、Setn 信号和 Clnr 信号的配置方式均与实验 5.1 中相应信号的配置方式相同; LED_Out[7:0] 输出信号分别与开发板上的 LED7~LED0 相连; SW_In[7:0] 输入信号分别与 SW7~SW0 相连。引脚连接信息已经在第一篇的 1.3 节中做过详细介绍, 这里就不再赘述。

Node Name	Direction	Location
in CLK	Input	PIN_E1
in Clnr	Input	PIN_J16
out LED_Out[7]	Output	PIN_C3
out LED_Out[6]	Output	PIN_A2
out LED_Out[5]	Output	PIN_B3
out LED_Out[4]	Output	PIN_A3
out LED_Out[3]	Output	PIN_B4
out LED_Out[2]	Output	PIN_A4
out LED_Out[1]	Output	PIN_B5
out LED_Out[0]	Output	PIN_A5
in Setn	Input	PIN_J14
in Sw_In[7]	Input	PIN_N13
in Sw_In[6]	Input	PIN_N14
in Sw_In[5]	Input	PIN_M12
in Sw_In[4]	Input	PIN_N16
in Sw_In[3]	Input	PIN_N15
in Sw_In[2]	Input	PIN_P16
in Sw_In[1]	Input	PIN_P15
in Sw_In[0]	Input	PIN_R16

图 5.10 8D 触发器实验 Pin Planner

六、实验结果

当按键开关 KEY0 按下时, LED7~LED0 均点亮; 当按键开关 KEY0 不按, KEY1 按下时, LED7~LED0 均熄灭; 当按键开关 KEY0, KEY1 均不按下时, 在每个时钟的上升沿, LED7~LED0 分别输出 SW7~SW0 的状态, SW_i 拨至 DOWN 时, LED_i 熄灭, 反之点亮。图 5.11 是当 KEY0, KEY1 均不按下时的实验现象。因篇幅有限, 置位与清零功能请自行验证。

七、思考与拓展

图 5.12 是使用 if 语句直接描述 D 触发器功能的实验代码，它不同于实验 5.1

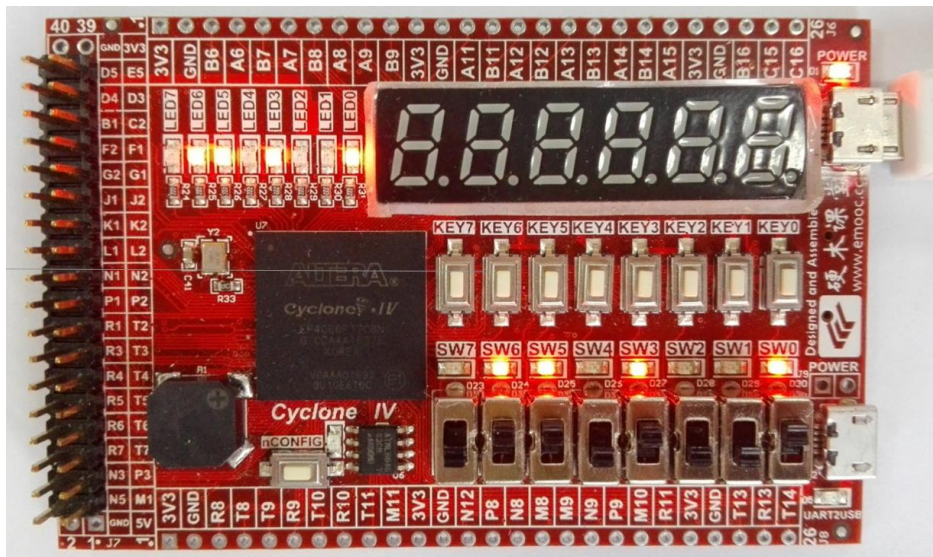


图 5.11 8D 触发器实验结果

节从触发器的门级结构出发，也不同于实验 5.3 节从触发器的真值表出发，实际上在设计更为复杂的 FPGA 系统时，往往结合这几种方式，择优处理。请参考图 5.12 所示代码自行建立工程实现 8D 触发器功能。

```

11      always @ ( posedge CLK or negedge Clrn or negedge Setn )
12      begin
13          if( Setn == 0 )
14              Q <= 8'b11111111;
15          else if( Clrn == 0 )
16              Q <= 8'b0;
17          else
18              Q <= D;
19      end
    
```

图 5.12 if 语句实现 8D 触发器参考代码

八、实验小结

(1) 在本实验中，我们使用了非阻塞语句运算符“<=”代替阻塞语句运算符“=”。当使用阻塞运算符“=”时，赋值语句立即就把当前值赋给变量；但是，当使用非阻塞运算符“<=”时，赋值语句要等到 always 块结束后，才完成对变量的赋值操作。

(2) 行为描述语句可描述顺序执行或并行执行的程序结构, 本节实验使用行为描述语言实现 8D 触发器功能, 相较于实验 5.1, 程序更加直观、简洁, 请结合实验 5.1 理解它们各自的特点和优势。

实验 5.3 实现 4JK 触发器

一、实验设计目标

- (1) 设计实现一个 4 输入的带置位和清零端的正边沿触发的 JK 触发器。
- (2) 通过此实验学习利用编程得到任意频率的时钟信号的基本方法。

二、实验设计思路

类似于实验 5.2 节, 本节实验设计实现一个带置位和清零端的正边沿触发的 JK 触发器。包含时钟信号 CLK1, 置位端 Setn, 清零端 Clrn, 四组数据输入信号 J3~J0、K3~K0 和四组数据输出信号 Q3~Q0。当 Setn 和 Clrn 都为高电平时, 输出 Q 在时钟信号 CLK1 的上升沿处随输入 J、K 的值变化, 在 J、K 端均为 1 时, 每遇到一个时钟的上升沿, 输出端的状态翻转一次。

表 5.2 是第 i 组输入输出信号的真值表, Q_{n+1} 表示该组输出的当前值, Q_n 表示该组输出上一刻的状态, 从真值表可以写出 Q_{n+1} 的逻辑表达式:

$$Q_{n+1} = (J_i \cdot \overline{Q_n} + \overline{K_i} \cdot Q_n) \cdot \text{Setn} \cdot \text{Clrn} + \overline{\text{Setn}} \quad (0 \leq i \leq 3, i \in N) \quad (5.2)$$

表 5.2 带置位与清零端的 JK 触发器真值表

置位	清零	输入			输出	
Setn	Clrn	CLK1	J_i	K_i	Q_{n+1}	$\overline{Q_{n+1}}$
0	X	X	X	X	1	0
1	0	X	X	X	0	1
1	1	↑	0	0	Q_n (保持)	$\overline{Q_n}$ (保持)
1	1	↑	0	1	0	1
1	1	↑	1	0	1	0
1	1	↑	1	1	$\overline{Q_n}$ (翻转)	Q_n (翻转)

三、功能模块图与输入输出引脚说明

JK 触发器工程包含顶层模块 triggerJK1 和底层模块 trigger_module, 图 5.13 是使用 Quartus 生成的顶层文件的 BSF 图, 图 5.14 是整个工程的模块功能图。下面介绍一下顶层模块各引脚的功能:

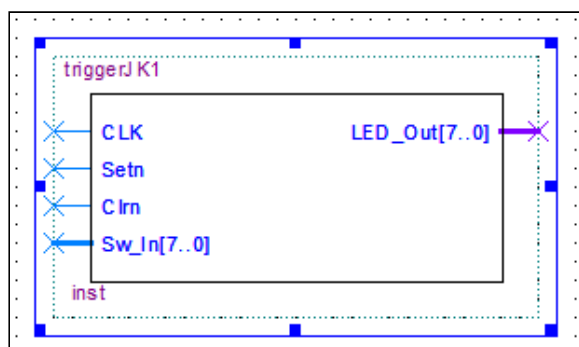


图 5.13 JK 触发器顶层文件 BSF 图

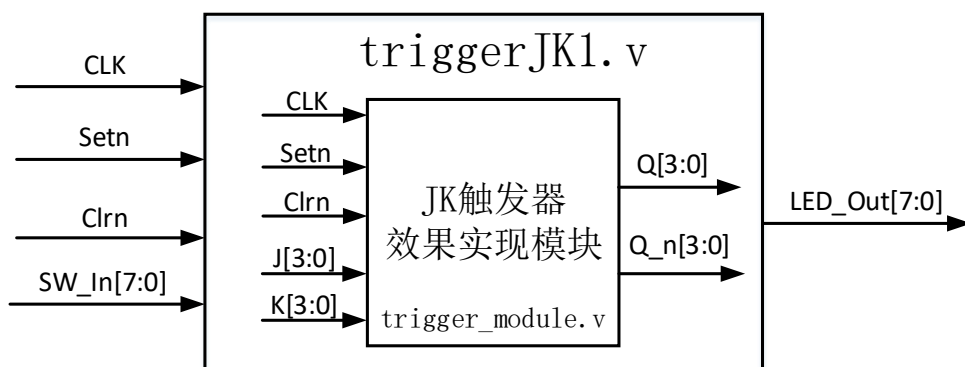


图 5.14 JK 触发器模块功能图

(1) CLK: 50MHz 的系统基准时钟输入。为了能观察到 JK 触发器的翻转效果, 必须降低时钟信号 CLK1 的频率, 将 CLK 适当分频可得到频率为 12.5HZ 的时钟信号 CLK1, 用 CLK1 的上升沿作为 JK 触发器的触发信号。

(2) Setn: 置位输入信号。当 Setn 为低电平时 JK 触发器输出 Q 恒为 1。

(3) Clrn: 清零输入信号。当 Setn 为高电平且 Clrn 为低电平时输出 Q 恒为 0。且 Setn 信号的优先级高于 Clrn 信号。

(4) SW_In: 拨动开关输入, 共有八位总线。SW_In[7:4] 分别连接 “JK 触发器” 的输入 J3~J0, SW_In[3:0] 分别连接 “JK 触发器” 的输入 K3~K0, 用于模拟输

入信号。

(5) LED_Out: 输出到 LED 灯, 共有八位总线。LED_Out[7:4] 分别连接 “JK 触发器” 的输出 $Q_3 \sim Q_0$, LED_Out[3:0] 分别连接 “JK 触发器” 的输出 $\sim Q_n0$ (对应于表 5.2 中的 Q_{n+1}), 通过 LED 灯的亮灭情况来观察触发器的输出效果。

四、程序设计

图 5.15 是截取自底层模块 pencode_module 的部分代码:

```

13      reg CLK1 = 0;
14      reg [20:0]Count;
15      parameter Timex = 21'd2000_000;
16
17      always @ ( posedge CLK )
18      begin
19          if( Count == Timex - 1'b1 )
20          begin
21              Count <= 0;
22              CLK1 <= ~CLK1;
23          end
24          else
25              Count <= Count + 21'b1;
26      end
27
28      always @ ( posedge CLK1 )
29      begin
30          Q[0] <= ( ( J[0] & Q_n[0] ) | ( ~K[0] & Q[0] ) ) & Setn & Clrn ) | ~Setn;
31          Q[1] <= ( ( J[1] & Q_n[1] ) | ( ~K[1] & Q[1] ) ) & Setn & Clrn ) | ~Setn;
32          Q[2] <= ( ( J[2] & Q_n[2] ) | ( ~K[2] & Q[2] ) ) & Setn & Clrn ) | ~Setn;
33          Q[3] <= ( ( J[3] & Q_n[3] ) | ( ~K[3] & Q[3] ) ) & Setn & Clrn ) | ~Setn;
34          Q_n[0] <= ~Q[0];
35          Q_n[1] <= ~Q[1];
36          Q_n[2] <= ~Q[2];
37          Q_n[3] <= ~Q[3];
38      end
    
```

图 5.15 JK 触发器实验核心代码

13-26: 一个分频程序的基本写法, 用于产生频率为 12.5Hz 的时钟信号 CLK1。
13-15 定义了该分频程序需用到的内部信号及参数; 19 行代码中的 “-1” 用于保证所得到的时钟信号 CLK1 的频率的准确性, 非常重要; 需要注意的是常量的书写格式; 时钟信号 CLK1 的频率计算公式为

$$\begin{aligned}
 f_{CLK1} &= f_{CLK} \div Timex \div 2 \\
 &= 50000000 \div 2000000 \div 2 = 12.5(Hz)
 \end{aligned}
 \tag{5.3}$$

28-38: 实现带异步置位和清零端的 JK 触发器的逻辑表达式, 见公式 (5.2)。
Setn 信号的优先级高于 Clrn 信号。

五、FPGA 管脚配置

图 5.16 是使用 Quartus 生成的 Pin Planner 管脚图, CLK 信号、Setn 信号、Clrln 信号、SW_In[7:0] 输入信号和 LED_Out[7:0] 输出信号的配置方式均与实验 5.2 中相应信号的配置方式相同, 但它们所行使的功能是不同的。引脚连接信息已经在第一篇的 1.3 节中做过详细介绍, 这里就不再赘述。

Node Name	Direction	Location
in CLK	Input	PIN_E1
in Clrn	Input	PIN_J16
out LED_Out[7]	Output	PIN_C3
out LED_Out[6]	Output	PIN_A2
out LED_Out[5]	Output	PIN_B3
out LED_Out[4]	Output	PIN_A3
out LED_Out[3]	Output	PIN_B4
out LED_Out[2]	Output	PIN_A4
out LED_Out[1]	Output	PIN_B5
out LED_Out[0]	Output	PIN_A5
in Setn	Input	PIN_J14
in Sw_In[7]	Input	PIN_N13
in Sw_In[6]	Input	PIN_N14
in Sw_In[5]	Input	PIN_M12
in Sw_In[4]	Input	PIN_N16
in Sw_In[3]	Input	PIN_N15
in Sw_In[2]	Input	PIN_P16
in Sw_In[1]	Input	PIN_P15
in Sw_In[0]	Input	PIN_R16

图 5.16 JK 触发器实验 Pin Planner

六、实验结果

本实验设计实现包含 4 组输入输出信号的 JK 触发器, 在验证实验时仅需挑选其中一组进行观察。如挑选第四组输入输出信号, SW7 和 SW3 分别连接 J3、K3 输入信号, LED7 和 LED3 分别连接 Q3、Q_n3 输出信号。当 SW7 和 SW3 均拨至“DOWN”时, 输出处于“保持”状态, LED7 和 LED3 的状态不改变; 当 SW7 和 SW3 分别拨至“DOWN、UP”时, 输出 Q 为 0, LED7 灭, LED3 亮; 当 SW7 和 SW3 分别拨至“UP、DOWN”时, 输出 Q 为 1, LED7 亮, LED3 灭; 当 SW7 和 SW3 均拨至“UP”时, 输出处于“翻转”状态, 每遇到一个时钟信号 CLK1 的上升沿, LED7 和 LED3 的状态翻转一次。

图 5.17 是当 SW7 和 SW3 分别拨至“UP、DOWN”时的实验现象 (仅考虑第四组信号), 因篇幅受限, 其他情况请自行验证。

七、思考与拓展

(1) 在观察现象时，发现实验结果不太理想，LED 灯的状态变换情况存在一定的延时，请问这是什么原因造成的？

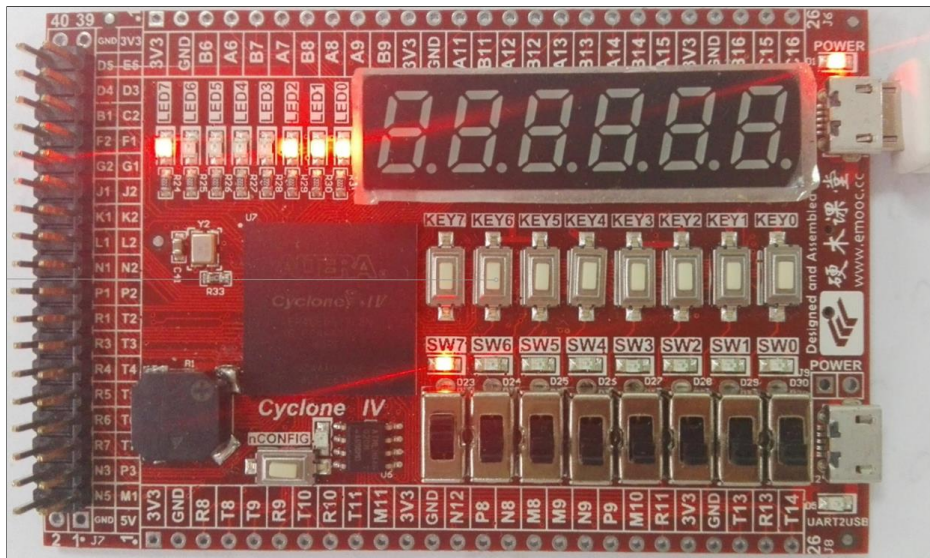


图 5.17 JK 触发器实验结果

(2) 为解决 (1) 中的延时问题，令 trigger_module.v 文件中的参数 Timex = 21'd200_000，当 SW7 和 SW3 均拨至 “UP” 时，LED7 和 LED3 快速闪烁。

(3) 参考实验 5.1 中 D 触发器实验思路，使用结构描述的方式实现带置位和清零端的 JK 触发器功能。

实验六 加法计数器

一、实验设计目标

- (1) 在 FPGA 中设计实现 24 进制加法计数器，并用数码管显示其结果。
- (3) 通过此实验学习 MINI_FPGA 开发板上的数码管的使用方法。

二、实验设计思路

本实验设计实现一个 24 进制的加法计数器，它由晶体振荡器、分频器、计数器和数码管显示器组成，图 6.1 是该加法计数器的示意图。

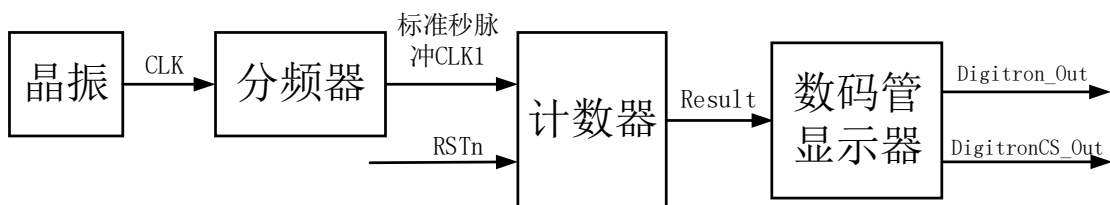


图 6.1 24 进制加法计数器示意图

晶体振荡器产生稳定的 50MHz 的脉冲信号 CLK，经过分频器后输出标准秒脉冲 CLK1，作为计数器的计数时钟。计数器按照“00-01-02...22-23-00-01”的规律计数，每增加 1 秒，计数器加 1，信号 Result[7:4] 代表计数器输出结果的十位，Result[3:0] 代表个位，RSTn 为复位输入信号。将计数器的结果 Result 送给数码管显示器显示，当 MINI_FPGA 开发板正放时，数码管从左往右数依次为 DIG1-DIG6，使用数码管 DIG5 和 DIG6 分别显示十位、个位。数码管显示器输出信号 Digitron_Out 和 DigitronCS_Out，其中，Digitron_Out 为七段码管的显示输出，DigitronCS_Out 为 6 个数码管的片选信号，它们的具体驱动方法和功能在第一篇的 1.3.2 节中已经详细介绍过，这里不再赘述。

三、功能模块图与输入输出引脚说明

该工程包含顶层模块 counter24 与底层模块 Accumulator_module、

Digitron_NumDisplay_module。其中, Accumulator_module 实现分频器和计数器的功能, Digitron_NumDisplay_module 实现数码管显示器的功能。晶振由 MINI_FPGA 开发板上的外设提供。图 6.2 是使用 Quartus 生成的顶层文件的 BSF 图, 图 6.3 是整个工程的模块功能图。下面介绍一下顶层模块各引脚的功能:

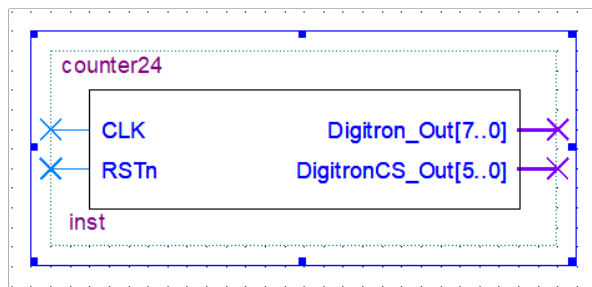


图 6.2 加法计数器顶层文件 BSF 图

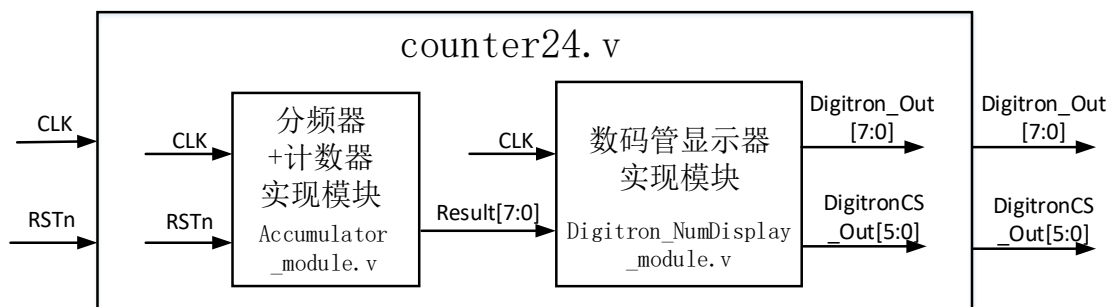


图 6.3 加法计数器模块功能图

- (1) CLK: 50MHz 的时钟信号输入。将其分频后产生标准秒脉冲 CLK1。
- (2) RSTn: 复位输入信号。低电平有效, 复位后计数器输出 Result 变为 00。
- (3) Digitron_Out: 七段数码管的显示输出, 共有八位总线。Digitron_Out[7:0] 分别控制字码段 “DP、G、F、E、D、C、B、A” 的点亮情况。
- (4) DigitronCS_Out: 数码管的片选信号, 因为开发板上有 6 个七段数码管, 所以共有六位总线。DigitronCS_Out[5:0] 分别对应于数码管 DIG1~DIG6, 其也称为七段数码管的扫描驱动, 在本实验中扫描频率为 250KHz, 由于人眼的视觉残留效果, 呈现在眼前的便是整体的 “十位-个位” 显示。

四、程序设计

- (1) 图 6.4 是截取自底层模块 Accumulator_module 的部分代码:

24: 触发条件中的 CLK1 是由之前的分频代码产生的标准秒脉冲，周期为 1s。

28-29: RSTn 为 0 时，系统复位，计数器输出 Result[7:0] 变为 “00”。

31-42: 这是一个 24 进制计数器，个位 (Result[3:0]) 满 10 后向十位进位，整体按照 “23” 翻 “00” 规律计数。

```

24      always @ ( posedge CLK1 or negedge RSTn )
25      begin
26          if( !RSTn )
27              begin
28                  Result[7:4] <= 0;      //ResultH
29                  Result[3:0] <= 0;      //ResultL
30              end
31          else if( Result[7:4] == 4'd2 && Result[3:0] == 4'd3 )
32              begin
33                  Result[7:4] <= 0;
34                  Result[3:0] <= 0;
35              end
36          else if( Result[3:0] == 4'd9 )
37              begin
38                  Result[3:0] <= 0;
39                  Result[7:4] <= Result[7:4] + 1'b1;
40              end
41          else
42              Result[3:0] <= Result[3:0] + 1'b1;
43      end

```

图 6.4 Accumulator_module 核心代码

(2) 图 6.5 是截取自底层模块 Digitron_NumDisplay_module 的部分代码，请结合第一篇的 1.3.2 节中关于数码管模块的介绍部分理解：


```

10     parameter Timex = 8'd200;
11     reg [7:0]Count;
12     reg [3:0]SingleNum;
13     reg [7:0]W_Digitron_Out;
14     reg [5:0]W_DigitronCS_Out;
15     parameter _0 = 8'b0011_1111, _1 = 8'b0000_0110, _2 = 8'b0101_1011,
16               _3 = 8'b0100_1111, _4 = 8'b0110_0110, _5 = 8'b0110_1101,
17               _6 = 8'b0111_1101, _7 = 8'b0000_0111, _8 = 8'b0111_1111,
18               _9 = 8'b0110_1111;
19
20     always @ ( posedge CLK )
21     begin
22         if( Count == Timex )
23         begin
24             Count <= 8'd0;
25             W_DigitronCS_Out = {W_DigitronCS_Out[5:2],W_DigitronCS_Out[0],W_DigitronCS_Out[1]};
26             if(W_DigitronCS_Out == 6'b11_1100)
27                 W_DigitronCS_Out = 6'b11_1110;
28
29             case(W_DigitronCS_Out)
30                 6'b11_1110: SingleNum = Result[3:0]; //Display ResultL
31                 6'b11_1101: SingleNum = Result[7:4]; //Display ResultH
32             endcase
33
34             case(SingleNum)
35                 4'd0: W_Digitron_Out = _0;
36                 4'd1: W_Digitron_Out = _1;
37                 4'd2: W_Digitron_Out = _2;
38                 4'd3: W_Digitron_Out = _3;
39                 4'd4: W_Digitron_Out = _4;
40                 4'd5: W_Digitron_Out = _5;
41                 4'd6: W_Digitron_Out = _6;
42                 4'd7: W_Digitron_Out = _7;
43                 4'd8: W_Digitron_Out = _8;
44                 4'd9: W_Digitron_Out = _9;
45                 default: W_Digitron_Out = 8'b1111_1111;
46             endcase
47
48         else
49             Count <= Count + 1'b1;

```

图 6.5 Digitron_NumDisplay_module 核心代码

10-11、22-24、48-49：分频程序，用于产生频率为 250KHz 的扫描信号，频率的计算方法参考实验 5.3 中的公式 (5.3)。

15-18：参数型常量定义。定义了“_0_1_2..._9”这九个输送到数码管的常量的值。

25-27：片选信号 DigitronCS_Out 的扫描程序。W_DigitronCS_Out[5:0]信号的值将在程序的最后赋给 DigitronCS_Out[5:0]信号。DigitronCS_Out[5:0]分别控制数码管 DIG1-DIG6，当它的某一位为 0 时，对应的数码管选中，反之则不选中。在本实验中仅使用数码管 DIG5 和 DIG6 显示计数器的十位和个位，因而 DigitronCS_Out[5:0]的值在“111110”和“111101”之间变换，变换频率为 250KHz。

29-32：当 DigitronCS_Out[5:0]的值为“111110”时，选中数码管 DIG6，将计数器个位的值赋给 SingleNum；当 DigitronCS_Out[5:0]的值为“111101”时，选中数码管 DIG5，将计数器十位的值赋给 SingleNum。

34-46：根据 SingleNum 信号的值选择输送到数码管的常量，控制字码段的点亮情况。例如当 SingleNum 的值为 4'b1000 时，输送到数码管的常量为“_8”，即

Digitron_Out[7: 0]的值为 8'b0111_1111，字码段 A、B、C、D、E、F、G 全部点亮，DP（小数点）熄灭，数码管显示数字“8”。

五、FPGA 管脚配置

图 6.6 是使用 Quartus 生成的 Pin Planner 管脚图，CLK 时钟输入信号与 MINI_FPGA 开发板上的 50MHz 的晶振时钟相连；复位输入信号 RSTn 与开发板上的 SW0 相连；数码管片选信号 DigitronCS_Out[5: 0]分别与数码管 DIG1~DIG6 的片选引脚相连；数码管输出显示信号 Digitron_Out[7: 0]分别与字码段“DP、G、F、E、D、C、B、A”的控制引脚相连。相应引脚连接信息已经在第一篇的 1.3 节中做过详细介绍，这里就不再赘述。

六、实验结果

图 6.7 是当 SW0 拨至“UP”时的实验现象，数码管显示“21”。当 SW0 拨至“DOWN”时，将显示“00”。因篇幅有限，复位功能请自行验证。

	Node Name	Direction	Location
in	CLK	Input	PIN_E1
out	DigitronCS_Out[5]	Output	PIN_D12
out	DigitronCS_Out[4]	Output	PIN_C11
out	DigitronCS_Out[3]	Output	PIN_F11
out	DigitronCS_Out[2]	Output	PIN_G11
out	DigitronCS_Out[1]	Output	PIN_D14
out	DigitronCS_Out[0]	Output	PIN_C14
out	Digitron_Out[7]	Output	PIN_C9
out	Digitron_Out[6]	Output	PIN_E9
out	Digitron_Out[5]	Output	PIN_D8
out	Digitron_Out[4]	Output	PIN_C8
out	Digitron_Out[3]	Output	PIN_D11
out	Digitron_Out[2]	Output	PIN_E8
out	Digitron_Out[1]	Output	PIN_E10
out	Digitron_Out[0]	Output	PIN_D9
in	RSTn	Input	PIN_R16

图 6.6 加法计数器实验 Pin Planner

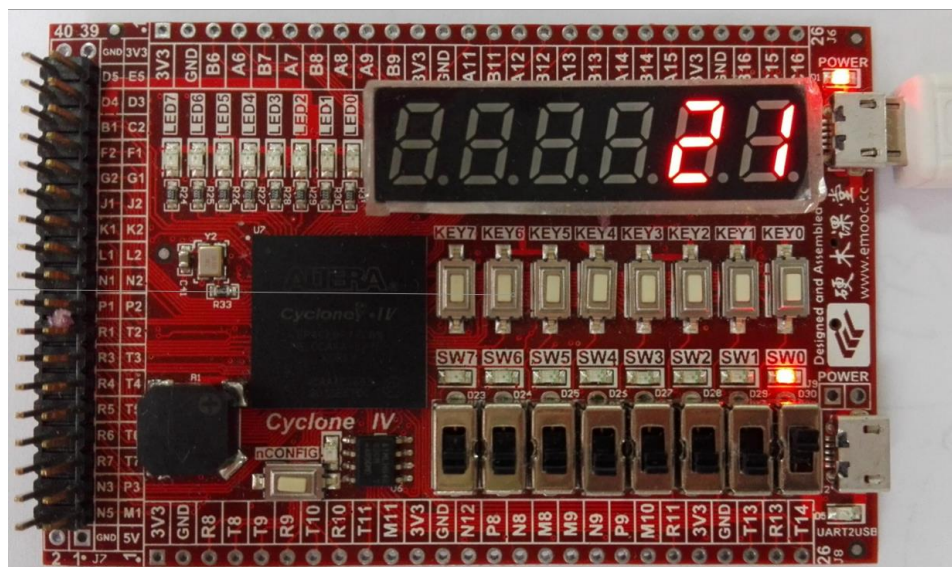


图 6.7 加法计数器实验结果

七、思考与拓展

- (1) 本实验初次使用数码管验证实验输出，认真阅读代码，理解数码管的驱动原理，这将在后面的实验中频繁出现。
- (2) 当删除图 6.5 中的 26-27 行代码时，实验现象及其形成原因是什么？

实验七 抢答器

一、实验设计目标

- (1) 设计实现一个可容纳四组选手参赛的抢答器系统，每组设一个抢答按钮。答题开始后，由主持人按下“开始”键后进入抢答环节，当某个小组抢答成功时，抢答器系统发出半秒的低频音，显示该组别序号并点亮该组“选手指示灯”直至系统复位。此时进入答题计时环节，若超过 30 秒仍未答出，抢答器系统发出 1 秒的高频音示警，同时点亮“超时灯”1 秒。由裁判员按下“复位”键，开始新一轮答题。
- (2) 通过此实验学习 MINI_FPGA 开发板上的蜂鸣器的发声原理。

二、实验设计思路

本实验设计实现一个抢答器系统，它由第一信号鉴别、锁存模块，答题计时模块，蜂鸣器发声模块和数码管显示模块组成，图 7.1 是该系统的示意图。

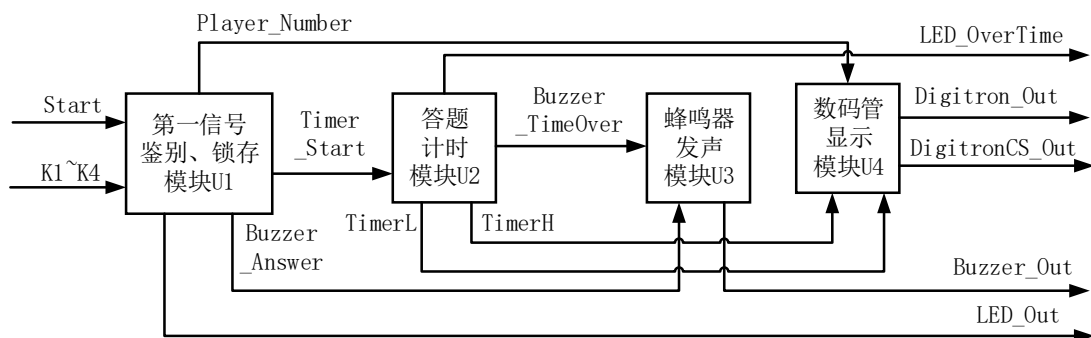


图 7.1 抢答器系统示意图

第一信号鉴别、锁存模块的关键在于准确判断出第一抢答者并将其锁存。设置一个主持人“开始”按钮 Start，四个抢答按钮 K1、K2、K3、K4，主持人开始后，抢答信号才能被有效识别，在某个小组抢答成功后，其他组的抢答信号无效。本模块输出 LED_Out[3:0]、Player_Number[3:0]、Buzzer_Answer 和 Timer_Start 信号。LED_Out 用于控制“选手指示灯”；Player_Number 存储抢答成功的小组序号；Buzzer_Answer 作为“抢答成功鸣笛标志位”，送到蜂鸣器发声模块，控制蜂鸣器的发声频率及时间；Timer_Start 作为答题计时模块的“启动标志位”，当某个小组抢答成功后，Timer_Start 变为 1，启动答题计时器。

答题计时模块的关键在于倒计时的启动，利用 Timer_Start 信号控制倒计时的开启。在实验六中设计实现了一个加法计数器，类似的，本模块只需实现一个“30-29-…-01-00”的减法计数器即可。本模块输出 TimerH[3:0]、TimerL[3:0]、Buzzer_TimeOver 和 LED_OverTime 信号。TimerH 和 TimerL 分别存储倒计时的十位和个位；Buzzer_TimeOver 作为“答题超时鸣笛标志位”，送到蜂鸣器发声模块；LED_OverTime 用于控制答题“超时灯”。

蜂鸣器发声模块用于控制蜂鸣器的发声频率、何时发声及发声时长。通过标志位 Buzzer_Answer 和 Buzzer_TimeOver 控制送到蜂鸣器上的电压值及电压变化频率。Buzzer_Out 即为最终送给蜂鸣器的信号。

数码管显示模块用于显示倒计时的时间和抢答成功的组别序号，驱动原理与实

验六相同。

三、功能模块图与输入输出引脚说明

该工程包含顶层模块 responder 与底层模块 Sel_module、Timer_module、Buzzer_module 和 Digitron_NumDisplay_module。底层模块依次对应于图 7.1 中从左至右四个功能模块。图 7.2 是使用 Quartus 生成的顶层文件的 BSF 图，整个工程的模块功能图参考图 7.1 即可。下面介绍一下顶层模块各主要引脚的功能：

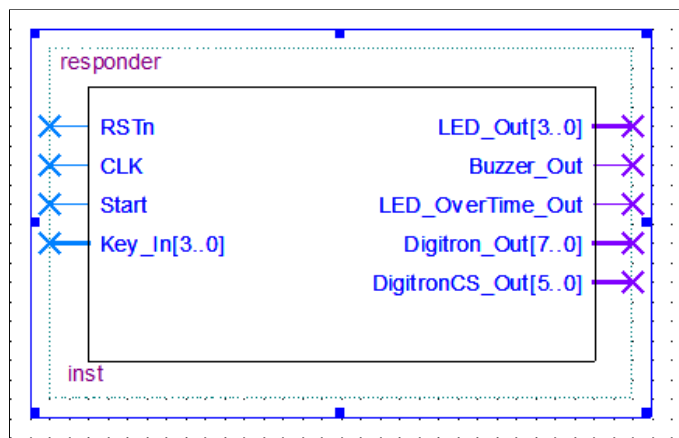


图 7.2 抢答器系统顶层文件 BSF 图

(1) CLK: 50MHz 的基准时钟信号输入。在 Sel_module 模块中将其分频后产生标准秒脉冲 CLK1。

(2) RSTn: 系统复位输入信号。低电平有效，由裁判员控制，复位后系统回到初始状态，“选手指示灯、超时灯”熄灭，蜂鸣器不响，数码管显示“030”。

(3) Start: 抢答开始输入信号。由主持人控制，当 Start 为 1 时，各个小组的抢答信号有效。

(4) Key_In: 抢答输入信号。Key_In [3:0] 分别连接抢答器电路的 K4~K1，当 Key_In 为 0 时表示存在抢答操作。

(5) LED_Out: “选手指示灯”输出信号，输出到 LED 灯，共有四位总线。LED_Out [3:0] 分别连接第四组~第一组选手的指示灯，例如若第四组选手抢答成功，LED_Out [3:0] 的值为“1000”。

(6) Buzzer_Out: 输出到蜂鸣器。

(7) LED_OverTime_Out: 答题“超时灯”输出信号。LED_OverTime_Out 信号与抢答器电路的 LED_OverTime 相连，控制超时灯。

(8) Digitron_Out: 七段数码管的显示输出, 共有八位总线。

(9) DigitronCS_Out: 数码管的片选信号, 共有六位总线。(8)、(9) 与实验六中相应引脚功能相同, 不再赘述。

四、程序设计

(1) 7.3 是截取自底层模块 Sel_module 的部分代码:

```

28         else if( Start == 1 )
29         begin
30             if( Timer_Start )
31             begin
32                 if( Count == 25'd24_999_999 )
33                 begin
34                     Buzzer_Answer <= 0;
35                     Count <= Count;
36                 end
37             else
38             begin
39                 Buzzer_Answer <= 1;
40                 Count <= Count + 25'b1;
41             end
42         end
43     else if( !K1 && !Block )
44     begin
45         LED_Out <= 4'b0001;
46         Block <= 1;
47         Timer_Start <= 1;
48         Player_Number <= 4'd1;
49     end

```

图 7.3 Sel_module 核心代码

30-42: Timer_Start 是“计时器启动标志位”, 当第一信号鉴别、锁存模块锁存了第一抢答信号后, 令 Timer_Start 变为 1, 启动答题计时器电路。当 Timer_Start 变为 1 后, 30-42 行代码将 Buzzer_Answer (“抢答成功鸣笛标志位”) 置为 1, 保持半秒后, 再置为 0。这里用到了一个很简单的计数器 Count, 用于控制半秒的时间。

43-49: 判断是否已经锁存以及第一组别是否有抢答操作, Block 是锁存信号, 高电平表示电路已锁存。若第一组抢答, 且在此之前未有其他组抢答成功 (Block 为 0), 45-48 行依次为点亮第一组“选手指示灯”, 锁存电路, 开启答题计时器, 将第一组别序号送给数码管。

(2) 图 7.4 是截取自底层模块 Buzzer_module 的部分代码, 请结合第一篇的

1.3 节中关于蜂鸣器模块的介绍内容进行理解:

```

17      always @ ( posedge CLK )
18      begin
19          if( Buzzer_Answer == 1)
20              Pulse_x <= _Answer;
21          else if( Buzzer_TimeOver == 1 )
22              Pulse_x <= _TimeOver;
23          else
24              Pulse_x <= 'd20000;
25      end
26
27      always @ ( posedge CLK )
28      begin
29          if( (Pulse_x == _Answer) | (Pulse_x == _TimeOver) )
30          begin
31              if( Count == Pulse_x )
32              begin
33                  Count <= 23'd0;
34                  W_buzzer <= ~W_buzzer;
35              end
36              else
37                  Count <= Count + 1'b1;
38          end

```

图 7.4 Buzzer_module 核心代码

17-25: 通过“鸣笛标志位”控制送到蜂鸣器上的电压和电压变化频率。例如, 当某个小组抢答成功时,“抢答成功鸣笛标志位” Buzzer_Answer 将被置为 1 且经过半秒后变回 0, 这半秒内将常量_Answer 的值 ('d95419) 赋给 Pulse_x, 半秒后 Pulse_x 的值变为'd20000。

29-37: 这是一个小型计数器。若 Pulse_x 的值为“_Answer”或“_TimeOver”, 则控制 W_buzzer 的值以一定频率在“0”和“1”之间翻转, 这样便形成了一定频率的脉冲信号。W_buzzer 的值即为将送给蜂鸣器的值, 该脉冲信号的频率即控制蜂鸣器发声的频率。

五、FPGA 管脚配置

图 7.5 是使用 Quartus 生成的 Pin Planner 管脚图, CLK 时钟输入信号、数码管片选信号 DigitronCS_Out 和数码管输出显示信号 Digitron_Out 的配置方式与实验六相同, 使用 DIG4 显示抢答成功组别的序号, 使用 DIG5 和 DIG6 显示倒计时的十位和个位; 复位输入信号 RSTn 和抢答开始信号 Start 分别与开发板上的 SW0 和 SW1 相连; Buzzer_Out 信号输出到蜂鸣器引脚; Key_In[3:0]分别与按键开关 KEY3~KEY0 相连; LED_Out[3:0]分别输出到 LED3~LED0, LED_OverTime_Out 输出到 LED4, LED4 点亮表明答题超过 30 秒。相应引脚连接信息已经在第一篇的 1.3 节中

做过详细介绍，这里就不再赘述。

	Node Name	Direction	Location
out	Buzzer_Out	Output	PIN_L3
in	CLK	Input	PIN_E1
out	DigitronCS_Out[5]	Output	PIN_D12
out	DigitronCS_Out[4]	Output	PIN_C11
out	DigitronCS_Out[3]	Output	PIN_F11
out	DigitronCS_Out[2]	Output	PIN_G11
out	DigitronCS_Out[1]	Output	PIN_D14
out	DigitronCS_Out[0]	Output	PIN_C14
out	Digitron_Out[7]	Output	PIN_C9
out	Digitron_Out[6]	Output	PIN_E9
out	Digitron_Out[5]	Output	PIN_D8
out	Digitron_Out[4]	Output	PIN_C8
out	Digitron_Out[3]	Output	PIN_D11
out	Digitron_Out[2]	Output	PIN_E8
out	Digitron_Out[1]	Output	PIN_E10
out	Digitron_Out[0]	Output	PIN_D9
in	Key_In[3]	Input	PIN_K16
in	Key_In[2]	Input	PIN_J15
in	Key_In[1]	Input	PIN_J16
in	Key_In[0]	Input	PIN_J14
out	LED_Out[3]	Output	PIN_B4
out	LED_Out[2]	Output	PIN_A4
out	LED_Out[1]	Output	PIN_B5
out	LED_Out[0]	Output	PIN_A5
out	LED_OverTime_Out	Output	PIN_A3
in	RSTn	Input	PIN_R16
in	Start	Input	PIN_P15

图 7.5 抢答器系统 Pin Planner

六、实验结果

当 SW0 和 SW1 均拨至“UP”时，系统进入抢答环节。若假设第二组选手抢答成功，此时蜂鸣器鸣低频音并持续半秒钟，灯 LED1 点亮，数码管 DIG4 将显示“2”，DIG5 和 DIG6 将显示倒计时数字；若答题超过 30 秒，灯 LED4 将点亮 1 秒，蜂鸣器鸣高频音并持续 1 秒钟，DIG5 和 DIG6 显示“00”并保持直到系统复位。图 7.6 即为上述假设中处于倒计时 19 秒时的现象。因篇幅受限，其他情况请自行验证。

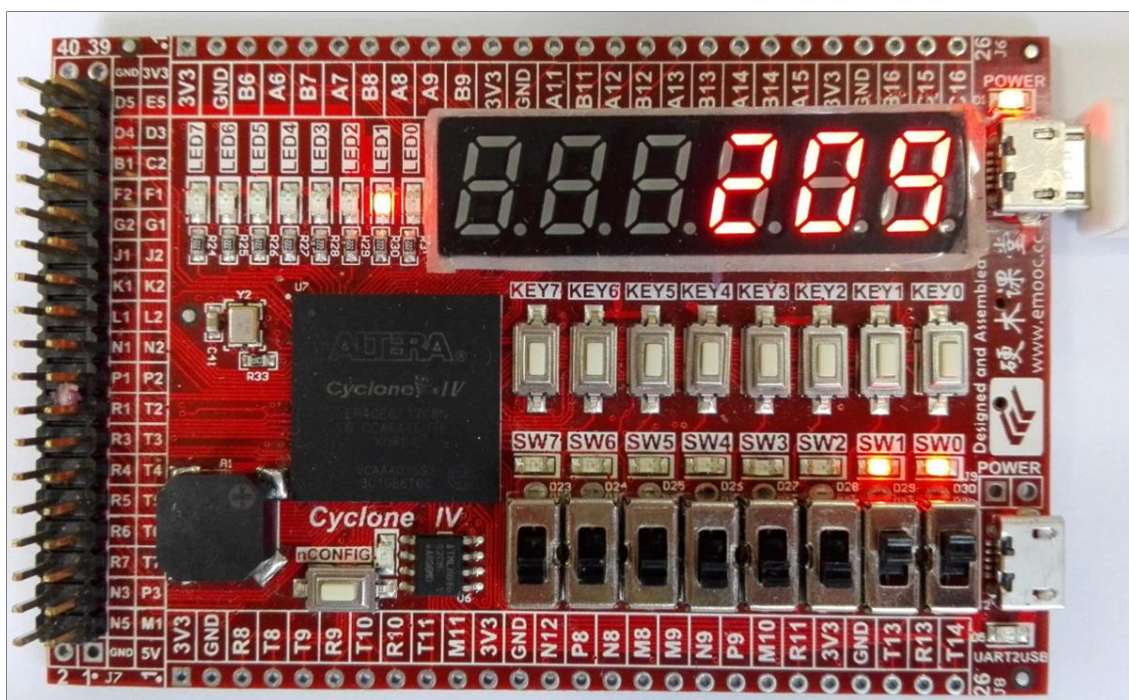


图 7.6 抢答器系统实验结果

七、思考与拓展

(1) 新建一个工程，自行设计程序或查询相关资料，实现以下功能：①计分功能，每组开始预置 5 分，由主持人计分，答对一次加 1 分，答错一次减 1 分；②抢答未开始前若有人抢答则报警并点亮相应选手指示灯。

(2) 本实验初次使用蜂鸣器显示实验结果，请仔细理解蜂鸣器的工作方法，并新建工程，实现按键控制蜂鸣器发“Do、Re、Mi、Fa、Sol、La、Si”音阶的功能。实际上本实验中的低频音和高频音即为“Do、Si”，频率的计算方法参考实验 5.3 中的公式 (5.3)。各个音阶的参考频率依次为 262、294、330、349、392、440 和 494Hz。

实验八 功能数字钟

一、实验设计目标

在 FPGA 中设计实现一个多功能数字钟，具备以下功能：

(1) 准确计时。能显示时、分、秒和星期，小时的计时为 24 进制，分和秒的计时为 60 进制，星期按“1、2、3、4、5、6、日”显示。

(2) 校时功能。时、分和星期可调。

(3) 准点报时。当“时-分-秒”为“XX-59-50、XX-59-52、XX-59-54、XX-59-56、XX-59-58”时，蜂鸣器发“嘀”；当“时-分-秒”为“XX-00-00”时，扬声器发“嗒”。

二、实验设计思路

本节实验是在综合了前面几节实验知识点的基础上开展的，容易理解和掌握，该数字钟系统由计时、校时模块，数码管显示模块和整点报时模块组成，图 8.1 是该系统的示意图。

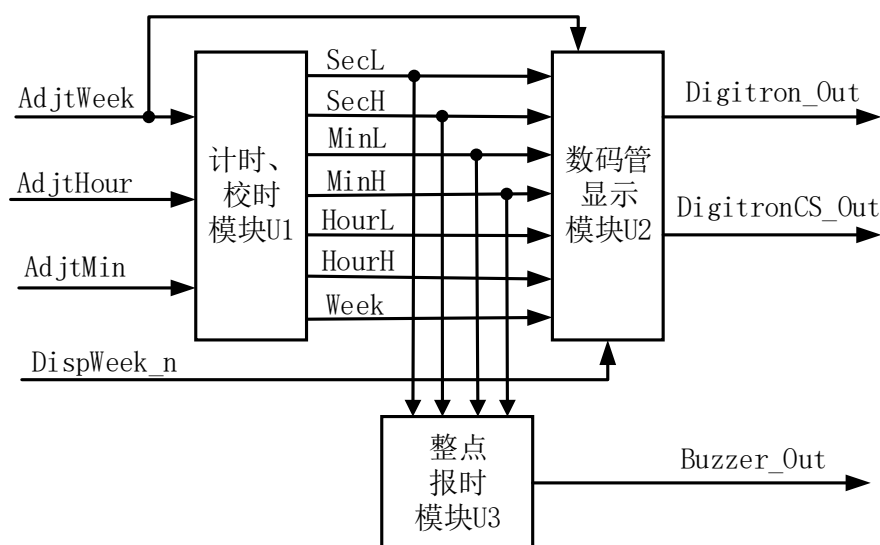


图 8.1 数字钟系统示意图

计时、校时模块能工作于“计时”和“校时”两个状态。本模块内含一个分频电路用于产生标准秒脉冲。设置 3 个开关用于调整时间，包括星期调整开关 AdjWeek、小时调整开关 AdjHour 和分钟调整开关 AdjMin，当其有效时，模块停止计时，工作于“校时”状态。输出信号包括秒个位 SecL、秒十位 SecH、分个位 MinL、分十位 MinH、时个位 HourL、时十位 HourH 和星期 Week。

数码管显示模块用于显示“时、分、秒、星期”。MINI_FPGA 开发板上只有 6 个数码管，但要显示包括“时、分、秒、星期”在内的 7 个时间信号，因此，我们增

加一个“星期显示输入 (DispWeek_n)”信号，用于查看当前星期，低电平有效。当要调整星期时，也需要使用数码管显示星期界面， AdjWeek 信号便是用于告知

这一点的。

整点报时模块的关键在于判断出当前时刻。使用 if 语句判断是否满足报时条件，通过改变输送到蜂鸣器上的导通信号的频率来改变蜂鸣器发声的频率，这在实验七中已经用到过，工作原理相同。

三、功能模块图与输入输出引脚说明

该工程包含顶层模块 clock 与底层模块 TimeKeeper_module、Digitron_TimeDisplay_module 和 Buzzer_module。底层模块依次对应于图 8.1 中的 U1、U2、U3。图 8.2 是使用 Quartus 生成的顶层文件的 BSF 图，整个工程的模块功能图参考图 8.1 即可。下面介绍一下顶层模块各主要引脚的功能：

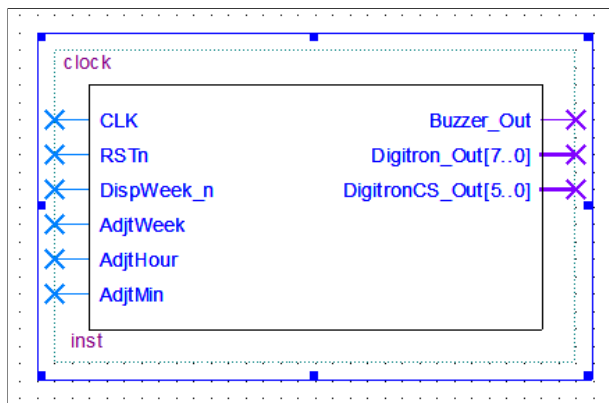


图 8.2 多功能数字钟顶层文件 BSF 图

(1) CLK: 50MHz 的基准时钟信号输入。将其 50_000_000 分频得到 1HZ 的数字钟工作频率 CLK_1HZ。将其 100_000 分频得到 500HZ 的蜂鸣器报时频率“Di”，将其 50_000 分频得到 1KHZ 的蜂鸣器报时频率“Da”。将其 200 分频得到七段数码管的扫描频率。

(2) RSTn: 系统复位输入信号，低电平有效。复位后“星期-小时-分钟-秒”置为“1-00-00-00”。

(3) DispWeek_n: 星期显示输入信号，低电平有效。当 DispWeek_n 或 AdjWeek 信号有效时，数码管 DIG1-DIG5 全灭，DIG6 显示当前星期；当 DispWeek_n 和 AdjWeek 信号均无效时，数码管 DIG1-DIG6 显示当前“时、分、秒”。

(4) AdjWeek、AdjHour、AdjMin: 星期、小时、分钟调节输入信号，高电平有效。当 AdjWeek=1 时，每来一个标准秒脉冲 CLK_1HZ 的上升沿，星期增加

一天。其他 2 个开关相似。AdjWeek、AdjHour、AdjMin 信号的优先级从左至右依次降低。

(5) Buzzer_Out: 输出到蜂鸣器, 通过改变输送到蜂鸣器的导通信号的频率, 来改变蜂鸣器发声的频率, 控制蜂鸣器产生“嘀、嗒”的报时声。

(6) Digitron_Out: 七段数码管的显示输出, 共有八位总线。

(7) DigitronCS_Out: 数码管的片选信号, 共有六位总线。也称七段数码管的扫描驱动信号, 扫描频率为 250KHz。(6)、(7) 与实验六中相应引脚功能相同, 不再赘述。

四、程序设计

(1) 图 8.3 是截取自底层模块 TimeKeeper_module 的部分代码, 理解了此段程序则可以相应理解该模块的所有程序:

```

47 | begin
48 |     if( AdjWeek == 1 )
55 |         else if( AdjHour == 1 )           //Adjust Hour
56 |             begin
57 |                 if( HourL == 'd9 )
58 |                     begin
59 |                         HourL <= 0;
60 |                         HourH <= HourH + 1;
61 |                     end
62 |                 else
63 |                     begin
64 |                         if( HourH == 'd2 && HourL == 'd3 )
65 |                             begin
66 |                                 HourL <= 0;
67 |                                 HourH <= 0;
68 |                             end
69 |                         else
70 |                             HourL <= HourL + 1;
71 |                         end
72 |                     end

```

图 8.3 TimeKeeper_module 代码

47-72: 当 AdjHour 为 1 且 AdjWeek 为 0 时, 系统进入“校时”状态, 开始调整“小时”。若时个位为 9, 则时个位清零并向时十位进位; 若时十位、时个位为“23”, 则时十位、时个位清零; 若以上均不满足, 则时个位加 1, 时十位不变。

(2) 图 8.4 是截取自底层模块 Digitron_TimeDisplay_module 的部分代码, 工作原理与实验六加法计数器中的类似:

```

32 |         if( (DispWeek_n == 0) || (AdjWeek == 1) )
33 |         begin
34 |             W_DigitronCS_Out = 6'b11_1110;
35 |             SingleNum = Week;
36 |
37 |         case(SingleNum)
38 |             'd0: W_Digitron_Out = _0;
39 |             'd1: W_Digitron_Out = _1;
40 |             'd2: W_Digitron_Out = _2;
41 |             'd3: W_Digitron_Out = _3;
42 |             'd4: W_Digitron_Out = _4;
43 |             'd5: W_Digitron_Out = _5;
44 |             'd6: W_Digitron_Out = _6;
45 |             'd7: W_Digitron_Out = _Ri;
46 |         endcase

```

图 8.4 Digitron_TimeDisplay_module 核心代码

32-35: 当要显示星期或者调整星期时, 数码管 DIG1~DIG5 全灭, DIG6 显示当前 Week 的值。

五、FPGA 管脚配置

图 8.5 是使用 Quartus 生成的 Pin Planner 管脚图, CLK 时钟输入信号、数码管片选信号 DigitronCS_Out 和数码管输出显示信号 Digitron_Out 的配置方式与实验六相同, DIG1~DIG6 依次显示“时、分、秒”, DIG6 也用于显示“星期”; 复位信号 RSTn 和星期显示信号 DispWeek_n 分别与 KEY0 和 KEY1 相连; 调整开关 AdjWeek、AdjHour、AdjMin 依次与 SW0、SW2、SW1 相连; Buzzer_Out 信号输出到蜂鸣器引脚。相应引脚连接信息已经在第一篇的 1.3 节中做过详细介绍, 这里就不再赘述。

六、实验结果

当 SW2、SW1、SW0 拨至“UP”时, 分别调整“时、分、星期”, 且优先级从高到低排列为 SW0、SW2、SW1; KEY1 按下时 DIG1~DIG5 灭, DIG6 显示“星期”; KEY0 按下时时钟复位, “时、分、秒、星期”置为“00、00、00、1”; 另有整点报时功能。

图 8.6 显示当前时刻为 3 点 10 分 33 秒, 其他功能请自行验证。

Node Name	Direction	Location
in AdjthHour	Input	PIN_P16
in AdjthMin	Input	PIN_P15
in AdjthWeek	Input	PIN_R16
out Buzzer_Out	Output	PIN_L3
in CLK	Input	PIN_E1
out DigitronCS_Out[5]	Output	PIN_D12
out DigitronCS_Out[4]	Output	PIN_C11
out DigitronCS_Out[3]	Output	PIN_F11
out DigitronCS_Out[2]	Output	PIN_G11
out DigitronCS_Out[1]	Output	PIN_D14
out DigitronCS_Out[0]	Output	PIN_C14
out Digitron_Out[7]	Output	PIN_C9
out Digitron_Out[6]	Output	PIN_E9
out Digitron_Out[5]	Output	PIN_D8
out Digitron_Out[4]	Output	PIN_C8
out Digitron_Out[3]	Output	PIN_D11
out Digitron_Out[2]	Output	PIN_E8
out Digitron_Out[1]	Output	PIN_E10
out Digitron_Out[0]	Output	PIN_D9
in DispWeek_n	Input	PIN_J16
in RSTn	Input	PIN_J14

图 8.5 多功能数字钟 Pin Planner

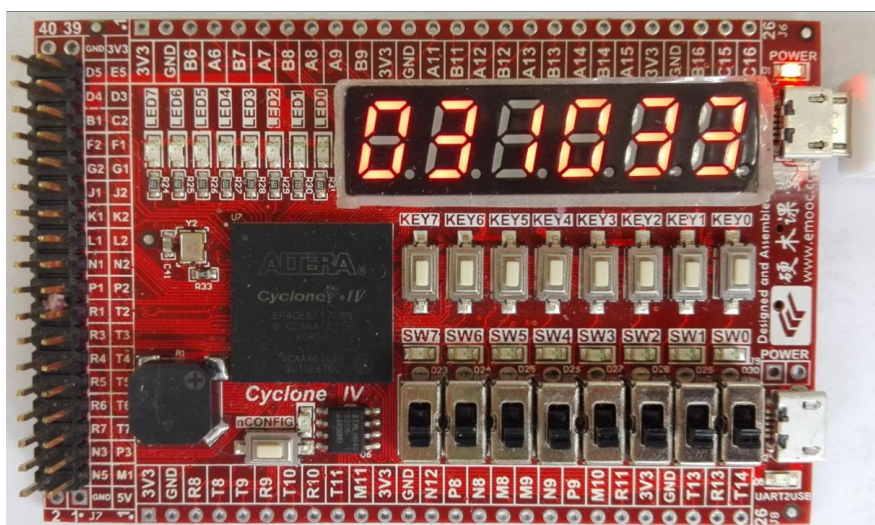


图 8.6 多功能数字钟实验结果

七、思考与拓展

- (1) 自行设计程序或查阅相关资料，实现以下功能：①显示“年-月-日”；②“定时闹钟”功能。

第三篇 综合性实验设计与实现

实验一 PWM

一、实验设计目标

(1) 编写程序，使用 FPGA 产生脉冲调制 (PWM) 信号，且信号的周期和占空比可通过按键调节。

(2) 通过此实验学习“按键消抖”的基本方法。

二、实验设计思路

PWM 信号可用于控制步进电机的工作，图 1.1 是一个 PWM 信号的示意图。这个脉冲的周期为 Period，宽度为 1 的那段时间称为脉冲宽度，占空比定义为高电平信号占整个脉冲周期的百分比，即：

$$\text{占空比Duty} = \frac{\text{脉冲宽度}}{\text{周期Period}} \times 100\% \quad (1.1)$$

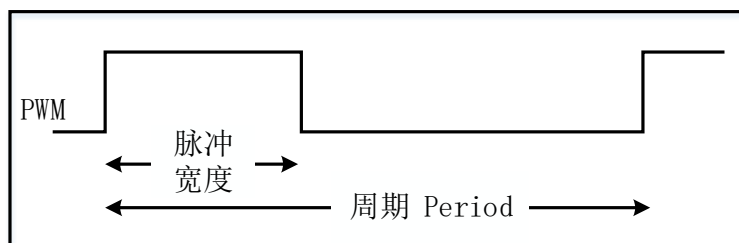


图 1.1 一个 PWM 信号

要产生这样的 PWM 信号，基本思想就是使用一个计数器，当计数值 Cnt1 小于脉冲宽度时，让 PWM 信号为 1；当 Cnt1 大于等于脉冲宽度时，让 PWM 信号为 0；当 Cnt1 的值等于 Period-1 时，计数器复位，Cnt1 变为 0。循环往复便产生了一个连续的 PWM 信号。

该 PWM 信号发生器系统由占空比、周期调整模块，PWM 信号产生模块和数码管显示模块组成，图 1.2 是该系统的示意图。

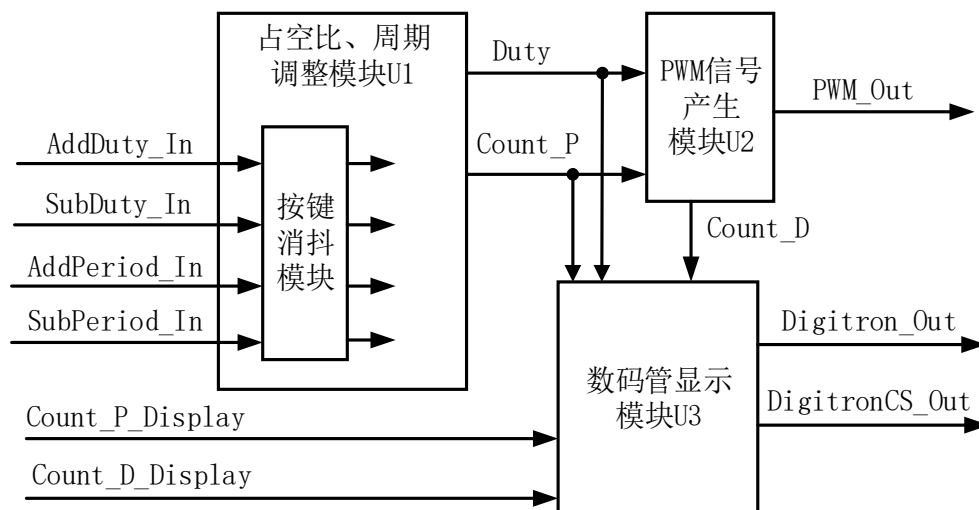


图 1.2 PWM 信号发生器示意图

占空比、周期调整模块用于调整 PWM 信号的占空比 Duty 和周期 Period，设置四个按键开关，分别控制占空比增加、占空比减少、周期增加和周期减少。输出信号 Duty 用于控制 PWM 波的占空比，取值范围为 0~100；Count_P 内存储了一个数值，用于表征周期 Period。需要注意的是，当按下一次按键开关时，FPGA 可能识别到多次操作或未识别到操作，所以，U1 模块内增加“按键消抖模块”用于准确识别按键操作。

PWM 信号产生模块用于产生 PWM 信号。Count_D 内存储了一个数值，用于表征脉冲宽度，计算公式为

$$Count_D = (Duty * Count_P) \div 100 \quad (1.2)$$

U2 模块内有一个计数器 Cnt1，按照前文提到的基本思想，比较 Cnt1 和 Count_P、Count_D 的数值，即可产生所需的 PWM 波。

数码管显示模块用于显示当前 PWM 信号的 Duty、Count_P 和 Count_D，便于观察验证实验结果。因数码管个数限制，增加开关信号 Count_D_Display 与 Count_P_Display 用于切换数码管界面，这与第二篇的实验八中的思想相同。

三、功能模块图与输入输出引脚说明

该工程较包含顶层模块 pwm 与底层模块 Duty_Period_Adjust_module、

PWM_Generate_module、Digitron_NumDisplay_module, 底层模块依次对应于图 1.2 的 U1~U3。图 1.3 是使用 Quartus 生成的顶层文件的 BSF 图, 整个工程的模块功能图参考图 1.2 即可。下面介绍一下顶层模块各主要引脚的功能:

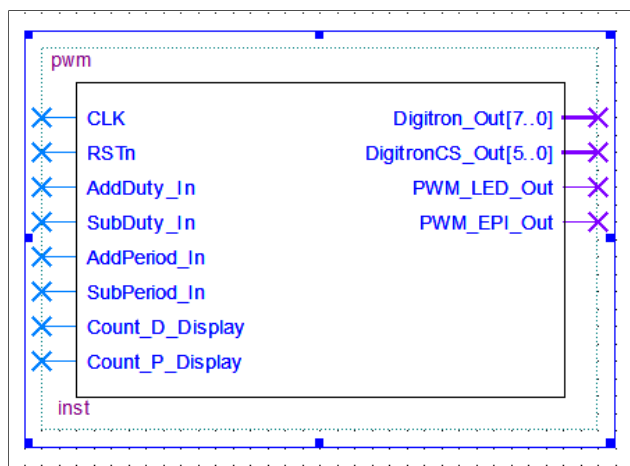


图 1.3 PWM 实验顶层文件 BSF 图

(1) CLK: 50MHz 的时钟信号输入。

(2) RSTn: 复位输入信号。当 RSTn 为低电平时, PWM 信号的周期、占空比分别置为 5ms 和 50%。

(3) AddDuty_In、SubDuty_In、AddPeriod_In、SubPeriod_In: 按键开关输入信号。依次控制占空比增加、占空比减少、周期增加和周期减少, 每按一下, 相应参数变化一次。

(4) Count_D_Display、Count_P_Display: 拨动开关输入信号, 高电平有效, 分别控制数码管显示 Count_D 和 Count_P。

(5) Digitron_Out、DigitronCS_Out: 分别为七段数码管的显示输出信号和扫描驱动信号, 与第一篇的实验六中相应引脚功能相同, 不再赘述。

(6) PWM_LED_Out: U2 模块产生的 PWM 信号, 输出到 LED 灯, 共有一位总线。因为 PWM 信号的平均直流值和占空比是成比例的, 可以通过 LED 灯的亮度变化来定性观察 PWM 信号的占空比的变化情况。

(7) PWM_EPI_Out: 与 (6) 相同, 也是所产生的 PWM 信号, 输送到 Pocket Instrument 中, 定量观察 PWM 信号。

四、程序设计

(1) 图 1.4 是截取自底层模块 PWM_Generate_module 的部分代码:

```

15      assign Count_D = (Duty * Count_P) / 'd100;
16
17      always @ ( posedge CLK or negedge RSTn )
18      begin
19          if( RSTn == 0 )
20              Cnt1 <= 0;
21          else if( Cnt1 == Count_P - 1'b1 )
22              Cnt1 <= 0;
23          else
24              Cnt1 <= Cnt1 + 1'b1;
25      end
26
27      always @( * )
28      begin
29          if( Cnt1 <= Count_D )
30              PWM_Out <= 1'b1;
31          else
32              PWM_Out <= 0;
33      end
    
```

图 1.4 PWM_Generate_module 核心代码

15: 计算 Count_D, 见公式 (1.2)。

29-32: 使用一个简易的计数器 Cnt1, 通过比较 Cnt1 与 Count_D 的值, 决定信号 PWM_Out 电平的高或低。

(2) 图 1.5 是截取自“消抖模块” Jitter_Elimination_module 的部分代码:

```

8      input Button_In;
9      output Button_Out;
10
11      reg neg1;
12      reg neg2;
13
14      always @ ( posedge CLK or negedge RSTn )
15      if( !RSTn )
16      begin
17          neg1 <= 1'b1;
18          neg2 <= 1'b1;
19      end
20      else
21      begin
22          neg1 <= Button_In;
23          neg2 <= neg1;
24      end
25
26      assign Button_Out = ( neg2 & (!neg1) ) ? 1'b1 : 1'b0;
    
```

图 1.5 Jitter_Elimination_module 核心代码

8-9: 定义按键输入信号和按键输出信号。

22-26: neg2 存储上一个时钟周期时 Button_In 的状态, neg1 存储这个时钟周期时的状态, 当 Button_In 从 1 变为 0 时, Button_Out 置为 1, 随后又变回 0。也就是说, Button_Out 只有在按键按下的一刻为 1。

五、FPGA 管脚配置

图 1.6 是使用 Quartus 生成的 Pin Planner 管脚图, CLK、DigitronCS_Out 和 Digitron_Out 信号已经多次用到, 不再赘述; 复位输入信号 RSTn 与开发板上的 SW0 相连; AddDuty_In、SubDuty_In、AddPeriod_In、SubPeriod_In 输入信号分别与开发板上的 KEY3~KEY0 相连; Count_D_Display、Count_P_Display 输入信号分别与开发板上的 SW1、SW2 相连; 输出信号 PWM_LED_Out 送到 LED0; PWM_EPI_Out 送到开发板上侧的 I/O 通道 “A6” 中, 通过 “A6” 口送往 Pocket Instrument。引脚连接信息已经在第一篇的 1.3 节中做过详细介绍, 这里就不再赘述。

Node Name	Direction	Location
 AddDuty_In	Input	PIN_K16
 AddPeriod_In	Input	PIN_J16
 CLK	Input	PIN_E1
 Count_D_Display	Input	PIN_P15
 Count_P_Display	Input	PIN_P16
 DigitronCS_Out[5]	Output	PIN_D12
 DigitronCS_Out[4]	Output	PIN_C11
 DigitronCS_Out[3]	Output	PIN_F11
 DigitronCS_Out[2]	Output	PIN_G11
 DigitronCS_Out[1]	Output	PIN_D14
 DigitronCS_Out[0]	Output	PIN_C14
 Digitron_Out[7]	Output	PIN_C9
 Digitron_Out[6]	Output	PIN_E9
 Digitron_Out[5]	Output	PIN_D8
 Digitron_Out[4]	Output	PIN_C8
 Digitron_Out[3]	Output	PIN_D11
 Digitron_Out[2]	Output	PIN_E8
 Digitron_Out[1]	Output	PIN_E10
 Digitron_Out[0]	Output	PIN_D9
 PWM_EPI_Out	Output	PIN_A6
 PWM_LED_Out	Output	PIN_A5
 RSTn	Input	PIN_R16
 SubDuty_In	Input	PIN_J15
 SubPeriod_In	Input	PIN_J14

图 1.6 PWM 实验 Pin Planner

六、实验结果

本实验使用 Pocket Instrument 观察 PWM 波形, 使用 LED0 定性观察 PWM 波的占空比变化情况, 使用数码管显示 Duty、Count_D 和 Count_P 的值。表 1.1 和表 1.2 分别是在调节 PWM 波时信号 Duty 和信号 Count_P 可能出现的数值, 十六进制数值即数码管上所显示的数值, 表 1.2 还列出了相应的 PWM 波的频率。

表 1.1 Duty 信号数值

十进制数值	00	10	20	30	40	50	60	70	80	90	100
十六进制数值	00	0A	14	1E	28	32	3C	46	50	5A	64

表 1.2 Count_P 信号数值

十进制 ($\times 10^3$)	50	100	150	200	250	300	350	400	450	500
十六进制 值	C350	186A0	249F0	30D40	3D090	493E0	55730	61A80	6DD00	7A120
PWM 的 频率 (Hz)	1000	500	333	250	200	167	143	125	111	100

图 1.7 是使用 EPI 观察到的初始状态时的 PWM 波形, 初始状态时, PWM 信号的周期、频率、占空比分别为 5ms、200Hz 和 50%, 和图 1.7 中波形参数吻合。当按下按键 SW3~SW0 后, 波形、LED0 的亮度、数码管上的数值都会有一定变化, 请结合程序及表 1.1、表 1.2 自行验证。

七、思考与拓展

- (1) 请仔细阅读代码, 计算 PWM 信号的周期的可调范围, 并与实际结果比对。
- (2) 请参照表 1.1, 列出当 PWM 信号的频率固定为 200Hz 时, Count_D 可能出现的数值。

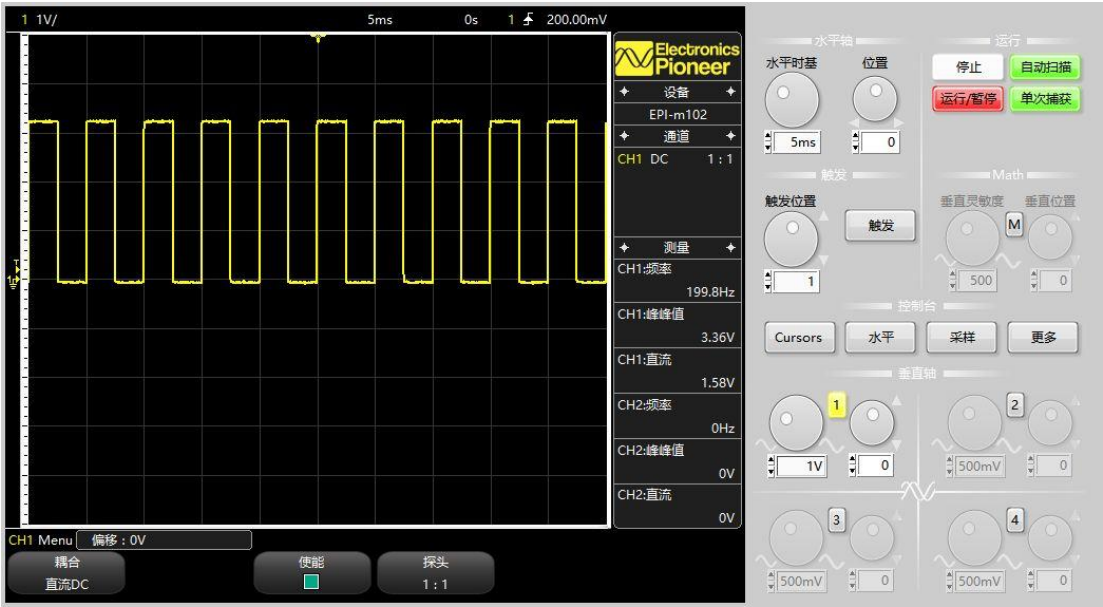


图 1.7 PWM 波形

实验二 DA 及 DDS

实验 2.1 DA 输出实验

一、实验设计目标

编写程序，使用 FPGA 驱动 DA 芯片，输出锯齿波。

二、实验设计思路

任意复杂的数字电路都可以使用 FPGA 实现，但当关联上模拟信号时，就需要 AD、DA 芯片作为转换器件。AD 是把模拟信号转变成数字信号，再送给 FPGA 做数字算法处理；DA 则是将 FPGA 处理过后的数字信号转变成模拟信号。

本实验设计使用 FPGA 驱动 DA 芯片，输出锯齿波。由 FPGA 向 DA 芯片输入一个数字值，经过转换后输出相应的模拟值，改变输入的数字值便可得到幅度变化的模拟信号输出，形成锯齿波。实验的关键在于编写 DA 的驱动程序和控制送入 DA 的数字值。

为方便实验，直接购买了一块集成了 DA 芯片 (AD9708)、滤波器、放大器等器件的高速并行 AD/DA 集成板。AD9708 是 8 位、125MSPS 的并行 DA 转换芯片，内置 1.2V 参考电压，差分电流输出；DA 集成板上有九个输入端和一个输出端（输入时钟 DACLK、数据输入 DADB7~DADBO 和数据输出 DA OUT）。图 2.1 是该芯片的工作时序图，图 2.2 是截取自芯片原理图上的部分参数说明。可以看出，DA 芯片的工作由 CLOCK 信号控制，在每个 CLOCK 时钟的上升沿后并延时 t_{PD} 时间后将 DA 转换结果输出；CLOCK 时钟的工作脉宽 t_{LPW} 最小为 3.5ns；每一个输入的数值必须满足最小建立时间 t_s 和最小保持时间 t_H 才能被有效读取。下面介绍一下转换规律，式 2.1 是截取自原理图上的 DA 芯片输出模拟电压的计算公式，具体参数含义可自行查阅原理图。

$$V_{DIFF} = \{(2DAC\ CODE - 255) \div 256\} \times (32R_{LOAD} \div R_{SET}) \times V_{REFIO} \quad (2.1)$$

其中“DAC CODE”即为送入 DA 芯片的值，因 DA 集成板上还集成了滤波器、反向放大器等器件，最终的计算公式可以等价于：

$$(255 - 2 \times \text{输入的数据}) \div 256 = \text{输出的模拟电压} \div V_{max} \quad (2.2)$$

$V_{\max}$ 可通过 DA 集成板上的电位器旋钮调节, 调节范围是 $-5V \sim 5V$ ($10V_{pp}$)。当输入的 8 位数据都为 1 时, 输出电压近似为 $-V_{\max}$; 当输入的 8 位数据都为 0 时, 输出电压近似为 $V_{\max}$ 。这样就能根据输入的 data 控制输出信号的波形。

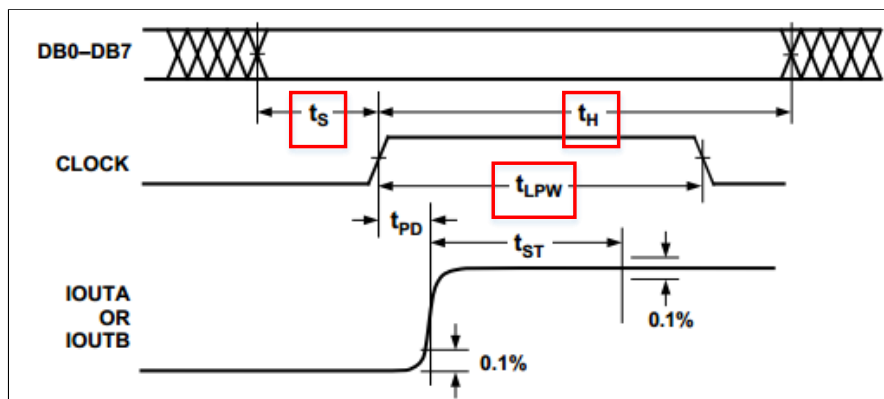


图 2.1 DA 芯片工作时序图

Parameter	Min	Typ	Max	Units
Input Setup Time (t_s)	2.0			ns
Input Hold Time (t_H)	1.5			ns
Latch Pulsewidth (t_{LPW})	3.5			ns

图 2.2 DA 芯片部分工作参数

三、功能模块图与输入输出引脚说明

该工程较为简单, 仅包含顶层模块 DA, 图 2.3 是使用 Quartus 生成的顶层文件的 BSF 图。下面介绍一下顶层模块各引脚的功能:

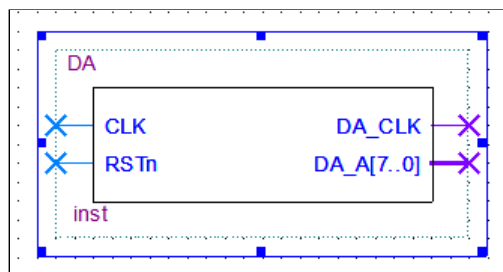


图 2.3 DA 实验顶层文件 BSF 图

(1) CLK: 50MHz 的时钟信号输入。CLK 时钟的周期为 20ns, 满足 DA 芯片的工作时钟 CLOCK 的要求, 因而直接将 CLK 赋给 CLOCK。

(2) RSTn: 复位输入信号。当 RSTn 为低电平时, 输出信号 DA_Data[7: 0]恒为 “00000000”。

(3) DA_CLK: DA 工作时钟信号。输送到 DA 集成板中, 即为图 2.1 中的 CLOCK 时钟信号。

(4) DA_Data: 输送到 DA 集成板的数字值, 共有八位总线。DA_Data[7: 0]分别对应与图 2.1 中的 DB7-DB0。

四、程序设计

图 2.4 是截取自顶层模块 DA 的部分代码:

```

13      assign DA_CLK = CLK ;
14
15      always @( posedge CLK or negedge RSTn )
16          if( !RSTn )
17              Count <= 0;
18          else if ( Count == 'd255 )
19              begin
20                  Count <= 0;
21                  DA_Data <= Count[7:0] ;
22              end
23          else
24              begin
25                  Count <= Count + 1'b1;
26                  DA_Data <= Count[7:0] ;
27              end

```

图 2.4 DA 输出实验核心代码

13: 给 DA_CLK 赋值。

18-27: 使用一个简易的计数器 Count, 改变送给 DA_Data 的数值。因共有八位, 取值范围介于 0~255 之间。

五、FPGA 管脚配置

图 2.5 是使用 Quartus 生成的 Pin Planner 管脚图, CLK 时钟输入信号与开发板上的 50MHz 的晶振时钟相连;复位输入信号 RSTn 与开发板上的 SW0 相连;DA_CLK 和 DA_Data[7: 0]分别送到开发板上左侧的 I/O 通道中, 再将 DA 集成板的相应引脚与该 I/O 通道相连, 数据流便从 FPGA 传送到了 DA 集成板中。引脚连接信息已经在第一篇的 1.3 节中做过详细介绍, 这里就不再赘述。

	Node Name	Direction	Location
in	CLK	Input	PIN_E1
out	DA_CLK	Output	PIN_P3
out	DA_Data[7]	Output	PIN_N3
out	DA_Data[6]	Output	PIN_T7
out	DA_Data[5]	Output	PIN_R7
out	DA_Data[4]	Output	PIN_T6
out	DA_Data[3]	Output	PIN_R6
out	DA_Data[2]	Output	PIN_T5
out	DA_Data[1]	Output	PIN_R5
out	DA_Data[0]	Output	PIN_T4
in	RSTn	Input	PIN_R16

图 2.5 DA 输出实验 Pin Planner

六、实验结果

注意，除时钟信号和输入数据外，还需提供“+5V”电压和“GND”，DA 芯片才能正常工作。因 Pocket Instrument 上可提供的电源功率不足，使用实验室的电源给 DA 集成板供电。DA 集成板需要与 MINI_FPGA 开发板“共地”。

当开关 SW0 拨至“DOWN”时，DA 集成板上的“DA OUT”口输出稳定的高电压，电压值可由电位器调节；当 SW0 拨至“UP”时，“DA OUT”口输出下行的锯齿波。图 2.6 是使用 EPI 观察到的锯齿波图形。其他现象请自行验证。

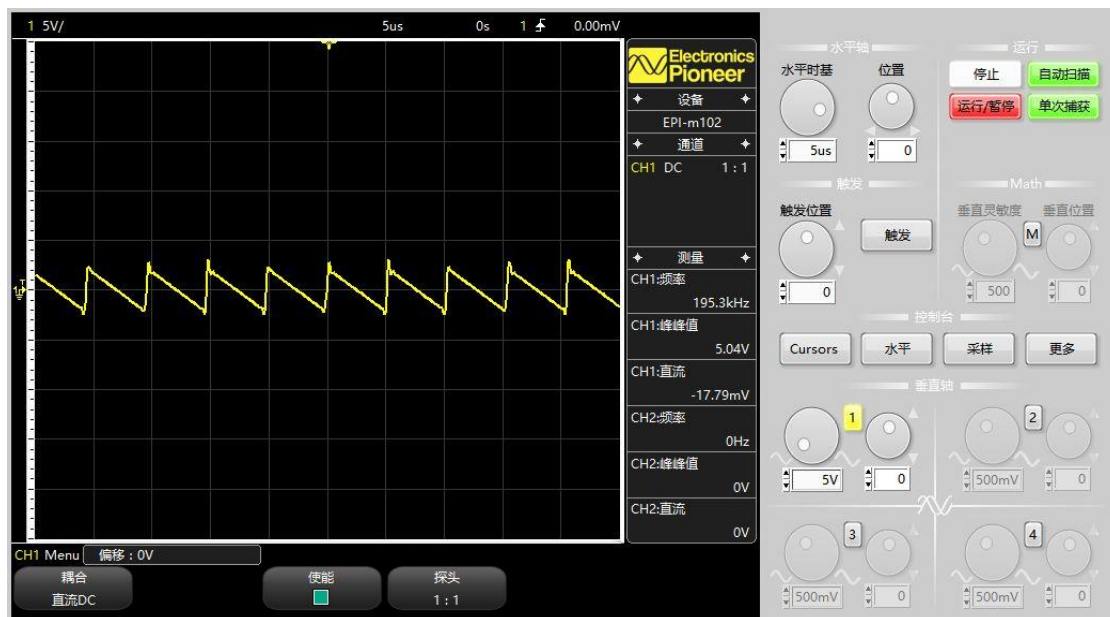


图 2.6 DA 输出实验 Pin Planner

七、思考与拓展

(1) 图 2.6 中的锯齿波电压在跳变点后是“从高变低”的，请结合式 (2.2) 解释原因。

(2) 当把图 2.6 中的波形放大时，可以看见在跳变点处有“毛刺”，当使用实验室示波器显示波形时，则没有“毛刺”，请思考其中的原因。

(3) 图 2.6 给出了波形的频率等参数，请结合程序，判断参数是否正确。

八、实验小结

使用 FPGA 驱动 DA 芯片工作的关键就在于编写它的时序驱动，本节实验使用的是并行 DA，在时序上要求不高。本篇的实验三将涉及到串行 AD 芯片的时序驱动原理。

实验 2.2 DDS 实验

一、实验设计目标

(1) 在学习了实验 2.1 的基础上, 使用 FPGA 驱动 DA 芯片, 设计一个直接数字频率合成器 (DDS)。要求能产生正弦波、方波和三角波, 并且波形的频率可在一定范围内调节。

(3) 通过此实验初步学习使用 Altera 中的 IP 核生成 ROM 查询表的方法。

二、实验设计思路

DDS 信号源是能直接合成所需波形的一种频率合成器件, 由频率合成器 (相位累加器、波形查询表) 和数模转换器构成。波形合成的基本思想与实验 2.1 中锯齿波的产生原理相同, 只要控制送入数模转换器的数字值以一定规律周期性变化, 便能得到所需的波形。

图 2.7 是 DDS 器件的核心部分流程图, 包含相位累加器和波形查询表。CLK 为工作脉冲, KW 代表频率控制字。相位累加器内包含一个 N 位全加器, 每来一个时钟脉冲 CLK, 相位累加器的输出结果 addr (N 位) 就以步长 KW 递增。波形查询表 ROM 即为波形存储器, 其内部存储了所需波形的一个周期内的 2^N 个采样点的数据。查询表把送入的地址信息 addr 映射成所需波形的数字幅度信号, 即输出 Wave_Data。

DDS 合成公式为:

$$f_{WAVE_DATA} = \frac{f_{CLK}}{2^N} * KW \quad (2.3)$$

可以看出, 当 CLK 的频率和 N 的值不变时, 只要改变输入的频率控制字 KW, 即可改变输出波形的频率。

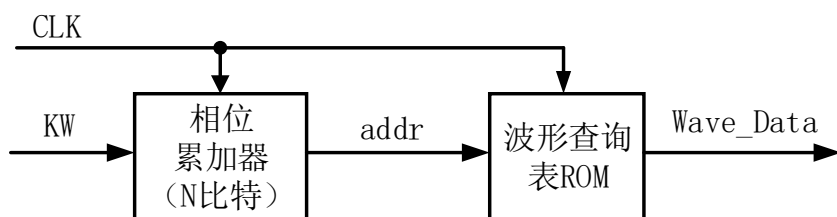


图 2.7 核心部分流程图

图 2.8 是本实验设计的 DDS 系统的示意图，包含频率控制字模块、波形选择模块和数模转换模块。

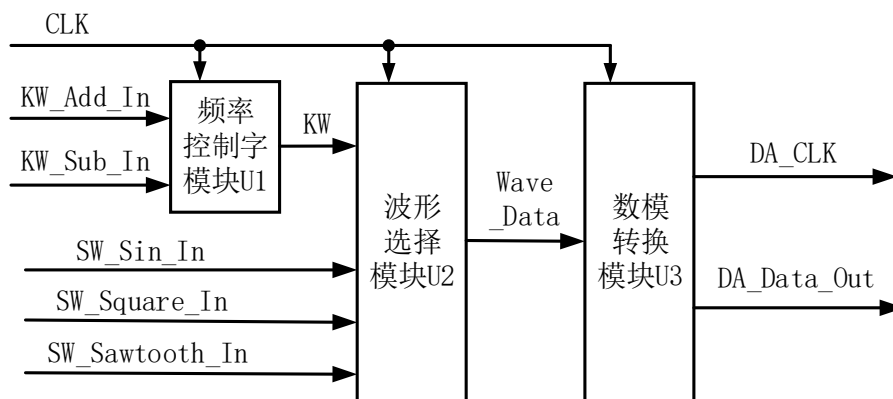


图 2.8 DDS 系统示意图

频率控制字模块是用于改变送往 U2 模块的 KW 信号的值。设置 2 个按键输入 KW_Add_In 和 KW_Sub_In，分别控制 KW 的增加和减少；U1 模块内部包含“按键消抖”电路。该模块的工作原理与实验 2.1 节相同，这里不再赘述。

波形选择模块内部包含了相位累加器和三个波形查询表，分别为正弦波、方波和三角波，功能如图 2.7 所示。设置 3 个开关输入 SW_Sin_In、SW_Square_In、SW_Sawtooth_In，用于选择输出的波形。

数模转换模块的功能与实验 2.1 节相同，向 DA 集成板输送 DA_CLK 信号和 DA_Data_Out 信号，用于驱动 DA 芯片工作。

三、功能模块图与输入输出引脚说明

该工程包含顶层模块 DDS 与底层模块 frequency_adjust_module、choose_wave_module 和 dac_module，底层模块从左至右依次对应于图 2.8 中的 U1~U3。图 2.9 是使用 Quartus 生成的顶层文件的 BSF 图，整个工程的模块功能图参考图 2.8 即可。下面介绍一下各主要引脚的功能：

(1) CLK：50MHz 的时钟信号输入。

(2) RSTn：复位输入信号。当 RSTn 为低电平时，KW 为 5，输出波形的频率约为 61KHZ。

(3) KW_Add_In、KW_Sub_In。通过按键开关输入，分别控制 KW 的增加和减少。每按一次，KW 的值相应变化一次。

(4) SW_Sin_In、SW_Square_In、SW_Sawtooth_In: 通过拨动开关输入, 分别

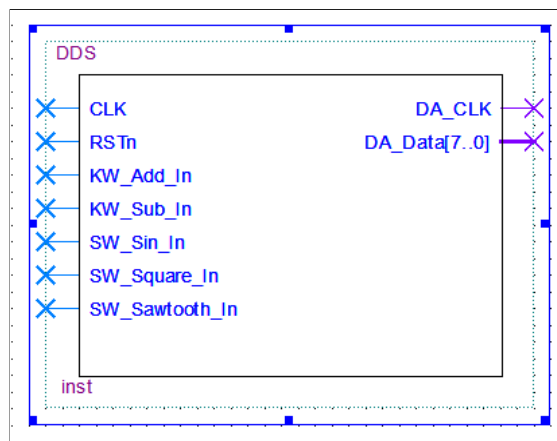


图 2.9 DDS 实验顶层文件 BSF 图

控制系统输出正弦波、方波、三角波。从左至右优先级依次降低, 均为高电平有效。

(5) DA_CLK: DA 工作时钟信号。

(6) DA_Data: 输送到 DA 集成板的数字值, 共有八位总线。(5) (6) 与实验 2.1 节相应信号功能相同。

四、程序设计

图 2.10 是截取自底层模块 choose_wave_module 的部分代码:

```

36      dds_sin_rom Sin_Rom
37      (
38          .address ( addr ),    // input - from top
39          .clock ( CLK ),      // input - from top
40          .q ( Sin_Out )       // output - to top
41      );
    
```

图 2.10 choose_wave_module 核心代码

36-41: 调用了一个类型为 dds_sin_rom 的查询表。每个波形数据都存储于一个的单元中, address 即为要输出的数据的单元地址; clock 为工作时钟; q 即为查询表根据 address 信号映射出的数据。

五、FPGA 管脚配置

图 2.11 是使用 Quartus 生成的 Pin Planner 管脚图, CLK、RSTn、DA_CLK 和

DA_Data[7: 0] 信号的配置方式均与实验 2.1 中相应信号相同；KW_Add_In、KW_Sub_In 分别与开发板上的 KEY0、KEY1 相连；SW_Sin_In、SW_Square_In、SW_Sawtooth_In 分别与 SW1~SW3 相连。引脚连接信息已经在第一篇的 1.3 节中做过详细介绍，这里就不再赘述。

	Node Name	Direction	Location
in	CLK	Input	PIN_E1
out	DA_CLK	Output	PIN_P3
out	DA_Data[7]	Output	PIN_N3
out	DA_Data[6]	Output	PIN_T7
out	DA_Data[5]	Output	PIN_R7
out	DA_Data[4]	Output	PIN_T6
out	DA_Data[3]	Output	PIN_R6
out	DA_Data[2]	Output	PIN_T5
out	DA_Data[1]	Output	PIN_R5
out	DA_Data[0]	Output	PIN_T4
in	KW_Add_In	Input	PIN_J16
in	KW_Sub_In	Input	PIN_J14
in	RSTn	Input	PIN_R16
in	SW_Sawtooth_In	Input	PIN_N15
in	SW_Sin_In	Input	PIN_P15
in	SW_Square_In	Input	PIN_P16

图 2.11 DDS 实验 Pin Planner

六、实验结果

MINI_FPGA 开发板与 DA 集成板之间的连接方式与实验 2.1 节相同，使用实验室的电源给 DA 集成板供电，使用口袋仪器的“示波器功能”观察 DA 集成板输出的波形。当 SW1、SW2、SW3 分别拨至“UP”时，可看到正弦波、方波和三角波，且 SW1、SW2、SW3 的优先级从左至右依次降低。按下 KEY1（KEY0）时，输出波形的频率增加（减少），且频率的变化是介于一定范围内的。

图 2.12、2.13、2.14 分别是初始状态下使用 EPI 观察到的正弦波、方波、三角波的波形。初始状态下，KW 值为 5，输出波形频率约为 61KHz，与图形吻合。其他现象请自行验证。

七、思考与拓展

（1）用口袋仪器看到的方波的波形存在很多毛刺，当使用实验室示波器观察时，“毛刺”现象大大改善，请思考毛刺产生及减少的可能原因。

（2）参考实验 2.1 节，自行编写程序，用数码管显示任意时刻的 KW 值，并通过公式（2.3）计算此时的波形频率，然后与口袋仪器显示的结果相比较。

(3) 除使用 IP 核建立 ROM 表外, 也可用 verilog 语言直接编写程序生成 ROM 表。感兴趣的同学可以自行阅读参考书籍, 实现此功能。

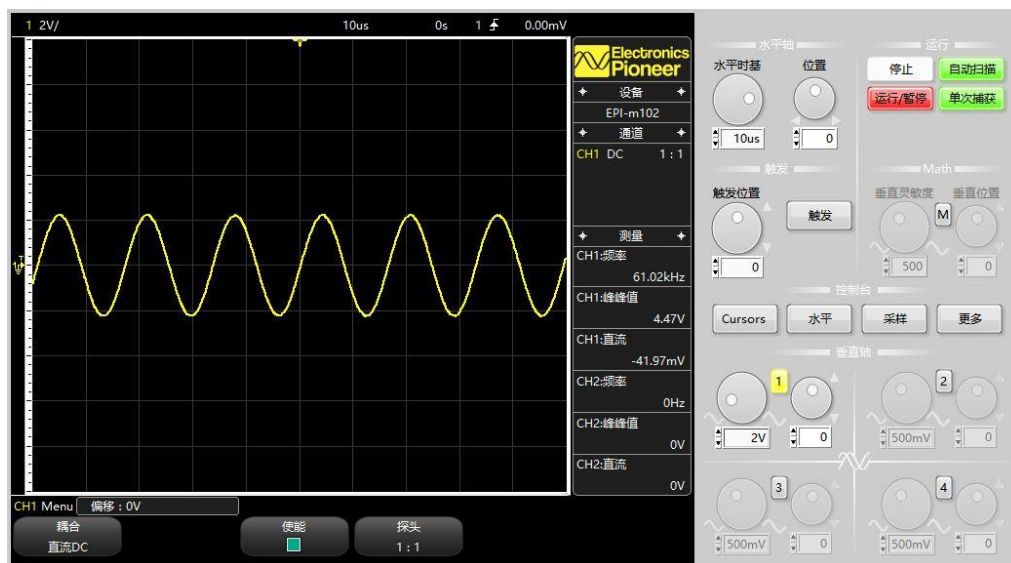


图 2.12 正弦波波形

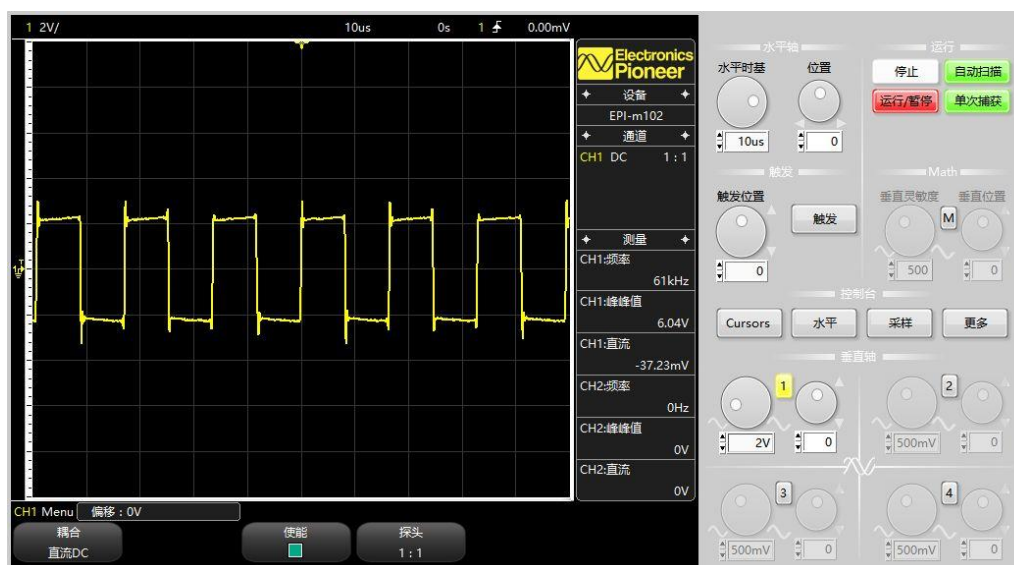


图 2.13 方波波形

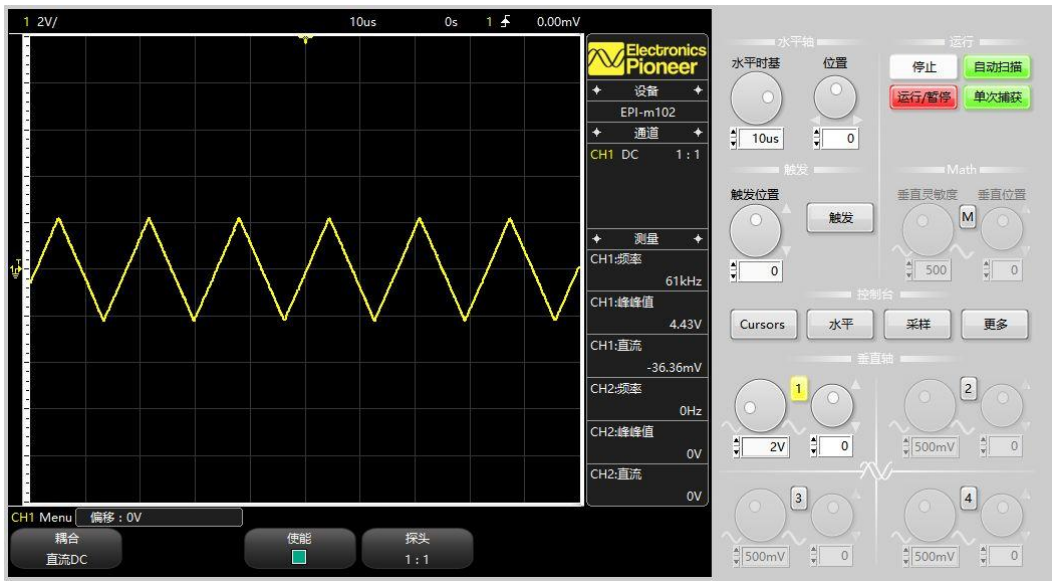


图 2.14 三角波波形

实验三 AD

实验 3.1 AD 采样实验

一、实验设计目标

(1) 编写程序，使用 FPGA 驱动 AD 采集器工作，并使用 SignalTap 显示得到的数字信号的波形，进行对比。

(2) 通过本实验学习 Altera 中嵌入式逻辑分析仪 SignalTap 的使用方法。

二、实验设计思路

本实验设计使用 FPGA 驱动 10 位串行 AD 芯片工作，将正弦模拟信号转化为数字信号，并由 FPGA 接收该串行数据流后存储为并行数据，观察得到的数字信号的波形。与实验二类似，AD 实验的关键在于编写 AD 芯片的驱动程序和正确接收 AD 采集器输出的串行数字信号。

为方便实验，直接购买了一块包含了 AD 芯片 (TLC1543)、滤波器、放大器等器件的串行 AD 集成板。TLC1543 是 10 位串行 AD 转换芯片，有三个输入端和一个 3 态输出端，分别为片选 (CS)、输入/输出时钟 (I/O CLOCK)、地址输入 (ADDRESS) 和数据输出 (DATA OUT)。

图 3.1 是该芯片工作于快速转换方式 1 时的工作时序图。

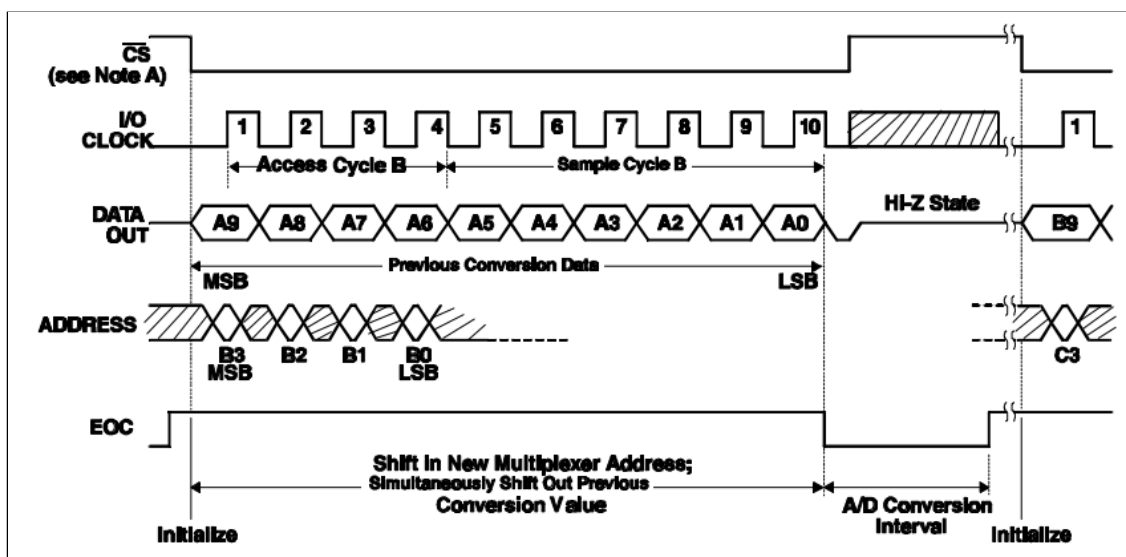


图 3.1 AD 芯片工作时序图

可以看出, 当 CS 变为低电平后, 芯片被选中, 但是 CS 拉低后需要等待一个设置时间 $t_{su(cs)}$ 才能响应其他控制输入信号, 图 3.2 中 (a) 为设置时间 $t_{su(cs)}$ 的示意图。CS 有效后, 从 I/O CLOCK 端输入 10 个外部时钟信号。前一次转换的最高有效位 (MSB) 在 CS 的下降边时被移到 DATA OUT 引脚上, 剩下的 9 位在第 1~9 个 I/O CLOCK 信号的下降边时被移出, 在 I/O CLOCK 的第十个下降边时, DATA OUT 端被驱动为逻辑低电平。在 I/O CLOCK 的前 4 个上升沿将 ADDRESS 端呈现的下一个转换周期的 4 位模拟通道选择位 (MSB 在前) 输入地址寄存器, 这个地址选择 14 个输入 (11 个模拟输入和 3 个内部测试电压) 中的 1 个, 但 ADDRESS 也需满足一定的设置时间 $t_{su(A)}$ 才能被有效读取, 图 3.2 中 (b) 为设置时间 $t_{su(A)}$ 的示意图。在第 4 个 I/O CLOCK 信号由高至低的跳变时, 开始对由 ADDRESS 信号选择的通道的输入模拟量进行采样, 第 10 个 I/O CLOCK 信号的下降沿时, 电路开始 AD 转换, 并将 EOC (转换结束端) 拉低直到转换完成。然后 CS 拉高。每一次的 AD 转换也需要一定的时间, 结合 datasheet, 最大为 21us。

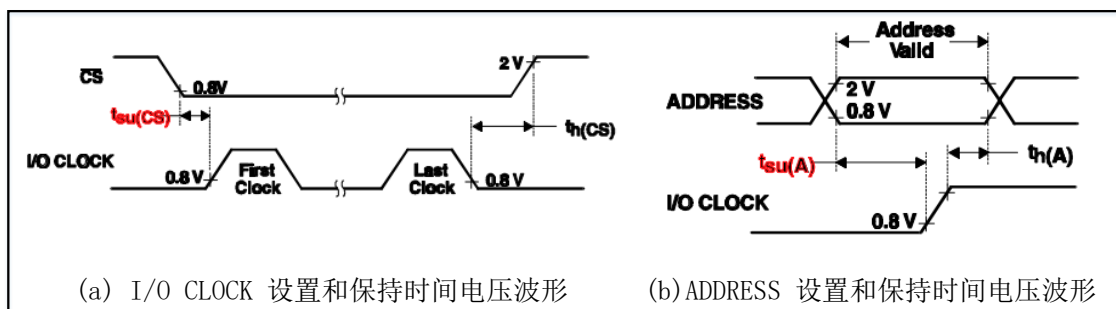


图 3.2 部分参数示意图

图 3.3 是截取自芯片原理图上的部分参数说明。总结如下:

	最小	典型	最大	单位
建立时间, I/O CLOCK 上跳之前在数据输入端地址位, $t_{su(A)}$	100			ns
保持时间, I/O CLOCK 上跳之后地址位, $t_{h(A)}$	0			ns
保持时间, 最后 1 个 I/O CLOCK 下跳之后 $\overline{CS}$ 为低, $t_{h(cs)}$	0			ns
建立时间, 在用时钟同步输入第 1 个地址位之前 $\overline{CS}$ 为低, $t_{su(cs)}$ (见注 3)	1.425			μs
I/O CLOCK 时钟频率, $f_{clock(I/O)}$ (见注 4)	0		2.1	MHz
I/O CLOCK 为高时的脉冲宽度, $t_{WH(I/O)}$	190			ns
I/O CLOCK 为低时的脉冲宽度, $t_{WL(I/O)}$	190			ns

图 3.3 AD 芯片部分参数说明

$$(1) f_{(I/O\ CLOCK)} \leq 2.1MHz, \text{ 实验时, 取 } T_{(I/O\ CLOCK)} = 520ns \quad (3.1)$$

$$(2) t_{su(cs)} \geq 1.425\mu s, \text{ 实验时, 取 } t_{su(cs)} = 1.44\mu s \quad (3.2)$$

$$(3) t_{su(A)} \geq 100ns, \text{ 实验时, 取 } t_{su(A)} = 120ns \quad (3.3)$$

$$(4) \text{ 连续读完 10 位的数据后需要等待 } 21\mu s \text{ 的时间才可以进行下一次数据的读取。取 } t_{oov} = 21\mu s \quad (3.4)$$

下面介绍一下转换规律，类似于实验 2.1 中 DA 转换公式，AD 转换公式如下，其中， V_{max} 的值由送给 AD 集成板的基准电压决定：

$$\frac{\text{输入的模拟电压}}{V_{max}} = \frac{\text{输出的data}}{1023} \quad (3.5)$$

三、功能模块图与输入输出引脚说明

在理解了上述 AD 采集过程及转换规律后，设计 AD 采样系统如图 3.4 所示，包含 AD 采集模块 U1 和数码管显示模块 U2。

U1 模块用于驱动 AD 芯片完成上述采集过程。AD 转换得到的数字信号以串行方式经 DATA OUT 引脚送入 FPGA 中，即输入信号 AD_DigData_In；再经过 U1 模块内的控制电路后存储为 10 位并行数据 Data_Out[9:0] 输出；CS_Valid_Sig 信号用于表征 CS 信号的下降沿是否有效（即 $t_{su(cs)}$ 是否满足条件），不送给 AD 芯片；AD_CS_n、AD_Clk、AD_Address 信号分别对应图 3.1 中各信号，输送到 AD 芯片上。

U2 模块用于显示经 U1 模块得到的并行数字信号 Data_Out。

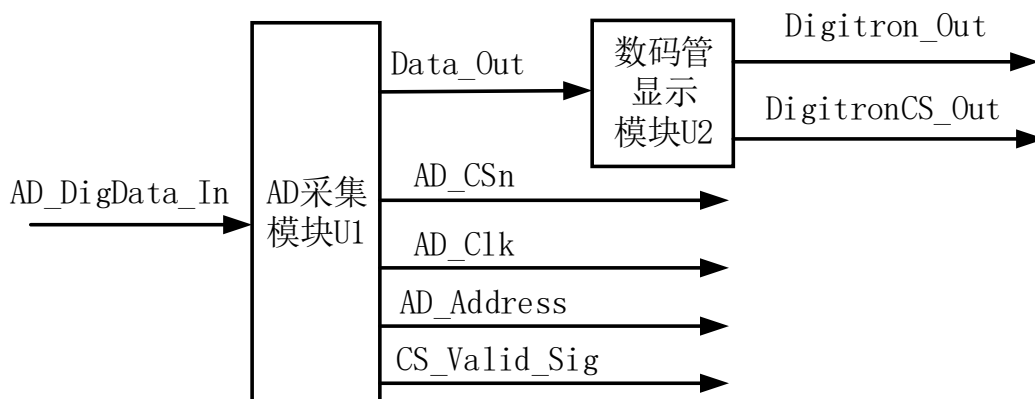


图 3.4 AD 采样系统示意图

在所建立的工程中，包含顶层模块 AD 与底层模块 AD_Data、Digitron_NumDisplay。底层模块依次对应于图 3.4 中的 U1、U2。此外，工程中还包含一个底层模块 p11_100K，不参与 AD 采样过程，是通过 Altera 的 IP 核生成的，用于产生频率为 100KHz 的时钟信号，作为时序分析仪（SignalTap）的工作时钟。图 3.5 是使用 Quartus 生成的顶层文件的 BSF 图，下面介绍一下顶层模块各主要引脚的功能：

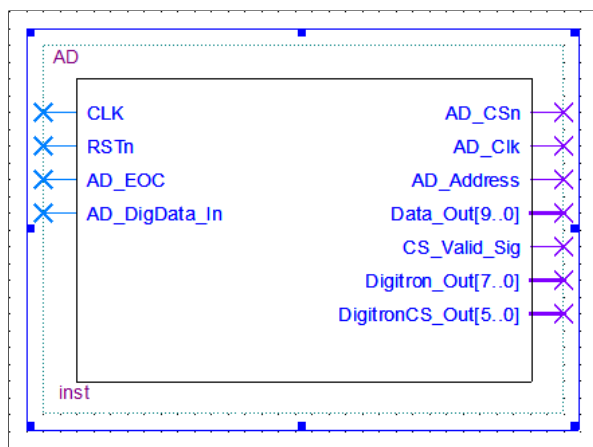


图 3.5 AD 采样系统顶层文件 BSF 图

- (1) CLK: 50MHz 的基准时钟信号输入。周期为 20ns，用于产生一定的延时。
- (2) RSTn: 系统复位输入信号，低电平有效。复位后 AD 芯片不工作，Data_Out[9:0]全为 0，数码管显示全为 0。
- (3) AD_DigData_In: 数据输入信号。与 AD 芯片的 DATA OUT 引脚相连，传送串行数据。
- (4) AD_CSn、AD_Clk、AD_Address: 控制输出信号。分别与 AD 芯片的 CS、I/O CLOCK、ADDRESS 相连。
- (5) Data_Out: 数据输出信号，共有 10 位总线。将 AD_DigData_In 中的串行数据存储为并行数据。
- (6) CS_Valid_Sig: 片选有效标志位。当 AD_CSn 信号由高变低时，CS_Valid_Sig 信号由高变低；当距 AD_CSn 信号的下降沿一定时间后，CS_Valid_Sig 再置高，表明此时 AD_CSn 信号的下降沿有效。
- (6) Digitron_Out、DigitronCS_Out: 数码管的输出显示信号及片选信号。

四、程序设计

图 3.6 是截取自底层模块 AD_Data 的部分代码,理解了此段程序则可以相应理解该模块的所有程序:

```

26      reg [7:0]Cnt1;
27      reg Cnt1_en;
28
29      always @ ( posedge CLK or negedge RSTn )
30      begin
31          if( !RSTn )
32          begin
33              Cnt1 <= 8'd0;
34          end
35          else if( Cnt1 == 8'd12 )
36          begin
37              Cnt1 <= 8'b0;
38          end
39          else if( Cnt1_en )
40          begin
41              Cnt1 <= Cnt1 + 1'b1;
42          end
43          else
44              Cnt1 <= 8'b0;
45      end

```

```

148      'd3: if( Cnt2 == 8'd71 )
149      begin
150          AD_Clk_r <= 1'b0;
151          CS_Valid_Sig <= 1'b1;
152          Cnt1_en <= 1'b1;
153          Cnt3_en <= 1'b1;
154          i <= i + 1'b1;
155      end

```

```

162      'd4: if( Cnt3 == 8'd6 )
163      begin
164          AD_Address_r <= Addr[3];
165          i <= i ;
166      end
167      else if( Cnt1 == 8'd12 )
168      begin
169          AD_Clk_r <= 1'b1;
170          Data_Out_r[9] <= AD_DigData_In;
171          i <= i + 1'b1;
172      end

```

图 3.6 AD_Data 代码

26-45: 这是一种计数器的改变,加了一个 Cnt1_en,目的是制造精确的延时时间。当 Cnt1_en 为 0 时,Cnt1 不计数;当 Cnt1_en 为 1 时,Cnt1 开始计数。此计数器用于产生频率符合要求的 AD_Clk 信号,系统基准时钟 CLK 的周期为 20ns,

则得到的 AD_Clk 信号的周期为 520ns。计算方法为 $2 \times 13 \times 20\text{ns} = 520\text{ns}$ 。

148-155: 计数器 Cnt2 用于产生准确的延时时间 $t_{su(cs)}$ 。当 Cnt2 为 71 时, 延时了 1440ns ($72 \times 20\text{ns}$), 此时 AD_CS_n 信号的下降沿有效, 151 行将 CS_Valid_Sig 信号置为 1; 152 行打开控制 AD_Clk 信号的计数器 Cnt1; 152 行打开控制 AD_Address 信号的计数器 Cnt3; 154 进入下一个状态。

164: Addr 信号中存储了要送外 AD 芯片的 ADDRESS 引脚的数值, AD_Address 信号与 ADDRESS 引脚相连, 164 行控制以串行方式向 ADDRESS 引脚送 Addr 信号的 MSB。

167-172: AD 芯片在 CS 的下降沿时将上次转换的数据的 MSB 移到 DATA OUT 引脚中, AD_DigData_In 信号与 DATA OUT 引脚相连。所以, 控制 FPGA 在时钟信号 AD_Clk 的第一个上升沿处, 将 MSB 存入 Data_Out[9] 中。

五、FPGA 管脚配置

图 3.7 是使用 Quartus 生成的 Pin Planner 管脚图, CLK 信号、Digitron_Out 信号、DigitronCS_Out 信号的配置方式与之前实验相同; RSTn 信号接 SW0; AD_EOC 信号、AD_DigData_In 信号、AD_CS_n 信号、AD_Clk 信号和 AD_Address 信号分别输送到开发板上方的 I/O 通道中, 继而与 AD 集成板的相应引脚相连; CS_Valid_Sig 信号和 Data_Out 信号用于在 SignalTap 中做时序分析使用, 不需连接到外部引脚。相应引脚连接信息已经在第一篇的 1.3 节中做过详细介绍, 这里就不再赘述。

out	AD_Address	Output	PIN_A7	out	DigitronCS_Out[5]	Output	PIN_D12
out	AD_CS _n	Output	PIN_A9	out	DigitronCS_Out[4]	Output	PIN_C11
out	AD_Clk	Output	PIN_B7	out	DigitronCS_Out[3]	Output	PIN_F11
in	AD_DigData_In	Input	PIN_B8	out	DigitronCS_Out[2]	Output	PIN_G11
in	AD_EOC	Input	PIN_A8	out	DigitronCS_Out[1]	Output	PIN_D14
in	CLK	Input	PIN_E1	out	DigitronCS_Out[0]	Output	PIN_C14
out	CS_Valid_Sig	Output		out	Digitron_Out[7]	Output	PIN_C9
out	Data_Out[9]	Output		out	Digitron_Out[6]	Output	PIN_E9
out	Data_Out[8]	Output		out	Digitron_Out[5]	Output	PIN_D8
out	Data_Out[7]	Output		out	Digitron_Out[4]	Output	PIN_C8
out	Data_Out[6]	Output		out	Digitron_Out[3]	Output	PIN_D11
out	Data_Out[5]	Output		out	Digitron_Out[2]	Output	PIN_E8
out	Data_Out[4]	Output		out	Digitron_Out[1]	Output	PIN_E10
out	Data_Out[3]	Output		out	Digitron_Out[0]	Output	PIN_D9
out	Data_Out[2]	Output		in	RSTn	Input	PIN_R16
out	Data_Out[1]	Output					
out	Data_Out[0]	Output					

图 3.7 AD 采样系统 Pin Planner

六、实验结果

在验证实验时, 使用实验室电源向 AD 集成板提供 “5V” 电压, 则式 (3.5) 中

的 $V_{\max}$ 的值为 5V，AD 集成板、电源和 MINI_FPGA 开发板需要“共地”，使用口袋仪器的“信号源”向 AD 芯片送入正弦信号，选择模拟输入端 AIN5。

(1) 图 3.8 是使用口袋仪器产生的正弦信号的波形及参数说明。

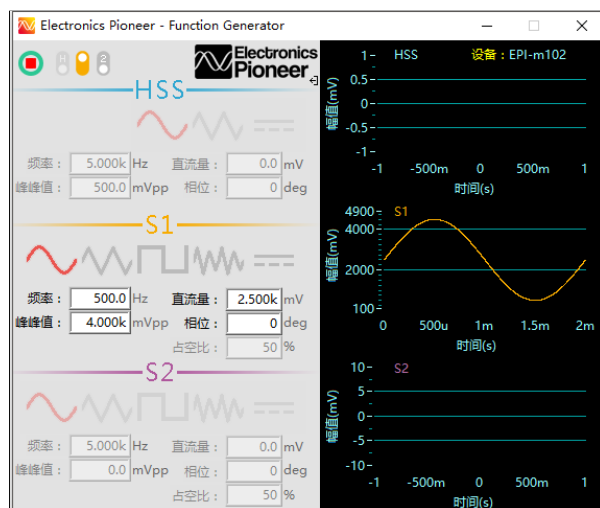


图 3.8 AD 转换前的正弦信号

(2) 图 3.9 是使用 SignalTap 观察到的经 AD 转换后的波形。新建一个 SignalTap 文件，命名为“stp1”，SignalTap 的工作时钟选择 100KHz，则每个采样点的时间间隔为 $10\mu s$ ，取“2k”采样点，使用“Unsigned Line Chart”类型显示 Data_Out[9:0]。在波形的 2 个峰值处插入时间条，左边的为设置为“主时间条”。如图所示，3 个周期的时间长度为 6.01ms，则计算得到该正弦波的频率约为 499.2Hz，与输入的模拟信号吻合；峰值处的数据为 911，根据式 (3.5) 计算得到的电压的峰值为 4.45V，略小于输入信号的峰值 4.5V，这是仪器本身的原因造成的。

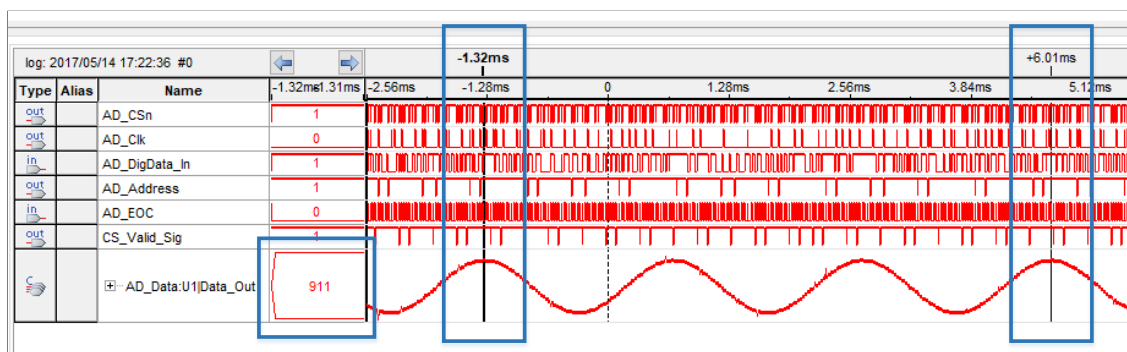


图 3.9 AD 转换后的波形图

七、思考与拓展

(1) 使用 SignalTap 观察波形的最低点的数据值，并根据式 (3.5) 计算相应的模拟电压值，与口袋仪器所产生的正弦信号进行比较。

(2) 参考 AD 芯片的 datasheet，修改程序中的 addr 信号值，将 AD 集成板的输入地址更改为内部自测试电压的选择地址，观察并验证数码管所显示的数值。

(3) 本节实验中 SignalTap 的工作时钟选择 100KHz，关于使用 IP 核生成分频模块

实验 3.2 时序分析实验

一、实验设计目标

在完成了实验 3.1 的基础上，使用 SignalTap 对实验 3.1 节中的信号进行时序分析，将分析结果与图 3.1 的 AD 芯片工作时序图进行对比。

二、实验过程

本实验是在实验 3.1 的基础开展的分析性实验，使用实验室的电源给 AD 集成板提供“5V”电压，式 (3.5) 中的 $V_{\max}$ 的值则为 5V，AD 集成板、电源和 MINI_FPGA 开发板需要“共地”，使用口袋仪器的“信号源”向 AD 芯片送入方波信号，选择模拟输入端 AIN5。图 3.30 是使用口袋仪器产生的方波信号的波形及参数说明。

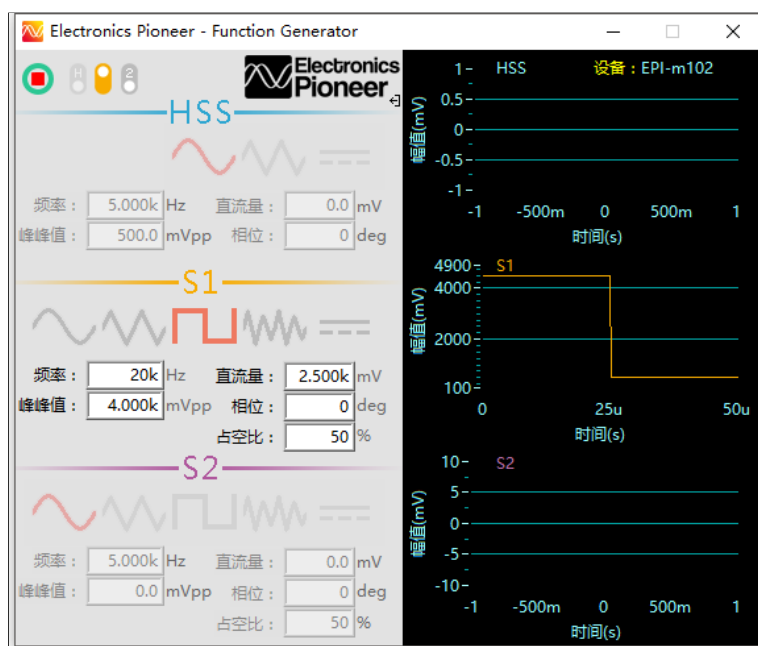


图 3.30 方波信号参数说明

新建一个 SignalTap 文件，命名为“stp2”，工作时钟选择 50MHz 的系统基准时钟，则每个采样点的时间间隔为 20ns，取“8k”采样点。相关设置如图 3.31 所示。

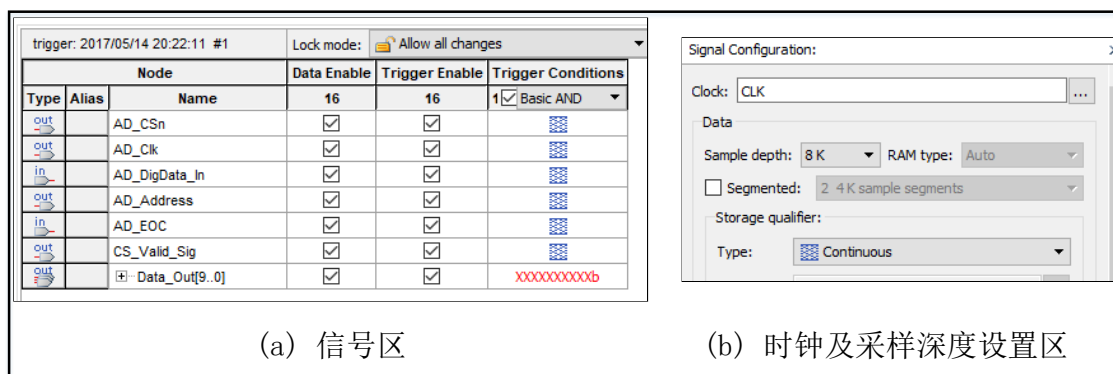


图 3.31 SignalTap 文件的相关设置

三、实验结果

(1) 时序分析仪的工作频率为 50MHz，口袋仪器产生的方波频率为 20KHz，所以，方波信号的一个周期内采集 2500 个点。因资源限制，SignalTap 的采样深度设置为 8k，则屏幕上共能显示 AD 采样到的 3.2 个周期的方波信号。图 3.32 为其

部分截图。程序中，仅在 AD_Clk 信号的上升沿时将 AD_DigData_In 的当前值赋给 Data_Out 信号，其他时刻 Data_Out 的值保持不变，所以 Data_Out 信号的波形仅在 AD_Clk 有脉冲时出现阶梯状的变化。



图 3.32 整体图

(2) 图 3.33 是 AD_CS 信号为低的一段时刻的时序放大图，通过此图可以简单直观地感受 AD 芯片的控制信号之间的时序关系。以图 3.33 为参考时刻，图 3.34 截取了从 AD_CS 信号的下降沿到第三个 AD_Clk 信号的上升沿的时序图，并在关键时刻插上时间条，命名为①~⑦；图 3.35 截取了 AD_CS 信号为低的一段时刻的时序图，并在关键时刻插上时间条，命名为⑧；图 3.36 截取了位于 AD_CS 信号的两个下降沿内的一段时刻的时序图，并在关键时刻插上时间条，命名为⑨、⑩、“11”。



图 3.33 时序分析图 1

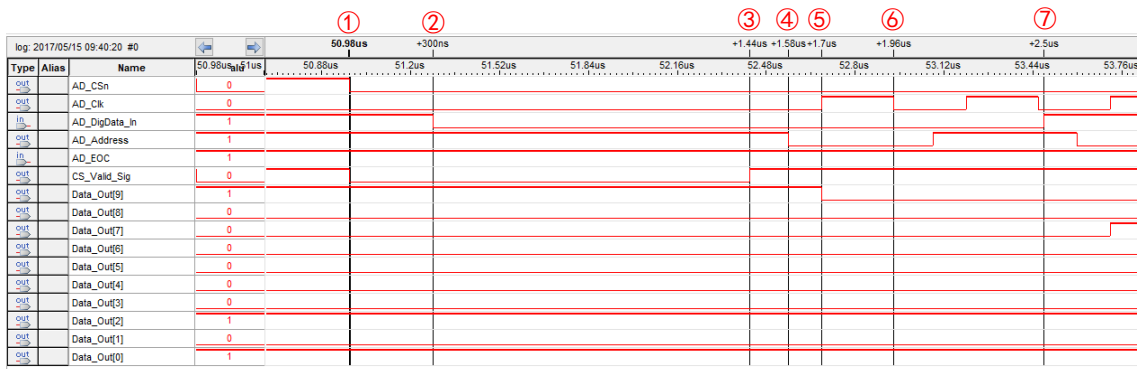


图 3.34 时序分析图 2



图 3.35 时序分析图 3

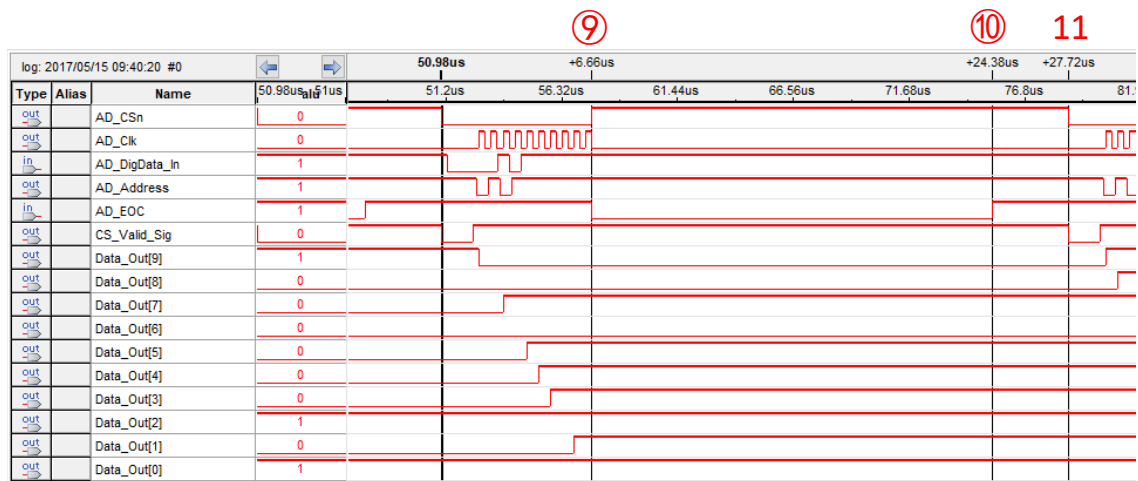


图 3.36 时序分析图 4

(3) 为方便观察, 将图 3.34~3.36 做一定处理, 得到图 3.37。下面对时间条 ①~⑩、“11” 进行分析。

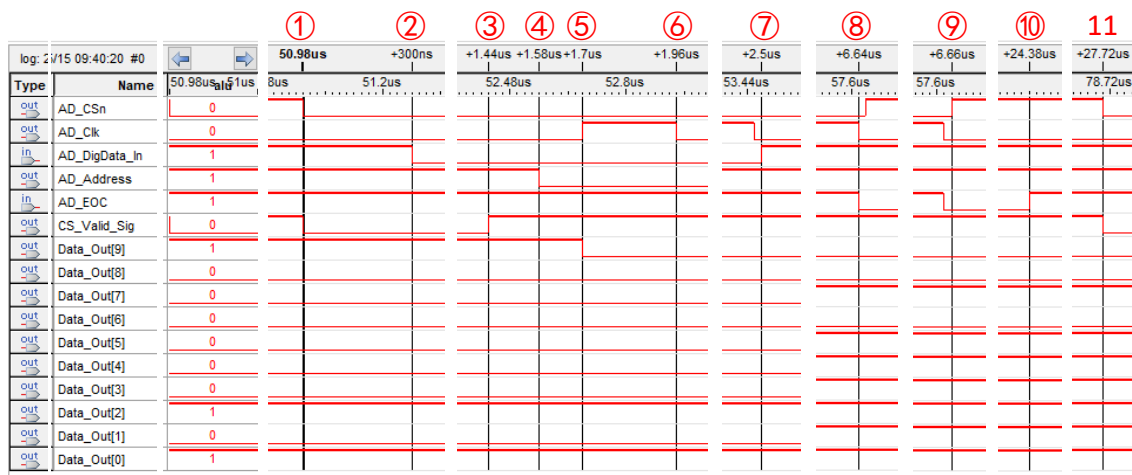


图 3.37 时序分析图 5

“①”是“主时间条”。在此刻，AD_CSn 信号由高至低，AD_Clk、AD_DigData_In、AD_Address 信号均不响应，分别保持为“0、0、1”。

“②”距离“①”延时 300ns。对于 AD 芯片来说，DATA OUT 引脚在 CS 为高时处于高阻抗状态，而当 CS 为低时处于激活状态。一旦 CS 的下降沿有效（延时一定时间），按照前一次转换结果的 MSB 值将 DATA OUT 从高阻抗状态转变成相应的逻辑电平。所以，此刻 AD_DigData_In 信号由低变高，表明上一次 AD 转换结果的 MSB 为高电平。

“③”距离“①”延时 $1.44\mu s$ 。在此刻 CS_Valid_Sig 信号由低变高，表明距离 AD_CSn 信号的下降沿已有 1440ns ($72 \times 20ns$)，符合设置的 $t_{su(cs)}$ ，见式 (3.2)。

“④”比“⑤”超前 120ns。本实验选择模拟通道 AIN5，所以送给 AD 芯片的 ADDRESS 引脚的值为“0101”。在此刻 AD_Address 信号由高变低，即将“0101”的最高有效位“0”送给 ADDRESS 引脚。且 120ns 符合设置的 $t_{su(A)}$ ，见式 (3.3)。

“⑤”距离“③”延时 260ns。在此刻 AD_Clk 信号由低变高，即将第一个时钟上升沿送给 I/O CLOCK 引脚，且 260ns 符合设置的 $T_{(I/O\ CLOCK)}$ ，见式 (3.1)。同时，在此刻 Data_Out[9] 信号由低至高，即在第一个时钟的上升沿将 AD_DigData_In 信号的当前值 (MSB) 赋给 Data_Out[9]。

“⑥”距离“⑤”延时 260ns。在此刻 AD_Clk 信号由高变低，即将第一个时钟下降沿送给 I/O CLOCK 引脚。且 260ns 符合设置的 $T_{(I/O\ CLOCK)}$ ，见式 (3.1)。

“⑦”是在 AD_Clk 信号的第二个下降沿后延时 20ns (时序图的顶栏的 1 小格) 的时间条。对于 AD 芯片来说，上一次转换结果的低 9 位在第 1~9 个 I/O CLOCK 信

号的下降沿时被移至 DATA_OUT 引脚。所以,理论上来说,AD_DigData_In 信号应在“⑦”时刻条前面的 AD_Clk 下降沿时由高变低,但图 3.37 中延时了 20ns,这是由 SignalTap 的采样时钟的离散性造成的。在实际中,DATA_OUT 引脚的输出信号并非是延时了 20ns 才发生变化,但 SignalTap 所能采集到的最近的点就是延时了 20ns 的时刻。

“⑧”距离“①”延时 $6.64\mu s$ 。此刻为第十个 AD_Clk 信号的下降沿,根据式 (3.1) 和式 (3.3),计算“①”与“⑧”之间的延时:

$$t_{①-⑧}=1.44\mu s+10\times 520ns=6.64\mu s \quad (3.6)$$

计算结果与时序图吻合。此外,对于 AD 芯片来说,第 10 个 I/O CLOCK 信号的下降沿时,电路开始 AD 转换,并将 EOC (转换结束端)拉低直到转换完成,所以,在此刻 AD_EOC 输入信号由高变低。

“⑨”距离“⑧”延时 20ns。这与 AD_Data 模块中 239-251 行代码相吻合,当 AD_Clk 信号的第十个下降沿到来后,AD_CS_n 信号将在下一个系统基准时钟 CLK 的上升沿时由低变高。且 AD_CS_n 信号的上升沿将禁止 AD 芯片的三个输入端。

“⑩”距离“①”延时 $17.74\mu s$ 。此时,AD_EOC 信号由低至高,表明此次 AD 转换完成。 $17.74\mu s$ 是此次转换过程实际所用时间。

“11”距离“⑨”延时 $21.06\mu s$ 。此刻为下一次 AD 采样过程中 AD_CS_n 信号的下降沿。根据式 (3.4), $t_{oonv}=21\mu s$,多出的 60ns 实际上也是符合代码的,延时过程与“⑨”“⑧”之间的延时过程相似。

“11”距离“①”延时 $27.72\mu s$,此即为包括采集与转换在内的整个 AD 采样过程的周期。根据式 (3.1)、(3.3)、(3.4),计算一次 AD 采样的周期:

$$t=1.44\mu s+10\times 520ns+21\mu s=27.64\mu s \quad (3.7)$$

多出的 80ns 是符合代码的,前面已经介绍过。

综上,实际 AD 采样的频率为 36.075KHz,时序与设计吻合。

实验四 UART 串行通信

实验 4.1 UART 串口接收实验

一、实验设计目标

(1) 在 FPGA 中实现串口协议, 通过 MINI_FPGA 开发板上的 “UART2USB” 口接收从计算机发来的数据。

二、实验设计思路

UART 串口是一种类似于 USB、VGA 的接口, 有固定的引脚和通信协议。使用 FPGA 实现串口通信, 可分为 “计算机发送数据给 FPGA” 和 “FPGA 发送数据给计算机” 两部分。本节为串口接收实验, 使用 FPGA 接收从计算机发来的数据。

进行串口接收实验首先需要了解串口的接收时序, 图 4.1 为接收一帧数据时的

简单示意图。串口接收只使用一条数据线 RX，特点如下：

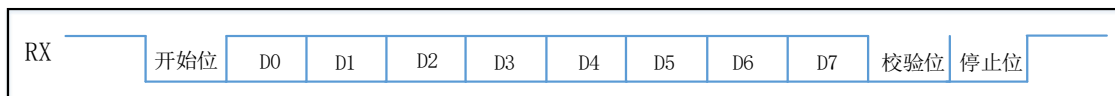


图 4.1 串口接收时序示意图

（1）当 RX 产生了一个下降沿后才开始一帧数据的接收，接收到的第一位为起始位。

（2）一帧周期内有 11 个时钟，第一个时钟是起始位，2~9 个时钟为数据位，并且数据是从低位到高位接收的。

（3）串口的接收时钟频率取决于它的波特率，常用的波特率是有几个固定值的。本实验的波特率选用 9600bps，表明 1 秒内传送 9600 位数据，则一位数据的周期为：

$$T = \frac{1}{\text{波特率}} \quad (4.1)$$

可以看出，进行串口接收实验的关键在于准确读取 RX 上的数据，需做到以下两点：①判断什么时候 RX 开始接收一帧数据，即判断起始位；②判断什么时候读取 D0~D7 上的数据，即判断数据位。

为实现第①点，借鉴实验一 PWM 实验中的“消抖程序”的思想检测 RX 线上的信号，在 RX 信号产生了一个下降沿后才启动一帧数据的接收；并且，为保证每一次检测到的都是起始位，必须确定在一次接收中经过了 11 个时钟后才开始下一帧数据的接收，这可以通过编程实现。

对于第②点，为保证数据的准确，应在数据位的正中间读取数据。图 4.2 为读取数据的简单示意图，当 BPS_CLK 信号为高时，读取 RX 线上的数据，BPS_CLK 信号的高电平出现的频率应为 9600Hz。设计一个计数器 Count_BPS，将 50MHz 的系统基准时钟进行分频，分频数为：

$$50000000 \div 9600 - 1 = 5207 \quad (4.2)$$

当 Count_BPS 计数到 2604 时，令 BPS_CLK 信号为高电平，其他时刻都置低，这样便能在一个接收时钟的正中间读取数据。

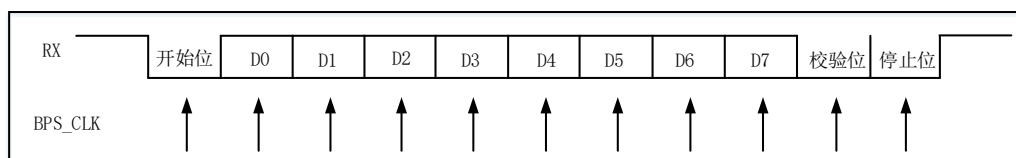


图 4.2 数据读取示意图

三、功能模块图与输入输出引脚说明

图 4.3 是本实验设计的串口接收系统的示意图，包含开始位检测模块 U1、波特率控制模块 U2、接收控制模块 U3 和 LED 显示模块 U4。

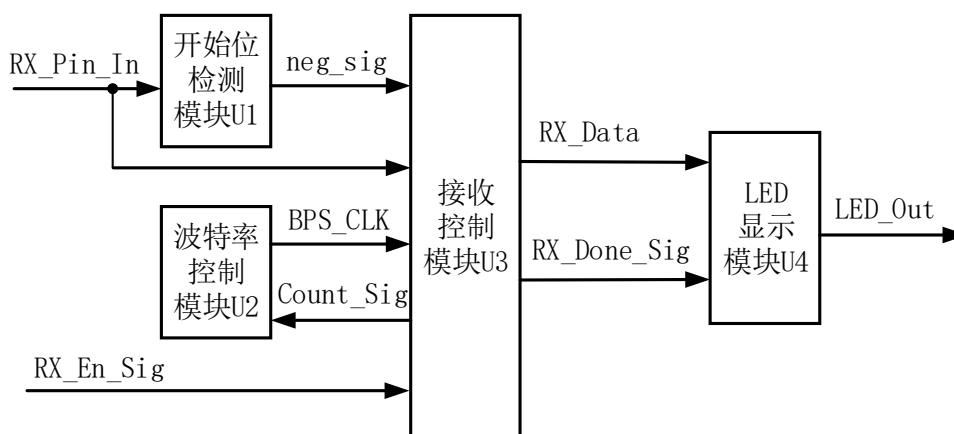


图 4.3 串口接收系统示意图

开始位检测模块用于检测一帧数据的开始位，功能如前所述。RX_In 与串口的 RX 线相连，在没有数据传输时，RX 线上为高电平；neg_sig 为“下降沿标志位”，当检测到 RX_In 的下降沿时，neg_sig 信号被置为 1，其他时刻为 0。

波特率控制模块内包含一个计数器 Count_BPS，功能如前文所述。BPS_CLK 为使能时钟信号，控制何时读取数据。Count_Sig 为计数标志位，当一帧数据开始时，Count_Sig 被置为 1，此时计数器 Count_BPS 才能开始计数；当一帧数据接收完毕后，Count_Sig 被置为 0，计数器 Count_BPS 停止工作。

接收控制模块从 RX_En_Sig 信号上的数据帧中读取数据位，将串行数据转换为并行 8 位数据并存储于 RX_Data 中。当一帧数据接收完毕时，“接收完成标志位”RX_Done_Sig 被置为 1。

LED 显示模块将接收到的八位数据输送到 LED 灯上。

在 FPGA 中实现此系统，工程包含顶层模块 DA 与底层模块 detect_module、

rx_bps_module、rx_control_module、LED_display_module，底层模块从左至右依次对应于图 4.3 中的 U1~U4。图 4.4 是使用 Quartus 生成的顶层文件的 BSF 图。下面介绍一下顶层模块输入输出引脚的功能：

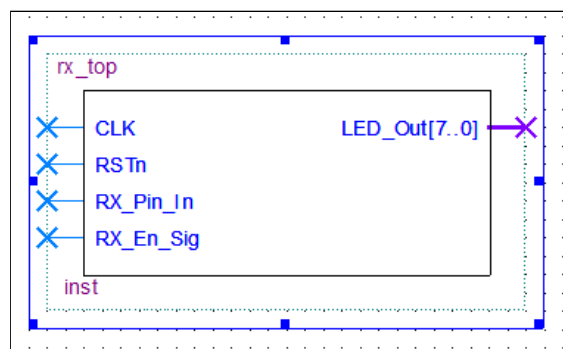


图 4.4 串口接收实验顶层文件 BSF 图

- (1) CLK: 50MHz 的时钟信号输入。
- (2) RSTn: 复位输入信号，低电平有效。当 RSTn 信号为 0 时，LED_Out 全为 0。
- (3) RX_In: 接收输入信号。连接到串口的 RX 线。
- (4) LED_Out: 输送到 LED 灯，共有八位总线。LED_Out [7: 0] 分别存储了一帧数据的 MSB 至 LSB 位。

四、程序设计

串口接收程序中的大部分代码功能已在前面的实验中使用过，容易理解，此处仅简单说明。

- (1) 图 4.5 是截取自底层模块 rx_control_module 的部分代码：

49	4'd2, 4'd3, 4'd4, 4'd5, 4'd6, 4'd7, 4'd8, 4'd9 :
50	if(BPS_CLK)
51	begin
52	State <= State + 1'b1;
53	rData[State - 2] <= RX_Pin_In;
54	end

图 4.5 rx_control_module 核心代码

49-54: 数据位存储。计算机发送数据时，是先发（最低有效位）LSB 的，53 行代码依次将 RX_In 信号的值存储于 rData[0]~rData[7]中，rData 的值最终将赋

给 RX_Data。

(2) 图 4.6 是根据 rx_control_module 模块中的代码并结合图 4.2 画出的过程示意图, State 信号内寄存当前状态值, 最后的“12→13→0”的状态改变是立刻发生的, 与前面不同, 请结合程序理解。时序设计满足串口通信协议。

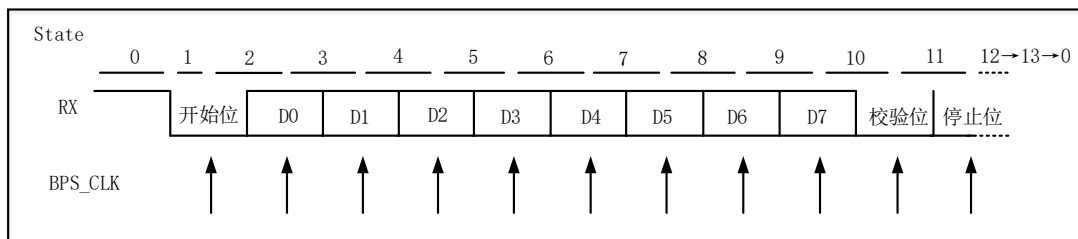


图 4.6 接收过程示意图

五、FPGA 管脚配置

图 4.7 是使用 Quartus 生成的 Pin Planner 管脚图, CLK 信号与开发板上的 50MHz 的晶振时钟相连; RSTn 信号与 SW0 相连; LED_Out[7: 0]分别与 LED7~LED0 相连; RX_En_Sig 信号与 SW1 相连; RX_In 信号与 F14 管脚相连, 串口接收引脚在第一篇的 1.3.7 节中已经做过介绍。其他引脚连接信息已经在第一篇的 1.3 节中做过详细介绍, 这里就不再赘述。

Node Name	Direction	Location
in CLK	Input	PIN_E1
out LED_Out[7]	Output	PIN_C3
out LED_Out[6]	Output	PIN_A2
out LED_Out[5]	Output	PIN_B3
out LED_Out[4]	Output	PIN_A3
out LED_Out[3]	Output	PIN_B4
out LED_Out[2]	Output	PIN_A4
out LED_Out[1]	Output	PIN_B5
out LED_Out[0]	Output	PIN_A5
in RSTn	Input	PIN_R16
in RX_En_Sig	Input	PIN_P15
in RX_Pin_In	Input	PIN_F14

图 4.7 串口接收实验 Pin Planner

六、实验结果

串口通信时需要通过“串口接口”传输数据, 但 MINI_FPGA 开发板内部已经集

成了 USB 转串口的功能，所以，直接使用 2 根 USB 数据线分别将开发板上的 JTAG 接口和 UART2USB 接口与计算机相连，在计算机上安装一个“串口调试助手”，即可进行实验。

图 4.8 是使用“串口调试助手”发送数据“AB”的参数界面，实验前需下载串口的驱动程序并安装，若出现图 4.8 中红框①的内容，则表明驱动安装成功。波特率选择 9600，选择 8 位数据位，1 位校验位和 1 位停止位。计算机向 FPGA 发送“AB”，选择“显示发送”，在显示区可以看到发送的“AB”。

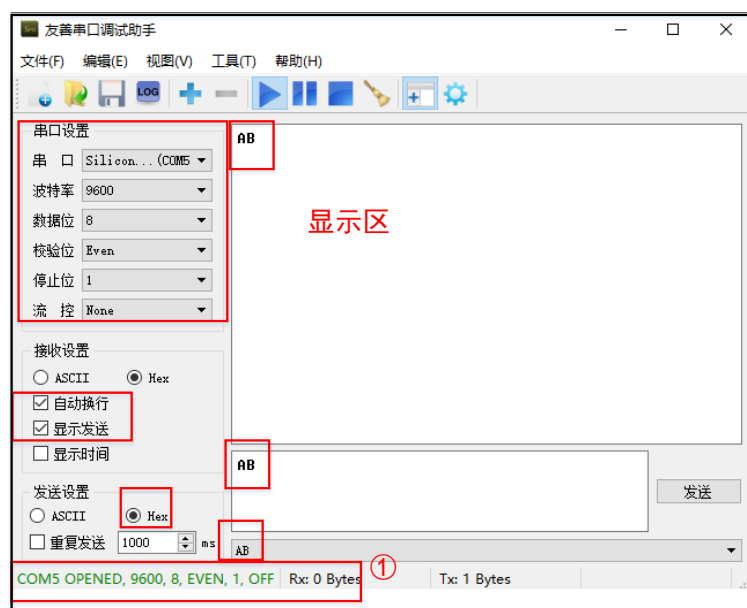


图 4.8 “串口助手”参数设置

图 4.9 是使用 FPGA 接收“AB”数据的实验结果。LED7~LED0 的状态依次为“亮、灭、亮、灭、亮、灭、亮、亮”，即 RX_Data[7:0]的值为“10101011”，换成 16 进制为“AB”，接收到的数据与所发送的相同。

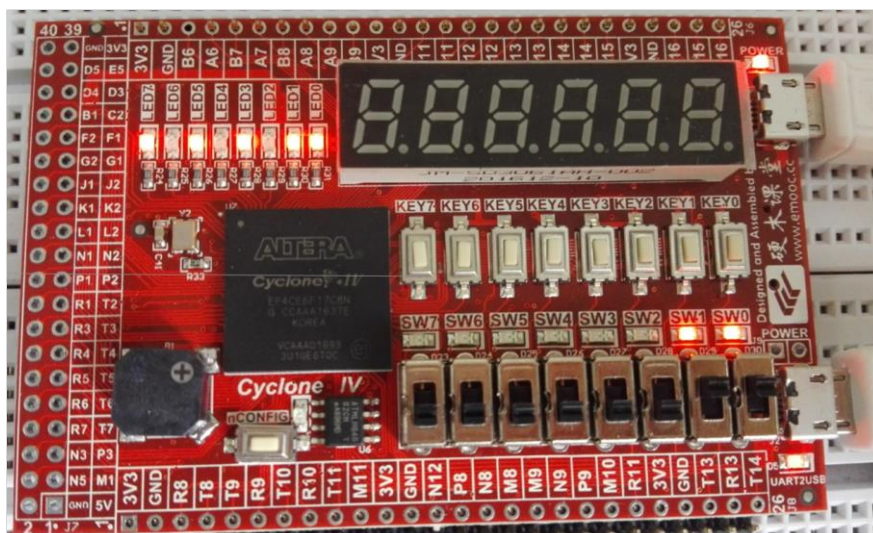


图 4.9 串口接收实验结果

七、思考与拓展

(1) 图 4.6 中, State 的值在发生“12→13→0”的变化时, 是随着基准时钟 CLK 的上升沿而变化的, 也就是说, 停止位的持续时间只有半个周期, 但依然得到了正确的实验结果, 请结合程序从“实际有用数据”的角度解释其原因。

(3) 在使用“串口调试助手”时, 若选择“无校验位”, 实验现象是怎样的? 请结合程序及“一帧数据”的格式解释其原因。

实验 4.2 UART 串口发送实验

一、实验设计目标

(1) 在 FPGA 中实现串口协议, 通过 MINI_FPGA 开发板上的“UART2USB”口向

计算机发送数据。

二、实验设计思路

在学习了实验 4.1 中 UART 串口接收实验的基础上，理解 UART 串口发送实验就较为简单了。本实验设计使用 FPGA 向计算机重复发送六组数据“0A、0B、0C、0D、0E、0F”，每隔 0.5 秒发送一组，在计算机上使用“友善串口助手”接收数据并验证是否正确。

图 4.10 为发送一帧数据时的简单示意图。可以看出，串口的发送时序和接收时序是一样的，而串口接收和发送的不同点就在于，接收是把数据从时序中提取出来，发送是把数据添加到时序中去。

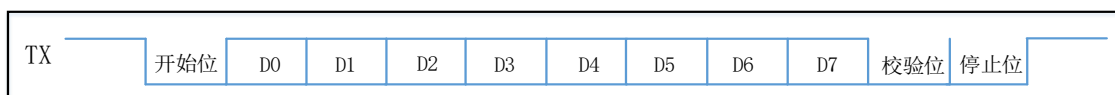


图 4.10 串口发送时序示意图

同串口接收一样，进行串口发送实验首先需要产生一个 BPS_CLK 信号，然后根据 BPS_CLK 实现 TX 的时序即可。以下为串口发送的简单步骤：

- (1) 由 TX 产生一个下降沿变化，作为一帧数据的开始，“开始位”时，TX 信号置为 0。
- (2) 把要发送的数据按照先低位后高位的顺序一位一位送给 TX。
- (3) “校验位”和“停止位”无操作，TX 信号置为 1。
- (4) 一帧数据发送完毕，等到下一次发送开始时，回到步骤 (1)。

图 4.11 是本实验设计的串口发送系统的示意图，包含数据控制模块 U1、波特率控制模块 U2 和发送控制模块 U3。

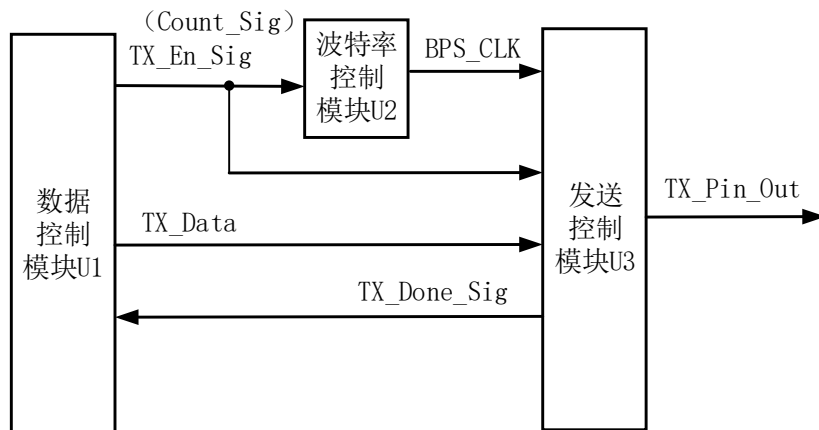


图 4.11 串口发送系统示意图

数据控制模块内有一个计数器 Count，控制系统每隔 0.5 秒发送一次数据。TX_Data 为要发送的 8 位并行数据；TX_En_Sig 为发送使能标志位，它的上升沿将驱动模块 U2 开始产生 BPS_CLK 信号，同时驱动 U3 模块开始发送数据；TX_Done_Sig 为发送完成标志位，当一帧数据发送完毕后，TX_Done_Sig 信号被置为 1，这将驱动 TX_En_Sig 信号变为 0 直到开始发送下一组数据。

波特率控制模块与实验 4.1 节串口接收实验中模块功能相同，U2 模块的输入信号 Count_Sig 即为 U1 模块的输出信号 TX_En_Sig。

发送控制模块将 TX_Data 的值一位一位（LSB 先导）的送给 TX_Out 信号，当一帧数据发送完毕时，TX_Done_Sig 被置为 1。

三、功能模块图与输入输出引脚说明

在 FPGA 中实现此系统，顶层模块为 tx_top，底层模块 data_control_module、tx_bps_module、tx_control_module 从左至右依次对应于图 4.11 中的 U1~U3。图 4.12 是使用 Quartus 生成的顶层文件的 BSF 图。下面介绍一下顶层模块输入输出引脚的功能：

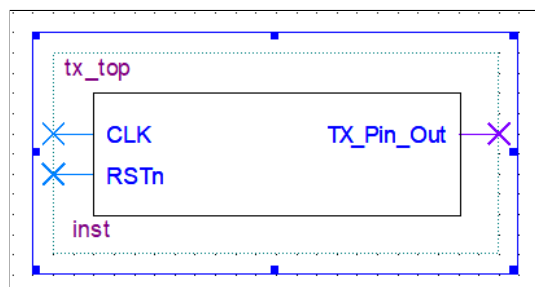


图 4.12 串口发送实验顶层文件 BSF 图

- (1) CLK: 50MHz 的时钟信号输入。
- (2) RSTn: 复位输入信号, 低电平有效。当 RSTn 信号为 0 时, TX_Out 恒为 1。
- (3) TX_In: 发送输出信号。连接到串口的 TX 线。

四、程序设计

串口发送程序中的大部分代码功能都已在实验 4.1 节串口接收实验中使用过, 容易理解, 此处仅简单说明。图 4.13 是截取自底层模块 data_control_module 的部分代码:

```

28      else if( TX_Done_Sig == 1 )
29      begin
30          isTX <= 0;
31          Count <= Count + 1'b1;
32      end
33      else if ( Count == T05S - 1 )
34      begin
35          isTX <= 1'b1;
36          Count <= 0;
37          if( i == 'd5 )
38              i <= 0;
39          else
40              i <= i + 1'b1;
41      end

```

图 4.13 data_control_module 核心代码

28-32: isTX 是一个寄存器型信号, 最终将赋给 TX_En_Sig。当一帧数据发送完毕后, 将 isTX 信号置为 0, 禁止系统发送数据。

33-35: 分频数 T05S 的值为 25000000, 当距离上一次发送数据过去 0.5 秒后, 将 isTX 信号置为 1, 开始新一轮数据的发送。

37-40: i 的取值范围是 0~5, 用于从六组数据中选择一组发送。

五、FPGA 管脚配置

图 4.14 是使用 Quartus 生成的 Pin Planner 管脚图, CLK 信号与开发板上的 50MHz 的晶振时钟相连; RSTn 信号与 SW0 相连; RX_In 信号与 F13 管脚相连, 串口发送引脚在第一篇的 1.3.7 节中已经做过介绍, 这里就不再赘述。

	Node Name	Direction	Location
in	CLK	Input	PIN_E1
in	RSTn	Input	PIN_R16
out	TX_Pin_Out	Output	PIN_F13

图 4.14 串口发送实验 Pin Planner

六、实验结果

类似于串口接收实验，直接使用 2 根 USB 数据线分别将开发板上的 JTAG 接口和 UART2USB 接口与计算机相连，在计算机上安装一个“串口调试助手”，即可进行实验。

图 4.15 是使用“串口调试助手”接收数据“0A、0B、0C、0D、0E、0F”的结果。波特率选择 9600，选择 8 位数据位，1 位校验位和 1 位停止位。选择“显示时间”，在显示区可以看到接收到数据的时间。实验结果与程序设计相符。

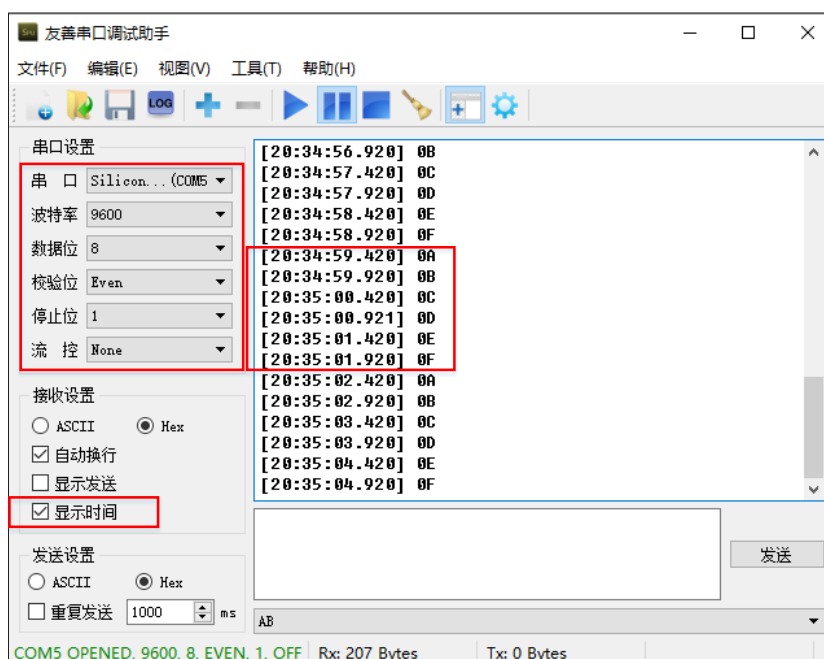


图 4.15 串口发送实验结果

七、思考与拓展

(1) 仔细阅读 tx_control_module 模块的程序，并参考图 4.6 绘出本实验中发送过程的简单示意图。

(2) 更改程序，实现发送完一组数据后立刻发送下一组数据的功能，并使用

“串口助手”验证实验。

八、实验小结

实验 4.1 与实验 4.2 节分别描述了串口接收实验与串口发送实验,但仍存在部分限制。串口接收实验使用 LED 灯来显示接收到的数据,当计算机连续发送多个数据帧时,速度是极快的,在 LED 灯上只能显示最后一帧数据结果。究其根本,这里缺少了一个寄存器用于存储接收到的数据,并以某种肉眼可见的方式显示出来。

本实验体系在实验 4.1 和实验 4.2 节程序的基础上,建立了一个工程,可接收到计算机发来的数据并返还给计算机。该实验使用了 Altera 自带的 IP 核,建立了两个 FIFO 存储器,将 FPGA 接收到的当前数据 B 存于接收存储器“rx_fifo”中,当发送存储器“tx_fifo”中的上一个数据 A 发送完成后,再将 B 移入“tx_fifo”中,同时“rx_fifo”接收下一个数据 C。请同学们在学习了实验 4.1 和实验 4.2 节的基础上,自行理解程序,学习 FIFO 存储器的使用。

实验五 FIR 滤波器

一、实验设计目标

- (2) 在 FPGA 中实现 FIR 数字滤波器功能。
- (2) 通过本实验学习利用 Altera 中的 IP 核来建立 FIR 数字滤波器的方法。

二、实验设计思路

数字滤波技术用于从带有噪声的信号中提取所需的有效信号，并抑制不需要的噪声信号。数字滤波器的输入输出均为数字信号。有限冲激响应（FIR）数字滤波器是一个线性时不变系统，它在时域中的输入-输出关系为：

$$y(n) = x(n)h(n) = \sum_{k=0}^N x(k)h(n-k) \quad (5.1)$$

可以看出，对于 FIR 数字滤波器，其基本结构是一个分节的延时线，每一节的输出与相应系数加权累加，便得到滤波器的输出结果。

本实验设计使用 DDS 产生 2 个频率不同的正弦信号，叠加后经 FIR 低通滤波器滤波，将滤波前与滤波后的信号分别送往 DA 芯片，观察并对比得到的模拟信号的频谱图。图 5.1 为本次实验设计的 FIR 滤波系统的示意图，包含 DDS 模块、低通滤波器模块和 DA 转换模块。

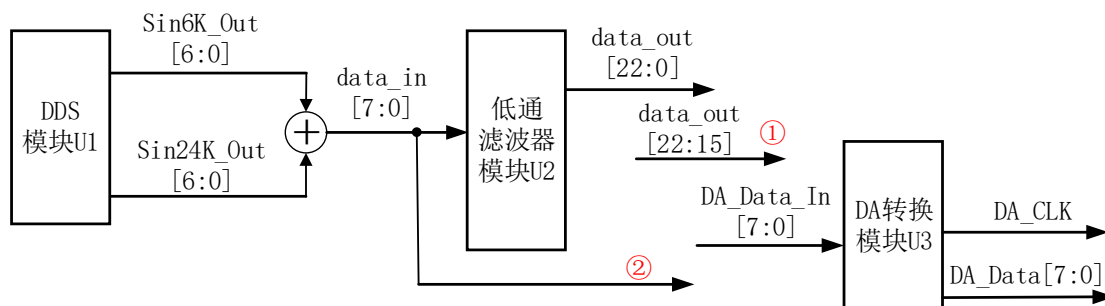


图 5.1 FIR 滤波器系统示意图

DDS 模块用于产生 2 个频率分别为 6.103KHz 和 24.414KHz 的正弦信号，工作原理与实验 2.2 中 DDS 信号的产生方法相同，图 5.2 与图 5.3 分别为产生这 2 个信号的 ROM 表的参数设置图。将 Sin6K_Out 信号和 Sin24K_Out 信号相加后作为低

滤波器的输入信号 data_in，需要注意的是，Sin6K_Out 信号和 Sin24K_Out 信号均为 7 位的，相加后的 data_in 信号为 8 位。

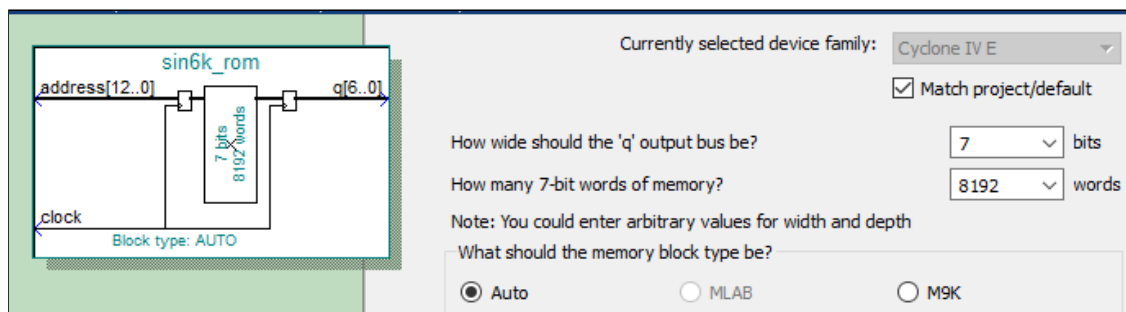


图 5.2 6.103KHz 正弦信号 ROM 表的参数设置

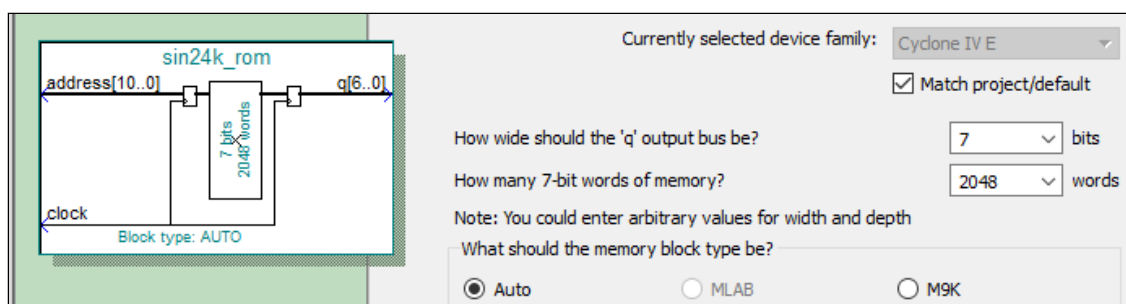


图 5.3 24.414KHz 正弦信号 ROM 表的参数设置

低通滤波器模块对输入的 data_in 信号进行算法处理，得到滤波后的信号 data_out。该 FIR 低通滤波器是使用 Quartus 自带的 IP 核来设置的，具体设置方法参看本实验的“八”。图 5.4 为滤波器的参数设置图，截止频率为 10KHz，选用“汉明窗”，采样率和阶数分别为 50000 和 50。需要注意的是，输入信号 data_in 的位数为 8 位，而输出信号 data_out 为 23 位，所以，截取 data_out 信号的高 8 位再送至 DA 芯片（八位并行）中。

DA 转换模块对输入的 DA_Data_In 信号进行数模转换。使用实验 2.1 节中的 DA 集成板来完成 DA 转换，工作方法与驱动原理与实验 2.1 节 DA 输出实验相同，这里不再赘述。需要注意的是，图 5.1 中的信号①和②均需进行 DA 转换，①得到的是滤波后的波形，②得到的是滤波前的波形。

三、功能模块图与输入输出引脚说明

在 FPGA 中实现此系统，工程包含顶层模块 fir_top，底层模块 dds_module、

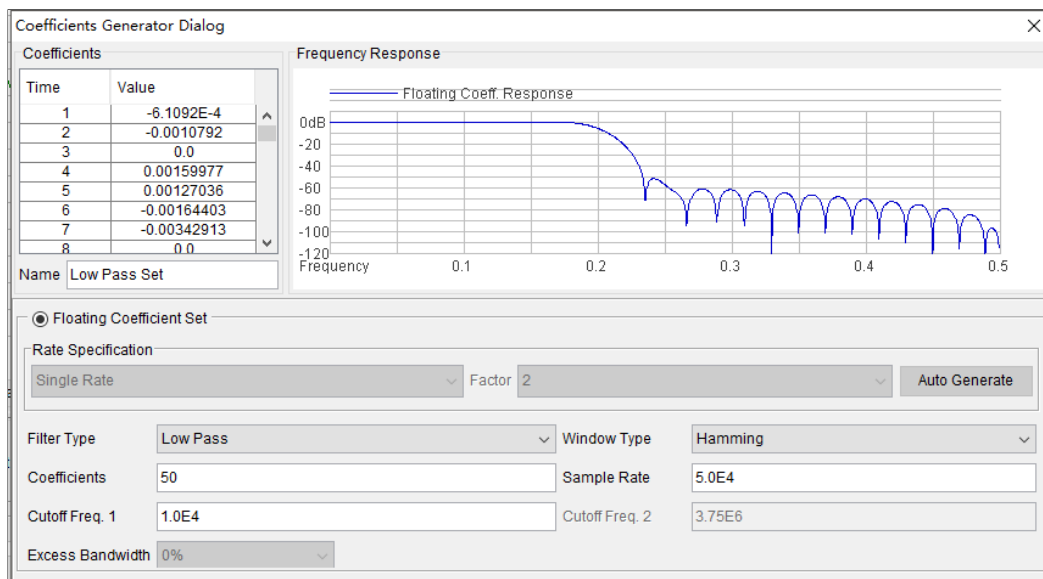


图 5.4 FIR 滤波器的参数设置

low_pass_filter1 和 DAC_module。底层模块依次对应于图 5.1 中的 U1~U3，系统的整体功能模块图参考图 5.1 即可。此外，工程中还包含一个分频模块 p11_70K，是通过 IP 核生成的，用于低通滤波器的时钟输入。图 5.5 是使用 Quartus 生成的顶层文件的 BSF 图，下面介绍一下顶层模块各主要引脚的功能：

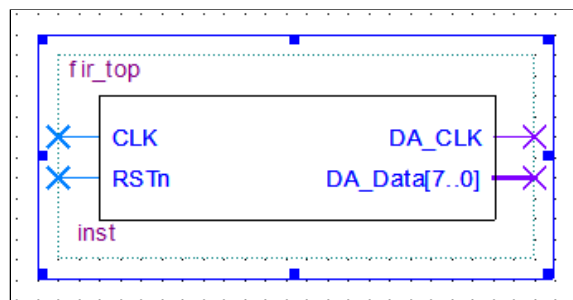


图 5.5 FIR 实验顶层文件 BSF 图

- (1) CLK: 50MHz 的基准时钟信号输入。用于分频。
- (2) RSTn: 系统复位输入信号，低电平有效。复位后 U1 模块输出信号全为 0，送往 DA 集成板的输入信号恒为 0，根据实验 2.1 中的 DA 转换公式 (2.2)，DA 转换结果保持高电平。
- (3) DA_CLK: 送往 DA 芯片的时钟引脚 CLOCK，向其提供时序。
- (4) DA_Data: 送往 DA 芯片的并行数据引脚，共有八位总线。DA_Data 内存储的可能是滤波前的信号也可能是滤波后的信号。

四、程序设计

DDS 模块与 DAC 模块在前面的实验中已经用到过，这里简单介绍下低通滤波器的相关代码。图 5.6 是截取自顶层模块 fir_top 的部分代码：

```

59     output [25:0]data_out;
60
61     wire ast_sink_valid = 1'b1;
62     wire ast_source_ready = 1'b1;
63     wire [1:0]ast_sink_error = 2'b0;
64
65     wire ast_sink_ready;
66     wire ast_source_valid;
67     wire [1:0]ast_source_error;
68
69     low_pass_filter1 U3
70     (
71         .clk( CLK_70K ) , // input
72         .reset_n( RSTn ) , // input
73         .ast_sink_data( data_in ) , // input [10:0]
74         .ast_sink_valid( ast_sink_valid ) , // input
75         .ast_source_ready( ast_source_ready ) , // input
76         .ast_sink_error( ast_sink_error ) , // input [1:0]
77         .ast_source_data( data_out ) , // output [25:0]
78         .ast_sink_ready( ast_sink_ready_sig ) , // output
79         .ast_source_valid( ast_source_valid ) , // output
80         .ast_source_error( ast_source_error ) // output [1:0]
81     );

```

图 5.6 fir_top 部分代码

59-67：定义了该低通滤波器的一些端口参数。

69-81：调用名为 U3 的低通滤波器元件并进行接口的连接。clk 是用于所有内部 FIR 滤波器寄存器的时钟信号，频率选择 70KHz；“FIR 准备好信号” ast_sink_valid，当 FIR 滤波器能在当前时钟周期接收数据时该信号置位，本实验中该信号恒保持为“1”；“数据有效信号” ast_source_ready，当端口数据有效时该信号置位，本实验中该信号恒保持为“1”；ast_sink_error 为“出错信号”，标识在“信宿端”出现违背 Avalon-ST 协议的错误，本实验中该信号恒保持为“00”。

五、FPGA 管脚配置

图 5.7 是使用 Quartus 生成的 Pin Planner 管脚图，CLK 信号与开发板上的 50MHz 的晶振相连；RSTn 信号接 SW0；DA_CLK 和 DA_A[7: 0]分别送到开发板上左侧的 I/O 通道中，再将 DA 集成板的相应引脚与该 I/O 通道相连，连接方式与实验 2.1 节相同。相应引脚连接信息已经在第一篇的 1.3 节中做过详细介绍，这里就不再赘述。

Node Name	Direction	Location
in CLK	Input	PIN_E1
out DA_A[7]	Output	PIN_N3
out DA_A[6]	Output	PIN_T7
out DA_A[5]	Output	PIN_R7
out DA_A[4]	Output	PIN_T6
out DA_A[3]	Output	PIN_R6
out DA_A[2]	Output	PIN_T5
out DA_A[1]	Output	PIN_R5
out DA_A[0]	Output	PIN_T4
out DA_CLK	Output	PIN_P3
in RSTn	Input	PIN_R16

图 5.7 FIR 滤波器实验 Pin Planner

六、实验结果

在验证实验时，使用实验室电源向 DA 集成板提供“5V”电压，DA 集成板、电源和 MINI_FPGA 开发板需要“共地”，使用口袋仪器的“频谱分析仪”和“示波器”功能观察实验结果。

(1) 图 5.8 与图 5.9 分别为滤波前和滤波后的信号波形。滤波前的信号由频率为 6.1KHz 和 24.4KHz 的正弦信号相加而成，滤波后得到了一个频率为 6.1KHz 的正弦信号。滤波器的截止频率为 10KHz, 实验结果与实验设计吻合。

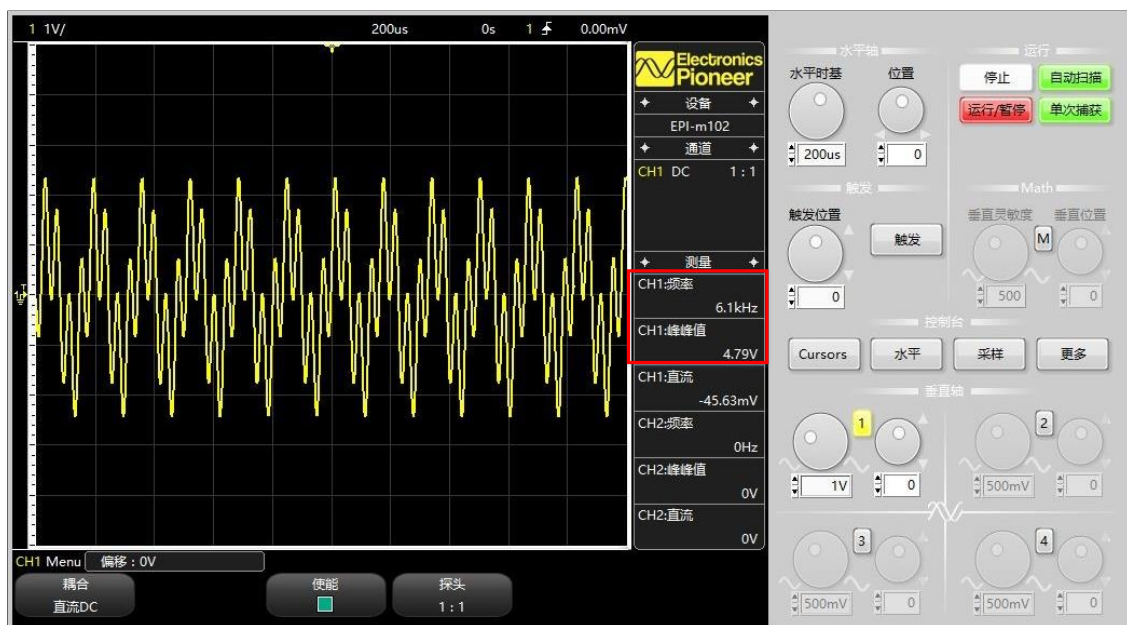


图 5.8 滤波前的信号波形

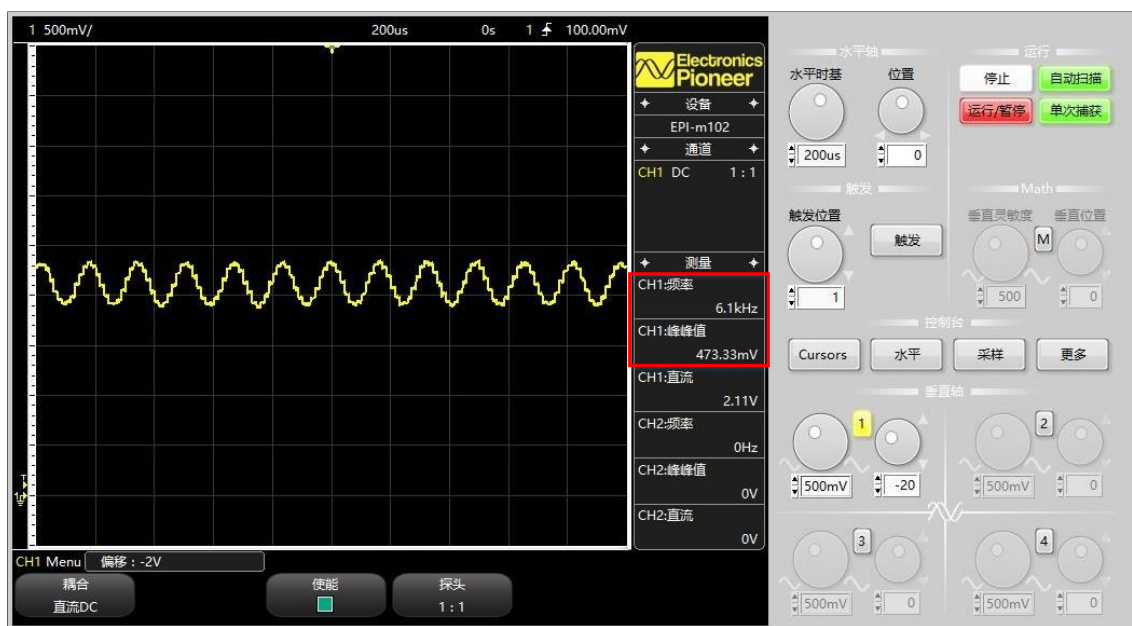


图 5.9 滤波后的信号波形

(2) 图 5.10~5.12 为使用“频谱分析仪”观察到的滤波前信号的频谱图，基波频率为 24.45KHz。如频谱图 2 和 3 所示，滤波前信号包含两个主要的强度相近的频率分量，分别为 6.12KHz 和 24.45KHz，它们的当前幅度分别为 1.55dBV 和 1.65dBV。

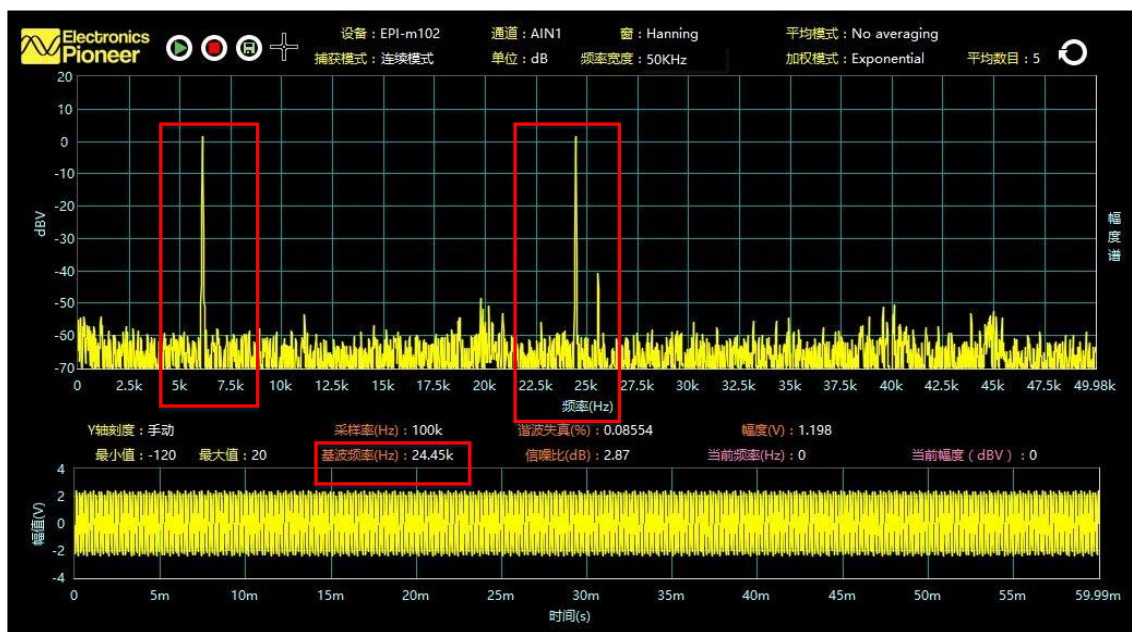


图 5.10 滤波前的信号频谱 1

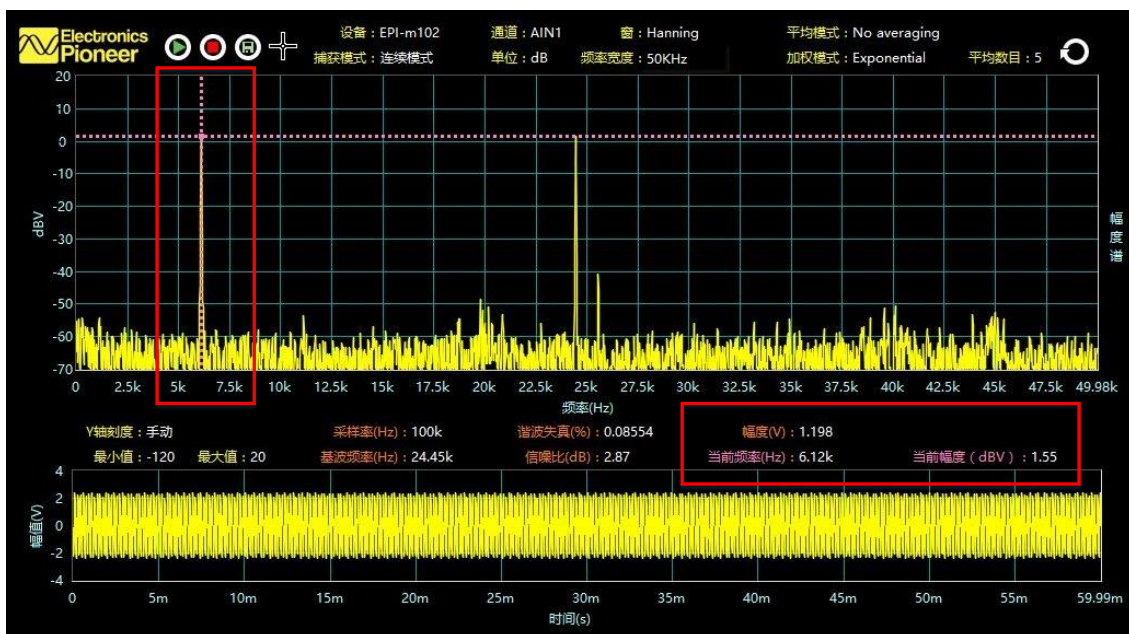


图 5.11 滤波前的信号频谱 2

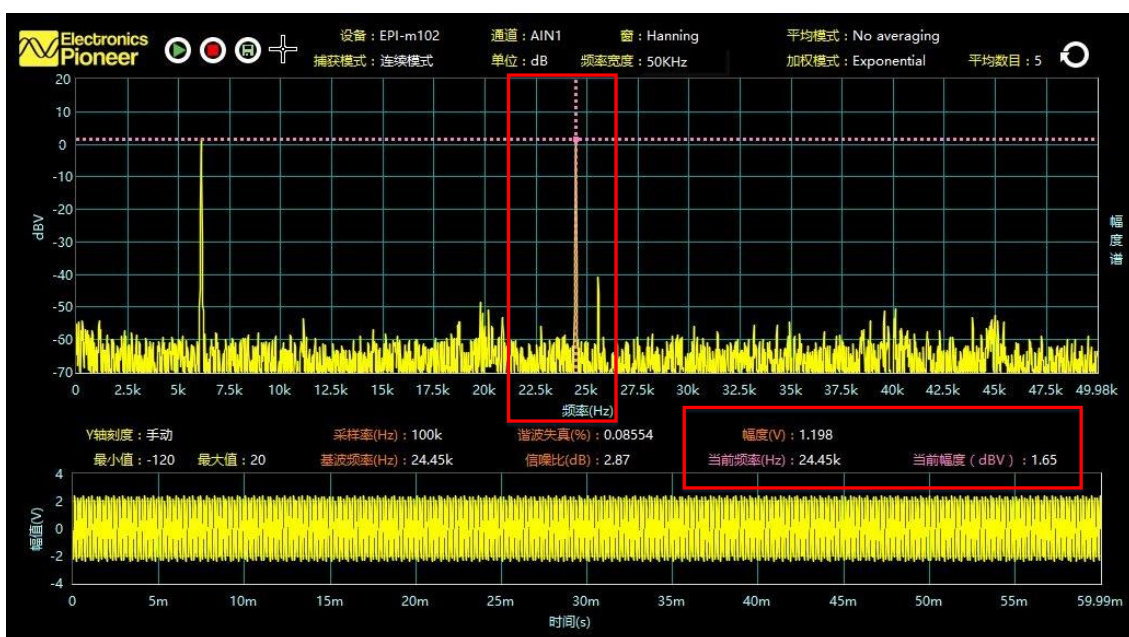


图 5.12 滤波前的信号频谱 3

(3) 图 5.13~5.15 为使用“频谱分析仪”观察到的滤波后信号的频谱图，基波频率为 6.12KHz。如频谱图 5 和 6 所示，滤波后信号的主频分量在 6.12KHz 处，而在频率为 24.45KHz 处，幅度已降至 -59.99dBV。

综上，低通滤波器实验结果基本吻合实验设计，实验效果良好。

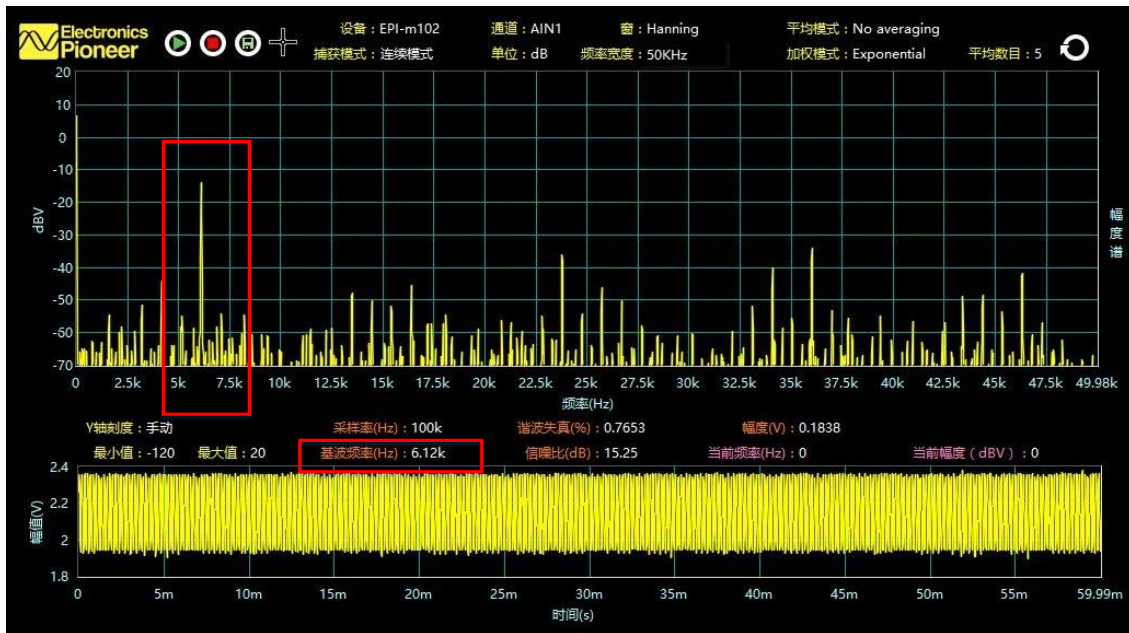


图 5.13 滤波后的信号频谱 4

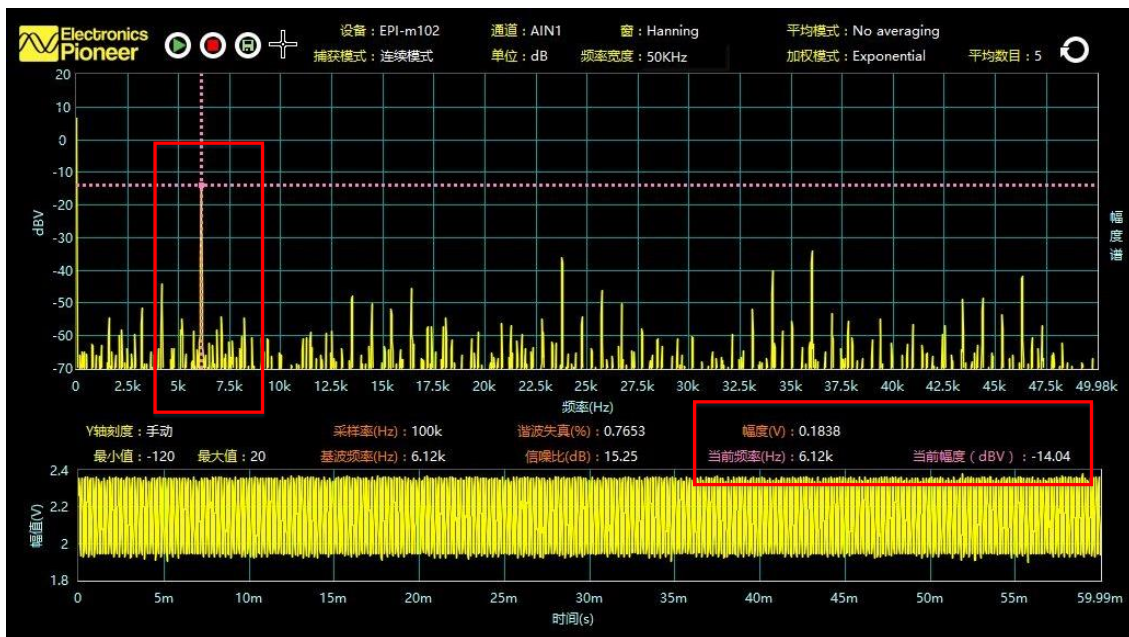


图 5.14 滤波后的信号频谱 5

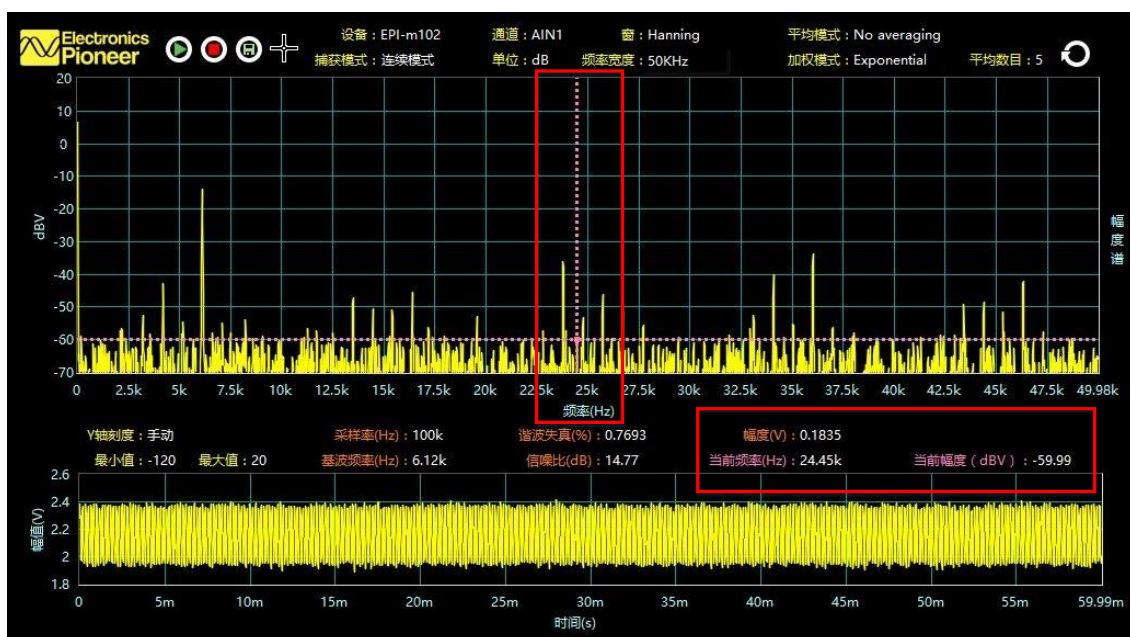


图 5.15 滤波后的信号频谱 6

七、思考与拓展

(1) 本实验使用的 DA 芯片为并行八位数据的，所以 DA_Data_In 信号也必须为八位。当将滤波器输出赋给 DA_Data_In 信号时，截取了它的高八位，请思考，这样做对实验结果（如波形的峰峰值）有什么影响，当截取它的低八位时，实验现象又是怎样的。

(2) 使用 SignalTap 采集 data_in、data_out 等信号的值，观察未经过 DA 转换的滤波前后的波形图，并参考实验 3.2 节时序分析的方法，对本实验中滤波器的相关端口参数进行时序分析。

(3) 调整示波器的纵坐标，放大观察图 5.9，会发现滤波后的波形图是由许多小阶梯组成的，并不光滑，请结合实验 2.1 中的 DA 转换公式 (2.2) 与滤波后送往 DA 芯片的实际值来解释其原因。